RENAR

Requirements Engineering & Normative Adaptive Regulation

Версия 1.0-draft | 22.05.2026

Авторы: Андрей Юмашев, Вадим Соглаев

CC BY-SA 4.0 | renar.tech

---

title: "RENAR Core" description: "Minimum Viable RENAR — 5 правил, ADAPT, 2 шлюза качества." lang: ru version: "1.0-draft" authors: - "Андрей Юмашев (Andrey Yumashev)" license: "CC BY-SA 4.0" site: "renar.tech" order: 1

RENAR Core

Minimum Viable RENAR (MVR)

Версия: 1.0-draft | **Дата:** 2026-05-15 **Авторы:** Андрей Юмашев (Andrey Yumashev)
Copyright: (C) 2026 Andrey Yumashev. Лицензия CC BY-SA 4.0. **Сайт:** renar.tech

---

> *5 правил, 1 обязательный артефакт (ADAPT), 2 шлюза качества и минимальный сквозной пример — всё, что нужно, чтобы вести требования как **источник истины** и не дать AI-ассистентам «дрейфовать» в свободные интерпретации.*
> ***Порядок чтения:** Core (вы здесь) → guide/00-quickstart → standard/00 §0.8 «Язык RU-корпуса» → полный standard/. Никакого специального инструментария не требуется. Время чтения: 20 минут.*

Для кого это

Вы пишете требования к ПО или работаете с AI-агентами, которые пишут код по этим требованиям. Вы заметили: AI красиво формулирует требования и красиво пишет тесты, но реализация всё равно «не то». Клиент при приёмке говорит: «мы это так не имели в виду». Тесты зелёные, продукт неверный.

RENAR Core — это лёгкая дисциплина инженерии требований для AI-нативной разработки. Она ловит расхождение между договорным ТЗ, инженерной интерпретацией и реализацией — **до** того, как разойтись стало дорого. Работает с любым носителем (distributed VCS, документо-ориентированное хранилище, любая БД с версионностью), любым AI-инструментом, любым процессом разработки.

Для команд, которым нужны полная трассировка, контрольный след (audit trail) и соответствие требованиям (compliance), есть полный RENAR Standard — 15 нормативных глав.

Маршруты по ролям

Роль	Старт	Затем

PM / RTE	Core (вы здесь)	guide/05 → guide/09 §E3
Legal / Compliance	guide/09 §E3	guide/06 → reference/07
Regulator / Auditor	reference/07	reference/08 → standard/13
RE engineer	Core → guide/00	standard/06 → standard/10

Подробнее — standard/00 §0.6.4 и guide/README.

Ключевые термины

ТЗ — техническое задание, договорной документ с клиентом. Подписано → immutable. Если в ТЗ ошибка/гэп/противоречие — ТЗ не редактируется; правки идут через ADAPT.

ADAPT (артефакт двусторонней адаптации ТЗ) — артефакт двусторонней адаптации ТЗ. Forward: инженерная интерпретация каждого раздела ТЗ. Backward: список вопросов/противоречий/гэпов, на которые клиент должен дать ответ перед началом реализации. Подписывается дважды: клиентом и архитектором.

SR (System Requirement, системное требование) — инженерное требование, выводимое из approved ADAPT. SR — единица, которая верифицируется контрольным примером (TC).

TC (Test Case, контрольный пример) — контрольный пример (TC), проверяющий **что** делает система, а не **как**. Позитивный + негативный сценарий парно.

Носитель (`substrate`) — система хранения и версионирования артефактов (distributed VCS, документо-ориентированное хранилище, БД с историей). RENAR не привязан к конкретному носителю — требуются только capabilities (immutable history, atomic change, links).

Источник истины — при расхождении кода и требования побеждает требование; код — следствие, а не авторитетное определение поведения.

---

5 правил MVR

Правило 1. ADAPT перед SR

Каждое ТЗ получает ADAPT — артефакт, в котором инженер фиксирует:

- **Forward** — как поняли каждый раздел ТЗ (цитата → инженерная интерпретация → достроенные сценарии → scope: входит/не входит).
- **Backward** — что неясно, противоречиво или отсутствует в ТЗ. Список вопросов клиенту.
- **Term mapping** — таблица «термин клиента → инженерное понимание».

SR не выводятся напрямую из ТЗ. SR ссылаются на разделы **approved** ADAPT.

Зачем это нужно. Без ADAPT инженерные интерпретации идут «молчком» в SR и код. Клиент при приёмке говорит «мы это не имели в виду» — и начинается переоткрытие ТЗ либо тихий компромисс со сдвигом сроков. ADAPT делает интерпретацию явной и подписанной — расхождение ловится до написания кода, не после.

Правило 2. Код не источник истины

При расхождении между кодом и требованием побеждает требование. Если код делает X, а ADAPT/SR говорит Y — это дефект кода, а не «фактическое требование изменилось».

Если требование действительно нужно изменить — изменение идёт через delta-ADAPT с подписью клиента, а не через «переписали код, потом подгоним требование».

> *В полном стандарте инверсия источника истины («код главнее требования») детализирована четырьмя обязательными следствиями: запрет reverse-engineering в SR, разделение code/spec review, drift-hook, запрет молчаливой адаптации SR под код — §2.3.3. Mandatory clause — §13.3.1.*

Зачем это нужно. «Код — это документация» работает только до первой переуступки команды или до первого судебного спора с клиентом. ТЗ — это контракт; ADAPT/SR — формальная интерпретация контракта; код — её реализация. Перевернуть иерархию = потерять провенанс.

Правило 3. Версионирование на носителе

Все артефакты (ТЗ, ADAPT, SR, TC) хранятся на носителе с capabilities:

Capability	Минимум
Immutable history	После approval запись не редактируется
Atomic change	Создание/обновление — одной транзакцией
Stable identifier	ID артефакта переживает rename
Cross-link	Артефакт ссылается на другой по ID
Version pin	SR может сослаться на конкретную ревизию ADAPT

Audit query	«Кто и когда подписал ADAPT-001» — отвечается одной командой

Конкретный носитель — на выбор: distributed VCS с PR-ревью; документо-ориентированное хранилище с API-подписями; любая БД с историей и подписями. RENAR Core не требует git — только перечисленные capabilities.

Зачем это нужно. AI-агент через 3 месяца спросит «откуда это требование». Если носитель не хранит историю — ответа нет. Если нельзя восстановить, кто подписал ADAPT — спор с клиентом затрудняется. Версионирование — не «git-привычка», а инженерное требование к системе хранения требований.

Правило 4. TC pos/neg парность

Каждый SR имеет минимум:

- **1 позитивный TC** — счастливый путь, удовлетворяет SR.
- **1 негативный TC** — граничное условие, ошибочный ввод, отказ внешней системы, недостаток прав.

> *В полном RENAR Standard норма формулируется точнее — на уровне каждого верифицируемого **утверждения** требования (а не «один SR = одна пара»); см. §9.7 и MVR-5. Для Core упрощение «1 SR = 1 pos + 1 neg» достаточно.*

TC проверяет **что** делает система (наблюдаемое поведение), а не **как** (детали реализации). Тест, который ломается при рефакторинге без изменения поведения — это тест реализации, его переписать.

Зачем это нужно. AI-агенты охотно пишут позитивные тесты и обходят негативные («счастливый путь покрыт, всё ок»). 80% инцидентов в продакшене — это именно негативные сценарии: пустой ввод, конкуренция, отказ внешнего сервиса. Без принудительной парности негативный путь остаётся непроверенным до первого инцидента.

Правило 5. Выборочная проверка (ручная верификация)

Раз в спринт (или раз в N задач, если нет спринтов) инженер вручную проверяет **5 случайных** passing TC:

- Запускает TC сам.
- Сверяет фактический результат с тем, что заявлено в SR.
- Проверяет, что TC действительно проверяет SR, а не подделан под код.

Если хотя бы одна из пяти выборочных проверок провалилась — расследовать причину (принцип root-cause: устраняй причину, не симптом).

Зачем это нужно. AI-агенты способны написать ТС, который **формально** зелёный, но **семантически** не проверяет SR (например, `assert True`, мок возвращает ожидаемое значение независимо от поведения). Автоматизация не ловит такие подделки — ловит только человек глазами. Выборочная проверка — дешёвый санитарный фильтр.

---

1 обязательный артефакт — ADAPT

Назначение

ADAPT (*Adaptive Document for Articulating Project's TZ*; по-русски — «адаптивный документ проработки ТЗ проекта») — формализованная двусторонняя интерпретация ТЗ. Один ADAPT на одно ТЗ (плюс delta-ADAPT на каждое delta-ТЗ).

Минимальная схема

Frontmatter

```
---
id: ADAPT-NNN                          # immutable; NNN — порядковый
номер в проекте
title: "Адаптация ТЗ <name>"
type: ADAPT

source-tz:
  id: TZ-YYYY-NNN
  signed-date: "<ISO-date>"
  signed-by-client: "<name-role>"
  document-ref: "<нативная для носителя ссылка>"   # ссылка на pinned
ревизию ТЗ

status: draft | review | approved | frozen

approval:
  client-signature:
    signed-by: "<name>"
    organization: "<client-org>"
    signed-at: "<ISO-datetime>"
    signature-ref: "<нативная для носителя ссылка>"
```

```yaml
  architect-signature:
    signed-by: "<name>"
    signed-at: "<ISO-datetime>"

open-questions-count: integer        # для approved обязательно 0
resolved-questions-count: integer

ai-provenance:
  generated-by: "<vendor>-<model>@<date>"
  human-edits: true                  # обязательно true для approved
---
```

Body — обязательные разделы

```
## Краткое содержание
<3-5 параграфов: о чём ТЗ, scope, основные acceptance критерии>

## Term mapping (клиент → инженер)
| Термин клиента | Инженерное понимание | Раздел ТЗ |
|---|---|---|

## Forward: интерпретация по разделам ТЗ

### ТЗ §N. <название>
<Цитата>
**Интерпретация**: <инженерное понимание>
**Достроенные сценарии**: <список>
**Scope**: входит/не входит
**Forward links** (после approval): SR-NN, SPEC-NN

## Backward: обнаруженные проблемы

### B-001: <короткое описание>
- **Категория**: contradiction | gap | hidden-assumption |
feasibility | regulatory | terminology | scope
- **Статус**: open → asked-to-client → answered → resolved → frozen
- **Описание проблемы**: <...>
- **Вопрос к клиенту**: <...>
- **Ответ клиента** (после answered): <кто, когда, что ответил>
- **Resolution** (после resolved): <как интегрирован ответ в Forward>
```

```
## Резюме backward findings
| Категория | Open | Asked | Answered | Resolved |
|---|---|---|---|---|
```

Lifecycle backward записи

```
open → asked-to-client → answered → resolved → frozen
                ↑                        |
                └──────── revised ─────────┘  (если ответ клиента
расплывчатый)
```

- `open` — инженер записал, клиента ещё не спрашивали.
- `asked-to-client` — вопрос передан клиенту (с датой и каналом).
- `answered` — клиент ответил; ответ зафиксирован в ADAPT с автором и датой.
- `resolved` — ответ интегрирован в Forward; обновлён term mapping и/или достроенные сценарии.
- `frozen` — после approval ADAPT все backward записи immutable.

Категории backward (закрытый список)

Только 7 категорий, новые добавляются только через изменение полного RENAR Standard:

`contradiction` | `gap` | `hidden-assumption` | `feasibility` | `regulatory` | `terminology` | `scope`.

Двойная подпись

ADAPT переходит в `approved` только когда:

1. **Все** backward записи в статусе `resolved` (`open-questions-count == 0`).
2. Подписал клиент (или представитель с полномочиями) — что forward совпадает с тем что хотели, и ответы на backward финальные.

3. Подписал архитектор — что backward отработаны полностью, forward технически реализуем.

Механизм на стороне носителя: на git — PR с двумя required reviewers; на document store — endpoint approval с двумя signature-полями. Конкретика задаётся выбранным носителем.

---

2 Шлюза качества

Два шлюза, через которые проходит каждое требование. Метод обеспечения — на выбор (вручную, автоматически, на рецензировании); важно, чтобы проверка состоялась.

> *Соответствие полному Standard: **QG-ADAPT-approve** — это ADAPT-specific gate одобрения ADAPT (glossary §14.4.2; в реализации может быть локальным псевдонимом QG-3 Architecture), а **QG-2 Verification Gate** совпадает с canonical QG-2 §10.3.3. Имена в Core — педагогические; в Standard нумерация шлюзов шире (QG-0..QG-4).*

Шлюз качества QG-ADAPT-approve

Применяется при попытке перевести ADAPT в статус `approved`.

#	Проверка	Правило
1	Forward охватывает все разделы ТЗ (нет пропущенных §)	Правило 1
2	Term mapping заполнен; нет неопределённых терминов клиента	Правило 1
3	Все backward записи в статусе `resolved` (`open-questions-count == 0`)	Правило 1
4	Подпись клиента присутствует, дата ≥ даты последнего `answered`	Правило 1
5	Подпись архитектора присутствует	Правило 1
6	`ai-provenance.human-edits == true` (текст видел человек)	Правило 1

Если хотя бы один пункт не выполнен — ADAPT остаётся в `review`, approval блокируется.

Шлюз качества QG-2 Verification Gate

Применяется при попытке перевести SR в статус `verified`.

#	Проверка	Правило
1	SR ссылается на approved ADAPT (раздел Forward)	Правила 1, 2
2	Для SR существует ≥ 1 позитивный TC, и он зелёный	Правила 3, 4
3	Для SR существует ≥ 1 негативный TC, и он зелёный	Правило 4
4	TC проверяют поведение из SR, а не детали реализации	Правило 4
5	В текущем спринте выборочная проверка пяти случайных passing TC пройдена	Правило 5
6	Версия носителя, к которой привязан TC, pinned к ревизии SR	Правило 3

Если что-то не выполнено — SR остаётся в `review`. Зелёные тесты без выборочной проверки (пункт 5) не основание для `verified`.

Когда переходить на RENAR Standard

RENAR Core достаточен, пока:

- Один договорной клиент или работа без внешнего клиента (внутренний продукт).
- Команда ≤ 5 человек.
- Нет жёстких compliance-требований (ФЗ-152, GDPR, AI Act, отраслевая регуляторика).
- Один носитель артефактов, без координации между несколькими проектами.

Триггеры перехода на полный RENAR Standard (15 глав):

Триггер	Что добавляет Standard
Появился договорной клиент с подписанным ТЗ → ADAPT нужен с юридически весомой двойной подписью	Глава 7 ADAPT — полная схема, signature requirements, errata workflow
Команда более 5 человек, нужны выделенные роли	Глава 5 Roles — Architect, Requirements Engineer, Reviewer, QA
Compliance (ФЗ-152, GDPR, AI Act)	guide/06 Compliance + reference/03 AI Risk Register

Координация между проектами или командами	Глава 10 Lifecycle + delta-ADAPT chains (Глава 7)
Нужен audit trail для регулятора	Глава 3 Версионирование носителя (V1-V6, включая V6 Audit query) + reference/05 Knowledge Graph Schema
Внедряется AI-агент в роли requirements engineer	reference/04 AI Style Guide + reference/03 AI Risk Register
Появились SPEC-* (API, DATA, ARCH, SEC, UI)	Глава 8 Specifications — closed list 9 типов

Переход не «всё или ничего» — Standard внедряется главами по мере роста зрелости.

Минимальный сквозной пример

Один сквозной пример: ТЗ → ADAPT → SR → TC. Демонстрирует Правила 1, 2, 4.

Шаг 1. ТЗ (фрагмент)

```
TZ-2026-001 §3.1 Регистрация пользователя
Система должна позволять клиенту зарегистрироваться через email.
После регистрации клиент получает доступ к личному кабинету.
```

Шаг 2. ADAPT-001 — Forward (фрагмент)

```
### ТЗ §3.1 Регистрация пользователя

**Цитата**: «Система должна позволять клиенту зарегистрироваться
через email. После регистрации клиент получает доступ к личному
кабинету.»

**Интерпретация**: User с ролью customer создаёт аккаунт через POST
/auth/sign-up
с email + password. Email верифицируется отдельным шагом перед первым
входом.
```

```
**Достроенные сценарии**:
- sign-up: email + password → создание User в статусе `unverified`
- email verification: клик по ссылке из email → перевод User в
`verified`
- первый вход: только для `verified` → редирект в личный кабинет

**Scope**:
- Входит: email/password, email verification, личный кабинет
(профиль).
- НЕ входит: OAuth-провайдеры (Google/Apple), SMS-верификация, 2FA.

**Forward links** (после approval): SR-03, SR-04, SR-05.
```

Шаг 3. ADAPT-001 — Backward (1 запись)

```
### B-007: gap — что происходит при попытке входа неверифицированного
пользователя

- **Категория**: gap
- **Статус**: resolved
- **Ссылка на ТЗ**: §3.1
- **Описание**: ТЗ не описывает поведение системы, если пользователь
  пытается войти до email-верификации.
- **Вопрос к клиенту**: показывать сообщение «подтвердите email» и
предложить
  отправить письмо повторно — или блокировать молча с 401?
- **Asked-to-client**: 2026-05-10
- **Ответ клиента**: 2026-05-12, Иванов И.И. (PM): «Показать
сообщение
  + кнопку повторной отправки. 401 без объяснения — плохой UX.»
- **Resolution**: добавлен сценарий в Forward §3.1; создан SR-05.
```

Шаг 4. SR-03 (derived from ADAPT-001 §4.1 Forward)

```
---
id: SR-03
title: "Sign-up через email/password"
```

```
source:
  adapt: ADAPT-001
  adapt-section: "Forward §3.1"
  tz-section: "§3.1"
status: verified
verified-by: [TC-15, TC-16]
---

Описание: POST /auth/sign-up принимает {email, password}.
При валидных данных создаёт User в статусе `unverified`, возвращает
201.
При невалидном email возвращает 422 с указанием поля.
```

Шаг 5. TC pos/neg парность

```
TC-15 (позитивный):
  given: новый email, валидный password (≥ 8 символов)
  when: POST /auth/sign-up {email, password}
  then: status 201; User создан в статусе unverified; email с
verification link отправлен

TC-16 (негативный):
  given: невалидный email "not-an-email"
  when: POST /auth/sign-up {email, password}
  then: status 422; body содержит {"field": "email", "error":
"invalid format"}
```

Цепочка трассировки полная: TC-16 → SR-03 → ADAPT-001 §4.1 Forward → TZ-2026-001 §3.1.

При аудите через год: «откуда взялось требование возвращать 422» — ответ из B-007 не возникает (B-007 о другом), но трассировка от TC к ТЗ восстанавливается без догадок.

Что дальше

RENAR Core — отправная точка. Когда команда растёт, объём ТЗ/SR увеличивается или появляются compliance-требования, переходите на полный RENAR Standard поэтапно

(см. таблицу триггеров выше).

Соответствие Core: если вы соблюдаете 5 правил, ведёте ADAPT на каждое Т3 и стабильно проходите оба шлюза — вы практикуете RENAR Core. Никакого официального подтверждения не требуется; самооценка.

Соответствие правил Core правилам Standard:

Правило Core	Эквивалент в Standard
1. ADAPT перед SR	Глава 7 ADAPT + Глава 10 Lifecycle и QG (ADAPT lifecycle)
2. Код не источник истины	Глава 2 §2.3 Source of Truth: требования, не код + Глава 13 §13.3.1 SoT inversion
3. Версионирование на носителе	Глава 3 Версионирование носителя — capabilities V1-V6
4. TC pos/neg парность	Глава 9 §9.7 Pos/neg pairing + QG-2 Verification Gate (Глава 10)
5. Выборочная проверка инженера	Глава 9 §9.14 Спот-чек инженера

Краткая справочная карта

Перед началом проекта:

1. Создайте ADAPT-001 на Т3. Forward по каждому §. Backward — всё что неясно.
2. Спросите клиента по backward. Зафиксируйте ответы с датой и автором.
3. Подпишите ADAPT дважды (клиент + архитектор). До approval не пишите SR.

При написании SR:

4. SR ссылается на approved ADAPT §, не на Т3 напрямую.
5. Каждый SR — минимум 1 позитивный + 1 негативный TC. Парно.

Перед закрытием SR (QG-2 Verification Gate):

6. Оба TC зелёные.
7. TC проверяют поведение, не реализацию.
8. В текущем спринте выборочная проверка пяти случайных passing TC прошла.

При изменении требований:

9. ТЗ не редактируете. Клиент даёт delta-ТЗ → создаёте delta-ADAPT с новой подписью.
10. SR/TC обновляются после approval delta-ADAPT, не до.

RENAR Core 1.0-draft — renar.tech Copyright (C) 2026 Andrey Yumashev. Лицензия CC BY-SA 4.0.

---

title: "Введение" order: 0 lang: ru

00. Введение

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

> ***Новичкам.*** *Эта глава — нормативная и плотная (RFC-2119, closed lists, mandatory clauses). Если вы впервые встречаете RENAR — начните с мягкого введения* `core/renar-core.md` *(≈20 мин) и* `guide/00-quickstart` *(≈30 мин), затем возвращайтесь сюда. Политика языка RU-корпуса — §0.8.*

0.1 Зачем эта глава

Когда код пишут AI-агенты, узким местом перестаёт быть скорость разработки — им становится **корректность требований**. Одно и то же требование живёт сразу в нескольких местах: договорное ТЗ клиента, инженерная декомпозиция, спецификации, тест-кейсы и, наконец, сам код. Всё это пишет и правит смесь людей и агентов — и без явных контрактов между артефактами смысл требования незаметно расходится. RENAR существует, чтобы этого расхождения не возникало.

Эта глава — карта всего стандарта. Она отвечает на четыре вопроса: **что** такое RENAR (§0.2), **зачем** он нужен (§0.3), **как** он соотносится с SENAR (§0.4) и **каков минимум**, ниже которого о соответствии говорить нельзя, — **Minimum Viable RENAR** (минимально жизнеспособный RENAR, далее MVR; §0.5). Дальше — навигация по корпусу (§0.6), политика закрытых списков (§0.7) и язык RU-корпуса (§0.8).

Глава **не** вводит новых нормативных требований. Каждое из семи утверждений MVR — это ссылка на главу §1–§13, где оно превращается в точную норму; здесь дано лишь связное целое, чтобы их было видно вместе. Дублировать нормирующие формулировки запрещено — только ссылаться.

> ***Конвенция модальных глаголов (для всего стандарта).*** *Нормативная сила выражается русскими модальными словами: «обязан» / «должен» — mandatory (RFC-2119* ***shall*** */* ***must****); «следует» / «рекомендуется» — recommended (****should****); «может» / «допустимо» — permitted (****may****); «запрещён» / «не должен» —* ***shall not****. Соответствие — RFC-2119 и ISO/IEC/IEEE 29148:2018 §5.2.1. Конвенция действует во всех нормативных главах и обязательна для EN-перевода.*

0.2 Что такое RENAR

0.2.1 Нормативный стандарт инженерии требований (requirements engineering)

RENAR (**R**equirements **E**ngineering & **N**ormative **A**daptive **R**egulation) — нормативный стандарт управления требованиями для разработки с AI-агентами. Стандарт нормирует:

- **Модель данных (data model)** артефактов требований (BR / SR / TR), ADAPT, девяти типов SPEC, TC.
- **Lifecycle** артефактов через закрытый список Quality Gates (QG-0 / QG-1 / QG-2 обязательные; QG-3 / QG-4 опциональные).
- **Возможности носителя** V1–V6, которым носитель проекта обязан удовлетворять.
- **Соответствие (conformance)** — уровни (RENAR-1..RENAR-5), mandatory clauses, manifest, процедуры оценки.

0.2.2 Специализация SENAR

RENAR — **специализация SENAR** (§1.6) в области инженерии требований (requirements engineering), не альтернатива SENAR и не самостоятельная методология. SENAR предоставляет методологическую базу (5 ценностей, 14 правил, общие Quality Gates, 5 базовых ролей); RENAR нормирует только те аспекты, которые SENAR оставляет на усмотрение доменного стандарта.

Реализация, соответствующая RENAR, **всегда** совместима с SENAR; обратное — неверно (§1.6.2).

0.2.3 Независимость от носителя

RENAR не привязан к конкретному носителю (система контроля версий, документная БД, wiki): нормативные главы используют язык, независимый от носителя (§3.1), и нормируют **возможности** V1–V6, а не конкретные продукты или команды. Указания, специфичные для носителя, вынесены в `guide/03-tool-guide-*.md`.

RENAR — стандарт, а **не** реализация: реализации могут существовать на разных носителях (single-SSoT правило, §3.3) при условии удовлетворения V1–V6 и mandatory clauses (§13.3).

0.3 Зачем существует RENAR

0.3.1 Проблема: дрейф требований

В разработке с AI-агентами требования живут одновременно в нескольких артефактах (ТЗ от клиента, decomposed BR/SR, SPEC, TC, описание задачи, реализация кода), создаваемых и поддерживаемых смесью human + AI authors. Без нормативных контрактов между артефактами возникает **дрейф** — расхождение между тем, что зафиксировано, тем, что верифицируется, и тем, что фактически реализовано.

RENAR закрывает восемь классов дрейфа:

#	Класс дрейфа	Что происходит	Нормативное закрытие
1	**Schema drift**	Поля артефактов расходятся между проектами и носителями	Closed list нормативных областей §1.2 + canonical schema (reference/02-schemas)
2	**Lifecycle drift**	Статусы (`draft` / `approved` / `verified`) значат разное у разных авторов	Canonical state machines + closed list QG (§10)
3	**Source-of-truth drift**	Одна сущность правится одновременно в нескольких местах	Single-SSoT designation per артефакт (§3.3 V3)
4	**Implementation drift**	Код реализует требование, которое уже удалено или переименовано	Immutable identifiers + V5 version-pin (§3.3); reference-validation hooks (§10.11.1)
5	**Terminological drift**	Термины (`verified` , `implemented` , `approved`) значат разное у разных людей	Canonical глоссарий (§4) — один термин = одно состояние lifecycle
6	**Order / provenance drift**	Delta-ТЗ применяется не в порядке, ссылается на несуществующее требование	Delta-ADAPT workflow + V6 author/timestamp (§7.6, §3.3)
7	**TC ↔ requirement provenance drift**	TC верифицирует устаревшее поведение (`requirement-version` отстаёт)	Pinned `verifies[].version` + QG-2 блок promote в `verified` (§9, §10.3.3)

8	**Test-fitting drift**	AI-агент ослабляет Pass/Fail-критерии ТС вместо исправления реализации	Adversarial review pattern + separate approval критериев ТС (§9.4, §10.3.3)

0.3.2 Норма как контракт между AI-агентами

Восемь классов дрейфа — структурные, не операционные: они возникают из самого факта совместного владения артефактом несколькими авторами (включая AI), а не из ошибок дисциплины. Закрыть их можно только нормативно — фиксацией контракта о том, как артефакты связаны, кто пишет какое поле, и какие предусловия обязаны быть выполнены при переходах (§13.3 mandatory clauses).

Без RENAR (или эквивалентного нормативного слоя над SENAR) дрейф неотвратим. С RENAR — нативные для носителя hooks (§10.11) обеспечивают enforcement, а conformance manifest (§13.4) фиксирует уровень формального соответствия.

0.4 Связь с SENAR

0.4.1 RENAR ⊂ SENAR

RENAR нормирует **только** аспекты инженерии требований (requirements engineering), которые SENAR оставляет на усмотрение domain-стандарта: data model артефактов, lifecycle requirement-артефактов, conformance manifest для RE. RENAR **не** дублирует и **не** переопределяет SENAR-конструкции: 5 ценностей, 14 правил, общие Quality Gates (§10.2.3), 5 базовых ролей (§5.2) — нормируются SENAR.

0.4.2 Conformance manifest объявляет обе версии

Conformance manifest проекта (§13.4) обязан декларировать одновременно `senar-version` и `renar-version`. Заявление о соответствии RENAR без указания SENAR-version или с указанием несовместимой SENAR-version — non-conformant (§13.8).

0.4.3 Несовместимость = баг стандарта

Если нормативное утверждение RENAR оказывается несовместимым с нормой SENAR — это баг стандарта RENAR. Разрешение — через formal change procedure SENAR + corresponding alignment в RENAR (§13.9), не через project-local переопределение. Реализация **не** имеет права заявить «следуем RENAR вместо SENAR» (§1.6.3).

0.4.4 Разделение ответственности

Аспект	SENAR (база)	RENAR (специализация RE)
Методологические ценности	5 ценностей разработки с AI-агентами	— (наследует)
Общие правила	14 правил агентской работы	— (наследует)
Общие Quality Gates	Общая методология гейтов / фреймворк контрольных точек	Closed list canonical gates QG-0..QG-4 (§10.3)
Базовые роли	5 ролей SENAR §4: Supervisor, AI-агент, Architect / Tech Lead, Reviewer, Stakeholder	RE-specializations поверх SENAR §4: Architect / Tech Lead (approver), Client representative (Stakeholder), AI-author (§5)
Модель данных (data model) артефактов	— (вне scope)	BR / SR / TR / ADAPT / 9 SPEC types / TC (§6, §7, §8, §9)
Lifecycle requirement-артефактов	— (вне scope)	Canonical state machines (§10)
Возможности носителя	— (вне scope)	V1–V6 (§3)
Conformance manifest	Типовой manifest SENAR	RENAR-CONFORMANCE.yaml + mandatory clauses (§13.3, §13.4)

Пересечение строк «— (наследует)» означает, что RENAR использует SENAR-конструкцию **как есть**, не модифицирует. Пересечение строк «— (вне scope)» означает, что SENAR оставляет аспект на усмотрение domain-стандарта, и RENAR его нормирует.

Что оригинально именно как RENAR (не унаследовано из SENAR): data model требований (BR / SR / TR + 9 типов SPEC + TC); артефакт ADAPT с двусторонней адаптацией Т3 и двойной подписью (§7); независимые от носителя возможности V1–V6 (§3); canonical lifecycle и QG для оси требований (§10); pos/neg-парность и judge ≠ production как блокирующие clauses (§9); модель уровней соответствия (conformance) RENAR-1..RENAR-5 с manifest (§13). Методологический каркас (ценности, правила, роли, базовые QG-концепты, классы дрейфа) наследуется из SENAR без переписывания. Это разграничение защищает от прочтения RENAR как тонкой обёртки над SENAR: вклад RENAR — нормативная data-model-ось и её enforcement, а не повторение методологии.

0.5 Minimum Viable RENAR (MVR)

Minimum Viable RENAR — закрытый список из семи нормативных утверждений, которым обязана удовлетворять любая RENAR-conformant реализация независимо от объявленного уровня (включая `RENAR-1`). MVR эквивалентен mandatory clauses §13.3; глава 0 представляет MVR как первичную сводку для читателя, §13.3 — как нормативную деталь для assessor-а.

0.5.1 Семь утверждений

#	Утверждение[1]	Нормативный источник
MVR-1	**SoT inversion**: иерархия артефактов требований обязана быть источником истины о поведении системы; код — производный артефакт реализации. Обратная разработка (reverse-engineering) поведения из кода в SR без обоснования bug-fix запрещена.	§2.3, §13.3.1
MVR-2	**V1–V6 capabilities**: носитель проекта обязан удовлетворять всем шести возможностям — immutable history (V1), atomic change unit (V2), diff & review (V3), branching / change-set (V4), cross-substrate version pin (V5), author + timestamp (V6).	§3.3, §13.3.2
MVR-3	**ADAPT per T3**: каждое T3 обязано иметь ровно один корневой ADAPT в статусе `approved` с двусторонней client-side validation и двойной подписью (Architect + Client representative). Delta-T3 обязано иметь свой delta-ADAPT.	§7, §13.3.3
MVR-4	**Closed list 9 SPEC types**: тип SPEC обязан принадлежать закрытому списку `SPEC-ARCH` / `API` / `DATA` / `INT` / `PROC` / `UI` / `AI` / `SEC` / `OPS`. Проект не имеет права создавать новые типы локально.	§8.3, §13.3.4
MVR-5	**TC pos/neg парность**: каждое нормативное утверждение верифицируемого артефакта (BR / SR / SPEC / TR), охватываемое хотя бы одним TC, обязано иметь парный negative TC. Исключение — утверждение само описывает negative invariant.	§9.7, §13.3.5
MVR-6	**Quality Gates closed list**: реализация обязана поддерживать QG-0 (Approval), QG-1 (Implementation), QG-2 (Verification) как `required`. QG-3 (Architecture) и QG-4 (Acceptance) — `declared` или `absent`. Проект не имеет права создавать новые gate-типы локально.	§10.3, §13.3.6

MVR-7	**Conformance manifest**: проект обязан содержать conformance manifest, объявляющий одновременно `renar-version` + `senar-version` + `level` (RENAR-1..RENAR-5) + подтверждение всех mandatory clauses §13.3. Manifest immutable в смысле V1.	§13.4

[1] Каждая строка MVR задаёт обязательный уровень требования (по семантике RFC-2119 «shall» в русских формулировках — «обязан» или «должен» по контексту) в трактовке ISO/IEC/IEEE 29148:2018 §5.2.1 — см. §14.4.2.

0.5.2 Minimum Viable RENAR — необходимое и достаточное условие соответствия

Реализация, удовлетворяющая всем семи утверждениям MVR, соответствует **минимум** уровню `RENAR-1` (§13.2.1). Уровни `RENAR-2..RENAR-5` добавляют **дополнительные** обязательства поверх MVR (§11). Реализация, нарушающая хотя бы одно утверждение MVR, **не имеет** соответствия стандарту RENAR в целом — независимо от заявленного уровня (§13.8).

0.6 Структура документа

0.6.1 Нормативные главы (`standard/`)

#	Глава	Назначение
00	Introduction	Эта глава — определение RENAR, MVR, навигация
01	Scope	Closed list нормативных областей и исключений; основной и негативный scope
02	Methodology positioning	SoT inversion (SDD); waterfall-form ≠ classical waterfall; независимое от носителя версионирование
03	Версионирование носителя	Возможности V1–V6; независимый от носителя нормативный язык; single-SSoT правило
04	Terms and definitions	Canonical глоссарий — закрытие terminological drift
05	Roles	Роли над SENAR §4; ответственность за артефакты; двойная подпись ADAPT

06	Requirements hierarchy	BR / SR / TR; декомпозиция система / подсистема / модуль
07	ADAPT	Двусторонняя адаптация T3; forward + backward findings; двойная подпись
08	Specifications	Closed list 9 типов SPEC; общая схема + type-specific extensions; `constrained-by[]`
09	Test cases	TC как полноценный артефакт (TC); pos/neg парность; adversarial review; VLM-judge
10	Lifecycle и QG	State machines; QG-0..QG-4; hooks enforcement
11	Maturity model	RENAR-1..RENAR-5; checklist по уровням
12	Metrics	REQ-специфичные метрики (RDLT, Hallucination Rate, DRA, ACR); бизнес-outcomes — в guide
13	Conformance	Mandatory clauses; manifest; self-assessment + third-party; loss-of-conformance
14	Normative references	Нормативные (ISO/IEC 29148, 25010, 5338, 23894) и информативные (BABOK, SAFe, NIST AI RMF, EARS/BDD, ISO 42001, EU AI Act)

0.6.2 Вспомогательные каталоги (Supporting directories)

Directory	Статус	Назначение
`core/`	**non-normative**	Gentle single-doc введение для read-первого знакомства + `core-mode` boundary case (§1.5.4)
`guide/`	**non-normative**	Practical guides: quickstart, walkthrough, transition guide, специфичные для носителя руководства по инструментам, compliance, failure modes
`reference/`	**non-normative (informative)**	Приложения: glossary, schemas, AI risk register, AI style guide, knowledge graph schema
`research/`	**вне публикации**	Внутренние обоснования при разработке стандарта; не часть нормативного корпуса

Терминология, зависящая от носителя (§1.3.1), присутствует **только** в `guide/` и `reference/` (примеры); нормативные главы — независимы от носителя.

0.6.3 Рекомендованный порядок чтения

Первое знакомство (on-ramp, non-normative): core/renar-core.md (≈20 мин) → guide/00-quickstart (≈30 мин) → standard/README → глава 06. Не начинайте normative reading с главы 0 без этого on-ramp — см. callout в начале главы 0.

Последовательное чтение standard/ (assessor / RE engineer): главы выстроены в порядке аргумента — читайте подряд **00** → **14**. Фундамент (положение в типологии, §2) и инфраструктура носителя (V1–V6, §3) сознательно вынесены вперёд: главы об артефактах (06–10) на них опираются.

0.6.4 Маршруты читателей

Роль	Старт	Затем	Фокус
PM / RTE	core	guide/05 → guide/09 §E3	BR, приоритеты, SAFe mapping
Legal / Compliance	guide/09 §E3	guide/06 → reference/07	Traceability, GDPR/ФЗ-152 evidence
Regulator / Auditor	reference/07	reference/08 → §13	ISO 29148 mapping, manifest, mandatory clauses
RE engineer / Architect	core → guide/00	§6 → §10	SoT, ADAPT, QG, schema

Полная таблица ролей — guide/README §Маршруты. Плотность глав — reference/09.

0.7 Политика закрытого списка (closed list)

0.7.1 MVR закрыт на v1

Список из семи утверждений §0.5.1 — closed на v1.0. Проект не имеет права расширять MVR локально (добавлять MVR-8 и далее) или сокращать список. Все семь утверждений обязательны для соответствия стандарту в целом (§13.3).

0.7.2 Declared-stricter допустимо

Реализация может **ужесточить** требования сверх MVR, декларируя в conformance manifest (§13.4) с маркером `declared-stricter` (§10.10.2). Примеры:

- Требовать QG-3 + QG-4 как `required` (вместо опциональных).
- Требовать adversarial review TC (§9.4) на всех типах SPEC, не только `SPEC-SEC` / `SPEC-AI`.
- Декларировать `RENAR-3` или выше как минимальный уровень для всех проектов организации.

Declared-stricter — допустимая локальная политика; соответствие нормативному минимуму сохраняется.

0.7.3 Declared-weaker запрещено

Реализация **не** имеет права declared-weaker относительно MVR:

- Выдать заявление о соответствии RENAR без поддержки одного из MVR-1..MVR-7.
- Объявить ADAPT опциональным (нарушение MVR-3).
- Допустить single-TC-coverage для нормативного утверждения без negative-invariant исключения (нарушение MVR-5).
- Опустить `senar-version` или `level` в manifest (нарушение MVR-7).

0.7.4 Путь расширения

Изменение MVR (добавление, удаление, переформулировка утверждения) — только через formal change procedure стандарта RENAR (§13.9):

1. **Research draft** — обоснование с практическими случаями.
2. **Public review** — открытое обсуждение партнёрами и сообществом.
3. **Minor-version bump** — изменение включается в `v1.X` или `v2.0`.
4. **Migration guidance** — для существующих conformant проектов.

0.8 Язык RU-корпуса

RU normative корпус RENAR **не является полностью русифицированным текстом**. Это осознанный гибрид: **canonical identifiers** (латиница и closed-list аббревиатуры) + **русская связующая проза**. Политика зафиксирована в reference/06-ru-style-guide.md §1 и standard/04-terms.md §4.2.

Что остаётся латиницей и почему:

- **Артефакты и ID** (`BR` , `SR` , `TR` , `ADAPT` , `SPEC-*` , `TC` , `QG-*`) — closed list §4.3–§4.7; замена на RU-only form в frontmatter, YAML и именах файлов **запрещена** (referential integrity с schemas и hooks).
- **Lifecycle statuses** (`draft` , `approved` , `verified` , …) — system-readable identifiers Bucket B; перевод в normative prose разрывает state machines §10.
- **VCS domain** (`commit` , `hook` , `runner` , `frontmatter` , …) — Bucket A; RU-эквиваленты теряют точность или конфликтуют с возможностями V1–V6 §3.
- **Accepted loanwords** (`traceability` , `provenance` , `adversarial review` , drift-классы §0.3.1) — domain terms международной RE/AI-практики; см. Style Guide §1.11.

> ***Носитель, не `substrate`.*** *Концепт хранилища артефактов по-русски — **«носитель»** (склоняется). Латиницей `substrate` остаётся только в именах полей (`substrate-capabilities`), коде/YAML и именах файлов; в связующей прозе — всегда «носитель» (решение сессии #27, reference/06 §1).*

Что переводится: организационная терминология (Bucket C), rewrite-кальки (Bucket E: `имплементация` → «реализация»), gratuitous EN в связующей ткани предложения. Editorial rule: **≤1 latin token на нормативное предложение**, кроме canonical identifiers и цитат.

UI-перевод ≠ canonical: multilingual UI может показывать «Бизнес-требование» вместо `BR` (§4.13.3); body normative paragraphs, frontmatter и ID — всегда canonical. Для onboarding см. core/renar-core.md (first-mention: русский эквивалент + `canonical` в скобках).

0.9 Перекрёстные ссылки

Источник	Применение в этой главе
SENAR (полная методология)	Методологическая база RENAR (§0.4.1); RENAR ⊂ SENAR.
§1 Scope	Closed list нормативных областей и исключений; основной и негативный scope; принцип независимости от носителя.
§4 Terms	Canonical глоссарий — закрытие terminological drift (§0.3.1 класс 5).

§5 Roles	Роли + двойная подпись ADAPT — основа MVR-3.
§2 Methodology positioning	SoT inversion — основа MVR-1.
§7 ADAPT	Forward + backward findings, двойная подпись, delta-ADAPT — основа MVR-3 и §0.3.1 класс 6.
§8 Specifications	Closed list 9 типов SPEC — основа MVR-4.
§9 Test cases	pos/neg парность, adversarial review — основа MVR-5 и §0.3.1 классы 7-8.
§10 Lifecycle и QG	Canonical state machines + closed list QG — основа MVR-6 и §0.3.1 классы 2, 7.
§3 Версионирование носителя	Возможности V1–V6 + single-SSoT правило — основа MVR-2 и §0.3.1 классы 1, 3, 4, 6.
§11 Maturity model	RENAR-1..RENAR-5 — уровни поверх MVR.
§13 Conformance	Mandatory clauses §13.3 ≡ MVR §0.5.1; manifest §13.4 — основа MVR-7.
core/renar-core.md	Core-mode boundary case (§1.5.4) для internal product без external client.

Оглавление · **Следующая: 01. Область применения →**

---

title: "Область применения" order: 1 lang: ru

01. Область применения

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

1.1 Где RENAR работает, а где нет

RENAR — не универсальная дисциплина «на любой проект». Он создан для разработки, где есть договорное ТЗ и сторона, перед которой за это ТЗ отвечают: заказная разработка, регулируемые отрасли, консалтинг по чужому ТЗ. На чистом продуктовом дискавери, где «сначала строим, потом понимаем, что строили», RENAR не просто избыточен — он структурно неприменим: нет неизменяемого ТЗ, не из чего строить ADAPT. Эта глава проводит обе границы — **предметную** (что стандарт нормирует) и **контекстную** (когда им вообще стоит пользоваться).

Границы заданы закрытыми списками: что RENAR нормирует (§1.2) и что явно оставляет вне себя (§1.3); где он применим как основной контекст (§1.4) и где неприменим в принципе (§1.5). Связь с SENAR — §1.6, политика закрытых списков — §1.7. Все списки закрыты на v1; расширить их можно только формальной процедурой изменения стандарта (§13.9).

Глава **не** определяет содержательные нормы артефактов и lifecycle — это область глав 06–14. Глава **не** определяет нативные для носителя механизмы реализации — это область главы 3 и `guide/03-tool-guide-*.md`.

1.2 Что нормирует RENAR (closed list)

Закрытый список нормативных областей RENAR на версии v1.0. Каждая область рассматривается в указанной главе:

#	Область	Глава
1	Иерархия требований (BR / SR / TR)	06
2	ADAPT (двусторонняя адаптация ТЗ): forward интерпретация + backward findings + двойная подпись	07

3	Closed list 9 типов спецификаций (SPEC-ARCH / API / DATA / INT / PROC / UI / AI / SEC / OPS)	08
4	Test Cases как полноценный артефакт (TC); pos/neg парность; spec-specific TC-обязательства	09
5	Lifecycle states + Quality Gates (QG-0 / QG-1 / QG-2 обязательные; QG-3 / QG-4 опциональные)	10
6	независимые от носителя возможности версионирования V1–V6	03
7	Maturity model (RENAR-1..RENAR-5)	11
8	Метрики requirements engineering (RDLT, Hallucination Rate, DRA, ACR и др.)	12
9	Conformance (mandatory clauses, manifest, self-assessment, third-party assessment)	13
10	Роли и ответственность за артефакты (specializations над SENAR §4)	05

Все перечисленные области имеют нормативное содержание: обязательные требования и запрет project-local расширений по соответствующим closed lists.

1.2.1 Что относится к нормативной области

К нормативной области RENAR относятся:

- **Модель данных (data model) артефактов** — обязательные frontmatter поля, типы, enum-значения, инварианты согласованности.
- **Lifecycle states + переходы** — state machines per артефакт-тип; pre/post-conditions; gate-id для каждого перехода.
- **Identity и provenance** — immutable identifiers; нативная для носителя фиксация авторства и времени (V6); version pin между артефактами (V5).
- **Cross-artifact constraints** — `verifies[]`, `constrained-by[]`, `parent`, `delta-of`, `verified-by`, `implements-spec` — нормативная семантика связей и правила консистентности.
- **AI-provenance** — обязательная фиксация модели-генератора, prompt-template, объёма tokens; правила human-edits для approval.
- **Conformance procedures** — mandatory clauses, manifest schema, assessment cadence, loss-of-conformance handling.

1.3 Что RENAR явно НЕ нормирует (closed list)

Закрытый список областей, которые **намеренно** оставлены за пределами стандарта. Реализация в этих областях — свободный выбор и не влияет на conformance.

#	Область	Что вне scope
1	**SENAR-методология в целом**	5 ценностей, 14 правил, общие Quality Gates, agent instrumentation — нормируются SENAR; RENAR не дублирует и не переопределяет.
2	**Конкретный носитель / VCS**	Выбор конкретного version-control / document-database / wiki-platform — на усмотрение реализации; RENAR нормирует только возможности V1–V6 (§3).
3	**Tech stack реализации**	Языки программирования, фреймворки, базы данных, инфраструктурные компоненты — вне scope; RENAR нормирует требования, не реализацию.
4	**Конкретный UI / IDE / редактор артефактов**	Web-интерфейсы, IDE-расширения, CLI-инструменты — вне scope; нормируется только формат хранения артефактов в носителе.
5	**Конкретные test runners**	pytest, jest, playwright, ragas, и др. — специфичны для носителя; RENAR нормирует только обязательность `automation.location` и `last-run` фиксации (V5+V6).
6	**Бизнес-процессы продаж / договоров**	Pre-sale, формулировка договорной стоимости, юридическая структура контрактов — вне scope; RENAR работает с ТЗ как с **входом** и не нормирует процесс его создания на стороне клиента.
7	**Project management practices**	Agile-ceremonies, sprint planning, kanban-boards, story-points estimation — вне scope; RENAR нормирует workflow артефактов требований, не management-overhead.
8	**AI model selection и prompt engineering**	Выбор LLM-провайдера, prompt-templates, fine-tuning стратегии — вне scope; RENAR нормирует только обязательную фиксацию `ai-provenance.generated-by` и adversarial review pattern (§9.4).
9	**Конкретные нативные для носителя команды и hooks**	специфичные для носителя CLI-команды (например, конкретные имена команд VCS), реализация hooks (§10.11) — специфичны для носителя и относятся к `guide/03-tool-guide-*.md` .
10	**Юридическая интерпретация artifact-подписей**	Электронная подпись, юридическая значимость, GDPR-обработка персональных данных в ТЗ — вне scope

		стандарта; нормируется применимым законодательством.

1.3.1 Принцип независимости от носителя

Нормативные главы стандарта RENAR используют независимую от носителя терминологию (§3.1). Зависящие от носителя имена (конкретные продукты, протоколы, команды) не появляются в нормативном тексте; они присутствуют только в `guide/` (специфичные для носителя инструменты) и `reference/` (примеры).

1.4 Область применимости (primary scope)

1.4.1 Контракт-ориентированная разработка

Нормативный primary-контекст RENAR — **контракт-ориентированная разработка**: проект, в котором:

1. **Существует ТЗ** (явно сформулированное требование со стороны заказчика), которое после подписания становится **immutable** (§7.4.1 ADAPT-source).
2. **Имеется идентифицируемая сторона клиента** (Stakeholder с полномочиями подписания, §5.3.6) — способная подтвердить acceptance (§10.4.2) и подписать ADAPT (§5.5).
3. **Двусторонняя client-side validation** обязательна — forward интерпретация ТЗ обсуждается с клиентом до начала декомпозиции (§7.3).
4. **Delta-ТЗ workflow** возможен — изменения scope формализуются через delta-ADAPT (§7.6), а не через silent reinterpretation.

1.4.2 Типичные представители контракт-ориентированной разработки

Контекст	Характеристики
Заказная разработка по ТЗ	Независимый vendor + идентифицируемый client; формальный договор; acceptance criteria.
Regulated industries	Compliance audit обязателен (медицина, финансы, госсектор); traceability requirements → tests → код требуется по нормативу.
Enterprise консалтинг	Third-party реализует по ТЗ клиента-корпорации; multi-stakeholder approval; audit trail.

Long-lived product с явным product owner-ом	Product Owner играет роль Client representative для внутренних feature-T3; внутренние SLA + audit обязательны.
Public-sector / government IT	Тендерные T3; формальная приёмка; multi-year contracts.

1.4.3 Spec-Driven Development (SDD) — современное имя

Контракт-ориентированная разработка с AI-ускорением — это форма **Spec-Driven Development** (§2.3.4). RENAR — нормативный стандарт для AI-native SDD; не альтернатива SDD, а его специализация в области управления требованиями.

1.5 Где RENAR неприменим (негативный scope)

RENAR нормативно неприменим в контекстах, где структурно отсутствуют предусловия §1.4.1. Заявление RENAR-conformance (§13.4) для проектов в этих контекстах — non-conformant (§13.8).

1.5.1 Lean startup / pure discovery

Контекст: продуктовая команда строит MVP в условиях неопределённости рынка; «требования» — гипотезы, валидируемые на пользователях, а не immutable договорённости.

Почему вне scope:

- Отсутствует immutable T3 — гипотезы перепроверяются после каждого pivot.
- Client representative структурно отсутствует (внутренний продакт = author + sole assessor — нарушение §5.5.3 о двух независимых лицах в подписи).
- Delta-T3 workflow не применим: pivot — это смена scope **целиком**, не дельта.

Альтернатива: lean startup команды могут заимствовать **отдельные практики** RENAR (например, AI-провенанс артефактов, adversarial review TC) без заявления о соответствии RENAR — это допустимо как inspiration, но не заявление о соответствии стандарту.

1.5.2 Pure R&D / исследовательские проекты

Контекст: научно-исследовательский проект без определённого результирующего scope (например, exploratory ML research, novel-algorithm prototyping без заказчика).

Почему вне scope:

- T3 как immutable артефакт отсутствует.

- Acceptance criteria (QG-4 §10.4.2) не формализуемы — критерий «успеха» научного исследования не подписывается заранее.
- Двойная подпись ADAPT неприменима.

1.5.3 Exploratory hackathon / proof-of-concept

Контекст: time-boxed exploratory работа без обязательной приёмки клиентом.

Почему вне scope: те же причины, что в §1.5.1 + явный отказ от формальной приёмки.

1.5.4 Internal product без external client (core-mode boundary)

Контекст: внутренний инструмент команды; «клиент» совпадает с автором; нет независимого Stakeholder (представителя клиента) для двойной подписи ADAPT.

Boundary case: для таких проектов существует **core-mode** (core/renar-core.md) — минимальное подмножество практик RENAR (immutable identifiers, lifecycle states, V1–V6, ADAPT с **опциональной** client-signature). Core-mode **не** является полным RENAR-conformance — manifest для core-mode проекта объявляет `mode: core` и не заявляет RENAR-N (§13.2.1).

Negative scenario: попытка объявить RENAR-N (с полным conformance) для проекта без независимого представителя клиента (Client representative) — non-conformant (§13.8), нарушение §5.5.3.

1.5.5 Негативные сценарии — конкретизация

Сценарий	Почему non-conformant
Проект без письменного ТЗ; требования — устные обсуждения	Нарушение §1.4.1 (1): ТЗ как immutable артефакт отсутствует; ADAPT не имеет source (§7.4.1).
Проект с author == client (одно лицо подписывает обе стороны)	Нарушение §5.5.3 о двух независимых лицах в подписях ADAPT.
Проект, в котором scope пересматривается без формальной фиксации delta-ТЗ	Нарушение §7.6 delta-ADAPT workflow; нарушение §13.3 mandatory clause «ADAPT для каждого ТЗ».
Manifest объявляет tech-stack-specific требования (например, «обязательно использовать Python»)	Нарушение §1.3 (3): tech stack вне scope стандарта; manifest non-conformant.

1.6 Связь с SENAR

1.6.1 RENAR как специализация SENAR

SENAR — **методологическая база**: 5 ценностей AI-native разработки, 14 правил, агенты-инструкции, общие Quality Gates (§10.2.3), 5 базовых ролей (§5.2).

RENAR — **специализация SENAR в области requirements engineering**: нормирует только те аспекты, которые SENAR оставляет на усмотрение domain-стандарта (data model артефактов, lifecycle, conformance manifest). RENAR не дублирует SENAR и не переопределяет SENAR-конструкции.

1.6.2 Совместимость

RENAR-conformant реализация **всегда** является SENAR-совместимой. Обратное — не обязательно: SENAR-совместимый проект может **не** заявлять RENAR-conformance, если работает вне primary scope §1.4.

Несовместимость RENAR с SENAR в любой нормативной точке — баг стандарта RENAR; разрешается через formal change procedure стандарта SENAR + corresponding alignment в RENAR.

1.6.3 RENAR не альтернатива SENAR

Реализация **не** имеет права заявить «следуем RENAR вместо SENAR» — это нарушение §1.6.1. RENAR используется поверх SENAR; conformance manifest проекта объявляет одновременно SENAR-version и RENAR-version (§13.4).

1.7 Политика закрытого списка (closed list)

1.7.1 Что закрыто на v1

- Список нормативных областей §1.2 — closed; project-local расширения нормативной области запрещены.
- Список исключений §1.3 — closed; project-local попытки нормировать через manifest исключённые области (§1.3 (3) tech stack, §1.3 (7) PM practices) — non-conformant.
- Primary scope §1.4 — closed; добавление новых primary-контекстов — только через formal change procedure стандарта.

- Negative scope §1.5 — closed; перевод сценария из §1.5 в §1.4 — только через formal change procedure (с обоснованием в research draft).

1.7.2 Declared-stricter допустимо

Реализация может **ужесточить** scope относительно нормативного минимума, явно декларируя в conformance manifest (§13.4) с маркером `declared-stricter` (§10.10.2):

- Применять RENAR только к подмножеству требований проекта (например, только к security-critical SR).
- Требовать дополнительные artifact-типы (например, threat-models) сверх нормативных §1.2.
- Запретить core-mode для всех проектов организации.

Declared-stricter — допустимая локальная политика; соответствие нормативному уровню сохраняется.

1.7.3 Declared-weaker запрещено

Реализация **не** имеет права declared-weaker относительно §1.2 / §1.3 / §1.4:

- Объявить RENAR-conformance для проекта в §1.5 negative scope.
- Применять RENAR без ADAPT (нарушение §13.3.3 mandatory clause).
- Нормировать через project-local manifest исключённые области §1.3.

1.7.4 Путь расширения

Изменение §1.2 / §1.3 / §1.4 / §1.5 — только через formal change procedure стандарта RENAR (§13.9):

1. **Research draft** — обоснование изменения scope с практическими случаями.
2. **Public review** — открытое обсуждение.
3. **Minor-version bump** — изменение включается в `v1.X` или `v2.0`.
4. **Migration guidance** — для существующих conformant проектов.

1.7.5 Master list of closed lists в RENAR

Настоящий параграф — единый индекс всех закрытых списков стандарта RENAR. Каждый перечисленный список нерасширяем project-local; изменения возможны только через formal change procedure (§13.9). Канонический источник для каждого списка — в указанной секции; остальные упоминания являются cross-references.

#	Closed list	Канонический источник	Cross-refs
1	Нормативные области стандарта (10 пунктов)	§1.2	§1.7.1
2	Исключения из нормативной области (10 пунктов)	§1.3	§1.7.1
3	Primary scope: контексты применимости (4 признака + 5 типичных представителей)	§1.4	§1.7.1
4	Negative scope: контексты неприменимости (5 контекстов + 4 negative scenario)	§1.5	§1.7.1
5	Роли RENAR (specializations над SENAR §4)	§5	§1.2 (10)
6	Категории backward findings ADAPT (7 категорий)	§7.4.4	§13.3.7
7	Типы спецификаций SPEC-* (9 типов: ARCH/API/DATA/INT/PROC/UI/AI/SEC/OPS)	§8.3	§4.4.1, §13.3.4
8	Нормативные принципы TC (closed list)	§9.2	§13.3
9	Типы TC (`tc-type` : 6 значений)	§9.5	§4.5.2
10	Lifecycle states per artifact type (BR / SR / SPEC / TC / ADAPT / T3)	§10.5–§10.8	§4.7–§4.10
11	Quality Gates: обязательные {QG-0, QG-1, QG-2}, опциональные {QG-3, QG-4}	§10.3, §10.4	§10.10, §13.3.6
12	независимые от носителя возможности версионирования V1–V6	§3.x	§1.2 (6)
13	Уровни maturity RENAR-1..RENAR-5 (5 уровней)	§11.3	§13.2, §13.9
14	REQ-специфичные метрики (10 метрик)	§12.3	§12.5
15	Drift classes (нарушения требовательной инфраструктуры)	§4.11	§4.2
16	Запрещённые / устаревшие термины (non-canonical)	§4.14	§4.2

Расширение любого закрытого списка требует:

- research draft с обоснованием;
- public review;
- minor- или major-version bump RENAR;

- formal change procedure (§13.9);
- migration guidance для существующих conformant реализаций.

Политики закрытого списка на уровне глав (§10.10, §12.3, §13.9) являются специализациями настоящего §1.7 для соответствующих списков и **не** вводят независимых процедур.

Verification pass (2026-05-22): 16 closed lists в таблице выше сверены с каноническими секциями; cross-refs в §4, §8, §9, §10, §13 и reference/01 — без orphan entries. Инструмент: `node scripts/check-md-links.js` (0 broken internal links).

1.8 Перекрёстные ссылки

Источник	Применение
SENAR (полная методология)	Методологическая база RENAR (§1.6.1); RENAR не дублирует SENAR-нормы.
§5 Roles	Ownership артефактов и роли — specializations над SENAR §4 (§1.2 (10), §1.6.1).
§2 Methodology positioning	Три фундаментальных утверждения (SoT inversion, waterfall-form ≠ classical waterfall, независимое от носителя версионирование) — обоснование scope §1.4.
§7 ADAPT	ADAPT как нормативное требование §1.4.1 (3) двусторонней client-side validation.
§10 Lifecycle и QG	Lifecycle + Quality Gates — нормативная область §1.2 (5).
§3 Версионирование носителя	возможности V1–V6 — нормативная область §1.2 (6); принцип независимости от носителя §1.3.1.
§13 Conformance	Mandatory clauses, manifest, loss-of-conformance — нормативная область §1.2 (9); negative scenarios §1.5.5.
`core/renar-core.md`	Core-mode boundary case для §1.5.4 (internal product без external client).
`guide/02-transition-guide.md`	Practical guidance для проектов, переходящих с lean-стиля на контракт-ориентированную разработку (специфично для носителя).

← Предыдущая: 00. Введение · Оглавление · Следующая: 02. Положение в типологии методологий →

title: "Положение в типологии методологий" order: 2 lang: ru

02. Положение в типологии методологий

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

2.1 Три утверждения, на которых всё держится

RENAR стоит на трёх фундаментальных утверждениях, и без любого из них остальные главы рассыпаются: иерархия требований (глава 6) теряет смысл источника истины, ADAPT перестаёт быть атомарной единицей передачи между договорным и инженерным контурами, спецификации лишаются параллельной оси, тест-кейсы — привязки к версии требования, lifecycle — атомарности переходов, версионирование носителя — нормативной опоры. Поэтому глава идёт так рано: это фундамент, и §2.3–§2.5 стоит читать подряд.

Все три — **нормативные положения**: каждое является mandatory clause для любого заявления о соответствии RENAR (глава 13).

2.2 Три фундаментальных утверждения

1. **Source of Truth inversion.** Источник истины о поведении системы — иерархия артефактов требований. Код — производный артефакт реализации. Это и есть разработка от спецификации (Spec-Driven Development, SDD).
2. **Waterfall-форма ≠ классический waterfall.** Процесс RENAR имеет последовательную слоистую форму с гейтами. Стандарт явно отстраивается от четырёх смертельных грехов классического waterfall.
3. **Независимое от носителя версионирование.** Версионирование — обязательное свойство носителя, реализующего RENAR. Конкретный инструмент версионирования взаимозаменяем. Шесть возможностей (V1–V6) нормируют требования к носителю; детали — в главе 3.

Три утверждения логически связаны (см. §2.6).

2.3 Утверждение 1 — Источник истины о поведении: требования, не код (Source of Truth)

2.3.1 Нормативная формулировка

Источник истины о поведении системы — иерархия артефактов требований: ТЗ → ADAPT → BR / SR / SPEC → TR → TC. Код — производный артефакт реализации этой иерархии. При расхождении между кодом и вышестоящим требованием — нормативно побеждает требование.

2.3.2 Распределение ролей

Уровень	Кто источник истины	Кто производный
ТЗ	Договор с клиентом	—
ADAPT	Двусторонняя интерпретация ТЗ	производный от ТЗ
BR / SR / SPEC	Инженерный стандарт системы	производный от ADAPT
TC	Контракт верифицируемого поведения	производный от SR / SPEC
TR (задача)	Акт выдачи работы исполнителю	производный от SR + SPEC
Код	Реализация	производный от всего вышестоящего

2.3.3 Контракт (обязательные следствия)

Стандарт требует следующего поведения от любой реализации, заявляющей соответствие RENAR:

1. **Запрет обратного восстановления поведения из кода в SR (reverse-engineering).** AI-агент или человек, создающий новый SR или дополнение существующего SR, использует в качестве источника ADAPT и существующие SR — но не наблюдаемое поведение реализации. Обратное восстановление допустимо только при создании bug-fix задачи (см. §2.3.4).
2. **Разделение ревью кода и ревью спецификации.** Ревью кода проверяет соответствие изменения коду. Ревью спецификации проверяет соответствие тестов и реализации требованиям. Это два разных гейта с разными артефактами и разными ревьюерами по умолчанию.

3. **Детектирование дрейфа как hook носителя.** При обнаружении в коде ссылки на SR/SPEC, отсутствующий в носителе требований, атомарная единица изменения в реализационном носителе блокируется (см. глава 10 §10.5, глава 3 §3.5).
4. **Запрет молчаливой адаптации SR под код.** Если реализация делает X, а SR требует Y, ровно одно из двух истинно: (a) реализация ошибочна — создаётся bug-fix задача с целью привести код к SR; (b) SR ошибочен — создаётся delta-ADAPT с обоснованием и подписями для изменения SR. Третий вариант («просто обновим SR под код») запрещён.

2.3.4 Положение в индустриальной типологии

Утверждение 1 является конкретной реализацией парадигмы **Spec-Driven Development (SDD)** — индустриального термина, появившегося в 2024–2025 как ответ на ускорение разработки с AI-агентами. SDD признаёт, что когда AI-агенты способны декомпозировать формальные спецификации в код за минуты, **корректность спецификации** становится критическим ограничением, а не корректность кода.

Стандарт / методология или фреймворк	Соответствующее положение	Связь с RENAR §2.3
ISO/IEC/IEEE 29148:2018 §6.4.5	Requirements management mandates traceability from requirements to implementation	RENAR §2.3 — конкретная реализация этого mandate
BABOK Guide v3 §6.5	Verify requirements before they drive solution work	RENAR §2.3 нормирует verify как два разных гейта (code/spec)
ISO/IEC 5338:2023	Жизненный цикл ИИ-систем — требования управляют генерацией артефактов AI	RENAR §2.3 вместе с reference/04 AI Style Guide и adversarial review (guide/07 §4.5) поддерживают этот мандат
Spec-Driven Development (industry, 2024-2025)	GitHub Spec Kit, Anthropic spec-first agents, Amazon Kiro, Tessl, BMAD-Method	RENAR — formal standard в этой парадигме

Что здесь ново для RENAR. Сама SoT-инверсия — определение парадигмы SDD, а не изобретение RENAR. Вклад RENAR — её **enforcement**: четыре контрактных следствия §2.3.3 (запрет обратного восстановления в SR, разделение ревью кода и спецификации, drift-hook, запрет молчаливой адаптации SR под код), переводящие парадигму из принципа в проверяемые нормативные требования.

2.3.5 Дифференциация от SDD-инструментов

Industry SDD-инструменты и RENAR — в одной парадигме, но в разных слоях:

Ось	Industry SDD-инструменты (Spec Kit, Kiro, Tessl, BMAD)	RENAR
Природа	Эталонная реализация плюс готовый инструментарий с заданным процессом	Формальный нормативный стандарт (capabilities, lifecycle, инварианты)
Носитель	привязан к конкретному инструменту / платформе	независим от носителя (V1–V6); VCS / document-oriented store / иной — взаимозаменяемы
Договорный контур	Обычно отсутствует	ADAPT: двусторонняя адаптация ТЗ + двойная подпись (§7)
Conformance	Не определён	Заявление о соответствии через manifest + mandatory clauses (§13)
Верификация	Тесты как практика	pos/neg-парность + judge ≠ production как блокирующие gates (§9)

RENAR не конкурирует с этими инструментами: industry SDD-инструмент может быть **нативной для носителя реализацией RENAR** без потери переносимости заявления о соответствии (§14.5.2).

2.4 Утверждение 2 — Waterfall-форма, не классический waterfall

2.4.1 Нормативная формулировка

Процесс RENAR имеет последовательную слоистую форму: ТЗ → ADAPT → BR/SR/SPEC → TR → реализация → ТС-прогон → accepted. Это waterfall-образная форма. Стандарт явно отстраивается от четырёх смертельных грехов классического waterfall и не должен интерпретироваться как классический waterfall в смысле Royce 1970.

Это утверждение — **позиционное разъяснение** (снятие ложной аналогии с classical waterfall), а не заявление о новизне: после четырёх отстроек §2.4.2 форма совпадает с известными V-model и ATDD. Новизна RENAR сосредоточена в ADAPT, V1–V6 и переводе

практик в нормативные clauses, а не в самой «waterfall-форме». Утверждение остаётся mandatory clause (§2.7), потому что без него ревьюер натягивает на RENAR неподходящий шаблон (§2.6), но в один ряд новизны с ADAPT / V1–V6 не ставится.

2.4.2 Четыре отстройки от классического waterfall

Классический waterfall (Royce 1970, индустриальная практика 1970–1990)	**RENAR**
Один большой проход «требования → дизайн → код → тесты» за квартал или год	**Дельта-ТЗ workflow**: каждое изменение — мини-цикл за дни или часы. Та же форма повторяется сотни раз с AI-ускорением. См. глава 7 §7.6 (delta-ADAPT)
Тесты в конце цикла, после реализации	**ТС — полноценный артефакт верификации** (глава 9). Парные pos/neg ТС создаются вместе с SR/SPEC, не после кода. Это ближе к V-model и ATDD, чем к waterfall
Одностороннее «throw spec over the wall»	**ADAPT — двусторонний документ** по построению. Forward (инженерная интерпретация) + backward (вопросы клиенту). Запрет на одностороннюю передачу
Спека написана один раз, потом неприкосновенна; реальность дрейфует	**Непрерывная сверка** (continuous reconciliation) через hooks носителя. Дрейф code ↔ spec детектируется автоматически и попадает в delta-ADAPT. Спека живая

2.4.3 Применимость

RENAR применим в следующих контекстах:

- Контракт-ориентированная разработка (наличие договора с клиентом, подписанного ТЗ).
- Regulated industries: compliance, медицина, финтех, госсектор, обработка PII.
- Enterprise консалтинг (третья сторона делает продукт по чужому ТЗ).
- Проекты с высокой стоимостью изменений требований поздно в цикле.
- Проекты, требующие audit trail для compliance ревизий (глава 13).

RENAR **не применим** в следующих контекстах:

- Чистый продуктовый дискавери без договорного контекста (lean startup, продукт-MVP «сначала строим, потом понимаем что строим»).
- Чистое R&D без определённых требований.

- Прототипирование с life cycle меньше срока написания ADAPT.

Применимость документируется как часть процедуры соответствия (глава 13).

2.4.4 Положение в индустриальной типологии

Методология	Связь с RENAR
Classical Waterfall (Royce 1970)	RENAR отстраивается по 4 пунктам §2.4.2
V-model	RENAR ближе к V-model: парные TC, тесты в начале каждого слоя
Scrum / Kanban	Не противоречит, но RENAR — другая ось (артефактная, не процессная). Scrum sprint может содержать RENAR delta-ADAPT cycle
SAFe Solution Intent	ADAPT — конкретная реализация Solution Intent для контракт-ориентированной разработки. См. guide/05-safe-comparison.md
BABOK Requirements Analysis	RENAR §2.3+§2.4 — формальная реализация BABOK §3+§6 для контекста разработки с AI-агентами

2.5 Утверждение 3 — Независимое от носителя версионирование

2.5.1 Нормативная формулировка

Версионирование — обязательное свойство носителя, реализующего RENAR. Стандарт нормирует шесть возможностей (V1–V6), которые носитель обязан обеспечить. Конкретный инструмент версионирования взаимозаменяем: носитель, удовлетворяющий V1–V6, реализует RENAR независимо от того, является ли он распределённым VCS, централизованным VCS, документ-ориентированной СУБД с разрешением конфликтов или иным механизмом.

2.5.2 Шесть обязательных возможностей (обзорно)

Полная нормативная формулировка V1–V6 — в главе 3. На уровне положения в типологии:

#	Возможность	Что обеспечивает

V1	Неизменяемая история	Любое прошлое состояние артефакта восстановимо
V2	Атомарная единица изменения	«Изменение» — одна транзакция; всё или ничего
V3	Сравнение различий и рецензирование	Человек видит изменение и утверждает или отклоняет до интеграции
V4	Ветвление / набор изменений	Черновики отделимы от утверждённой истины
V5	Сквозная фиксация версии	Реализационный носитель фиксирует версию носителя требований
V6	Автор и отметка времени	Каждое изменение имеет автора и время

2.5.3 Носитель, не удовлетворяющий V1–V6, не реализует RENAR

Невозможность реализации RENAR на носителе без V1–V6 — структурная, не операционная: утверждение 1 (SoT inversion) физически не работает без версионирования, потому что:

- Невозможно сказать «реализация собрана против требований по состоянию на дату X» — пропадает provenance (нарушение V1, V6).
- Невозможно построить delta-ADAPT — нет базовой версии для отсчёта дельты (нарушение V1).
- Невозможно верифицировать TC — поле `verifies[].requirement-version` теряет смысл без stable identifier версии (нарушение V5).
- Невозможен переход требования `verified → accepted` — гейту не на что опереться (нарушение V1, V5).
- Невозможен audit trail «что мы сдавали клиенту по контракту 2025-Q3» (нарушение V1, V6).

Поэтому носитель без V1–V6 (плоский файловый сервер с переименованием файлов как механизмом версии; document store без разрешения конфликтов; иные системы без неизменяемой истории) **не реализует RENAR** независимо от других свойств.

2.5.4 Независимый от носителя нормативный язык

Нормативные параграфы RENAR используют независимый от носителя язык: «атомарная единица изменения», «фиксация версии», «автор и отметка времени», «сравнение различий и рецензирование» — а не названия конкретных примитивов конкретных инструментов. Это обеспечивает применимость стандарта к любому будущему носителю, удовлетворяющему V1–V6.

Специфичные для носителя детали (отображение V1–V6 на конкретные инструменты) — неотъемлемая часть реализации, но описываются в:

- Глава 3 §3.4 — нормативная таблица отображения V1–V6 × {основные носители}.
- guide/03-tool-guide-git.md — практическая реализация на distributed VCS.
- guide/04-document-store-substrate.md — практическая реализация на document-oriented store.

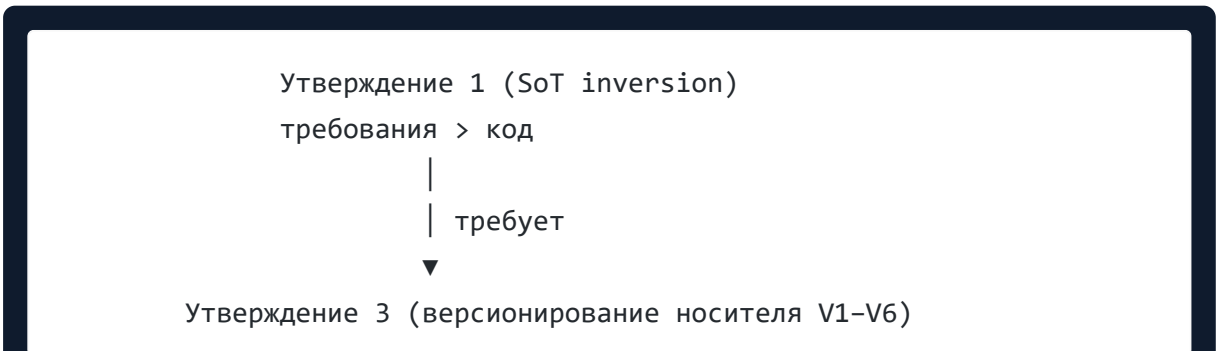
2.5.5 Положение в индустриальной типологии

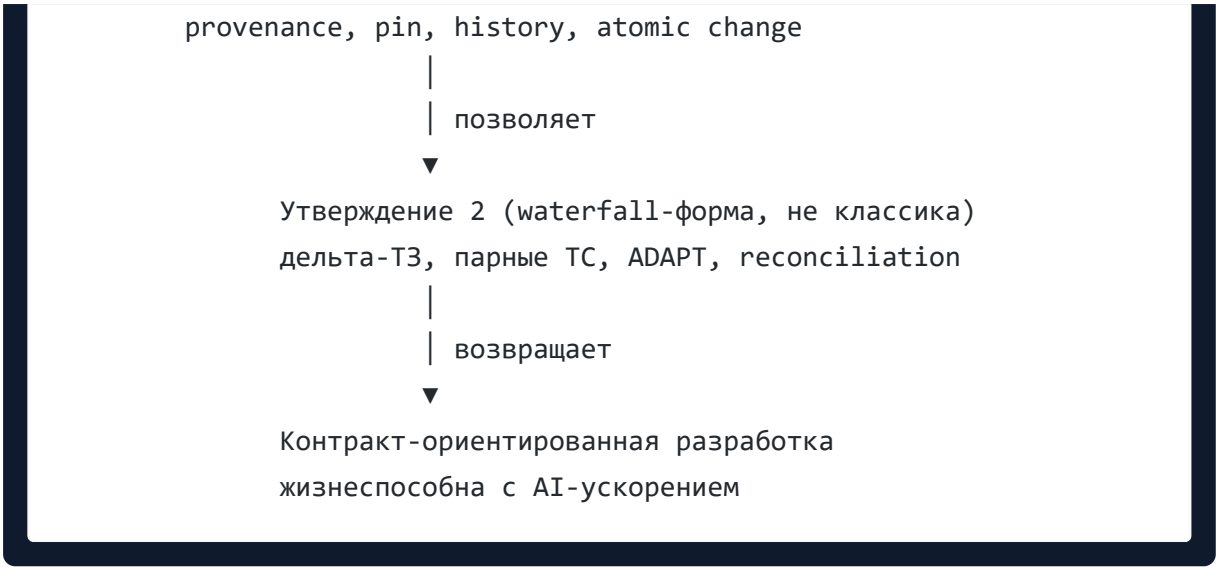
Стандарт	Соответствующее положение	Нейтральность к носителю
ISO/IEC/IEEE 29148:2018 §6.4.5	Configuration management of requirements	нейтрален к носителю; нормирует возможности, не инструменты
BABOK Guide v3 §5.3	Maintain requirements (для прослеживаемости)	нейтрален к носителю
CMMI-DEV CM SG2	Track and Control Changes	нейтрален к носителю
SAFe Solution Intent	Versioned artifact	Не нормирует носитель
ISO/IEC 5338:2023	Происхождение артефактов ИИ (AI artifact provenance)	нейтрален к носителю; provenance — возможность

RENAR следует той же нейтральности и явно фиксирует V1–V6 как контракт, что эти стандарты оставляли неявным.

2.6 Логическая связь трёх утверждений

Три утверждения связаны направленно:





Без утверждения 1: SoT диффузен, спека дрейфует, audit невозможен. **Без утверждения 3**: утверждение 1 декларативно, физически не работает. **Без явного утверждения 2**: ревьюер натягивает на RENAR неподходящие шаблоны (agile sprint без waterfall-form, или classical waterfall без 4 отстроек) и отвергает стандарт по неверной аналогии.

Все три утверждения являются mandatory clauses (глава 13). На уровне v1 заявление о соответствии RENAR требует принятия всех трёх утверждений; без любого из них оно несостоятельно.

2.7 Следствия для процедуры соответствия (Conformance)

Утверждения §2.3, §2.4, §2.5 являются **mandatory clauses** для любого заявления о соответствии уровням RENAR (RENAR-1 .. RENAR-5; см. глава 11, глава 13).

Это означает:

- Команда не может заявить «реализуем RENAR-1 без инверсии источника истины (SoT)» — это противоречие в терминах.
- Команда не может заявить «реализуем RENAR на носителе без V1–V6» — носитель не соответствует стандарту.
- Команда может выбрать **не заявлять соответствие RENAR** в контексте неприменимости (§2.4.3) — это нормативно допустимо.

Конкретный self-assessment checklist для каждого утверждения — в главе 13.

2.8 Связь с другими главами стандарта

Глава	Связь
06 Требования	Иерархия BR/SR/TR — конкретизация SoT chain из §2.3.2
07 ADAPT	Двусторонняя адаптация — конкретизация утверждения 2 (отстройка от «переброса спецификации через забор»)
08 Specifications	SPEC-* как параллельная ось — следствие SoT inversion для структурного описания
09 Test cases	TC как полноценный артефакт верификации — конкретизация утверждения 2 (отстройка от «тесты в конце»)
10 Lifecycle и QG	Гейты обеспечивают enforcement утверждения 1 (drift hooks) и утверждения 2 (continuous reconciliation)
03 Версионирование носителя	Детальная нормативка V1–V6 (§2.5 — обзорно)
11 Maturity	Уровни RENAR-1..RENAR-5 расширяют утверждения дополнительными требованиями
13 Conformance	Self-assessment по трём mandatory clauses

---

title: "Версионирование носителя (V1–V6)" order: 3 lang: ru

03. Версионирование носителя — V1–V6

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

3.1 Шесть возможностей носителя

Инверсия источника истины из главы 2 — побеждает требование, а не код — держится на одном физическом условии: носитель обязан честно помнить, что и когда менялось. Если прошлое состояние можно переписать задним числом, фраза «реализация прошла приёмку против требований версии X» теряет смысл, а вместе с ней рушится весь договор о приёмке. Поэтому шесть возможностей V1–V6 — не инфраструктурная деталь, а фундамент, на котором идея RENAR вообще стоит; их и выносим вперёд, до глав об артефактах.

Глава даёт **детальную нормативную формулировку** каждой возможности: предусловия, постусловия, связь с цепочкой источника истины (SoT) и примеры отображения на распространённые носители. Концептуальное обоснование — §2.5 (утверждение 3 «Положения в типологии методологий»).

Конкретный механизм версионирования — распределённый или централизованный VCS, document store с разрешением конфликтов — **взаимозаменяем**. RENAR нормирует возможности, а не инструменты.

3.2 Обоснование обязательности

Разберём по одной, что именно ломается без каждой возможности. Связь здесь структурная, а не операционная: дело не в дисциплине команды, а в том, что без возможности соответствующее свойство истины просто нечем выразить.

Нет возможности	Что становится невозможным	Связь в RENAR
V1 (неизменяемая история)	«Реализация собрана против требований по состоянию на дату X»	provenance, журнал аудита, гейт приёмки

V2 (атомарная единица изменения)	Гарантированная согласованность изменений	delta-ADAPT как атомарная единица изменения (см. §7.6)
V3 (сравнение различий и рецензирование)	Утверждение требований и спецификаций	QG QG-ADAPT-approve, QG-spec-approved
V4 (ветвление / набор изменений)	Черновик отделим от утверждённой правды	переходы `draft → review → approved`
V5 (сквозная фиксация версии)	Привязка версии требования из реализации	поле `verifies[].requirement-version` в TC (§9)
V6 (автор и отметка времени)	provenance каждого изменения	подписи в ADAPT, `ai-provenance` в SR/SPEC

Поэтому носитель без V1–V6 — плоский файловый сервер без истории, document store без разрешения конфликтов, любой механизм без неизменяемого отслеживания изменений — **не реализует RENAR** независимо от прочих достоинств. Это структурное ограничение, а не вопрос дисциплины команды.

3.3 Нормативные определения V1–V6

Параграфы §3.3.1–§3.3.6 сформулированы **независимо от конкретного носителя**. Имена конкретных продуктов — только в §3.4 (пример) и §3.6 (иллюстрации языка).

3.3.1 V1 — неизменяемая история

Возможность (immutable history): носитель обязан обеспечить, что для любого артефакта любое прошлое состояние адресуемо и восстановимо без потерь.

Предусловия: артефакт зарегистрирован в носителе как объект версионирования.

Постусловия: для любого момента времени T в истории артефакта существует стабильный идентификатор версии, через который состояние на момент T полностью восстанавливается.

Без V1 невозможно:

- Восстановить состояние артефакта на дату подписания контракта.
- Сравнить текущее состояние с опорным.
- Построить журнал аудита (audit trail) для соответствия (глава 13 §13.4).

- Задать базовую точку для delta-ADAPT.

Конкретные реализации на разных носителях — §3.4.

3.3.2 V2 — атомарная единица изменения

Возможность (atomic change unit): носитель обязан обеспечить, что любое изменение артефакта (или согласованной группы артефактов) фиксируется как одна транзакция: всё или ничего. Промежуточные несогласованные состояния снаружи не наблюдаемы.

Предусловия: у носителя есть понятие транзакции (atomic change).

Постусловия: после атомарного изменения либо все правки видны наблюдателю, либо ни одна. Промежуточное «рассогласованное» состояние недопустимо.

Без V2 невозможно:

- Согласованно обновить BR и связанные SR одной транзакцией.
- Гарантировать согласованность требования и метаданных `linked-tasks[]`.
- Оформить утверждение ADAPT как единое атомарное действие (двойная подпись + переход `client-ready → approved` за одну транзакцию).
- Откатить изменение без «полуприменённого» состояния.

3.3.3 V3 — сравнение различий и рецензирование

Возможность (diff & review): носитель обязан обеспечить, что предложенное (но ещё не интегрированное) изменение представимо как diff против опорного состояния, и человек или AI-агент с правом рецензирования может одобрить или отклонить его **до** включения в утверждённую правду.

Предусловия: предложенное изменение существует отдельно от утверждённой правды (см. V4).

Постусловия: до утверждения изменение существует, но не считается частью SoT. После утверждения становится частью SoT как атомарная единица изменения (V2).

Без V3 невозможно:

- Quality gates QG-ADAPT-approve, QG-spec-approved, QG-sr-approved (см. глава 10).
- Двойная подпись ADAPT (см. §7.5).
- Независимое рецензирование кода и спецификации (см. §2.3.3 (2)).
- Adversarial review как гейт.

3.3.4 V4 — ветвление / набор изменений

Возможность (branching / change-set): носитель обязан разделить **работу в процессе** (WIP) и **утверждённую правду** (SoT) так, что несколько независимых изменений могут разрабатываться параллельно без влияния на SoT.

Предусловия: артефакт зарегистрирован в носителе.

Постусловия: для любого артефакта в данный момент могут существовать (a) ровно одна утверждённая версия (SoT) и (b) ноль или более черновиков — каждый либо интегрируется через V3, либо отклоняется.

Без V4 невозможно:

- Отделить lifecycle-статус `draft` от `approved` (глава 10).
- Параллельно вести несколько delta-ADAPT.
- Держать backward findings в `asked-to-client` без блокировки утверждённой правды.
- Эволюционировать SPEC-* экспериментально без влияния на требования, производные от реализации.

3.3.5 V5 — сквозная фиксация версии между носителями

Возможность (cross-substrate version pin): носитель A, использующий артефакт носителя B, обязан иметь возможность зафиксировать конкретную версию артефакта носителя B как стабильный сквозной идентификатор. Этот идентификатор должен однозначно восстанавливать состояние артефакта в носителе B.

Предусловия: носитель A и носитель B удовлетворяют V1 (у каждого есть стабильный идентификатор версии).

Постусловия: для каждой зафиксированной ссылки в носителе A на артефакт носителя B существует пара `(artifact-id, version-id)`, и по ней восстанавливается полное состояние артефакта носителя B на момент фиксации.

Без V5 невозможно:

- Поле `verifies[].requirement-version` в TC (глава 9 §9.4).
- Привязать носитель реализации (код) к конкретной версии носителя требований (SR/SPEC).
- Гарантировать: «эта реализация прошла приёмку против требований версии X».
- Метрика свежести TC (зафиксированная версия старше текущей — TC устарел).

3.3.6 V6 — автор и отметка времени

Возможность (author + timestamp): носитель обязан для каждой атомарной единицы изменения (V2) зарегистрировать идентифицируемого автора (человек или AI-агент с уникальным id) и отметку времени с точностью не хуже секунды.

Предусловия: у носителя есть система идентификации авторов.

Постусловия: для любой атомарной единицы изменения запрос «кто? когда?» даёт однозначный ответ.

Без V6 невозможно:

- Двойная подпись ADAPT (подпись = автор + отметка времени в нативной для носителя форме).
- `ai-provenance` во frontmatter артефактов.
- Журнал аудита для соответствия.
- Метрика adversarial-found rate (распределение backward findings по авторам).

3.4 Отображение V1–V6 на конкретные носители (пример)

> ***Информативно.*** *Таблица показывает, как V1–V6 обычно реализуются на распространённых носителях. Она **не** часть нормативного контракта. Любой носитель, удовлетворяющий V1–V6, реализует главу 3 — независимо от того, есть ли он в таблице.*

Возможность	Git	Mercurial	SVN	Perforce	Document sto (пример)
V1 — неизменяемая история	commits, hash-chain	changesets, hash-chain	revisions, sequential numbering	changelists	revision tree p doc (`_rev`)
V2 — атомарная единица изменения	commit	commit	atomic revision	changelist submit	document upc (single _rev advance)
V3 — diff и рецензирование	merge request / pull request	hg phabricator, mq	svn diff + commit gate	swarm review	API workflow - UI approval

V4 — ветвление / набор изменений	branches	named branches, bookmarks	branches (copy semantic)	branches / streams	conflict branch WIP docs / dra status
V5 — сквозная фиксация версии	submodule SHA	subrepo changeset	externals (peg rev)	branches / streams reference	`_rev` referenc `created_by_c`
V6 — автор и отметка времени	commit metadata	commit metadata	revision properties	changelist metadata	doc fields (`aut` `updated_at`)

Практические workflow для каждого носителя:

- guide/03 — git
- guide/04 — document store

3.5 Носитель, не удовлетворяющий V1–V6

Следующие конфигурации **не реализуют RENAR**, потому что нарушают одну или более возможностей:

Конфигурация	Нарушение
Плоский файловый сервер; «версия» = переименование файла	V1 (нет стабильной истории; переименование = потеря provenance)
Document store без разрешения конфликтов	V2 (возможно рассогласованное состояние)
Wiki без истории ревизий	V1, V6
Wiki с историей ревизий, но без процедуры утверждения	V3
VCS с `mtime` вместо неизменяемого идентификатора	V1, V5
носитель с правкой исторических ревизий на месте	V1 (история изменяема — журнал аудита невозможен)

Команда, **внедряющая** RENAR на носителе из этого списка, **должна** либо перейти на подходящий носитель, либо построить **компенсирующий слой**, обеспечивающий V1–V6 поверх базового хранения. В обоих случаях manifest соответствия обязан явно документировать, как реализованы V1–V6.

3.6 Язык, независимый от конкретного носителя

Нормативные параграфы RENAR используют **независимые от носителя** формулировки: «атомарная единица изменения» (V2), «фиксация версии» (V5), состояние `approved` (после V3), «черновик» (V4). Термины, **специфичные для носителя** (имена инструментов и их примитивы), допустимы только:

- в иллюстративных таблицах с явной пометкой (как §3.4);
- в перекрёстных ссылках на guide/ по конкретному носителю;
- в приложениях к нормативным главам.

3.6.1 Примеры: нормативная форма и иллюстрация на git

Нормативная (независимая от носителя) формулировка	Иллюстрация на git (не норма)
«Изменение требования регистрируется как атомарная единица изменения с автором и временем (V2, V6)»	«Изменение регистрируется как commit»
«Изменение проходит сравнение различий и рецензирование до интеграции (V3)»	«Изменение проходит review merge request до merge в main»
«носитель реализации фиксирует конкретную версию носителя требований (V5)»	« `.src` фиксирует submodule SHA `.req` »
«Двойная подпись ADAPT — два независимых события автор+отметка времени (V6)»	«Двойная подпись — два approve в UI merge request»
«Правка уже утверждённого требования вне атомарной единицы изменения с рецензированием — нарушение»	«Force-push на approved branch блокируется hooks»

3.6.2 Носитель как параметр соответствия

Каждая команда фиксирует **выбор носителя** в manifest соответствия с явным отображением V1–V6 → нативные для носителя примитивы. При смене носителя (например, с git на document store) manifest обновляют и проводят регрессионную проверку V1–V6 (§3.8).

3.7 Манифест соответствия

Каноническая схема — §13.4, файл `RENAR-CONFORMANCE.yaml` (YAML 1.2) в корне носителя требований. Глава 3 нормирует часть, относящуюся к носителю: декларацию выбранного носителя и отображение V1–V6 (поле `substrate-capabilities`, §13.4.2).

```
# Фрагмент RENAR-CONFORMANCE.yaml — substrate-специфичная часть
# (полная схема — §13.4.2)
substrate:
  requirements:   { tool: "<name>", version: "<version>" }
  implementation: { tool: "<name>", version: "<version>" }
v1-v6-mapping:
  v1-immutable-history:   { primitive: "<substrate-specific>",
validation: "<CI check / manual>" }
  v2-atomic-change-unit:  { primitive: "<substrate-specific>",
validation: "<CI check / manual>" }
  v3-diff-review:         { primitive: "<substrate-specific>",
validation: "<CI check / manual>" }
  v4-branching:           { primitive: "<substrate-specific>",
validation: "<CI check / manual>" }
  v5-cross-substrate-pin: { primitive: "<substrate-specific>",
validation: "<CI check / manual>" }
  v6-author-timestamp:    { primitive: "<substrate-specific>",
validation: "<CI check / manual>" }
```

Подтверждение mandatory clauses (§2.3 SoT inversion, §7 ADAPT, §8 closed list 9 SPEC types, §9 pos/neg и др.) — в поле `mandatory-clauses-confirmed` (§13.3–§13.4).

Manifest **обязателен** для любого заявления о соответствии уровня RENAR-1 и выше (глава 13).

3.8 Миграция носителя

При смене носителя команда **обязана** выполнить шаги по порядку:

1. **Аудит до миграции:** целевой носитель удовлетворяет V1–V6; иначе миграция запрещена до компенсирующего слоя.
2. **Черновик manifest:** обновлённый `RENAR-CONFORMANCE.yaml` с новым отображением.

3. **Проверка изоморфности:** все артефакты (BR, SR, SPEC, ADAPT, TC) переносимы без потери полей, id и provenance. Все id **неизменяемы** — переименование при миграции запрещено.
4. **Атомарное переключение:** миграция — атомарная единица изменения на уровне процесса; одномоментный перевод всех артефактов. **Параллельное использование двух носителей как SoT запрещено** (см. §2.3.3 (1)).
5. **Проверка после миграции:** регрессионная проверка V1–V6 на реальных артефактах.
6. **Регистрация manifest:** обновлённый `RENAR-CONFORMANCE.yaml` регистрируется в новом носителе как атомарная единица изменения (V2).

После переключения старый носитель — архив (снимок только для чтения) или упразднён. Параллельная работа на двух носителях как SoT запрещена.

3.9 Связь с другими главами

Глава	Связь
02 Положение в типологии §2.5	Концептуальное обоснование V1–V6
06 Иерархия требований	frontmatter опирается на V1 (неизменяемый id), V5 (фиксация версии)
07 ADAPT	Утверждение двойной подписью — V3 + V6; delta-ADAPT — V1 + V2 + V4
08 Спецификации	`constrained-by[]`, `depends-on[]`, `referenced-by[]` — через V5
09 Тест-кейсы	`verifies[].requirement-version` — прямое применение V5
10 Lifecycle и QG	Переходы статусов — V2 + V3 + V4
11 Модель зрелости	RENAR-3+: V5 обязателен; RENAR-4+: V6 + ai-provenance
13 Соответствие	`RENAR-CONFORMANCE.yaml` — обязательный артефакт
guide/03	Практика на git
guide/04	Практика на document store

---

title: "Термины и определения" order: 4 lang: ru

04. Термины и определения

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

4.1 Зачем единый словарь терминов

Эта глава — справочник по формулировкам: её читают при согласовании артефактов, проверке носителя и подготовке оценки соответствия. Она фиксирует уникальное имя каждой концепции и предотвращает терминологический дрейф между командами и инструментами. После оглавления переходите к §4.3–§4.5 по типам артефактов, к §4.6–§4.7 для состояний и гейтов и к индексу закрытых списков в §1.7.5.

Глава нормирует **canonical терминологию RENAR**: одно определение на одну концепцию; единый источник истины для других глав, реализационных носителей, конформант-инструментов. Терминологический drift (§4.11) — отдельный класс нарушений conformance (§13.3.1 косвенно).

Глава фиксирует:

- **Артефакты требований и спецификаций** (§4.3–§4.5): BR, SR, TR, ADAPT, SPEC-*, TC и их типы.
- **Lifecycle статусы** (§4.6) — закрытый список per тип артефакта.
- **Quality Gates** (§4.7) — canonical из §10.3.
- **Термины носителя** (§4.8) — возможности V1–V6 и сопутствующие понятия.
- **Системная иерархия** (§4.9) — система / подсистема / модуль.
- **Provenance термины** (§4.10) — ai-provenance, source-citation, traceability chain.
- **Drift классы** (§4.11) — closed list нарушений требовательной инфраструктуры.
- **Термины полей связи (Connection terms)** (§4.12) — поля frontmatter, фиксирующие связи между артефактами.
- **Сопоставление с родственными стандартами (mapping)** (§4.13) — SENAR, ISO/IEC 29148, BABOK, SAFe.
- **Запрещённые / устаревшие термины** (§4.14) — closed list non-canonical.
- **Authority** (§4.15) — порядок разрешения разногласий.
- **Master index closed lists** — §1.7.5 (16 списков); канонический источник каждого списка — в указанной секции стандарта.

Глава **не дублирует** reference/01-glossary.md: эта глава содержит **canonical normative** определения короткой формы; `reference/01` — развёрнутые объяснения с anti-

patterns, history, и industry context (информационный материал).

4.2 Принцип «только canonical»

RENAR выбирает **один canonical термин** на одну концепцию. Внутри носителя (frontmatter, ID, body normative paragraphs, scripts, CI hooks) используется **только canonical**. Сопоставление с родственными стандартами (§4.13) — для документации, migration и интеграции с внешними системами; внутри носителя замены canonical на эквивалент из таблицы сопоставления **не происходит**.

При обнаружении non-canonical термина (по §4.14) в нормативном артефакте — нативный для носителя hook (§10.11.1) обязан выдать warning при change-set; для RENAR-4+ (§11.7) — blocking error.

Multilingual проекты могут отображать canonical терминологию в UI на язык клиента (§4.13.3); это **UI-перевод**, не canonical-замена.

4.3 Артефакты требований

4.3.1 ТЗ — Техническое задание

ТЗ (`ТЗ-YYYY-NNN`) — **immutable** (§7.4.2) договорный артефакт, фиксирующий обязательства между клиентом и инженерной командой. После регистрации в носителе не редактируется. Изменения — через delta-ТЗ как новый immutable артефакт (§7.6).

4.3.2 ADAPT — мостовой артефакт (bridge artifact)

ADAPT (`ADAPT-NNN`) — обязательный мостовой артефакт (§7.4.1) между ТЗ и иерархией требований. Содержит forward (инженерная интерпретация) + backward (вопросы клиенту) разделы. Каждое ТЗ обязано иметь ровно один корневой ADAPT в статусе `approved` (§13.3.3). Lifecycle: §4.6.4.

4.3.3 BR — Business Requirement

BR (`BR-NN`) — артефакт бизнес-уровня. Описывает наблюдаемый бизнес-эффект (`business-outcome`), а не способ его достижения. Декомпозируется в SR. Frontmatter — §6.5.2. Lifecycle: §4.6.1.

4.3.4 SR — System Requirement

SR (`SR-NN`) — артефакт системного уровня. Описывает обязательное поведение системы в рамках одного бизнес-эффекта. Имеет родительский BR (`parent: BR-N`) или родительский SR (при `level: subsystem` или `level: module` — §6.7). Frontmatter — §6.6.2. Lifecycle: §4.6.1.

4.3.5 TR — Task Requirement

TR (`TR-NN`) — артефакт уровня задачи исполнителя. Описывает actionable работу с goal + AC. Имеет родительскую цепочку `implements: SR-N` (или BR для тривиальных задач). Frontmatter — §6.7.2. Lifecycle: §4.6.2.

4.3.6 Иерархия

Иерархия требований:

```
ТЗ → ADAPT → BR → SR → TR → реализация
                  |
                  └── SPEC-* (параллельная ось, §4.4)
```

SR может реализовывать SPEC через `implements-spec[]` (§8.6.2) — это **связь**, а не parent-edge.

4.4 SPEC артефакты

SPEC — артефакт структурного описания системы как параллельная ось требований (§8.2). Не parent-edge с SR; связан через `constrained-by[]` / `implements-spec[]` (§8.6). Lifecycle: §4.6.3.

4.4.1 Closed list 9 типов SPEC

Закрытый список (§8.3):

Тип	Назначение
`SPEC-ARCH`	Архитектура системы (контексты, контейнеры, компоненты, deployment view, quality attributes)

`SPEC-API`	API contracts (REST / GraphQL / gRPC / async events)
`SPEC-DATA`	Data model (schema, ERD, migrations, retention, PII classification)
`SPEC-INT`	Integration (взаимодействие между подсистемами и внешними системами)
`SPEC-PROC`	Process / workflow (бизнес-процессы, state machines, saga)
`SPEC-UI`	UI / UX (экраны, навигация, accessibility, baselines)
`SPEC-AI`	AI / ML (model cards, RAG, prompt engineering, eval strategy)
`SPEC-SEC`	Security (authn / authz, threat model, secrets management)
`SPEC-OPS`	Operations (deployment, observability, SLO/SLA, runbook)

Проект не имеет права создавать новые типы SPEC локально (§13.3.4).

4.5 Артефакты тестирования

4.5.1 TC — Test Case

TC (`TC-NN`) — артефакт верифицируемого критерия. Покрывает нормативные утверждения BR / SR / SPEC (через `verifies[]` с version pin, §9.4). Lifecycle: §4.6.5.

4.5.2 Closed list типов TC (`tc-type`)

Закрытый список (§9.5):

`tc-type`	Назначение
`acceptance`	E2E-тесты бизнес-цели для BR (§9.5); runner-family: E2E + AI-валидатор
`ux`	UX-тесты с VLM-judge (§9.6.1) для SPEC-UI
`system`	Системные тесты общего назначения для SR / SPEC-PROC / SPEC-ARCH (§9.5)
`contract`	Контрактные тесты (§9.6.3) для SPEC-API / SPEC-INT / SPEC-DATA
`eval`	Eval-тесты для SPEC-AI с LLM-judge (§9.6.2); judge-модель обязана отличаться от модели реализации

`security`	Security-тесты (§9.6.4) для SPEC-SEC по STRIDE-категориям

4.5.3 Pos / neg парность

Нормативное требование (§9.7): каждое нормативное утверждение покрывается **парой** TC (positive scenario + negative scenario). Single-TC-coverage допускается только для утверждений-инвариантов (security STRIDE).

4.6 Lifecycle статусы

4.6.1 BR / SR

Закрытый список (§10.5): `draft → approved → verified → accepted → deprecated`. `accepted` — терминальный недеградируемый (§10.4.2, опционально); `deprecated` — терминальный.

4.6.2 TR

Закрытый список (§10.6): `draft → approved → done`; `obsolete` — альтернативный терминальный.

4.6.3 SPEC

Закрытый список (§10.7): `draft → review → approved → verified`; `obsolete` — терминальный.

4.6.4 ADAPT

Закрытый список (§10.8): `draft → review → client-ready → answered → approved → frozen`. `frozen` — терминальный immutable; изменения только через delta-ADAPT или errata.

4.6.5 TC

Закрытый список (§10.9): `draft → ready → passing / failing → obsolete`. `passing ↔ failing` — runner-managed transitions (§10.9.3), не gate-passages.

4.6.6 Backward findings (sub-state в ADAPT)

Закрытый список (§7.4.5): `open → asked-to-client → answered → resolved → frozen` ; `revised` — возврат в `asked-to-client` .

4.6.7 Закрытые списки lifecycle — нерасширяемы project-local

Любой статус вне закрытого списка для соответствующего типа артефакта — нарушение conformance (§10.10.2).

4.7 Quality Gates

Canonical список из §10.3 + §10.4. Проект не имеет права создавать новые gate-типы локально (§10.10.2, §13.3.6).

Gate	Назначение	Статус соответствия
`QG-0` — Approval (§10.3.1)	Approval артефакта для разработки / реализации	**Required**
`QG-1` — Implementation (§10.3.2)	Implementation valid (только для TC `draft → ready`)	**Required**
`QG-2` — Verification (§10.3.3)	Promote артефакта в `verified`	**Required**
`QG-3` — Architecture (§10.4.1)	Approval ADAPT (двойная подпись) / SPEC-ARCH	Опционально (`declared` или `absent`)
`QG-4` — Acceptance (§10.4.2)	Приёмка BR в `accepted`	Опционально

Runner-managed transitions (`ready → passing` , `passing → failing` , и иные) — **не** Quality Gates (§10.9.3).

4.8 Термины носителя

4.8.1 Носитель

Носитель (в полях и коде — `substrate`) — система хранения и версионирования артефактов RENAR. RENAR независим от носителя: нормирует возможности (§4.8.2), а не инструменты.

4.8.2 Возможности V1–V6

Закрытый список из §3.3. Все шесть обязательны абсолютно для соответствия (§13.3.2):

Возможность	Семантика
`V1` — Неизменяемая история	Любое прошлое состояние артефакта восстановимо
`V2` — Атомарная единица изменения	Изменения фиксируются как «всё или ничего»
`V3` — Сравнение различий и рецензирование	Предложенное изменение представимо как diff против опорного состояния и проходит утверждение до интеграции в SoT
`V4` — Ветвление / набор изменений	Черновики отделимы от утверждённой истины; параллельные изменения независимы
`V5` — Сквозная фиксация версии	Ссылка между носителями фиксирует конкретную версию артефакта
`V6` — Автор и отметка времени	Каждая атомарная единица изменения имеет идентифицируемого автора и отметку времени ≥ секундной точности

4.8.3 Атомарная единица изменения

Изменение носителя (V2), удовлетворяющее свойству «всё или ничего» — нативная для носителя транзакция; промежуточные несогласованные состояния не наблюдаемы наружу. Конкретная реализация (атомарная запись в распределённом VCS, транзакция в document store, иной механизм) специфична для носителя; описание форм выносится в `guide/`.

4.8.4 Фиксация версии

Нативный для носителя механизм (V5), фиксирующий конкретную версию артефакта одного носителя из другого через пару `(artifact-id, version-id)`.

4.8.5 Журнал аудита (audit trail)

Нативная для носителя append-only коллекция событий gate-passage и transitions (§10.13). Каждое событие содержит timestamp, artifact-id, artifact-version, from-status, to-status, gate-id, actor, evidence-refs. Удаление не допускается (V1 retention, §10.13.3).

4.9 Системная иерархия

Закрытый список уровней (§6.7, §6.8):

Уровень	Назначение
`system`	Весь продукт целиком; верхний уровень; редко используется (кросс-подсистемные задачи)
`subsystem`	Подсистема (например, отдельный сервис, фронтенд-приложение)
`module`	Модуль внутри подсистемы

Поле `level` фиксируется во frontmatter артефакта (BR / SR / TR). Иерархия может расширяться вниз через эволюцию подсистема → модуль (§6.9.1) или обратно (§6.9.3).

4.10 Термины provenance

4.10.1 ai-provenance — каноническая схема

Frontmatter-блок (§11.7.1, RENAR-4 mandatory) фиксирующий происхождение AI-генерируемого артефакта. Настоящая секция — **единственный канонический источник** схемы; chapter-level YAML примеры в §6.5.2, §6.6.2, §8.5, §9.3 ссылаются сюда и **не** определяют независимых полей.

Поле	Mandatory?	Семантика
`ai-provenance.generated-by`	mandatory	Идентификатор модели (`<vendor>-<model>-<version>@<date>`)
`ai-provenance.generated-at`	mandatory	UTC timestamp генерации (ISO-8601)
`ai-provenance.prompt-template`	mandatory	нативный для носителя pointer на prompt template (`<path>@<version>`)

`ai-provenance.context-tokens`	mandatory	Размер контекста на входе (integer)
`ai-provenance.output-tokens`	mandatory	Размер вывода (integer); источник для метрик §12.3
`ai-provenance.human-edits`	mandatory	Boolean — были ли ручные правки после генерации (информационное поле; см. §4.10.1.1)
`ai-provenance.generation-time-ms`	optional	Latency генерации в миллисекундах; рекомендуется на RENAR-5 для cost/latency budget мониторинга (§11.8.1)
`ai-provenance.cost-budget`	optional на RENAR-4, mandatory на RENAR-5	Запланированный budget стоимости генерации
`ai-provenance.cost-actual`	optional на RENAR-4, mandatory на RENAR-5	Фактическая стоимость; источник для §12.3.9 Cost per Approved Requirement

Добавление новых полей в схему — только через formal change procedure стандарта (§13.9). Локально определённые ai-provenance.* поля проектом-реализацией являются `declared-stricter` extension (§10.10.2) и не нарушают conformance, но не считаются canonical.

4.10.1.1 Семантика `human-edits`

`human-edits` — **информационное** поле для traceability и observability, не gating-флаг. Значение `human-edits: true` означает, что артефакт был отредактирован вручную после первичной AI-генерации; оно **не** запускает auto-rejection. Нормативное правило P3 (§9.2) — «инженер не пишет TC вручную» — нормирует **происхождение**, не последующие правки. Реализация на носителе **может** (`declared-stricter`) дополнительно требовать review для артефактов с `human-edits: true` ; это локальное ужесточение, не часть RENAR-N base conformance.

4.10.2 Указание источника (source citation)

Inline pointer в body артефакта (RENAR-4 mandatory, §11.7.1) на источник конкретного нормативного утверждения. Формат специфичен для носителя; типичные паттерны:

`[TZ-XXX §Y line Z]`, `[ADAPT-NNN §A.B]`, маркер `derived` с pointer на parent-артефакт.

4.10.3 Цепочка прослеживаемости (traceability chain)

Цепочка артефактов от ТЗ до прогона ТС, фиксирующая происхождение каждого assertions:

```
ТЗ → ADAPT → BR → SR → TR / SPEC → TC.last-run
```

Каждое звено связано через canonical frontmatter-поля (§4.12). Цепочка прослеживаемости — источник истины для compliance audit (§12.5.3).

4.11 Drift классы (closed list)

Закрытый список восьми классов нарушений требовательной инфраструктуры. Изменение списка — formal change procedure (§13.9.3). Нативный для носителя hook (§10.11.1) обязан обнаруживать каждый класс на соответствующей точке enforcement.

#	Класс	Что нарушено	Точка enforcement
3.11.1	Schema drift	frontmatter артефакта не соответствует обязательной schema гл. 06/08/09	hook носителя на change-set; RENAR-3+ — blocking (§11.6.1)
3.11.2	Lifecycle drift	Артефакт вне closed list статусов (§4.6) или прошёл forbidden transition (§10.12)	hook носителя на promote-transition
3.11.3	Source-of-truth drift	Реализационный код / производный артефакт расходится с SR / SPEC, на который ссылается через `verifies[].version` (V5)	Reconciliation hook RENAR-4+; регистрация как backward finding в delta-ADAPT
3.11.4	Implementation drift	Реализационный носитель перестал ссылаться на актуальную `version` носителя требований (V5 pin устарел)	Auto-invalidate `verified` (§10.5.4)
3.11.5	Terminological drift	Использование non-canonical термина (§4.14) в нормативном	hook носителя на change-set; RENAR-4+

		артефакте	— blocking
3.11.6	Order / provenance drift	Артефакт ссылается на источник в более низком статусе чем требует §10.3.1 reference-validation	hook носителя (§10.11.1) блокирует change-set
3.11.7	TC ↔ requirement provenance drift	TC потеряло `verifies[]` ссылку или `last-run.requirement-version` не совпадает с текущей `version`	runner-managed: TC переводится в `failing` (§10.9.3) до повторного прогона
3.11.8	Test-fitting drift	Pass / Fail-критерии TC изменены одновременно с реализационным кодом, чтобы failing TC стал passing без устранения корня (§9.13)	hook носителя через `[test-spec-change]` маркер; единое лицо не может одобрить оба change-set (§10.11.3)

4.12 Термины полей связи (frontmatter, Connection terms)

Canonical имена полей, фиксирующих связи между артефактами:

Поле	Артефакт-источник	Семантика
`parent`	BR / SR	Один родитель в иерархии (BR-NN или SR-NN)
`children[]`	BR / SR	Auto-derived обратное ребро (§6.x)
`implements`	TR	Цепочка реализации (`SR-N` или `BR-N`)
`implements-spec[]`	TR	Реализация спецификаций SPEC-* (§8.6.2)
`constrained-by[]`	SR	Ограничения от SPEC-* (§8.6.1)
`depends-on[]`	SPEC	Граф зависимостей между SPEC (§8.6.3)
`verifies[]`	TC	Закрытый список верифицируемых артефактов с `version` (§9.4)
`verified-by[]`	BR / SR / SPEC	Auto-derived обратное ребро от `verifies[]`
`source.adapt`	BR / SR / SPEC	ADAPT, из которого выведен артефакт

`replaces` / `replaced-by`	Любой	Замена при deprecation (§10.5.3)
`supersedes`	Новая версия артефакта	Какой артефакт заменяется (взамен «реанимации» obsolete)
`last-run`	TC	Результат последнего прогона runner (§9.12); bot-managed only

Полная schema каждого артефакта — в reference/02-schemas.md.

4.13 Сопоставление с родственными стандартами (Mapping)

4.13.1 Артефакты требований

RENAR canonical	SENAR (RU)	ISO/IEC 29148	BABOK v3	SAFe
`BR` (Business Requirement)	БТ (Бизнес-требование)	Business Requirement	Business Need	Portfolio Epic / Strategic Theme
`SR` (System Requirement)	СТ (Системное требование)	System Requirement / Software Requirement	Solution Requirement (Functional)	Feature
`TR` (Task Requirement)	ТЗ (Требование к задаче)	(нет прямого класса; детализация system / system-element requirement)	Solution Requirement (detailed)	Story
`ADAPT`	(RENAR-extension)	(n/a — formalised bridge artefact)	Stakeholder Requirement workshop output	(n/a)
`TC` (Test Case)	ТК	Test Case (verifiable item)	Verification artefact	Story acceptance test

`SPEC-*` (9 types)	(RENAR-extension)	Design Description (subset)	Solution Component (subset)	Enabler tech spec (subset)

4.13.2 Lifecycle статусы

RENAR canonical	ISO/IEC 29148	CMMI
`draft`	proposed	identified
`approved`	agreed-to / baselined	committed
`verified`	verified	validated
`accepted`	accepted	accepted
`deprecated` / `obsolete`	retired / superseded	obsolete / superseded

4.13.3 Multilingual UI

Multilingual проекты могут отображать canonical терминологию в UI на язык клиента (RU пример):

English (canonical)	Russian (UI-перевод)
Business Requirement	Бизнес-требование
System Requirement	Системное требование
Test Case	Тест-кейс
Quality Gate	Контрольная точка качества
Acceptance	Приёмка
Verified	Проверено
Approved	Утверждено
Deprecated	Устарело

Это **только UI-перевод**. Frontmatter, ID, имена файлов, body normative paragraphs — всегда canonical латиница / canonical RU из этой главы.

4.14 Запрещённые / устаревшие термины

Closed list non-canonical терминов; hook носителя RENAR-4+ — blocking при обнаружении в нормативном артефакте (§4.2).

Запрещённый термин	Canonical замена	Почему
«User Story» как требование	`SR`	Story — единица планирования, не требование; story может реализовывать SR через `implements`
«Use Case» (формально как артефакт)	`SPEC-UI + SR`	Use case mixes UX и behavior; RENAR разделяет SPEC-UI (UX) и SR (поведение)
«Spec» (как родовой термин)	Конкретный `SPEC-*` или `requirement` / `SR`	«Spec» неоднозначно; используем точные термины
«Бизнес-логика»	`SR`	Термин кода, не требований
«Функциональность»	`SR` / `TR`	Слишком широкое
«Фича» / «Feature» (как требование)	`BR` (бизнес-уровень) или `Feature` в SAFe-context (не RENAR canonical)	Mixed Russian / English; RENAR использует BR
«Хотелка»	(никогда)	Договорный документ так не пишется
«Эпик» (как требование)	`BR` (бизнес-уровень)	Epic — единица планирования, не требование

4.14.1 Устаревшие RENAR-specific labels

При migration с pre-v1.0 draft material встречаются устаревшие labels:

Устаревший label	Canonical v1.0 замена
`UIC` (UI Concept)	`SPEC-UI` (§8.5.6)
`AIC` (AI Concept)	`SPEC-AI` (§8.5.7)
`TS` (Technical Specification)	`SPEC-ARCH` или `SPEC-OPS` в зависимости от содержания
`INT-SR` (Integration SR)	`SR` с `constrained-by: [SPEC-INT-N]`

`INT-TC` (Integration TC)	`TC` с `tc-type: contract`
`TM` (Module/Submodule SR)	`SR` с `level: module` (§6.7)
`QG-0 Context Gate` (старое)	`QG-0 Approval Gate` (canonical v1.0, §10.3.1)
`QG-1 Requirements Gate` (старое)	`QG-1 Implementation Gate` (canonical v1.0, §10.3.2) — semantic shift: ранее approval BR/SR, теперь только TC `draft → ready`
`QG-2 Implementation Gate` (старое)	`QG-1 Implementation Gate` (canonical v1.0, §10.3.2)
`QG-3 Verification Gate` (старое)	`QG-2 Verification Gate` (canonical v1.0, §10.3.3)

Migration существующего носителя требований со старыми labels — отдельный однократный процесс; нативный для носителя hook на change-set должен auto-detect устаревшие labels и предлагать canonical замены.

4.15 Порядок разрешения разногласий (Authority)

При разногласиях по термину порядок обращения:

1. **Эта глава (§4)** — canonical для RENAR-стандарта.
2. **Соответствующая глава стандарта** (06–14) — для специфичной семантики артефактов (например, ADAPT-специфика — в §7).
3. **SENAR §3** (терминология родительского стандарта).
4. **ISO/IEC 29148:2018** — для общеинженерной терминологии requirements.
5. **BABOK v3** — для business-аналитики терминов.
6. **Фиксация через formal change procedure** стандарта (§13.9.3) — если все вышеперечисленные молчат.

Не использовать как источник терминологии:

- Тикеты ticket-систем (часто противоречивые).
- Чаты команды (slang ≠ canonical).
- Презентации и старые draft-материалы.
- Маркетинговые материалы.

4.16 Связь с другими главами

Глава	Связь
02 Положение в типологии методологий	§2.3 SoT inversion + §2.5 независимое от носителя версионирование — фундамент для §4.8 терминов носителя
06 Иерархия требований	frontmatter артефактов BR / SR / TR — §4.3, §4.9, §4.12 связи
07 ADAPT	ADAPT-специфика — §4.3.2, §4.6.4, §4.6.6 backward sub-states
08 Specifications	SPEC-* типы — §4.4, §4.6.3 lifecycle SPEC
09 Test cases	TC — §4.5, §4.6.5; pos/neg парность — §4.5.3
10 Lifecycle и QG	Quality Gates canonical — §4.7; state machines per type — §4.6; политика закрытого списка — §10.10 параллельно §4.11 / §4.14
03 Версионирование носителя	V1–V6 определения — §4.8.2
11 Maturity model	ai-provenance mandatory на RENAR-4+ — §4.10 (источник критерия — §11.7.1)
12 Metrics	Drift классы §4.11 — источник для метрик типа Reconciliation Findings (§12.3.10)
13 Conformance	§13.3 mandatory clauses ссылаются на canonical терминологию этой главы
reference/01-glossary.md	Развёрнутые объяснения, anti-patterns, history — non-normative

---

title: "Роли" order: 5 lang: ru

05. Роли

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

5.1 Кто за что отвечает

RENAR не заводит свою иерархию ролей — он наследует пять базовых ролей из SENAR §4 и добавляет к ним специализации для работы с требованиями: кто владеет каждым артефактом (ADAPT, BR, SR, SPEC, TR, TC) и кто отвечает за каждый Quality Gate. Здесь же — правило, ради которого глава во многом существует: **двойная подпись ADAPT** (клиент + архитектор), без которой ADAPT не переходит в `approved`.

Дальше: пять базовых ролей SENAR (§5.2), специализации-владельцы RENAR (§5.3), делегирование architect-полномочий (§5.4) и двойная подпись (§5.5).

- **RACI-матрица** (§5.6) — закрытая матрица ответственности по типам артефактов RENAR и каноническим gates.
- **Политика закрытого списка** (§5.7) — список ролей-owners закрыт на v1; project-local расширения запрещены.

Глава **не** нормирует нативные для носителя механизмы реализации ролевых проверок (нативный для носителя ACL, V6 author identifiers, нативные для носителя signature events) — это область главы 3 (capabilities) и `guide/03-tool-guide-*.md` (специфичный для носителя tooling). Зависящая от носителя терминология (имена конкретных review-механизмов, authorship-идентификаторов и task-routing систем) в этой главе не используется.

5.2 Базовые роли (SENAR §4)

5.2.1 Closed list

RENAR ссылается на пять базовых ролей SENAR §4 как на источник истины. Список закрыт на стороне SENAR; RENAR не добавляет и не удаляет роли.

Роль	Краткая семантика (для RENAR-контекста)

Supervisor	Лицо, инициирующее и контролирующее работу AI-агента; в RENAR — типичный заказчик задачи elicitation, draft generation, ADAPT review.
AI-агент	Программный actor, выполняющий генерацию артефактов требований (forward интерпретация ADAPT, draft BR/SR/SPEC/TC) и review-операции; включает primary-генератора и adversarial-критика.
Architect / Tech Lead	Лицо, нормативно ответственное за технические решения и одобрение артефактов требований (QG-0 §10.3.1); подписывающая сторона двойной подписи ADAPT (§5.5).
Reviewer	Лицо или AI-агент, выполняющий adversarial review артефактов; в RENAR — обязательная роль для QG-0 (§10.3.1).
Stakeholder	Внешнее заинтересованное лицо — клиент, представитель клиента с полномочиями, product owner, end-user representative; источник T3 и сторона signature на стороне клиента (§7.5) и QG-4 (§10.4.2).

Полное определение semantics каждой роли — обязанности, ограничения, взаимодействия — изложено в SENAR §4 и применяется к RENAR без изменений.

5.2.2 Запрет переопределения

Реализация **не** имеет права:

- Переименовывать базовые роли SENAR в нормативной части conformance manifest (§13.4).
- Объединять две базовые роли в одну так, чтобы исчезала возможность независимого подписания (например, объединение Architect и Stakeholder делает невозможной двойную подпись §5.5 и **не допускается**).
- Добавлять новые базовые роли вне SENAR §4 без formal change procedure стандарта SENAR.

Project-local **псевдонимы** (например, локальное название «Lead Engineer» как синоним SENAR Architect / Tech Lead) допустимы в коммуникации, но conformance manifest (§13.4) обязан использовать нормативные имена ролей SENAR §4.

5.3 Специализации RENAR: ответственность за артефакты

Каждый тип артефакта RENAR имеет нормативно зафиксированного owner-а — роль, ответственную за инициирование артефакта, его содержательную целостность и one-click promote через QG-0 (§10.3.1).

5.3.1 Owner ADAPT

Параметр	Нормативное значение
Owner	Architect (со стороны исполнителя) **и** Client representative (Stakeholder с полномочиями)
Тип ответственности	Совместный (joint); обе роли обязаны для approval
QG-0 actor	Architect
QG-3 actor (опционально)	Двойная подпись (§5.5): Client + Architect

ADAPT — единственный артефакт RENAR с **обязательной** совместной ответственностью: одна сторона не может довести ADAPT до `approved` без участия другой (§7.5). Это нормативная защита от односторонней интерпретации ТЗ.

5.3.2 Owner BR / SR

Параметр	Нормативное значение
Owner	Architect
QG-0 actor	Architect или authorized role-holder (§5.4)
QG-2 actor	Автоматический runner (без участия role-holder)
QG-4 actor (опционально)	Stakeholder (Client + Product Owner) §10.4.2

Architect нормативно ответственен за декомпозицию ADAPT → BR → SR и за согласованность дерева требований. AI-агент допустим как primary-генератор draft BR/SR, но **не** является owner-ом — owner всегда человек или явно делегированная authorized role-holder (§5.4).

5.3.3 Owner SPEC-*

Параметр	Нормативное значение
Owner	Architect **и** domain tech lead
Тип ответственности	Joint по типу SPEC: Architect — общая ответственность; domain tech lead — содержательная экспертиза для конкретного типа SPEC
QG-0 actor	Architect или authorized role-holder (§5.4)

Domain tech lead — нормативный термин для лица с экспертизой по конкретному типу SPEC из закрытого списка (§8.3): SPEC-ARCH (системный архитектор), SPEC-API (API designer), SPEC-DATA (data architect), SPEC-INT (integration lead), SPEC-PROC (process owner), SPEC-UI (UX lead), SPEC-AI (ML lead), SPEC-SEC (security lead), SPEC-OPS (operations lead).

В малых проектах одно лицо может совмещать domain tech lead роли для нескольких типов SPEC; объединение Architect и всех domain tech leads в одно лицо допустимо при условии явной декларации в conformance manifest (§13.4).

5.3.4 Owner TR

Параметр	Нормативное значение
Owner approval	Architect или authorized role-holder (§5.4)
Owner execution	Engineer (исполнитель TR)
QG-0 actor (`draft → approved`)	Architect или authorized role-holder
QG-2 actor (`approved → done`)	Engineer (с подтверждением автоматического runner)

TR имеет разделённую ответственность: одобрение задачи — у architect / role-holder, выполнение — у engineer. Engineer не может одобрить собственный TR (нарушение §10.2.2).

5.3.5 Owner TC

Параметр	Нормативное значение
Owner	Engineer **и** QA
Тип ответственности	Joint; engineer — авторство, QA — pos/neg парность и достаточность (§9.7)
QG-1 actor (`draft → ready`)	Одобряющий actor (engineer или QA) + автоматический runner (dry-run pass)
QG-2 actor	Автоматический runner с version-pin (V5)

AI-агент допустим как primary-генератор TC, но **обязан** проходить QA-проверку перед `ready` (§10.3.2). В проектах без явной QA-роли — обязательным QA-actor становится engineer, отличный от автора TC.

5.3.6 Owner accepted gate (QG-4)

Параметр	Нормативное значение
Actor QG-4	Stakeholder с полномочиями: Client + Product Owner
Применимость	Только если реализация объявила QG-4 в conformance manifest (§10.4.2)

QG-4 — единственный gate, для которого Architect **не является** достаточным actor. Подтверждение post-release бизнес-результата требует Stakeholder-а с полномочиями. При отсутствии QG-4 в manifest этот gate не применяется, и роль Client/Product Owner на этом этапе lifecycle артефакта не нормируется.

5.3.7 Политика закрытого списка для owner-распределений

Список owner-распределений §5.3.1–§5.3.6 — закрыт на v1. Project-local расширения (новый owner для нового типа артефакта; перераспределение owner-полномочий между ролями SENAR §4) **не допускаются** без formal change procedure стандарта (§13.9).

Negative scenario: попытка реализации объявить, что owner ADAPT — только Architect (без Client representative) — нарушение §5.3.1 и §13.3; manifest с таким объявлением не conformant.

5.4 Authorized role-holder

5.4.1 Нормативное определение

Authorized role-holder — лицо с явно делегированными architect-полномочиями для конкретного scope (один артефакт, один тип артефактов, или весь проект). Делегирование фиксируется нативно для носителя (V6 author identifier + ACL §3.3.6); конкретный механизм делегирования специфичен для носителя и не нормируется в стандарте.

Authorized role-holder — нормативный термин, идентичный по применению в §10.2.2, §10.3.1 и §13.5.

5.4.2 Допустимые сценарии

Authorized role-holder допустим как actor для:

- **QG-0 Approval Gate** (§10.2.2) — для BR / SR / SPEC / TR / ADAPT (на стороне architect) / TC.
- **Self-assessment** (§13.5) — как assessor с ролью `authorized-role-holder` в conformance manifest.

5.4.3 Недопустимые сценарии

Authorized role-holder **не** заменяет:

- **Client signature в ADAPT** (§5.5) — двойная подпись ADAPT нормативно требует client-signature от Stakeholder-а, не от authorized role-holder со стороны исполнителя.
- **Stakeholder в QG-4** (§10.4.2) — accepted gate требует Stakeholder с полномочиями на стороне клиента.
- **External assessor в third-party assessment** (§13.6) — внешняя оценка conformance требует независимости от проекта.

5.4.4 Авторизация на стороне носителя

Делегирование authorized role-holder обязано фиксироваться нативно для носителя через V6 (author + timestamp §3.3.6). Конкретный механизм специфичен для носителя:

- Носитель с ACL-моделью — через role-based access list.
- Носитель с capability-токенами — через выпуск делегирующего токена.
- Носитель без явной авторизации — через декларацию делегирования в нативном для носителя артефакте проекта (например, отдельный файл с подписью architect).

Специфичные для носителя детали реализации делегирования — в `guide/03-tool-guide-*.md` для соответствующих семейств носителей.

5.5 Двойная подпись ADAPT

5.5.1 Нормативное правило

ADAPT переходит в `approved` (QG-3 §10.4.1, §10.8.2) **только** при наличии обеих подписей:

1. `client-signature` — со стороны Client representative (Stakeholder с полномочиями подписания от клиента).
2. `architect-signature` — со стороны Architect исполнителя.

Структура полей подписей фиксирована в §7.5; каждая подпись содержит `signed-by`, `role`, `signed-at`, `signature-ref` (нативный для носителя pointer на signature event).

5.5.2 Семантика каждой подписи

Подпись	Подтверждает
`client-signature`	Forward интерпретация совпадает с намерением клиента; все backward findings отвечены и ответы — финальные; ADAPT представляет согласованное с клиентом понимание Т3.
`architect-signature`	Forward интерпретация технически реализуема; все backward findings в состоянии `resolved` (§7.4.5); декомпозиция в BR / SR / SPEC безопасна для запуска.

Подписи **не** взаимозаменяемы. Architect не подтверждает client-side семантику; Client не подтверждает technical feasibility. Оба подтверждения нормативно обязательны.

5.5.3 Негативные сценарии

- **Одна подпись отсутствует**: ADAPT с заполненной только `client-signature` или только `architect-signature` нормативно **не** переходит в `approved`. QG-3 запрещён (§10.8.2); попытка принудительного перехода — нарушение §13.3.
- **Одно лицо в обеих подписях**: реализация, в которой `client-signature.signed-by == architect-signature.signed-by`, нарушает §5.5.1 (две роли требуют двух независимых лиц). Случай `core` minimal-mode, где `client-signature` опционален (см. `core/renar-core.md`), — вне scope conformance к полному RENAR (§13.3).
- **Authorized role-holder вместо Architect**: допустимо (§5.4.2); подпись фиксируется с `role: authorized-role-holder` и `signature-ref` указывает на нативное для носителя доказательство делегирования.
- **Authorized role-holder вместо Client**: **не допустимо** (§5.4.3); client-signature обязана быть от Stakeholder-а на стороне клиента.

5.5.4 Связь с другими gates

Двойная подпись ADAPT — единственный нормативный случай dual signature в RENAR. Остальные gates (§10.3) выполняются одним actor-ом (architect, runner, stakeholder). Это сознательное архитектурное решение: ADAPT — точка согласования с клиентом, остальные gates — внутренние проверки реализации.

5.6 RACI-матрица

Матрица фиксирует Responsible / Accountable / Consulted / Informed по типам артефактов RENAR и каноническим gates. Сокращения: **R** = Responsible (исполняет), **A** = Accountable (одобряет / отвечает за результат), **C** = Consulted (обязан быть проинформирован до решения), **I** = Informed (уведомляется после решения).

5.6.1 Матрица артефактов

Активность	AI-агент	Architect	Domain TL	Engineer	QA	Reviewer	C
Импорт ТЗ	R	A	—	—	—	C (adversarial)	C
Создание ADAPT (forward + backward)	R	A	C	—	—	C (adversarial)	C
Approval ADAPT (двойная подпись)	—	**A (architect-signature)**	—	—	—	C (adversarial)	**A s**
Декомпозиция ADAPT → BR	R	A	C	—	—	C	I
Декомпозиция BR → SR	R	A	C	—	—	C	I
Создание SPEC-*	R	A	**R/A (per type)**	—	—	C	I
Создание TR	R	A	C	C	—	—	I

Реализация TR (execution)	—	C	C	R	—	—	I
Создание TC	R	C	C	R/A	A	—	I
Adversarial review артефакта	R (AI-критик)	A	—	—	—	R	–

5.6.2 Матрица Quality Gates

Соответствие нормативно фиксированной таблице actor-ов §10.2.2.

Gate	Артефакты	Responsible	Accountable	Consulted	I
QG-0 Approval Gate	BR, SR, TR, SPEC, TC, ADAPT (architect-side)	Architect или authorized role-holder	Architect	AI-критик (Reviewer)	K
QG-1 Implementation Gate	TC (`draft` → `ready`)	Engineer / QA	Engineer	Автоматический runner	K
QG-2 Verification Gate	BR, SR, TR, SPEC, TC	Автоматический runner (V5 + V6)	Architect	—	K
QG-3 Architecture Gate (опционально, ADAPT)	ADAPT (`answered` → `approved`)	Двойная подпись (Client + Architect)	Architect	AI-критик	K
QG-4 Acceptance Gate (опционально, BR)	BR (`verified` → `accepted`)	Stakeholder (Client + PO)	PO	Architect	K

5.6.3 Closed list ролей в RACI

Список ролей-колонок матрицы §5.6.1 / §5.6.2 — закрыт на v1 RENAR:

AI-агент, **Architect**, **Domain tech lead** (per SPEC-type), **Engineer**, **QA**, **Reviewer** (adversarial), **Client** (Stakeholder с полномочиями), **Product Owner**.

Project-local роли (например, «release Manager», «Compliance Officer») допустимы как **Informed** или **Consulted** в локальной practical-RACI реализации, но **не** появляются как **Responsible** или **Accountable** в нормативной матрице conformance — иначе manifest не conformant (§13.3).

5.7 Политика закрытого списка (closed list)

5.7.1 Что зафиксировано на v1

- Список базовых ролей SENAR §4 (§5.2.1) — closed на стороне SENAR; RENAR не расширяет.
- Owner-распределение по типам артефактов (§5.3.1–§5.3.6) — closed.
- Authorized role-holder как нормативный термин (§5.4) — closed; semantics и scope изменяются только через formal change procedure стандарта.
- Правило двойной подписи ADAPT (§5.5.1) — closed; не подменяется и не отменяется project-local.
- Список ролей-колонок RACI (§5.6.3) — closed.

5.7.2 Declared-stricter (допустимо)

Реализация может **ужесточить** требования относительно нормативного минимума, явно декларируя это в conformance manifest (§13.4) с указателем `declared-stricter` (§10.10.2):

- Требовать двойную подпись для BR/SR (не только для ADAPT).
- Требовать external adversarial reviewer на QG-0 для SPEC-SEC и SPEC-AI.
- Запретить совмещение Architect и domain tech lead в одном лице.

Declared-stricter — допустимая локальная политика; соответствие нормативному уровню сохраняется.

5.7.3 Declared-weaker (запрещено)

Реализация **не** имеет права declared-weaker относительно §5 (одностороннее ослабление):

- Объявить, что ADAPT approve возможен с одной подписью.

- Объявить, что engineer-author TC может самостоятельно одобрить TC без QA-actor.
- Объявить, что QG-4 actor может быть исполнитель проекта вместо Stakeholder-a.

Manifest с любым declared-weaker относительно §5 — non-conformant (§13.8 loss-of-conformance).

5.7.4 Путь для расширений

Добавление новой роли в нормативный список (§5.2, §5.3, §5.6.3) или новой owner-комбинации (§5.3.7) производится **только** через formal change procedure стандарта RENAR (§13.9): research draft → public review → minor-version bump. Project-local extensions остаются за пределами conformance.

5.8 Перекрёстные ссылки

Источник	Применение
SENAR §4	Closed list 5 базовых ролей; semantics и обязанности (нормативный источник)
§7.5 ADAPT approval schema	Структура полей `client-signature` и `architect-signature`
§10.2.2 QG actor table	Нормативное соответствие gate → actor; согласовано с §5.6.2
§10.3.1 QG-0 Approval Gate	Architect или authorized role-holder как actor §5.4
§10.4.1 QG-3 Architecture Gate	Dual signature requirement (§5.5) для ADAPT
§10.4.2 QG-4 Acceptance Gate	Stakeholder (Client + PO) — §5.3.6
§3.3.6 V6 Author + timestamp	нативная для носителя фиксация авторства для делегирования role-holder §5.4.4
§13.4 Conformance manifest	Использование нормативных имён ролей §5.2.2
§13.5 Self-assessment	Assessor role enum (architect / authorized-role-holder / external-assessor) — §5.4 anchor

`core/renar-core.md`	Minimal-mode: client-signature optional для ADAPT (вне scope полного conformance §5.5.3)
`guide/03-tool-guide-*.md`	специфичные для носителя механизмы делегирования (§5.4.4) и signature implementations

---

← Предыдущая: 04. Термины и определения · **Оглавление** · **Следующая: 06. Иерархия требований →**

---

title: "Иерархия требований" order: 6 lang: ru

06. Иерархия требований

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

> ***Плотная глава:*** *перед frontmatter — guide/00 quickstart; плотность глав — reference/09.*

6.1 Три типа требований и три уровня

Глава нормирует ось требований RENAR — closed list трёх типов (BR, SR, TR) и три уровня организационной декомпозиции (система, подсистема, модуль). На этих понятиях держится Source of Truth-иерархия из главы 2 §2.3: ТЗ → ADAPT → BR / SR / SPEC → TR → TC. Параллельная ось спецификаций SPEC-* нормирована в главе 8; BR / SR / TR ссылаются на SPEC через типизированные рёбра графа связей.

Дальше по тексту: закрытый список типов BR / SR / TR и родительское дерево (§6.2), уровни «система / подсистема / модуль» и допустимые типы требований (§6.3–6.4), нормативные профили и frontmatter BR / SR / TR (§6.5–6.7), расширения для составных систем и эволюция иерархии (§6.8–6.9), связь с ADAPT и SPEC (§6.10), схема хранения и контрольные точки качества для оси требований (§6.11–6.12).

Положения главы являются нормативными. Закрытые списки типов и уровней — mandatory clauses RENAR conformance (глава 13); расширение возможно только через formal change procedure стандарта.

Глава опирается на ISO/IEC/IEEE 29148:2018 «Requirements engineering» в части понятий business / system / task requirements и принципов traceability, но фиксирует closed list ровно из трёх типов оси требований v1.0 и явно отстраивается от свободно расширяемого набора типов, характерного для классических подходов.

6.2 Закрытый список трёх типов оси требований

6.2.1 Нормативная формулировка

Ось требований RENAR v1.0 содержит ровно три типа: BR, SR, TR. Список закрыт. Новые типы добавляются только через formal change procedure стандарта (глава

13).

Тип	Расшифровка	Вопрос	Содержит	Не содержит
BR	Business Requirement	Кто, что и зачем?	Бизнес-цель, роль, ценность	Технологии, экраны, контракты, поля данных
SR	System Requirement	Что делает система?	Поведение системы, ограничения	Имена таблиц, фреймворки, конкретные структуры
TR	Task Requirement	Что именно реализовать?	Конкретика реализации: поля, условия, ошибки	Архитектурные решения

Каждый тип отвечает на свой вопрос и принадлежит своему слою waterfall-формы стандарта (глава 2 §2.4).

6.2.2 Дерево родителей

```
BR
 └── SR              # parent = BR (единственный)
      └── TR         # parent = SR (единственный)
                ↓
          Goal + Acceptance Criteria
          в системе управления задачами
```

`SR.parent` — единственный BR. `TR.parent` — единственный SR. Это **дерево**, не граф; множественные родители на оси требований запрещены. Граф связей — между требованиями и спецификациями (§6.10).

6.2.3 Что не является типом оси требований

Артефакты, исторически встречавшиеся в проектах под именами UIC (UI Concept), AIC (AI Concept), INT-SR (интеграционное требование), TS (техническая спецификация), **не являются типами оси требований в v1.0**. Они перенесены в параллельную ось спецификаций как соответствующие типы SPEC (см. §8.3 closed list 9 типов SPEC и §8.7 migration):

Legacy артефакт	RENAR v1.0 тип
UIC	SPEC-UI
AIC	SPEC-AI
INT-SR	SPEC-INT
TS (architecture / data / API / process / security / ops)	SPEC-ARCH / SPEC-DATA / SPEC-API / SPEC-PROC / SPEC-SEC / SPEC-OPS

SPEC-* связан с осью требований не как «ещё один тип требования», а как параллельная ось с типизированными рёбрами `SR.constrained-by[]` и `TR.implements-spec[]` (§6.10).

Тест-кейсы (TC) — отдельный класс артефактов верификации, нормируемый главой 9. TC не являются требованиями: они проверяют поведение, описанное в BR / SR / SPEC.

6.3 Системы, подсистемы, модули

Уровни организационной декомпозиции — closed list трёх элементов: **система**, **подсистема**, **модуль**. Каждому элементу соответствует допустимый набор типов требований (см. §6.4).

6.3.1 Система

Система — верхний уровень иерархии. Целостный продукт или платформа, которая поставляется и эксплуатируется как единое целое и за которую отвечает организация.

Признаки системы:

- единый владелец со стороны бизнеса (Product Owner, директор, заказчик);
- поставляется и эксплуатируется как единое целое;
- клиент видит и оценивает систему в целом, а не её части по отдельности;
- единый верхнеуровневый ТЗ-документ и один корневой ADAPT.

Допустимые типы требований: **BR, SR, TR**.

6.3.2 Подсистема

Подсистема — крупный самостоятельный компонент системы, обладающий собственной бизнес-ценностью или отдельным бизнес-владельцем.

Подсистема выделяется, если выполняется **хотя бы одно** из условий:

Условие	Пример
Своя команда или технический владелец	Команда фронтенда vs команда AI-pipeline
Отдельная база данных или независимо разворачиваемый сервис	Изолированный микросервис с отдельным деплоем
Возможность быть заменённой независимо от других	Аналитический модуль заменяется без изменения операционного контура
Другой бизнес-домен или отдельная заинтересованная сторона	Финансовый директор vs операционный директор
Добавляется позже как отдельная инициатива с отдельным бюджетом	Партнёрская программа

Ключевой нормативный критерий: существует ли отдельный человек со стороны бизнеса, отвечающий за ценность этой части системы?

- да → подсистема, BR оправданы;
- нет → модуль, только SR.

Допустимые типы требований: **BR (если есть своя заинтересованная сторона) + SR + TR**.

6.3.3 Модуль

Модуль — техническое деление внутри подсистемы. Реализует часть поведения подсистемы, но не имеет самостоятельной бизнес-ценности.

Признаки модуля:

- отсутствует отдельная бизнес-заинтересованная сторона;
- не существует и не используется в отрыве от своей подсистемы;
- выделяется по техническому принципу: функциональная область, слой, домен;
- не упоминается отдельно в ТЗ верхнего уровня.

Допустимые типы требований: **SR + TR**. BR не создаётся.

6.3.4 Сводная таблица

	Система	Подсистема	Модуль
Бизнес-заинтересованная сторона	да	да (свой)	нет

Независимое развёртывание	возможно	да	нет
Упоминается в ТЗ отдельно	да	да	редко
Существует отдельно	да	возможно	нет
BR	да	да (если своя заинтересованная сторона)	нет
SR	да	да	да
TR	да	да	да

6.3.5 Эволюция «модуль → подсистема»

Граница между модулем и подсистемой не является навечно зафиксированной. Если модуль вырос, получил собственную команду или бизнес-владельца — он становится подсистемой и получает BR. Нормативно: **BR пишется в момент появления бизнес-владельца, не авансом**.

Обратная эволюция (подсистема → модуль) — также допустима, если бизнес-владелец ушёл, бизнес-ценность стала производной; в этом случае BR подсистемы получает статус `deprecated` (§6.5.4), а его SR переподчиняются BR родительской системы.

6.4 Допустимые типы требований по уровням декомпозиции

Уровень	BR	SR	TR	Пояснение
Система	mandatory	mandatory	mandatory	Верхний BR — обязателен (без бизнес-цели нет проекта)
Подсистема	optional	mandatory	mandatory	BR — только при наличии собственной заинтересованной стороны
Модуль	not allowed	mandatory	mandatory	BR запрещён нормативно

Нормативное обоснование запрета BR на уровне модуля: бизнес-требование без бизнес-заинтересованной стороны и без независимой бизнес-ценности приводит к

ложной декомпозиции и порождает «технические BR», не отличимые от SR. Это размывает SoT-иерархию (глава 2 §2.3).

6.5 BR — Business Requirement

6.5.1 Нормативное определение

BR фиксирует **что нужно бизнесу и зачем**. Описывает роль, действие и бизнес-ценность без отсылок к технологиям, экранам, контрактам, структурам данных.

6.5.2 frontmatter BR (mandatory поля)

```
---
id: BR-NN                              # immutable; NN sequential в
рамках scope
title: "<short, descriptive>"
type: BR
slug: "<kebab-case>"                   # auto-derived

# === Scope (mandatory) ===
level: system | subsystem              # BR на уровне модуля запрещён
(§6.4)
scope:
  system: "<system-id>"
  subsystem: "<subsystem-id>"          # null если level=system

# === Lifecycle (mandatory) ===
status: draft | approved | verified | deprecated
owner: "<role / responsible person>"

# === Source: provenance from ADAPT (mandatory) ===
source:
  adapt: ADAPT-NNN                     # ссылка на ADAPT (глава 7)
  adapt-section: "Forward §N"          # canonical интерпретация
  tz-section: "§N.N"                   # optional, для traceability

# === Граф связей (auto-managed) ===
children: []                           # auto-derived; SR со ссылкой
```

```
parent.id=<этот BR>
verified-by: []                        # auto-derived; TC
верифицирующие через SR

# === AI provenance (mandatory на RENAR-4+; canonical schema —
§4.10.1) ===
ai-provenance:
  generated-by: "<vendor>-<model>@<date>"
  generated-at: "<ISO-8601>"
  prompt-template: "<template-path>@<version>"
  context-tokens: integer
  output-tokens: integer
  human-edits: boolean
  # optional на RENAR-4, mandatory на RENAR-5 (см. §4.10.1):
  # cost-budget, cost-actual, generation-time-ms

# === Замена (mandatory если применимо) ===
replaces: "<old-id>"
replaced-by: "<new-id>"
deprecated-date: "<ISO date>"
---
```

Поле `source.adapt` — нормативно обязательное. Производство BR без ссылки на approved ADAPT — нарушение стандарта; hooks lifecycle (глава 7 §7.4.1) обязаны блокировать создание BR с `source.adapt` на ADAPT в статусе ниже `approved`.

Поле `parent` отсутствует в BR: BR — корневой узел дерева требований.

6.5.3 Body BR (обязательные разделы)

Раздел	Обязательность	Содержание
Потребность	mandatory	Кто (роль), что (действие), зачем (бизнес-цель). Формулировка в одном предложении.
Критерии успеха	mandatory	Измеримые результаты (3–7 пунктов); каждый поддаётся независимой проверке.
Контекст	mandatory	Откуда взялось требование (со ссылкой на раздел ADAPT), какие альтернативы рассматривались.

Ограничения	optional	Бизнес-ограничения (бюджет, сроки, регуляторика), не технические.

Технические детали (UI, API, схема данных) в BR запрещены — для этого существуют типы SPEC (глава 8) и SR.

6.5.4 Статусы BR

Статус	Смысл	Триггер перехода
`draft`	В проработке	Создаётся автором
`approved`	Утверждено, можно декомпозировать в SR	После QG-0 (глава 10)
`verified`	Все производные SR / TR / TC выполнены, бизнес-результат подтверждён	После QG-2; все `verified-by` TC имеют `last-run.result = pass` на текущей версии
`deprecated`	Устарело; заменено другим BR или утратило актуальность	Архитектором / Product Owner, обязательно с `replaced-by` (если замена)

BR в статусе `deprecated` **не удаляется** — остаётся как исторический след для аудита.

6.6 SR — System Requirement

6.6.1 Нормативное определение

SR фиксирует **что делает система** (на уровне системы, подсистемы или модуля). Описывает наблюдаемое поведение и ограничения. Не описывает имена таблиц, фреймворков и конкретных структур данных — это ответственность SPEC (глава 8).

6.6.2 frontmatter SR (mandatory поля)

```
---
id: SR-NN                              # immutable
title: "<short, descriptive>"
type: SR
```

```yaml
slug: "<kebab-case>"

# === Scope (mandatory) ===
level: system | subsystem | module
scope:
  system: "<system-id>"
  subsystem: "<subsystem-id>"        # null если level=system
  module: "<module-id>"              # null если level ≠ module

# === Lifecycle (mandatory) ===
status: draft | approved | verified | deprecated
owner: "<role / responsible person>"

# === Parent (mandatory) ===
parent:
  id: BR-NN                          # единственный родитель

# === Source: provenance from ADAPT (mandatory) ===
source:
  adapt: ADAPT-NNN
  adapt-section: "Forward §N"
  tz-section: "§N.N"                 # optional

# === Граф связей (mandatory ones + auto-managed) ===
constrained-by:                      # типизированные рёбра к SPEC
(глава 8)
  - SPEC-UI-NN
  - SPEC-API-NN
  - SPEC-DATA-NN
children: []                         # auto-derived; TR со ссылкой
parent.id=<этот SR>
verified-by: []                      # auto-derived; TC
верифицирующие SR

# === AI provenance (mandatory на RENAR-4+) ===
ai-provenance:
  generated-by: "<vendor>-<model>@<date>"
  prompt-template: "<template-path>@<version>"
  context-tokens: integer
  output-tokens: integer
  human-edits: boolean
```

```
# === Замена (mandatory если применимо) ===
replaces: "<old-id>"
replaced-by: "<new-id>"
deprecated-date: "<ISO date>"
---
```

Ключевые поля. `parent.id` — единственный BR; это дерево родителей. `constrained-by[]` — типизированные ссылки на SPEC-*; это **граф**, не дерево. SR может ссылаться на произвольное количество SPEC любых типов; SPEC, в свою очередь, может быть `referenced-by` множеством SR (глава 8 §8.2).

6.6.3 Body SR (обязательные разделы)

Раздел	Обязательность	Содержание
Требование	mandatory	Одно предложение нормативной формы: «Система должна …» (модальность — по конвенции §0.5.1: «должна» / «обязана» = mandatory).
Поведение	mandatory	Детальное описание наблюдаемого поведения; функциональные сценарии.
Ограничения	mandatory если применимо	Нефункциональные ограничения (производительность, безопасность); полные ограничения — в `constrained-by[]` SPEC.
Связь с SPEC	mandatory если присутствует `constrained-by[]`	Краткое объяснение, какие аспекты поведения нормированы какими SPEC.

6.6.4 Статусы SR

Идентичны статусам BR (§6.5.4) — `draft → approved → verified → deprecated`. Переход `approved → verified` — после QG-2 (глава 10); требует, чтобы все `verified-by` TC имели `last-run.result = pass` на текущей версии SR.

6.7 TR — Task Requirement

6.7.1 Нормативное определение

TR — атомарная единица работы исполнителя. Фиксирует **что именно реализовать** в рамках одного SR. TR декомпозирует SR до уровня, пригодного для прямой реализации (одна задача — одна TR).

6.7.2 frontmatter TR (mandatory поля)

```yaml
---
id: TR-NN                              # immutable
title: "<short, descriptive>"
type: TR
slug: "<kebab-case>"

# === Scope (mandatory) ===
level: system | subsystem | module   # system — редко, кросс-подсистемные задачи
scope:
  system: "<system-id>"
  subsystem: "<subsystem-id>"        # null если level=system
  module: "<module-id>"              # null если level ≠ module

# === Lifecycle (mandatory) ===
status: draft | approved | done | obsolete
owner: "<assignee role / agent>"

# === Parent (mandatory) ===
parent:
  id: SR-NN                          # единственный родитель

# === Source: trace chain (auto-derived) ===
source:
  adapt: ADAPT-NNN                   # auto-derived from parent SR
  sr-version: "<version-ref>"        # pinning к версии SR (возможность носителя V5; глава 3)

# === Граф связей ===
implements-spec:                     # типизированные рёбра к SPEC
  - SPEC-API-NN
  - SPEC-UI-NN
```

```yaml
verified-by: []                       # auto-derived; TC верифицирующие через SR

# === Goal + Acceptance Criteria ===
goal: "<one-sentence outcome>"
acceptance-criteria:
  - "<numbered, falsifiable, без двусмысленности>"
  - "..."

# === AI provenance (mandatory на RENAR-4+) ===
ai-provenance:
  generated-by: "<vendor>-<model>@<date>"
  human-edits: boolean
---
```

Ключевые поля. `parent.id` — единственный SR. `implements-spec[]` — типизированные рёбра к SPEC; конкретизирует, какие SPEC должны быть учтены при реализации именно этого TR (подмножество `constrained-by[]` родительского SR или его расширение типами SPEC, не указанными у SR прямо). `acceptance-criteria` — закрытый numbered список falsifiable утверждений.

6.7.3 Body TR (обязательные разделы)

Раздел	Обязательность	Содержание
Goal	mandatory	Один параграф; результат, который TR делает наблюдаемым.
Acceptance Criteria	mandatory	Numbered list; каждый пункт falsifiable; покрывает positive и negative сценарии.
Scope	mandatory	Что входит / что не входит в TR (соответствует SENAR Rule 2).
Ссылки	mandatory если применимо	На SPEC из `implements-spec[]` и разделы родительского SR.

6.7.4 Статусы TR

Статус	Смысл	Триггер

`draft`	TR создан, AC ещё не финализированы	Авторство
`approved`	AC утверждены, разрешён старт работы	QG-0 (глава 10): goal + AC присутствуют
`done`	AC верифицированы, TC прошли	QG-2; `verified-by` TC `pass`
`obsolete`	TR утратил актуальность до завершения (например, SR изменился)	Архитектором, обязательно с пометкой

6.7.5 TR не ссылается на ADAPT напрямую

Исполнитель TR работает в рамках SR / SPEC и **не обращается к ADAPT напрямую** — все нужные интерпретации T3 уже зафиксированы в approved ADAPT и протянуты в SR / SPEC через `source.adapt` (глава 7 §7.7.3). Если исполнитель обнаружил неясность в SR — это сигнал либо для новой обратной находки в ADAPT (если корень неясности в T3), либо для уточнения SR (если корень в декомпозиции).

6.8 Расширенная иерархия для составных систем

Базовая схема `BR → SR → TR` справедлива для большинства проектов. Для составных систем стандарт нормирует два варианта расширенной иерархии.

6.8.1 Подсистема — техническое деление, не самостоятельный продукт

BR относятся к системе в целом; подсистемы — техническое деление по командам, компонентам или другим архитектурным границам:

```
BR (система)
 └── SR (система)          # опционально, если есть кросс-подсистемные SR
       └── SR (подсистема)
             └── TR

SPEC-INT (между подсистемами)    # параллельная ось; см. главу 8
```

`SPEC-INT` не принадлежит ни одной из подсистем — это спецификация интеграции на уровне системы.

6.8.2 Подсистема — самостоятельный продукт со своей заинтересованной стороной

Подсистема имеет собственный BR со своим бизнес-владельцем:

```
BR (система)
 └┄┄ BR (подсистема)       # ┄┄ = scope containment, НЕ parent-edge
      └── SR (подсистема)
           └── TR
```

`└┄┄` между BR (система) и BR (подсистема) обозначает принадлежность подсистемы к scope системы, **не parent-edge дерева требований**. `BR (подсистема).parent` отсутствует — каждая такая подсистема является корневым узлом своего дерева требований (как и BR системы). Связь между деревьями системы и подсистемы — через раздел `Контекст` BR подсистемы и через общий ADAPT системы.

6.8.3 Запрет множественных родителей

Стандарт не допускает множественное `parent` для SR или TR. Кросс-функциональные требования, которые могли бы выглядеть как «дочерние сразу двух SR», нормируются одним из двух способов:

- разделяются на несколько SR, каждое с единственным parent BR;
- декомпозируются в SR более высокого уровня (родительская подсистема или система), от которой эти кросс-функциональные сценарии и зависят.

6.9 Эволюция иерархии

6.9.1 Модуль → подсистема

Сценарий из §6.3.5: модуль получает бизнес-владельца. Нормативная последовательность:

1. Появление бизнес-владельца фиксируется через обратную находку в ADAPT (категория `scope` — изменение границ работ; глава 7 §7.4.4).
2. После approved delta-ADAPT — модуль переводится в статус подсистемы; создаётся BR подсистемы с `source.adapt: <delta-ADAPT>`.

3. Существующие SR модуля сохраняются (immutable IDs); поле `parent` SR обновляется на новый BR подсистемы атомарным изменением.
4. TR / TC, ссылающиеся на эти SR, не требуют изменений (parent SR неизменен).

6.9.2 Запрет авансовой иерархии

Создание BR подсистемы «на вырост», без существующего бизнес-владельца — нарушение стандарта. BR без заинтересованной стороны превращается в «технический BR», размывающий SoT-инверсию и подменяющий собой SR. Hooks lifecycle (глава 10) обязаны блокировать переход BR в `approved`, если в ADAPT не зафиксирован identified бизнес-владелец.

6.9.3 Подсистема → модуль

Симметричный сценарий §6.3.5: подсистема утратила бизнес-владельца или бизнес-ценность стала производной от родительской системы. Нормативная последовательность:

1. Утрата бизнес-владельца / переоценка бизнес-ценности фиксируется через обратную находку в ADAPT (категория `scope`; глава 7 §7.4.4).
2. После approved delta-ADAPT — BR подсистемы переводится в статус `deprecated` с указанием причины в `Контексте` (бизнес-владелец отозван / бизнес-ценность поглощена системой). BR не удаляется (immutable IDs, V1).
3. SR подсистемы сохраняются (immutable IDs); поле `parent` SR обновляется атомарным изменением на BR родительской системы (или на BR другой подсистемы, если scope SR относится к ней).
4. Подсистема переименовывается в модуль на уровне scope и storage layout (§6.11.2); существующие IDs SR / TR остаются неизменными.
5. TR / TC, ссылающиеся на эти SR, не требуют изменений.

Если ни одна BR родительской системы не покрывает поведение SR — это сигнал того, что подсистема в действительности не утратила самостоятельную бизнес-ценность; обратная эволюция в этом случае невозможна, и BR подсистемы остаётся `approved`.

6.10 Связь иерархии с ADAPT и SPEC

Стандарт фиксирует связи между осью требований, ADAPT и параллельной осью SPEC через нормативные поля frontmatter и типизированные рёбра графа связей.

6.10.1 Связь с ADAPT

`BR.source.adapt`, `SR.source.adapt`, `SPEC-*.source.adapt` — mandatory ссылки на ADAPT, из которого артефакт был выведен (глава 7 §7.7.1). TR не имеет прямого `source.adapt` — наследует его через `parent SR` (§6.7.5).

6.10.2 Граф связей с SPEC

```
Дерево требований (ось поведения):        Параллельная ось
спецификаций:
BR
 └── SR  ──── constrained-by[] ────►    SPEC-ARCH  SPEC-API  SPEC-
DATA
      └── TR ─── implements-spec[] ──►   SPEC-INT   SPEC-PROC SPEC-UI
                                          SPEC-AI    SPEC-SEC  SPEC-
OPS
```

Ребро	Тип	Обязательность
`SR.constrained-by[] → SPEC-*`	Граф (множественное)	Optional; присутствует при наличии нормирующих SPEC
`TR.implements-spec[] → SPEC-*`	Граф (множественное)	Optional; конкретизирует SPEC для исполнителя
`SPEC-*.referenced-by[] → SR / TR`	Auto-derived inverse	Авто-вычисляется нативной для носителя индексацией
`SPEC-*.depends-on[] → SPEC-*`	Граф между SPEC	См. глава 8 §8.2

Связь оси требований и оси SPEC — нормативная: ни один SR с нетривиальным UI / API / data behavior не должен оставаться без `constrained-by[]` (отсутствие — сигнал либо для написания SPEC, либо для обоснования отсутствия в Контексте SR).

6.10.3 Полная trace chain (read-side)

```
TC-NN  →  verifies SR-12 v1.4
              │
```

```
                ├─ parent:         BR-03 v2.0
                ├─ source.adapt:   ADAPT-001 §Forward §3.2
                ├─ constrained-by: SPEC-UI-04, SPEC-API-02
                └─ children:       TR-101, TR-102, TR-103
                                       └─ implements-spec: SPEC-API-02
                                       └─ verified-by: TC-NN
```

При аудите от любого артефакта (TC / TR / SR / SPEC) восстанавливается полная цепочка до раздела ТЗ через approved ADAPT.

6.11 Схема хранения

Требования хранятся в подпапках носителя требований. Нативная для носителя реализация хранения специфична для носителя (см. guide/03 для distributed VCS; guide/04 для document-oriented store).

6.11.1 На уровне системы

```
[requirements-substrate]/        # корень носителя требований (layout
— guide/03 или guide/04)
  br/                            # BR-NN-*.md
  sr/                            # SR-NN-*.md, level=system
  tr/                            # TR-NN-*.md, level=system (редко)
  specs/                         # SPEC-* (глава 8)
  adapt/                         # ADAPT (глава 7)
  tz/                            # immutable исходники ТЗ
  REQUIREMENTS.md                # auto-generated index
```

6.11.2 На уровне подсистемы / модуля

```
[subsystem-substrate]/           # scope подсистемы внутри носителя
требований
  br/                            # если у подсистемы своя
заинтересованная сторона
  sr/
  tr/
```

```
modules/
  [module-substrate]/
    sr/                          # модуль имеет только SR + TR
    tr/
specs/                           # глава 8
adapt/
REQUIREMENTS.md
```

6.11.3 REQUIREMENTS.md — auto-generated index

`REQUIREMENTS.md` — auto-generated реестр всех BR / SR / TR в scope: ID, тип, уровень, заголовок, статус, parent, ссылка на файл. Помечается нативным для носителя флагом auto-generated. Триггеры перегенерации — каждое изменение frontmatter или каждый approve / verify gate (глава 10).

6.12 Контрольные точки качества для оси требований

Детальные определения gates — в глава 10. Краткая сводка для BR / SR / TR:

Gate	Применим к	Предусловие	Постусловие
QG-0 (Approval)	BR / SR / TR (`draft → approved`)	frontmatter валиден, mandatory fields заполнены, идентификатор уникален (V1); `source.adapt` указывает на approved ADAPT (BR / SR); `parent` указывает на approved BR / SR (SR); body разделы соответствуют §6.5.3 / §6.6.3; adversarial review произведён	Артефакт переведён в `approved` ; immutable до версии следующего change; разрешена декомпозиция в дочерние артефакты
QG-2 (Verification)	BR / SR / TR (`approved → verified` / `done`)	Все `verified-by` TC `pass` на текущей версии артефакта	Артефакт `verified` (BR/SR) или `done` (TR); цепочка до родительского BR — обновляется к verified, если все дети verified

QG-1 (Implementation) к оси требований **не применяется**: это отдельный gate только для TC (§10.3.2) — промежуточного «QG-1 implementation» для BR / SR / TR не существует, переход `approved → verified` / `done` управляется единым QG-2.

TR переходит `draft → approved` тем же QG-0 единым шагом (frontmatter + goal + AC). `ready` и аналогичные термины — не статусы оси требований и не присутствуют в state machine `draft → approved → verified | done | obsolete | deprecated` (см. §6.5.4 / §6.6.4 / §6.7.4).

Hooks носителя (глава 3 §3.3) обязаны блокировать переходы, нарушающие предусловие соответствующего гейта.

6.13 Связь с другими главами

Глава	Связь
02 Положение в типологии методологий	Иерархия BR / SR / TR — несущая структура SoT-инверсии (Утверждение 1); waterfall-форма (Утверждение 2) задаёт слои BR → SR → TR
07 ADAPT	`BR.source.adapt`, `SR.source.adapt` — mandatory; ось требований выводится только из approved ADAPT
08 Specifications	Параллельная ось SPEC; `SR.constrained-by[]`, `TR.implements-spec[]` — типизированные рёбра графа
09 Test cases	TC верифицируют BR / SR / TR; `verified-by[]` — auto-derived inverse
10 Lifecycle и QG	State machines BR / SR / TR; QG-0 / QG-2 для оси требований (QG-1 — только для TC)
03 Версионирование носителя	Immutable IDs (V1); atomic change unit при переподчинении (V2); diff & review для approve (V3); версионирование без потери истории (V4); pinning SR-version в TR (V5); нативная для носителя подпись approve с author + timestamp (V6)
11 Maturity model	RENAR-1: ось BR / SR / TR обязательна; RENAR-3+: `constrained-by[]` для всех SR, где применимо
13 Conformance	Closed list типов (BR / SR / TR) — mandatory clause v1.0; closed list уровней (система / подсистема / модуль) — mandatory clause v1.0
reference/02 — schemas	Полная machine-readable schema BR / SR / TR frontmatter

---

title: "ADAPT — двусторонняя адаптация ТЗ" order: 7 lang: ru

07. ADAPT — двусторонняя адаптация ТЗ

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

7.1 Что такое ADAPT и зачем он

Глава нормирует **ADAPT** — обязательный артефакт жизненного цикла RENAR, расположенный между immutable ТЗ (договорным документом) и инженерными требованиями BR/SR/SPEC. ADAPT обеспечивает двустороннюю адаптацию: **прямая интерпретация** ТЗ (от инженерии к заказчику) и **обратные находки** (вопросы, противоречия и пробелы — от исполнителя к клиенту, с регламентом ответов и статусов).

Дальше: типичные провалы без ADAPT и цикл двусторонней адаптации (§7.2–7.3), нормативные требования к составу и категориям обратных находок (§7.4), двойное утверждение (§7.5), дельты и errata (§7.6), связность с BR/SR/SPEC/TC и схема frontmatter (§7.7–7.8), контрольные точки качества и ИИ-сценарии (§7.9–7.10), схема хранения (§7.11).

ADAPT — следствие Утверждения 2 из §2.4 (RENAR — waterfall-form ≠ classical waterfall, потому что ADAPT обеспечивает двустороннюю адаптацию вместо «переброса спецификации через забор»).

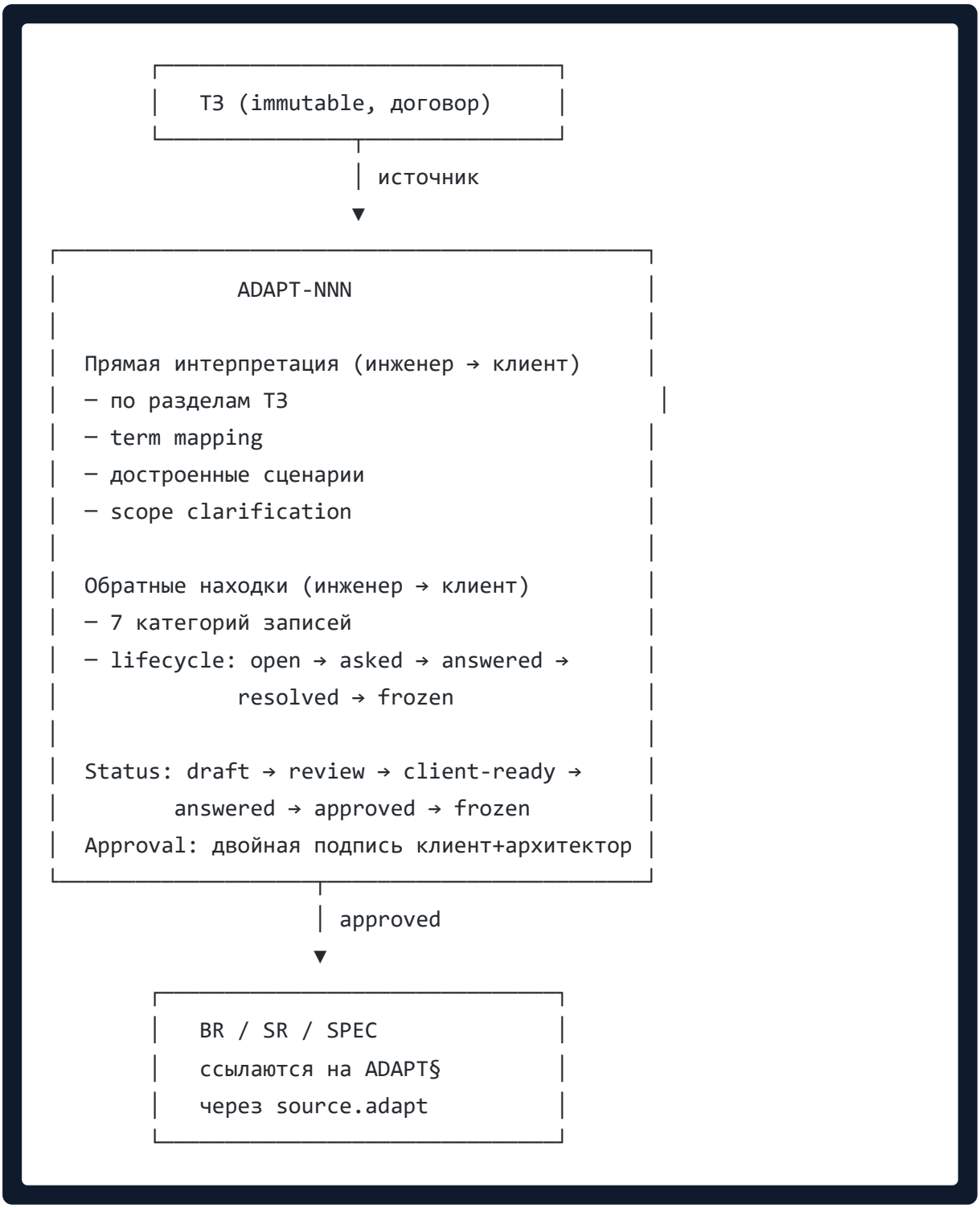
7.2 Проблема, которую нормирует ADAPT

Без формализованного промежуточного артефакта между ТЗ и BR/SR возникает один из двух негативных исходов:

Исход	Что происходит
Drift ТЗ	ТЗ редактируется после подписания → нарушается договор
Скрытая интерпретация	Инженерные предположения молча просачиваются в BR/SR/SPEC → разрушается trace chain и provenance

ADAPT исключает оба исхода: ТЗ остаётся immutable договорным документом, **все** интерпретации, уточнения и обратные находки регистрируются в ADAPT, который сам становится immutable после двойной подписи (§7.5).

7.3 Цикл двусторонней адаптации



ТЗ — первоисточник; ADAPT — canonical источник интерпретации; BR/SR/SPEC ссылаются на ADAPT, а не на ТЗ напрямую.

7.4 Нормативные требования к ADAPT

7.4.1 Обязательность

Каждое ТЗ обязано иметь ровно один корневой ADAPT (`ADAPT-NNN`). При наличии delta-ТЗ — каждый delta-ТЗ обязан иметь свой delta-ADAPT (§7.6).

Производство BR / SR / SPEC из ТЗ без прохождения через approved ADAPT — нарушение стандарта. Hooks lifecycle (глава 10) обязаны блокировать создание BR / SR / SPEC, ссылающегося на ADAPT в статусе ниже `approved` .

7.4.2 Immutability ТЗ

ТЗ — договорной документ. После регистрации ТЗ в носителе его содержимое **не редактируется**. Если в ходе работы выясняется, что в ТЗ ошибка / пропуск / противоречие — это регистрируется как обратная находка в ADAPT (§7.4.4), клиент даёт ответ, ответ становится частью ADAPT. ТЗ остаётся immutable.

При большом количестве правок или при изменении scope — клиент подписывает delta-ТЗ как новый immutable документ (§7.6).

7.4.3 Прямая адаптация ТЗ

Раздел прямой интерпретации ADAPT для каждого раздела ТЗ обязан содержать:

Элемент	Обязательность	Цель
Точная ссылка на ТЗ§N.N	mandatory	provenance
Цитата из ТЗ (или paraphrase с явной пометкой)	mandatory	Контекст
Инженерная интерпретация раздела	mandatory	Перевод языка клиента → язык требований
Term mapping (клиент → инженер)	mandatory если применимо	Дезамбигуация терминов
Достроенные сценарии	mandatory если ТЗ их подразумевает	Явная фиксация implicit cases

Scope clarification (входит / не входит)	mandatory	Граница работ
Ссылки из прямой интерпретации на BR/SR/SPEC	auto-derived	Trace chain (§7.7)

7.4.4 Обратные находки и их категории

Раздел обратных находок ADAPT фиксирует обнаруженные проблемы по семи нормативным категориям. **Список категорий закрыт на v1.0**; добавление новых категорий — через formal change procedure стандарта (см. глава 13). Общая closed-list политика и master-индекс — §1.7.5.

ID	Категория	Что регистрируется
`contradiction`	Противоречие	Внутренние противоречия в ТЗ (§A vs §B)
`gap`	Пропуск	ТЗ молчит о том, без чего невозможна реализация
`hidden-assumption`	Скрытое предположение	Предположение инженера, которое может быть неверно
`feasibility`	Реализуемость	Технически нереализуемое или непропорционально дорогое требование
`regulatory`	Регуляторика	Требование, затрагивающее законодательство / compliance
`terminology`	Терминология	Неясный термин ТЗ с несколькими возможными значениями
`scope`	Скоп	Неясная граница работ

Каждая запись об обратной находке имеет **stable ID** (`B-NNN`), immutable после создания. ID не переиспользуется даже для отозванных записей (audit trail).

7.4.5 Жизненный цикл одной записи об обратной находке

```
open → asked-to-client → answered → resolved → frozen
              ↑                          |
              └──────── revised ─────────┘  (если ответ требует уточнения)
```

Статус	Что значит
`open`	Инженер записал; не отправлено клиенту
`asked-to-client`	Вопрос отправлен клиенту, ожидается ответ; зафиксирована дата
`answered`	Клиент ответил; ответ записан в документ с author + timestamp (возможность носителя V6)
`resolved`	Инженер интегрировал ответ в прямую интерпретацию
`revised`	Ответ клиента расплывчатый; повторный вопрос. Переход обратно в `asked-to-client`
`frozen`	После approval ADAPT — изменения невозможны

Approval ADAPT (§7.5) запрещён при наличии хотя бы одной записи об обратной находке в статусе `open` / `asked-to-client` / `answered` / `revised`. Все такие записи обязаны быть в `resolved` перед approval.

7.5 Утверждение ADAPT — двойная подпись

ADAPT переходит из статуса `answered` в `approved` только после **двойной подписи** (атомарная единица изменения, возможности носителя V2 + V3 из §3.3):

Подпись	Кто	Что подтверждает
Client signature	Клиент или представитель клиента с полномочиями	Прямая интерпретация T3 совпадает с тем, что заказчик имел в виду; ответы на вопросы по обратным находкам — финальные
Architect signature	Архитектор со стороны исполнителя	Все обратные находки отработаны (нет нерешённых); прямая интерпретация технически реализуема

Нативная для носителя реализация подписи — комбинация V3 (diff & review) + V6 (author + timestamp). Конкретный механизм (digital signature / approval workflow / двойная review-аттестация) выбирается реализацией и фиксируется в conformance manifest (§3.7).

После approval ADAPT **immutable** наравне с T3. Дальнейшие изменения — только через delta-ADAPT (§7.6) или errata (§7.6.3).

7.6 Delta-ТЗ и delta-ADAPT

7.6.1 Рабочий процесс

При появлении delta-ТЗ от клиента:

1. Клиент регистрирует delta-ТЗ в носителе как новый immutable документ (`TZ-YYYY-NNN-delta-N`).
2. Клиент подписывает delta-ТЗ (V6 author + timestamp).
3. Архитектор / AI-агент создаёт delta-ADAPT (`ADAPT-NNN-delta-N`) с frontmatter полем `parent-adapt: ADAPT-NNN` и `source-tz: TZ-YYYY-NNN-delta-N` .
4. Раздел прямой интерпретации delta-ADAPT покрывает только разделы delta-ТЗ (не дублирует основной ADAPT).
5. Раздел обратных находок delta-ADAPT фиксирует записи конкретно по delta-ТЗ.
6. Approval delta-ADAPT — двойная подпись (§7.5).
7. Из approved delta-ADAPT — дельта в BR / SR / SPEC (новые / обновлённые / deprecated).

7.6.2 Цепочка delta-ADAPT

```
ADAPT-001 (от TZ-YYYY-NNN main)
  └─ ADAPT-001-delta-1 (от TZ-YYYY-NNN-delta-1)
       └─ ADAPT-001-delta-2 (от TZ-YYYY-NNN-delta-2)
            └─ ADAPT-001-delta-3 (от TZ-YYYY-NNN-delta-3)
```

Цепочка строго последовательна: применение delta-ADAPT обязано идти по порядку (см. сквозную фиксацию версии V5 в §3.3.5). Перенумеровать или переставлять delta-ADAPT в цепочке запрещено.

7.6.3 Errata для уже approved ADAPT

Если через продолжительное время после approval обнаруживается, что в ADAPT-NNN неверна прямая интерпретация ТЗ или некорректно зафиксировано разрешение обратной находки — два допустимых исхода:

Исход	Артефакт

ТЗ содержит ambiguity, обнаруженную поздно	delta-ADAPT с новой записью об обратной находке и ответом клиента
Incorrectly interpreted (ошибка инженера)	errata-ADAPT-NNN-M как отдельный артефакт с подписью клиента (если меняет contractual outcome) или только архитектора (если cosmetic)

В обоих исходах **frozen ADAPT не редактируется**. Только добавление новых артефактов с явной типизированной связью.

7.7 Связь ADAPT с другими артефактами

7.7.1 BR / SR / SPEC ссылаются на ADAPT через `source.adapt`

```
# Frontmatter SR (пример)
source:
  adapt: ADAPT-001
  adapt-section: "Forward §3"   # прямая интерпретация; канонический идентификатор секции — Forward §*
  tz-section: "§3.4"            # для traceability; первоисточник остаётся ТЗ
```

Поле `source.adapt` — нормативно обязательное для BR, SR, SPEC. Поле `source.tz-section` — рекомендуемое (для двойной trace chain).

7.7.2 Полная trace chain

```
TC-NN  →  verifies SR-12  →  выведено из ADAPT-001 §4 (прямая интерпретация)
                                  |     |
                                  |     └─ resolves B-007 (was: contradiction,
                                  |            answered by client 2026-03-15)
```

```
                                    │
                                    └─ interprets TZ-YYYY-NNN §3.4
```

При audit от тест-кейса можно дойти до: (a) исходного раздела ТЗ, (b) прямой интерпретации этого раздела, (c) вопросов исполнителя по обратным находкам, (d) ответов клиента, (e) производных BR / SR / SPEC.

7.7.3 TR не ссылается на ADAPT напрямую

TR (задача) ссылается на SR / SPEC, а они уже ссылаются на ADAPT. Исполнитель задачи **не обращается к ADAPT напрямую** — все нужные интерпретации уже содержатся в SR / SPEC. Если исполнитель обнаружил неясность в SR — это сигнал либо для регистрации обратной находки в ADAPT (если корень в ТЗ), либо для уточнения SR (если корень в декомпозиции).

7.8 ADAPT schema

7.8.1 frontmatter (обязательные поля)

```yaml
---
id: ADAPT-NNN                          # immutable; NNN sequential per
project
title: "Адаптация ТЗ <name>"
type: ADAPT

source-tz:
  id: TZ-YYYY-NNN
  signed-date: "<ISO-date>"
  signed-by-client: "<name + role>"
  document-version-ref: "<нативный для носителя идентификатор
версии>"   # V5 pin (см. §3.3.5)

parent-adapt:                          # для delta-ADAPT
  id: ADAPT-NNN
  delta-tz: TZ-YYYY-NNN-delta-N

status: draft | review | client-ready | answered | approved | frozen
```

```
| obsolete
created: "<ISO-date>"
last-updated: "<ISO-date>"

approval:                              # обязательно для approved
  client-signature:
    signed-by: "<name>"
    role: "<role>"
    organization: "<client-org>"
    signed-at: "<ISO-datetime>"        # V6 timestamp
    signature-ref: "<нативная для носителя ссылка>"
  architect-signature:
    signed-by: "<name>"
    role: architect
    signed-at: "<ISO-datetime>"

generates-requirements: []             # auto-derived; BR/SR из этого
ADAPT
generates-specs: []                    # auto-derived; SPEC-* из этого
ADAPT
open-questions-count: integer          # auto-derived; обязательно 0
для approved
resolved-questions-count: integer

ai-provenance:                         # обязательно если ADAPT draft
был AI-сгенерирован
  generated-by: "<vendor>-<model>@<date>"
  prompt-template: "<template-path>@<version>"
  context-tokens: integer
  output-tokens: integer
  human-edits: boolean                 # обязательно true для approved
— клиент видел текст
---
```

7.8.2 Body structure (обязательные разделы)

Обязательные разделы тела ADAPT:

1. **Краткое содержание** — 3–5 параграфов для одностраничного чтения клиентом.
2. **Term mapping** — таблица «клиент → инженер».

3. **Прямая интерпретация (раздел Forward)** — раздел на каждый раздел ТЗ с обязательными элементами из §7.4.3.
4. **Обратные находки** — все записи с lifecycle из §7.4.5.
5. **Резюме по обратным находкам** — статистическая таблица по категориям и статусам.
6. **Таблица производных артефактов** — auto-derived список BR / SR / SPEC.
7. **История изменений ADAPT** — нативно для носителя, auto-generated.

Опциональные разделы — на усмотрение реализации, не нормированы.

7.9 Контрольные точки качества для ADAPT

ADAPT имеет dedicated state machine. Детали в главе 10 §10.4. Краткая сводка:

Gate	Предусловие	Постусловие
QG-ADAPT-draft	ADAPT создан; присутствует frontmatter	Прямая интерпретация охватывает все разделы ТЗ
QG-ADAPT-review	Прямая интерпретация заполнена; первичные обратные находки в `open`	Все обратные находки в `open` или `asked-to-client`
QG-ADAPT-client-ready	Все обратные находки в `asked-to-client`; пакет вопросов сформирован	Готов к отправке клиенту
QG-ADAPT-answered	Все обратные находки в `answered`; начато разрешение	Готов к финализации
QG-ADAPT-approve	Все обратные находки в `resolved`; двойная подпись готова	ADAPT immutable; разрешена генерация BR / SR / SPEC
QG-ADAPT-frozen	`approved`	Дальнейшие изменения — только через delta-ADAPT или errata

Hooks носителя (§3.3.3 V3, §3.3.5 V5) обязаны:

- Блокировать переход в `approved` при `open-questions-count > 0`.
- Блокировать создание BR / SR / SPEC с `source.adapt` на ADAPT в статусе ниже `approved`.

- Пересчитывать `open-questions-count` / `resolved-questions-count` после каждого изменения.

7.10 ADAPT и AI-генерация

7.10.1 AI-агент создаёт draft ADAPT

При импорте ТЗ AI-агент создаёт **draft ADAPT** автоматически: прямая интерпретация по разделам, попытка обнаружить contradictions / gaps / terminology ambiguities, первая версия term mapping. Этот draft — стартовая точка для архитектора, не финальный артефакт.

7.10.2 Adversarial reviewer

Применение **принципа adversarial review** (отдельный AI-агент-критик с **другой моделью**; процедура — guide/07 §4.5): ищет, что primary агент пропустил — недостающие обратные находки. Если adversarial reviewer находит не менее трёх серьёзных новых находок — primary draft возвращается на доработку.

7.10.3 Клиент не общается с AI напрямую

Вопросы по обратным находкам агрегируются архитектором в человеческий формат **перед отправкой клиенту**. Архитектор может убрать дубликаты, переформулировать на язык клиента, объединить связанные. Клиент видит подготовленный список вопросов, не raw output AI-агента. Ответ клиента (в любой форме: текст, видеозвонок, письмо) транскрибируется архитектором в ADAPT с указанием канала и аутентификации.

7.11 Схема хранения

ADAPT-документы хранятся в подпапке `adapt/` носителя требований:

```
[project]/
  adapt/
    ADAPT-001-main.md
    ADAPT-001-delta-1.md
    ADAPT-001-delta-2.md
    ADAPT-002-main.md
```

```
    errata/
      errata-ADAPT-001-1.md
  tz/
    TZ-YYYY-NNN.md
    TZ-YYYY-NNN-delta-1.md
```

Конкретная файловая структура на конкретном носителе специфична для носителя (см. guide/03 для distributed VCS; guide/04 для document-oriented store).

7.12 Связь с другими главами

Глава	Связь
02 Methodology positioning	ADAPT — следствие Утверждения 2 (двусторонняя адаптация вместо «переброса спецификации через забор»)
06 Requirements hierarchy	BR / SR ссылаются на ADAPT через `source.adapt`
08 Specifications	SPEC-* также ссылаются на ADAPT через `source.adapt`
09 Test cases	TC через SR / SPEC возвращается в ADAPT для полной trace chain
10 Lifecycle и QG	ADAPT state machine + QG-ADAPT-* gates
03 Версионирование носителя	Двойная подпись (V6) + atomic approval (V2 + V3); immutability после approved (V1); delta-ADAPT через V4
13 Conformance	ADAPT для каждого T3 — mandatory clause

---

title: "Спецификации (9 типов SPEC)" order: 8 lang: ru

08. Спецификации — 9 типов SPEC

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

8.1 Зачем отдельная ось спецификаций

Требования (глава 6) говорят, **что** система делает. Но как она при этом устроена — какие у неё API, схемы данных, экраны, модель безопасности — через требования не выразить, не превратив их в кашу. Поэтому RENAR разводит описание на две оси: **поведение** (BR / SR / TR) и **структуру** (спецификации, SPEC). Спецификация — не «более детальный SR» и не его ребёнок: один SR обычно опирается сразу на пять-семь спецификаций, так что связь между осями — граф типизированных рёбер (`constrained-by[]` , `implements-spec[]`), а не дерево.

Типов спецификаций ровно девять, и список закрыт: `SPEC-ARCH` , `API` , `DATA` , `INT` , `PROC` , `UI` , `AI` , `SEC` , `OPS` . Это нормативное требование v1.0 — новый тип можно ввести только через формальную процедуру изменения стандарта (глава 13).

Дальше глава обосновывает разделение на две оси (§8.2), перечисляет девять типов (§8.3), задаёт общую и типовые схемы (§8.4–§8.6), описывает миграцию унаследованного (§8.7) и гейты с хранением (§8.8–§8.10).

8.2 Архитектурное решение: SPEC — параллельная ось, не дети SR

8.2.1 Две оси описания системы

Требования и спецификации отвечают на разные вопросы:

Ось	Артефакты	Вопрос
Поведенческая	BR / SR / TR (глава 6)	Что система должна делать

Структурная	SPEC-* (9 типов)	Как система структурно устроена для выполнения этих требований

8.2.2 SPEC как параллельная ось: связи через типизированный граф

Связи между осью требований (BR / SR / TR) и осью SPEC организованы как **граф зависимостей**, не дерево родителей. У SR ровно один родитель в дереве требований (BR), но множество типизированных рёбер `constrained-by[]` на SPEC. Один SR обязан ссылаться на каждую SPEC, ограничивающую его поведение в осях API / data / UI / process / security / ops; обратно — одна SPEC может ограничивать множество SR.

Пример: SR «создать заказ» опирается на SPEC-ARCH (где живёт компонент заказов), SPEC-API (контракт endpoint), SPEC-DATA (схема таблицы), SPEC-PROC (workflow), SPEC-SEC (правила доступа), SPEC-UI (форма).

Нормативно: каждое ребро `SR.constrained-by[]` и `TR.implements-spec[]` **должно** ссылаться на одну из закрытых категорий SPEC, перечисленных в §8.3; ad-hoc категории не допускаются (см. §1.7 closed-list policy).

```
Дерево требований (поведенческая ось):     Параллельная ось
спецификаций:

BR                                           SPEC-ARCH    SPEC-API
 └── SR  ←──── constrained-by[] ────►        SPEC-DATA    SPEC-INT
      └── TR ─── implements-spec[] ────►     SPEC-PROC    SPEC-UI
                                             SPEC-AI      SPEC-SEC
                                             SPEC-OPS


Дерево требований:
  SR.parent              → BR              (единственный родитель)
  TR.parent              → SR              (единственный родитель)

Граф связей (типизированные рёбра):
  SR.constrained-by[]    → SPEC-*
  TR.implements-spec[]   → SPEC-*
  SPEC-*.depends-on[]    → SPEC-*          (между спецификациями)
  SPEC-*.referenced-by[] → SR / TR          (auto-derived inverse)
```

8.2.3 Обоснование

Аргумент	Следствие
SPEC и SR отвечают на разные вопросы	SPEC не уточняет SR на более глубоком уровне — это отдельная категория описания
Один SR опирается на 5–7 SPEC	Дерево «SPEC как родитель SR» приводит к множественному родительству
Industry standards (arc42, C4, OpenAPI, BPMN, ERD) живут параллельно требованиям	RENAR следует этой проверенной практике
AI-агент может параллелить генерацию SR и SPEC	Без блокировки одного типа другим

8.3 Закрытый список из девяти типов SPEC

Тип	Назначение	Industry reference
`SPEC-ARCH`	Архитектура системы / подсистемы: контексты, контейнеры, компоненты, deployment view, quality attributes	arc42, C4 model (Brown), ISO/IEC/IEEE 42010
`SPEC-API`	API contracts: REST / GraphQL / gRPC / async events; версионирование, error model, rate limits	OpenAPI 3.x, AsyncAPI 2.x, gRPC IDL
`SPEC-DATA`	Data model: schema, ERD, indices, миграции, retention, PII classification	ISO/IEC 11179, JSON Schema
`SPEC-INT`	Integration: взаимодействие между подсистемами и внешними системами; протоколы, контракты, SLA	Enterprise Integration Patterns (Hohpe)
`SPEC-PROC`	Process / workflow: бизнес-процессы, state machines, saga, choreography, orchestration	BPMN 2.0, ISO/IEC 19510
`SPEC-UI`	UI / UX: экраны, навигация, user journeys, accessibility, i18n, baseline images	Material Design / Apple HIG, WCAG 2.2
`SPEC-AI`	AI / ML: model cards, RAG, prompt engineering, eval strategy, cost budget	ISO/IEC 23894, NIST AI RMF
`SPEC-SEC`	Security: authn / authz, threat model, secrets management, data classification	STRIDE, OWASP ASVS, ISO/IEC 27001

`SPEC-OPS`	Operations: deployment, observability, SLO / SLA, runbook, disaster recovery	Google SRE, ITIL v4, ISO/IEC 20000

8.3.1 Что НЕ вошло в v1.0 (с обоснованием)

Кандидат	Решение	Обоснование
`SPEC-EVENT`	Не отдельный тип	События / очереди — раздел SPEC-API (asynchronous APIs)
`SPEC-CONFIG`	Не отдельный тип	Feature flags / env vars / secrets — раздел SPEC-OPS
`SPEC-PERF`	Не отдельный тип	Performance / NFR — раздел SPEC-ARCH (quality attributes) или SPEC-OPS (SLO)
`SPEC-TEST-ENV`	Не отдельный тип	Тестовые окружения — раздел SPEC-OPS
`SPEC-DOMAIN`	Не отдельный тип	Domain model — поглощён SPEC-ARCH (decomposition) + SPEC-DATA (entities)
`SPEC-MIGRATION`	Не отдельный тип	Migration — раздел SPEC-DATA (lifecycle)
`SPEC-COMPLIANCE`	Не отдельный тип	Compliance — связи между SR/SPEC и нормативами через `compliance-refs[]`, не отдельный артефакт

Closed list policy: если в дальнейшей работе обнаружится, что какой-то из исключённых типов реально нужен — он добавляется через formal change procedure стандарта с обоснованием.

8.4 Общая схема (общие поля frontmatter)

Все 9 типов SPEC делят общий набор frontmatter полей. Type-specific поля добавляются как extensions поверх (§8.5). Полная machine-readable schema — в reference/02-schemas.md.

```
---
# === Identity (mandatory) ===
id: SPEC-<TYPE>-NN[.N]              # immutable; TYPE ∈
{ARCH,API,DATA,INT,PROC,UI,AI,SEC,OPS}
```

```
title: "<short, descriptive>"
type: SPEC-ARCH | SPEC-API | SPEC-DATA | SPEC-INT | SPEC-PROC | SPEC-UI | SPEC-AI | SPEC-SEC | SPEC-OPS
slug: "<kebab-case>"                # auto-derived

# === Scope (mandatory) ===
level: system | subsystem | module
scope:
  system: "<system-id>"
  subsystem: "<subsystem-id>"       # null если level=system

# === Lifecycle (mandatory) ===
status: draft | review | approved | verified | obsolete
priority: must | should | could     # not all types use; mostly SPEC-SEC / SPEC-OPS

# === Source: provenance from ADAPT (mandatory) ===
source:
  adapt: ADAPT-NNN                  # link to ADAPT artifact (см. глава 7)
  adapt-section: "Forward §N"       # прямая интерпретация; канонический идентификатор секции — Forward §*
  tz-section: "§N.N"                # optional, для traceability

# === Граф связей (auto-managed except mandatory ones) ===
referenced-by: []                   # auto-derived; SR/TR/SPEC ссылающиеся сюда
depends-on: []                      # mandatory если есть; SPEC-* на которые опирается этот SPEC
verified-by: []                     # auto-derived; список TC IDs верифицирующих

# === AI provenance (mandatory на RENAR-4+; canonical schema — §4.10.1) ===
ai-provenance:
  generated-by: "<vendor>-<model>@<date>"
  generated-at: "<ISO-8601>"
  prompt-template: "<template-path>@<version>"
  context-tokens: integer
  output-tokens: integer
  human-edits: boolean
  generation-time-ms: integer       # optional; см. §4.10.1
```

```
  # optional на RENAR-4, mandatory на RENAR-5:
  # cost-budget, cost-actual

# === Замена (mandatory если применимо) ===
replaces: "<old-id>"
replaced-by: "<new-id>"
deprecated-date: "<ISO date>"

# === Compliance (optional) ===
compliance-refs: []                # ссылки на ISO/GDPR/AI Act/NIST
AI RMF
---
```

8.4.1 Обязательные разделы body

Body любого SPEC обязательно содержит:

1. **Назначение** — 1–3 параграфа.
2. **Scope** — что входит, что не входит.
3. **Разделы, специфичные для типа** — см. §8.5.
4. **Связь с требованиями** — какие SR/BR ссылаются.
5. **Связь с другими SPEC** — `depends-on[]`.
6. **Verification** — какие TC верифицируют этот SPEC.

8.5 Расширения схемы по типам SPEC

Краткое описание type-specific полей и обязательных body-разделов. Полная machine-readable extension schema — в reference/02-schemas.md. Industry references детально — в указанных стандартах.

8.5.1 SPEC-ARCH

Type-specific frontmatter: `arch-style`, `deployment-model`, `tech-stack`, `quality-attributes`.

Mandatory body: системный контекст (C4 L1), контейнеры (C4 L2), компоненты (C4 L3) для критических контейнеров, quality attributes (latency / throughput / availability), ADR-журнал.

Spec-specific TC (глава 9): architecture conformance tests (zoning), quality attribute baseline tests.

8.5.2 SPEC-API

Type-specific frontmatter: `api-style` (rest / graphql / grpc / async-events), `api-version`, `versioning-strategy`, `authentication`, `rate-limits`, `contract-file` (location of machine-readable contract).

Mandatory body: endpoints / operations с payload / response / errors, versioning rules (breaking vs non-breaking), error model, authn/authz reference на SPEC-SEC, rate limits, 2–3 example запросов на endpoint.

Spec-specific TC: contract tests, authentication negative, rate limit tests.

8.5.3 SPEC-DATA

Type-specific frontmatter: `data-style` (relational / document / graph / columnar), `storage-engine`, `schema-version`, `pii-classification[]`, `retention-policies[]`, `migration-strategy`.

Mandatory body: domain entities, ERD (text / Mermaid / link), поля сущностей (type / constraints / indices / defaults), связи (FK / cardinality / cascade), PII / sensitive data classification + encryption at-rest + retention, migration approach, index strategy.

Spec-specific TC: migration tests, constraint tests (FK / NOT NULL / unique), PII handling tests, data retention tests.

8.5.4 SPEC-INT

Type-specific frontmatter: `integration-pattern` (request-response / event-driven / message-queue / webhook / file-transfer), `direction`, `counterparty`, `sla`, `idempotency`.

Mandatory body: интегрируемые системы, контракт обмена, failure modes + retry strategy, идемпотентность + dedup, security между системами, observability (correlation IDs).

Spec-specific TC: contract tests с моком counterparty, failure injection, idempotency, end-to-end TC `tc-type: contract`.

Замечание: SPEC-INT заменяет существующий `INT-SR` (§8.7 migration).

8.5.5 SPEC-PROC

Type-specific frontmatter: `process-style` (bpmn / state-machine / saga / choreography / orchestration), `state-count`, `participants[]`, `sla` end-to-end и по шагам, `compensation` (defined / not-applicable / manual).

Mandatory body: process diagram (BPMN-flavor / Mermaid / link), состояния и переходы (для state machine), participants и их роли, happy path, альтернативные сценарии и исключения, timeouts и compensation (для saga), SLA.

Spec-specific TC: happy path E2E, alternative paths, compensation tests (для saga), SLA tests.

8.5.6 SPEC-UI

Type-specific frontmatter: `ui-platform`, `target-users[]` (с ссылками на persona разделы ADAPT), `design-system`, `accessibility-level` (WCAG-A / AA / AAA), `i18n`, `mockup-links[]`, `baseline-images[]` для VLM-judge тестов.

Mandatory body: общая структура интерфейса, ключевые экраны, user journeys без технических деталей, сквозные элементы (права доступа / уведомления / error / empty states), тон и стиль, accessibility, i18n.

Spec-specific TC: VLM-judge с baseline (judge ≠ production изоляция), accessibility (axe-core / Pa11y), i18n (string overflow / RTL), user journey E2E.

Замечание: SPEC-UI заменяет существующий `UIC` (§8.7 migration).

8.5.7 SPEC-AI

Type-specific frontmatter: `ai-pattern` (rag / fine-tuning / prompt-engineering / tool-use / multi-agent), `production-model` (vendor / model / version), `judge-model` (обязан отличаться от production), `context-strategy`, `eval-strategy` (metric / threshold / baseline-dataset), `cost-budget`.

Mandatory body: архитектура AI-компонент (pipeline / orchestration / fallback), model card (capabilities / limits / known failure modes), context strategy, eval strategy с изоляцией judge ≠ production, cost management, hallucination mitigation, adversarial considerations.

Spec-specific TC: eval против baseline (judge isolated), adversarial (prompt injection как negative TC), cost regression, hallucination tests.

Замечание: SPEC-AI заменяет существующий `AIC`. Изоляция judge ≠ production model — обязательное требование стандарта для всех eval-TC.

8.5.8 SPEC-SEC

Type-specific frontmatter: `security-domains[]`, `auth-model` (authn / authz strategies), `data-classification[]` (PII-high / PCI / internal), `threat-model-method` (STRIDE / PASTA / OCTAVE), `compliance[]`, `incident-response` reference в SPEC-OPS.

Mandatory body: auth model (authn flow / authz rules), data classification с защитой, threat model (STRIDE-таблица с mitigation на каждую угрозу), secrets management, audit (что логируется / retention / доступ), encryption (at-rest / in-transit / key management), compliance mapping (ссылки на конкретные пункты).

Spec-specific TC: authn (pos + neg), authz (RBAC matrix), threat-test (каждая STRIDE-угроза → минимум 1 negative TC), audit log, secrets leakage.

8.5.9 SPEC-OPS

Type-specific frontmatter: `deployment-style`, `environments[]` (dev / staging / prod с purpose и scale), `slo`, `observability` (logs / metrics / traces / alerting), `runbook-link`, `disaster-recovery` (rto / rpo / backup-strategy).

Mandatory body: environments, deployment process (CI/CD pipeline / gating / rollout strategy), SLO (availability / latency / error budget), observability, alerting (критические алерты / escalation), runbook, capacity planning, disaster recovery.

Spec-specific TC: deployment tests (smoke), SLO regression (load testing), failover (DR drills), observability (alerts срабатывают когда ожидается).

8.6 Связь с требованиями и задачами

8.6.1 SR.constrained-by[]

SR в frontmatter получает поле `constrained-by[]` — типизированные ссылки на SPEC. Это **граф**, не дерево родителей. Родитель SR в дереве требований — единственный (BR).

```yaml
# Frontmatter SR (пример)
id: SR-05
parent:
  id: BR-02
constrained-by:
  - SPEC-UI-01
  - SPEC-API-02
  - SPEC-DATA-03
  - SPEC-PROC-01
  - SPEC-SEC-01
verified-by:
  - TC-12
  - TC-13
source:
  adapt: ADAPT-001
  adapt-section: "Forward §3"       # см. канонический идентификатор §8.4
```

8.6.2 TR.implements-spec[]

TR (задача) ссылается на SR (родитель в дереве) + одна или более SPEC через `implements-spec[]`:

```yaml
id: TR-42
title: "Реализовать endpoint POST /orders"
parent:
  id: SR-05
implements-spec:
  - SPEC-API-02
  - SPEC-DATA-03
verified-by:
  - TC-14
```

8.6.3 SPEC.depends-on[]

SPEC может опираться на другой SPEC:

```yaml
id: SPEC-API-02
title: "REST API заказов"
type: SPEC-API
depends-on:
  - SPEC-DATA-03          # стабильная схема данных
  - SPEC-SEC-01           # auth model для endpoints
```

При изменении upstream SPEC (например SPEC-DATA-03) все downstream (SPEC-API-02 и через него связанные SR) обязаны быть пересмотрены: либо `verified` остаётся в силе, либо downstream артефакт получает `obsolete-pending` статус до пересмотра.

8.6.4 Auto-derived обратные рёбра

`SPEC.referenced-by[]` пересчитывается хуком носителя после каждого изменения SR / TR / SPEC. Orphan SPEC (без `referenced-by[]` и без активного status) — предупреждение в отчёте качества.

8.7 Миграция UIC / AIC / INT-SR / TS → SPEC-*

8.7.1 Mapping таблица

Старый тип	Новый тип	Тип миграции
`UIC-NN`	`SPEC-UI-NN`	Переименование ID + перенос в `specs/ui/`
`AIC-NN`	`SPEC-AI-NN`	Переименование ID + перенос в `specs/ai/`
`INT-SR-NN`	`SPEC-INT-NN`	Переименование ID + перенос в `specs/int/`
`TS-NN`	`SPEC-<TYPE>-NN` (распределение)	Manual review каждого TS; AI-агент классифицирует body, архитектор approve через one-click

8.7.2 Атомарная миграция

Миграция — одна atomic change unit (V2) на уровне проекта. Параллельное существование старых типов (UIC / AIC / INT-SR / TS) и SPEC-* как SoT запрещено.

Процедура (независимо от носителя):

1. Подготовка: AI-агент классифицирует каждый существующий TS-NN в один из 9 типов SPEC.
2. Архитектор approve классификации.
3. Atomic change: переименование IDs (UIC→SPEC-UI; AIC→SPEC-AI; INT-SR→SPEC-INT; TS→SPEC-*), перенос файлов в `specs/<type>/`, обновление всех ссылок в BR / SR / TR / TC frontmatter (`parent: UIC-NN` → `constrained-by: [SPEC-UI-NN]`).
4. Регенерация auto-derived файлов (REQUIREMENTS.md, SPECS.md, обратные рёбра).
5. CI-проверка: отсутствие orphan ссылок и старых IDs.

8.7.3 ID immutability

После миграции SPEC IDs **immutable** (см. V1, §3.3.1). Переименование `SPEC-API-02` → `SPEC-API-08` запрещено. Замена — через `deprecated` + новый ID с `replaces[]`.

8.8 Контрольные точки качества для SPEC

SPEC имеет dedicated state machine (глава 10 §10.3):

State	Условие перехода
`draft`	Создан; обязательные поля frontmatter заполняются
`review`	Готов к рецензированию; обязательные body-разделы (§8.4.1) и type-specific (§8.5) присутствуют
`approved`	Архитектор подтвердил; `depends-on[]` консистенция проверена
`verified`	Все обязательные spec-specific TC (глава 9 §9.7) зелёные
`obsolete`	Заменён или больше не актуален; `replaced-by` обязателен

Связь с QG-0 / QG-2 SENAR:

- QG-0 (есть goal/AC у задачи) расширяется: для задач реализующих SPEC обязательны `implements-spec[]` в TR frontmatter.
- QG-2 (есть evidence у `done`) расширяется: для задач реализующих SPEC обязательны TC соответствующего spec-specific вида (глава 9 §9.7).

8.9 Схема хранения

8.9.1 На уровне системы

```
[requirements-substrate]/      # корень носителя требований (layout — guide/03 или guide/04)
  br/
  sr/
  specs/
    arch/   SPEC-ARCH-NN-*.md
    api/    SPEC-API-NN-*.md
    data/   SPEC-DATA-NN-*.md
    ui/     SPEC-UI-NN-*.md
    ai/     SPEC-AI-NN-*.md
    int/    SPEC-INT-NN-*.md
    proc/   SPEC-PROC-NN-*.md
    sec/    SPEC-SEC-NN-*.md
    ops/    SPEC-OPS-NN-*.md
  adapt/
  tz/
  SPECS.md                # auto-generated index
```

> *Все 9 типов SPEC (§8.3) допустимы на любом `level`; подпапки `specs/<type>/` создаются по мере необходимости — не все обязательны на уровне системы.*

8.9.2 На уровне подсистемы

```
[subsystem-substrate]/         # scope подсистемы
  br/                     # если своя бизнес-сторона
```

```
sr/
specs/
  arch/   SPEC-ARCH-NN-*.md      # архитектура подсистемы
  api/    SPEC-API-NN-*.md
  data/   SPEC-DATA-NN-*.md
  ui/     SPEC-UI-NN-*.md
  ai/     SPEC-AI-NN-*.md
  int/    SPEC-INT-NN-*.md
  proc/   SPEC-PROC-NN-*.md
  sec/    SPEC-SEC-NN-*.md
  ops/    SPEC-OPS-NN-*.md
adapt/
SPECS.md
```

Нативная для носителя реализация хранения специфична для носителя (см. guide/03, guide/04).

8.9.3 SPECS.md — auto-generated index

`SPECS.md` — auto-generated реестр всех SPEC: ID, тип, заголовок, статус, ссылка на верифицируемое требование, ссылка на файл. Помечается `linguist-generated=true`. Триггеры перегенерации — каждое изменение SPEC frontmatter или каждый approve / verify gate.

8.10 Связь с другими главами

Глава	Связь
02 Methodology positioning	SPEC как параллельная ось — следствие SoT inversion
06 Requirements hierarchy	`SR.constrained-by[]`, `TR.implements-spec[]`
07 ADAPT	SPEC ссылается на ADAPT через `source.adapt`
09 Test cases	Spec-specific TC types (таблица обязательных видов TC для каждого типа SPEC)
10 Lifecycle и QG	SPEC state machine + QG расширения для SPEC
03 Версионирование носителя	SPEC IDs immutable (V1); migration атомарно (V2)

11 Maturity model	RENAR-3+: все 9 типов SPEC где применимо
reference/02 — schemas	Полная machine-readable schema для каждого type-specific extension
reference/05 — knowledge graph schema	`constrained-by[]`, `implements-spec[]`, `depends-on[]` как edge types в графе

---

title: "Тест-кейсы (TC)" order: 9 lang: ru

09. Тест-кейсы

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

9.1 Тест-кейс — полноценный артефакт

Глава нормирует TC (Test Case) как **полноценный** артефакт стандарта: по жизненному циклу и версионированию он наравне с требованием. TC верифицирует требования (глава 6: BR / SR / TR) и спецификации (глава 8: SPEC-ARCH / API / DATA / INT / PROC / UI / AI / SEC / OPS), замыкая trace chain: T3 → ADAPT → BR / SR / SPEC → TR → TC (см. §2.3).

Структура главы: принципы TC (§9.2), общая схема frontmatter и body (§9.3–9.4), типы и расширения (§9.5–9.6), парность pos/neg и обязательные виды по SPEC (§9.7–9.8), машина состояний и контрольные точки качества (§9.9–9.10), критерии и защита от подгонки (§9.11–9.13), спот-чек и матрица покрытия (§9.14–9.15), влияние дельты T3 и схема хранения (§9.16–9.17).

Глава фиксирует:

- закрытый список нормативных принципов TC (§9.2);
- общую схему TC (§9.3) и расширения по типам (§9.6);
- нормативное требование pos/neg pairing (§9.7);
- закрытую таблицу «тип SPEC → обязательные виды TC» (§9.8);
- защиту от подгонки тестов через тег `[test-spec-change]` (§9.13);
- спот-чек инженера (§9.14) — единственная регулярная точка ручного контроля качества TC.

Глава опирается на ISO/IEC/IEEE 29119 «Software testing» в части концепций test design, test execution, test result reporting и pos/neg coverage, но фиксирует closed list типов TC, mandatory pos/neg pairing и judge ≠ production isolation как нормативные требования v1.0, не присутствующие в ISO 29119 в формализованном виде.

От практик Specification by Example (Adzic) и BDD / Gherkin — где исполняемые примеры также служат спецификацией — RENAR отличается тем, что переводит pos/neg-парность (§9.7), judge ≠ production isolation (§9.13) и version-pin TC к версии требования (V5, §3.3.5) из рекомендации в блокирующие нормативные clauses (§14.5.2).

Положения главы являются нормативными. Закрытые списки (принципы, типы TC, обязательные виды TC по типу SPEC) — mandatory clauses RENAR conformance (глава 13);

расширение — только через formal change procedure стандарта.

> ***Плотная глава:*** *reference/09 · decision tree ниже — informative routing.*

9.1.1 Дерево решений: выбор `tc-type` (informative)

```
flowchart TD
  A[Новый TC для SR/SPEC] --> B{SPEC type?}
  B -->|SPEC-UI| C[tc-type: ux]
  B -->|SPEC-AI| D[tc-type: eval + adversarial negative]
  B -->|SPEC-API / INT / DATA| E[tc-type: contract]
  B -->|SPEC-SEC| F[tc-type: security — negative only]
  B -->|SPEC-ARCH / PROC / OPS| G[tc-type: system]
  B -->|BR acceptance| H[tc-type: acceptance]
  C --> I{pos/neg pair?}
  D --> I
  E --> I
  F --> J[security: negative mandatory; positive via system TC]
  G --> I
  H --> I
  I -->|SR normative claim| K[§9.7: pos + neg required]
  I -->|negative invariant only| L[§9.7 exception: single TC OK]
```

Процедура adversarial review TC — guide/07 §4.5; agent panel (informative) — там же §4.5.

9.2 Закрытый список нормативных принципов TC

#	Принцип	Нормативная формулировка
P1	Полноценный артефакт (TC)	TC — самостоятельный артефакт стандарта, по жизненному циклу и версионированию равный требованию. Хранится отдельным файлом в подпапке `tests/` носителя требований (§9.17).
P2	Документ ≠ реализация	TC описывает **что и как проверяется** (decoupled от implementation). Реализация (код) адресуется полем `automation.location` и хранится в носителе кода. Один TC — одна реализация.

P3	AI-generated	TC создаются и редактируются AI-агентом по заданию инженера; инженер не пишет TC вручную (см. глава 11 §11.N).
P4	AI-executed	Все TC в статусе `ready` и выше — автоматизированы. Прогон выполняет автоматический runner (CI / AI-runner / specialized executor). Результаты в `last-run` записывает только runner (bot-managed актор) по факту прогона.
P5	Pos/neg pairing	На каждое утверждение требования (BR / SR / SPEC) создаётся минимум одна пара positive + negative TC (§9.7).
P6	`last-run` — bot-managed	Поле `last-run` (date / result / runner-id / requirement-version / judge-report) заполняет только автоматический runner. Ручное редактирование `last-run` любым актором запрещено стандартом (§9.12).
P7	Judge ≠ production isolation	Для типов TC, использующих LLM-as-judge (ux, eval), judge-модель **не должна совпадать** с production-моделью, генерирующей оцениваемый артефакт. Совпадение блокирует hook носителя (§9.6.2).

Список закрыт. Новые принципы добавляются только через formal change procedure (глава 13).

9.3 Общая схема TC (frontmatter)

Все типы TC делят общий набор frontmatter-полей. Type-specific поля добавляются как extensions поверх (§9.6). Полная machine-readable schema — в reference/02-schemas.md.

```
---
# === Identity (mandatory) ===
id: TC-NN                              # immutable; NN sequential в
рамках scope
title: "<short, descriptive>"
type: TC
slug: "<kebab-case>"                   # auto-derived

# === Classification (mandatory) ===
tc-type: acceptance | ux | system | contract | eval | security
negative: boolean                      # true для парного негативного
TC
```

```yaml
# === Scope (mandatory) ===
level: system | subsystem | module
scope:
  system: "<system-id>"
  subsystem: "<subsystem-id>"        # null если level=system
  module: "<module-id>"              # null если level ≠ module

# === Lifecycle (mandatory) ===
status: draft | ready | passing | failing | obsolete

# === Verification target (mandatory; хотя бы один из) ===
verifies:
  - id: SR-NN | BR-NN | SPEC-<TYPE>-NN
    requirement-version: "<нативный для носителя version-ref>"   # V5
pinning (см. глава 3)
  - id: ...

# === Pair link (mandatory если negative=false и существует парный)
===
paired-with:                         # ID парного TC (positive ↔
negative)
  - TC-NN

# === Automation (mandatory) ===
automation:
  status: automated | manual-pending
  location: "<нативный для носителя pointer to implementation>"  #
mandatory если automated
  manual-pending-until: "<ISO date>"                          #
mandatory если manual-pending
  manual-pending-reason: "<text>"                              #
mandatory если manual-pending

# === Execution (mandatory если type=ux | eval) ===
judge:
  vendor: "<provider>"               # mandatory; см. P7 isolation
  model: "<model-id>"
  prompt-template: "<template-path>@<version>"

baseline:                            # mandatory для ux | eval
  artifact: "<нативный для носителя pointer>"
  perceptual-diff-threshold: float   # для ux
```

```yaml
  metric-thresholds: {}              # для eval

# === Last run (auto-managed; bot-only) ===
last-run:
  date: "<ISO-datetime>"
  result: pass | fail | skipped | n/a
  runner-id: "<runner-name@version>"
  run-ref: "<нативная для носителя ссылка>"
  requirement-version: "<version-ref of verified artifact>"
  judge-report: "<inline or pointer>"

# === AI provenance (mandatory на RENAR-4+; canonical schema —
§4.10.1) ===
ai-provenance:
  generated-by: "<vendor>-<model>@<date>"
  generated-at: "<ISO-8601>"
  prompt-template: "<template-path>@<version>"
  context-tokens: integer
  output-tokens: integer
  human-edits: boolean
  # optional на RENAR-4, mandatory на RENAR-5 (см. §4.10.1):
  # cost-budget, cost-actual, generation-time-ms

# === Замена / obsolescence ===
obsolete-pending: boolean            # true при detected delta-T3
инвалидации
replaces: "<old-id>"
replaced-by: "<new-id>"
obsoleted-date: "<ISO date>"
---
```

`verifies[]` — закрытый список ссылок на верифицируемые артефакты (BR / SR / SPEC). TR прямо не указывается в `verifies` — TR верифицируется через свой parent SR (см. §6.7). `verifies[].requirement-version` — нативный для носителя pinning артефакта (V5 capability, см. глава 3 §3.3.5); QG-2 (§9.10) требует совпадения `verifies[].requirement-version` с текущей версией артефакта.

9.4 Body разделы TC

Body любого TC обязательно содержит следующие разделы (независимо от носителя):

Раздел	Обязательность	Содержание
Контекст	mandatory	На какой пункт верифицируемого артефакта ссылается TC; цитата или paraphrase утверждения.
Предусловия	mandatory	Состояние системы и данных, требуемое для прогона; обеспечивается seed-механизмом.
Шаги	mandatory	Действия runner; для `tc-type: ux` — намерения, не селекторы (см. §9.6.1).
Pass-критерий	mandatory	Бинарный, наблюдаемый, воспроизводимый (см. §9.11).
Fail-критерий	mandatory	Перечень наблюдаемых признаков нарушения (не отрицание Pass); включает утечки, side-effects, race conditions.
Постусловия	mandatory	Какое состояние ожидается после прогона; cleanup-механизм.
Out of scope	mandatory	Что **намеренно** не проверяется, с указанием парного TC, где это покрыто.
Связанные TC	optional	Ссылки на семантически связанные TC.

Раздел «Out of scope» — нормативно обязательный: защищает от ложного ощущения покрытия. Отсутствие раздела блокирует переход TC в `ready`.

Имена секций body — machine-detectable заголовки уровня `##`. Канонические идентификаторы секций критериев — `## Pass-критерий` и `## Fail-критерий`; именно их детектирует enforcement-хук change-of-criteria (§10.11.3), поэтому имена этих секций фиксированы и не подлежат локальной замене.

9.5 Закрытый список типов TC

```
tc-type ∈ { acceptance, ux, system, contract, eval, security }
```

Тип	Что проверяет	Применяется к	runner-семейство
`acceptance`	Достигнута ли бизнес-цель?	BR	E2E + AI-валидатор
`ux`	Соответствует ли UX заявленному опыту?	SPEC-UI	AI-driver + VLM-judge
`system`	Ведёт ли система себя как описано?	SR, SPEC-PROC, SPEC-ARCH	xUnit-семейство
`contract`	Соблюдён ли контракт?	SPEC-API, SPEC-INT, SPEC-DATA	Contract-testing framework
`eval`	Достигнуто ли качество AI-компонент?	SPEC-AI	Eval-runner с baseline dataset
`security`	Соблюдены ли security-инварианты?	SPEC-SEC	Authz/threat-test framework

Список закрыт. Новые типы добавляются только через formal change procedure стандарта (глава 13). Конкретные технологии runner специфичны для носителя и фиксируются в conformance manifest реализации.

9.6 Type-specific extensions

9.6.1 `tc-type: ux` — UX тесты на основе SPEC-UI

UX-тест нормативно построен как двухслойная структура:

Слой	Содержание	Исполнитель
Сценарий (намерение)	«<актор> хочет <результат> после <условие>»	AI-driver: переводит намерение в действия, находит элементы по семантике (без жёстких селекторов)
Перцептивная проверка	«На отрисованном состоянии видно <критерий>»	Perceptual judge (VLM): принимает рендер + критерий, возвращает pass/fail с обоснованием

Mandatory frontmatter extension: `judge.vendor`, `judge.model`, `baseline.artifact`, `baseline.perceptual-diff-threshold`.

Mandatory body sections дополнительно к §9.4: Сценарий (намерение, не селекторы); Perceptual критерий (что должен увидеть judge); Парный негативный (пустое состояние / ошибка / отсутствие прав).

Регрессия по визуалу. Дополнительно к VLM-judge — perceptual diff против `baseline.artifact` с порогом `perceptual-diff-threshold`. Превышение порога блокирует переход TC в `passing`.

Обновление baseline. Изменение `baseline.artifact` требует нативного для носителя approval-механизма с тегом `[baseline-update]` (см. §9.13); автоматическое обновление запрещено.

9.6.2 `tc-type: eval` — Eval-тесты на основе SPEC-AI

Eval-тест нормативно проверяет качество AI-компонент через dataset с метриками и порогами.

Mandatory frontmatter extension: `judge.vendor`, `judge.model`, `baseline.artifact` (versionable dataset), `baseline.metric-thresholds`.

Mandatory body sections: dataset provenance (как собран, какая разметка); метрика-кластер (одна eval-TC = одна семантически связная группа метрик; разные семейства — разные TC); regression rule (что считается провалом — выход за threshold или регрессия ≥ N% против baseline).

Judge ≠ production isolation (нормативно). Поле `judge.vendor` + `judge.model` обязано отличаться от `production-model` спецификации SPEC-AI, нормирующей оцениваемое поведение. Нативный для носителя hook (глава 3 §3.3.3) обязан блокировать merge change unit при совпадении.

Dataset versioning. Eval-dataset — управляемый носителем versionable artifact: каждое изменение фиксируется как atomic change unit с описанием (что добавлено / убрано / переразмечено) и authorship (генератор-агент / критик-агент / human spot-check).

Cost gating. Eval не запускается на каждое изменение реализации (cost): runner запускается при изменении SPEC-AI, production-model, или dataset, либо по расписанию. Триггеры фиксируются в нативной для носителя runner-configuration.

Двухступенчатая разметка dataset. Generator-agent создаёт кандидатов; critic-agent проверяет по чек-листу; engineer spot-check ≥ 10% случайных примеров до merge dataset.

9.6.3 `tc-type: contract` — Контрактные тесты на основе SPEC-API / SPEC-INT / SPEC-DATA

Mandatory body sections: machine-readable контракт (ссылка на OpenAPI / GraphQL SDL / Protobuf / JSON Schema из SPEC); сторона генератор / сторона потребитель; мок counterparty (для SPEC-INT — sandbox / реальная среда отдельным TC).

Mandatory extension для SPEC-INT. Контрактные TC обязательно сочетаются с интеграционным TC (`tc-type: contract` , `level: subsystem | system`) против реальной или sandbox-counterparty — мокированного контракта недостаточно для verify SPEC-INT.

9.6.4 `tc-type: security` — Security-тесты на основе SPEC-SEC

Mandatory body sections: атрибуты модели угроз (STRIDE-категория или эквивалент); subject под тестом (authn / authz / data classification / secrets / audit / encryption); negative scenarios (попытка обхода, неавторизованный доступ, leakage); ожидаемое поведение системы при нарушении.

Security-TC нормативно содержит **только negative scenarios** (попытка обхода защиты). Positive «дать корректный доступ корректному актору» покрывается `tc-type: system` со scope SPEC-SEC.

9.7 Pos/neg pairing — нормативное требование

На каждое утверждение требования (BR / SR / SPEC), описывающее наблюдаемое поведение, создаётся минимум одна пара positive + negative TC.

Положительный TC	Парный отрицательный TC
`negative: false`	`negative: true`
Описывает happy path / success behavior	Описывает граничные условия, нарушения, обходы
`paired-with: [TC-<neg-id>]`	`paired-with: [TC-<pos-id>]`

Negative TC нормативно описывает наблюдаемые признаки нарушения (что **не должно** произойти), а не отрицание Pass-критерия позитивного TC. Примеры:

Утверждение	Pos TC	Neg TC

«Аутентификация по email + password»	Корректные credentials → 200 + JWT	Неверный пароль → 401 без раскрытия, какое поле неверно; отсутствие записи в session-store; rate-limit после N попыток
«Создание заказа»	Валидный payload → 201 + order-id	Невалидный price < 0 → 422 с явной ошибкой; отсутствие записи в БД; отсутствие side-effect (уведомление, начисление)

QG-2 (§9.10) обязан блокировать promote артефакта в `verified`, если хотя бы одно нормативное утверждение покрыто только положительным TC.

Single-TC-coverage допускается **только** в одном случае: артефакт описывает запрет / negative invariant сам по себе (например, security-TC по STRIDE-категории — он негативный по природе).

9.8 Spec-specific TC — обязательные виды по типу SPEC

Закрытая нормативная таблица: каждый тип SPEC обязан иметь минимум по одному TC каждого «обязательного вида» перед переходом в `verified` (глава 8 §8.8).

Тип SPEC	Обязательные виды TC	Дополнительные виды TC
SPEC-ARCH	Conformance (zoning / dependency rules)	Quality attribute baseline (latency / throughput / availability)
SPEC-API	Contract (по контракту из `contract-file`)	Auth negative; rate-limit; versioning compatibility
SPEC-DATA	Constraint (FK / NOT NULL / unique); Migration (прямой проход + откат)	PII handling; retention; index regression
SPEC-INT	Contract (mocked counterparty); контрактный TC `tc-type: contract` (real / sandbox counterparty)	Failure injection; idempotency; observability (correlation IDs)
SPEC-PROC	Happy path E2E; Alternative paths E2E	Compensation (для saga); SLA end-to-end
SPEC-UI	VLM-judge с baseline (judge ≠ production); Accessibility (WCAG-AA минимум)	i18n (string overflow / RTL); Journey E2E

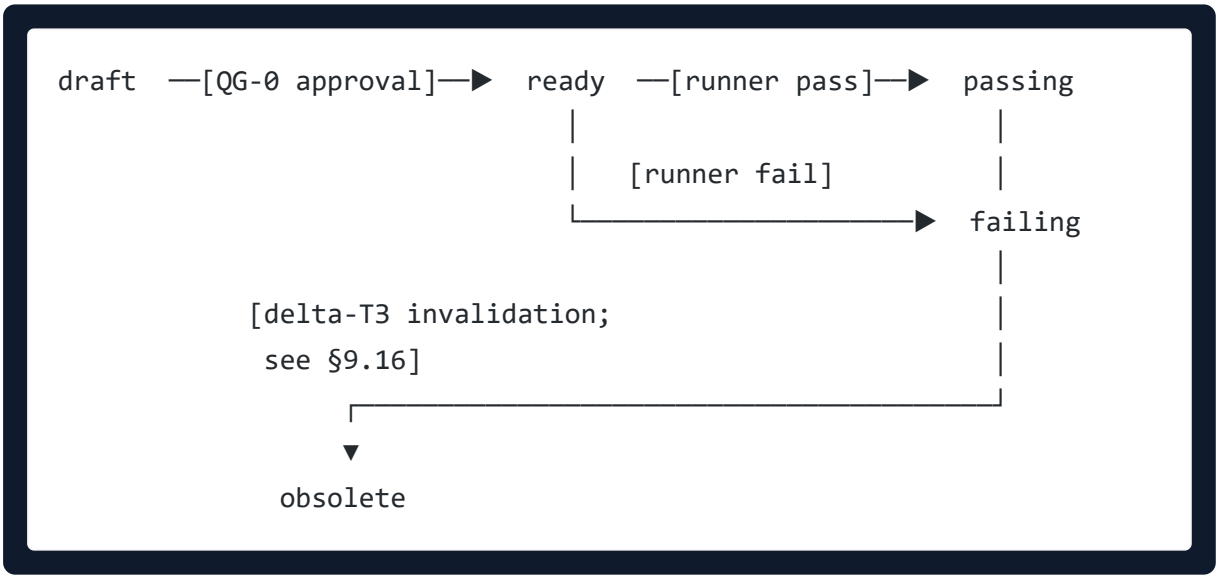
SPEC-AI	Eval против baseline dataset (judge isolated)	Adversarial (prompt injection как negative TC); Cost regression; Hallucination tests
SPEC-SEC	Authz / RBAC matrix; Threat-test per STRIDE-категории	Audit log; Secrets leakage; Encryption invariants
SPEC-OPS	Smoke after deploy; SLO regression (load test)	Failover / DR drill; Observability (alert firing correctness)

Нативный для носителя hook `promote SPEC → verified` (глава 3 §3.3.3) обязан проверять наличие минимум по одному TC каждого обязательного вида и блокировать переход при отсутствии.

Таблица закрыта на v1.0. Расширение — только через formal change procedure (глава 13).

9.9 Lifecycle TC

9.9.1 Машина состояний



Статус	Смысл	Триггер перехода
`draft`	Создан, реализация в работе	Создание AI-агентом
`ready`	Реализация валидна; dry-run runner прошёл; pos/neg парность подтверждена	QG-0 (§9.10): one-click approval

`passing`	Последний прогон `last-run.result = pass` на текущей `requirement-version`	Bot-managed по факту прогона
`failing`	Последний прогон `last-run.result = fail`	Bot-managed по факту прогона
`obsolete`	Терминальный; покрываемое поведение больше не существует	Delta-T3 инвалидация (§9.16) или deprecation родительского артефакта

`obsolete` — терминальный статус. TC в `obsolete` нативно для носителя сохраняется как исторический след: реализация носителя **должна** обеспечивать неизменность идентификатора TC и **не должна** допускать его переиспользование для нового TC (V1 capability; см. глава 3 §3.3.1).

9.9.2 Связь со статусом верифицируемого артефакта

Перевод BR / SR / SPEC в `verified` нормативно требует: все TC из `verified-by` верифицируемого артефакта имеют `last-run.result = pass` и `last-run.requirement-version` совпадает с текущей версией артефакта (см. §9.10 QG-2).

9.10 Контрольные точки качества для TC

Канонические определения Quality Gates — в главе 10 §10.3. Эта секция — TC-локальная сводка gates, применимых к TC напрямую (QG-0, QG-1) или использующих TC как evidence (QG-2). Нумерация и семантика обязаны совпадать с канонической §10.3; project-local переопределение запрещено (§10.10.2).

Gate (канонический)	Роль TC	Предусловие (кратко; полная формулировка — гл. 10)	Постусловие

QG-0 — утверждение (§10.3.1)	TC (`draft → ready`) — часть «утверждение»	Ссылка на `verifies[]` — артефакт есть в носителе в состоянии не ниже `approved` ; общие предусловия §10.3.1; решение утверждающего зафиксировано нативно для носителя (V3 + V6)	TC допускается к проверкам гейта реализации (QG-1)
QG-1 — реализация проверки (§10.3.2)	TC (`draft → ready`) — часть «реализация»	`automation.status = automated` (или `manual-pending` с дедлайном и причиной); pos/neg парность (§9.7); dry-run runner прошёл; обязательные секции body TC (§9.4) заполнены	TC переходит в `ready` ; допускается production-прогон runner
QG-2 — верификация (§10.3.3)	Артефакт (BR / SR / SPEC / TR) `→ verified` / `→ done` ; TC — доказательная база	Все TC из `verified-by` верифицируемого артефакта в статусе `passing` ; pos/neg парность по каждому нормативному утверждению; обязательные spec-specific виды TC (§9.8) присутствуют; `last-run.requirement-version` совпадает с текущей `version` артефакта	Верифицируемый артефакт переходит в `verified` (TR — в `done`); TC остаётся `passing`

Нативный для носителя one-click promote `draft → ready` атомарно проверяет предусловия обоих QG-0 и QG-1 как единый bundle (см. §10.3.2 «Триггер») — диаграмма §9.9.1 показывает агрегированное прохождение гейта как «утверждение QG-0».

Проверка совпадения `last-run.requirement-version` с текущей `version` верифицируемого артефакта при каждом последующем прогоне TC — runner-managed consistency check (§10.9.3), **не** отдельный Quality Gate в смысле §10.2.1. При mismatch носитель автоматически переводит TC в `failing` до повторного прогона на актуальной версии артефакта; record в audit-trail (§10.13) фиксируется типом `runner-fail` , не `gate-passage` .

Нативные для носителя hooks (глава 3 §3.3) обязаны блокировать gate-переходы, нарушающие предусловие.

9.11 Pass / Fail / Out of scope — нормативные критерии

9.11.1 Pass-критерий

Pass-критерий обязан быть:

- **Бинарным** — да или нет, без интерпретации.
- **Наблюдаемым** — фиксируется без доступа к внутренним структурам системы.
- **Воспроизводимым** — повторный прогон в тех же условиях даёт тот же результат.

Плохо	Хорошо
«Логин работает корректно»	« `POST /auth/login` с верными credentials возвращает 200 и JWT с `exp = now + 24h ± 1м` »
«Производительность приемлемая»	«p95 latency < 200 мс при 100 RPS на `/search` в течение 5 минут»
«Ошибка обрабатывается»	«При невалидном email возвращается 422 с телом `{"field":"email","code":"invalid_format"}` »

9.11.2 Fail-критерий

Fail-критерий — **не отрицание** Pass. Перечисляет наблюдаемые признаки нарушения, в том числе те, которые Pass-критерий явно не покрывает:

- Какой ответ / состояние / событие признаётся отказом.
- Утечки информации (например, в 401 не должно быть указано, какое именно поле credentials неверно).
- Side-effects, которых не должно быть — запись в лог, отправка письма, мутация других записей.
- Race conditions: одновременные запросы не приводят к нарушению инвариантов.

9.11.3 Out of scope

Каждый TC обязательно содержит секцию «Out of scope» с явным перечислением:

- Что намеренно не проверяется этим TC;
- Где это покрыто (ссылка на парный TC или другой TC).

Раздел защищает от ложного ощущения покрытия (см. также глава 11 — coverage matrix). Отсутствие явного «Out of scope» нормативно блокирует QG-0 (§9.10).

9.12 `last-run` — bot-managed only

Поле `last-run` (date / result / runner-id / run-ref / requirement-version / judge-report) заполняет **только** автоматический runner (CI-bot / AI-runner / specialized executor). Ручное редактирование `last-run` любым человеком — нарушение стандарта; hook носителя (глава 3 §3.3.6) обязан блокировать change unit, изменяющий `last-run` от non-bot author.

Состав `last-run`:

Поле	Mandatory	Содержание
`date`	да	ISO-datetime прогона
`result`	да	`pass | fail | skipped | n/a`
`runner-id`	да	Идентификатор runner + версия
`run-ref`	да	нативный для носителя pointer на полный лог прогона
`requirement-version`	да	Версия верифицируемого артефакта на момент прогона (V5 pinning)
`judge-report`	да для `ux | eval`	Inline или pointer на отчёт VLM/eval-judge

9.13 Защита от подгонки тестов

9.13.1 Нормативное правило

AI-агент не может одновременно изменить implementation-код и Pass / Fail-критерии существующего TC в одном change unit так, чтобы `failing` TC становился `passing`, без явного approval инженером на изменение TC.

Без этого правила AI-агент имеет тривиальный путь к зелёному прогону — ослабить критерий вместо исправления кода.

9.13.2 Механизм `[test-spec-change]`

Класс изменения	Тег	Approval
Изменение Pass / Fail-критериев существующего TC	`[test-spec-change]`	Mandatory: явный approval инженером change unit отдельно от любого изменения implementation-кода
Изменение `automation.location` (перенос реализации без изменения проверяемого поведения)	—	Без отдельного approval
Изменение implementation-кода без правок TC	—	Standard workflow
Обновление `baseline.artifact` / dataset / `mockup-baseline`	`[baseline-update]`	Mandatory: явный approval инженером
Создание нового TC	—	QG-0 (§9.10)

Нативная для носителя реализация тегов специфична для носителя (см. guide/03, guide/04); нормативное требование — атомарность change unit, явное обозначение класса изменения, и опциональная (но рекомендуемая) изоляция таких изменений в отдельный change unit от изменений implementation-кода.

9.13.3 Аудит

Все change units с тегом `[test-spec-change]` агрегируются в нативный для носителя audit-feed для архитектора. Цель — отследить паттерн: если AI-агент часто запрашивает изменение критериев, это сигнал о проблеме формулировок исходного требования (глава 7 ADAPT, категории обратных находок `terminology` или `gap`).

9.13.4 Judge isolation (P7) — частный случай защиты

`judge.vendor` + `judge.model` обязательно отличается от production-модели верифицируемого SPEC-AI (§9.6.2). Совпадение блокирует hook носителя.

9.14 Спот-чек инженера

9.14.1 Нормативная процедура

Раз в итерацию (по умолчанию — регулярный цикл реализации; конкретный интервал фиксируется в conformance manifest проекта) инженер выполняет спот-чек 5 случайных TC в статусе `passing`. Цель — поймать ситуацию, когда AI-агент сгенерировал «зелёный» TC, который ничего значимого не проверяет (`assert True`-эквивалент; VLM-prompt, проходящий на пустом экране; eval-критерий, всегда возвращающий pass на baseline).

9.14.2 Sampling

Параметр	Нормативное требование
Размер выборки	5 TC (по умолчанию); может быть увеличено в conformance manifest проекта
Распределение по типам	Равномерное по `tc-type` (acceptance / ux / system / contract / eval / security) — каждый тип имеет шанс быть выбранным
Статус	Только `passing`
Кто выбирает	AI-агент случайным образом (нативная для носителя случайность; seed фиксируется в audit-feed)

9.14.3 Что инженер проверяет

1. Соответствует ли Pass / Fail-критерий заявленному поведению в верифицируемом артефакте?
2. Не слишком ли мягкий критерий VLM-judge или eval-judge?
3. Реальны ли предусловия (не подменены ли через seed, маскирующий bug)?
4. Покрывает ли Out-of-scope именно то, что должен покрывать парный TC?

9.14.4 Фиксация результата

Результат спот-чека фиксируется нативно для носителя:

```
last-spot-check:
  date: "<ISO-date>"
  by: "<engineer-id>"
```

```
  sampled-tests: [TC-NN, TC-NN, ...]
  issues-found: integer
  issues:
    - test: "TC-NN"
      issue: "<краткое описание>"
```

9.14.5 Реакция на находки

При `issues-found > 0`:

- Архитектор регистрирует change unit на исправление найденных TC.
- AI-агент обязан учесть выявленный паттерн в следующих генерациях TC; паттерн добавляется в system prompt агента или в meta-style guide.
- При повторном обнаружении того же паттерна — эскалация на пересмотр шаблона генерации TC.

9.15 Матрица покрытия (auto-generated)

9.15.1 COVERAGE-artifact

`COVERAGE.md` (нативное для носителя имя artifact) — auto-generated сводный отчёт покрытия требований и спецификаций тест-кейсами на уровне носителя требований. Помечается нативным для носителя флагом auto-generated.

9.15.2 Mandatory метрики

Метрика	Цель	Действие при нарушении
`coverage-percent` (verified / total artifacts)	Целевой порог фиксируется в conformance manifest	нативный для носителя gate блокирует промоушн
`approved без verified`	0 перед промоушн	Бэклог AI-агента на следующую итерацию
Покрытие парным negative TC	100% утверждений	AI-агент создаёт change unit с парным негативом
`passing-tests / total-tests`	100% перед промоушн change unit	Блокирует QG-2 (§9.10)

`manual-pending` overdue	0	Уведомление архитектору; блокировка затронутых артефактов
Stale (`last-run.requirement-version` < текущей)	0	AI-агент перезапускает прогон

9.15.3 Триггеры перегенерации

`COVERAGE.md` перегенерируется автоматически при:

- Завершении change unit с изменением артефактов требований / SPEC / TC;
- Промоушн change unit в основную линию носителя;
- Каждом успешном прогоне runner (обновление `last-run`);
- По расписанию (нативный для носителя scheduler).

9.16 Delta-ТЗ и ТС

9.16.1 Impact analysis по тестам

При delta-ТЗ (глава 7 §7.6) AI-агент выполняет impact analysis по ТС одновременно с impact analysis по требованиям:

1. Находит все ТС, у которых `verifies[].requirement-version` ниже новой версии верифицируемого артефакта.
2. Помечает их `obsolete-pending: true` .
3. Формирует таблицу затронутых ТС в frontmatter delta-ADAPT или связанном change unit:

ТС	Verifies	Старая версия	Новая версия	Действие
ТС-NN	SR-NN	v1.1	v1.2	Update (новый шаг)
ТС-NN	SR-NN	v1.1	v1.2	Без изменений (актуален)

TC-NN	BR-NN	v1.0	deprecated	Перевести в `obsolete`

4. Генерирует обновлённые версии TC в том же change unit, что delta-ADAPT.
5. После прогона обновлённых TC и их перехода в `passing` — снимает `obsolete-pending` и обновляет `verifies[].requirement-version` на новую версию артефакта.

9.16.2 Переход TC в `obsolete`

TC переходит в `obsolete` (терминально), если:

- Родительский артефакт (BR / SR / SPEC) переведён в `deprecated` без замены;
- Родительский артефакт заменён новым (`replaced-by`), для которого создан новый набор TC, **и** старый набор не покрывает поведения нового артефакта;
- Поведение, покрываемое TC, в новой версии артефакта больше не существует.

`obsolete` TC immutable и не удаляется (V1; см. глава 3 §3.3.1).

9.17 Схема хранения

Тест-кейсы хранятся в подпапке `tests/` носителя требований. Нативная для носителя реализация хранения специфична для носителя (см. guide/03 для distributed VCS; guide/04 для document-oriented store).

9.17.1 На уровне системы / подсистемы

```
[requirements-substrate]/      # system или subsystem scope (глава 6 §6.11)
  br/   sr/   tr/                # глава 6
  specs/                          # глава 8
  adapt/                          # глава 7
  tests/
    acceptance/  TC-NN-*.md
```

```
    system/      TC-NN-*.md
    ux/          TC-NN-*.md
    contract/    TC-NN-*.md
    eval/        TC-NN-*.md
    security/    TC-NN-*.md
    baselines/                    # для ux / eval
      <baseline-artifact>.png
      <eval-dataset>.jsonl
  COVERAGE.md                     # auto-generated (см. §9.15)
```

9.17.2 Реализация в субстрате кода

Реализация TC (код) живёт в носителе кода отдельно от носителя требований; адресуется полем `automation.location` (нативный для носителя pointer). Связь TC↔implementation: 1:1.

9.18 Связь с другими главами

Глава	Связь
02 Положение в типологии методологий	TC — нижний слой trace chain (Утверждение 1 SoT inversion); pinning требования через `verifies[].requirement-version` (Утверждение 3 версионирование носителя)
06 Иерархия требований	TC верифицирует BR / SR; `verified-by[]` — auto-derived inverse edge на стороне требования
07 ADAPT	TC → SR → ADAPT → T3 — полная trace chain; обратные находки (`terminology` , `gap`) питаются паттернами из `[test-spec-change]` audit (§9.13.3)
08 Specifications	Spec-specific TC по таблице §9.8; SPEC → `verified-by[]` auto-derived; type-specific extensions §9.6
10 Lifecycle и QG	TC state machine; QG-0 + QG-1 bundle для TC (`draft → ready`); QG-2 для верифицируемого артефакта требует pos/neg парность и spec-specific виды TC

03 Версионирование носителя	Immutable IDs (V1); atomic change unit и hooks (V2 + V3); diff & review для `[test-spec-change]` (V3); версионирование TC без потери истории (V4); pinning `verifies[].requirement-version` (V5); author + timestamp для `last-run` (V6)
11 Maturity model	RENAR-1: TC обязательно; RENAR-3+: pos/neg pairing 100%, spec-specific TC table обязательно; RENAR-4+: AI-generated + AI-executed
13 Conformance	Closed list типов TC (§9.5) — mandatory clause v1.0; spec-specific TC table (§9.8) — mandatory clause v1.0; pos/neg pairing — mandatory clause v1.0; judge ≠ production isolation — mandatory clause v1.0
reference/02 — schemas	Полная machine-readable schema TC frontmatter + type-specific extensions

---

title: "Lifecycle и Quality Gates" order: 10 lang: ru

10. Lifecycle и Quality Gates

> ***Плотная глава:*** *читать после §6–§9; walkthrough QG — guide/00; плотность — reference/09.*

10.1 Как артефакты движутся: статусы и гейты

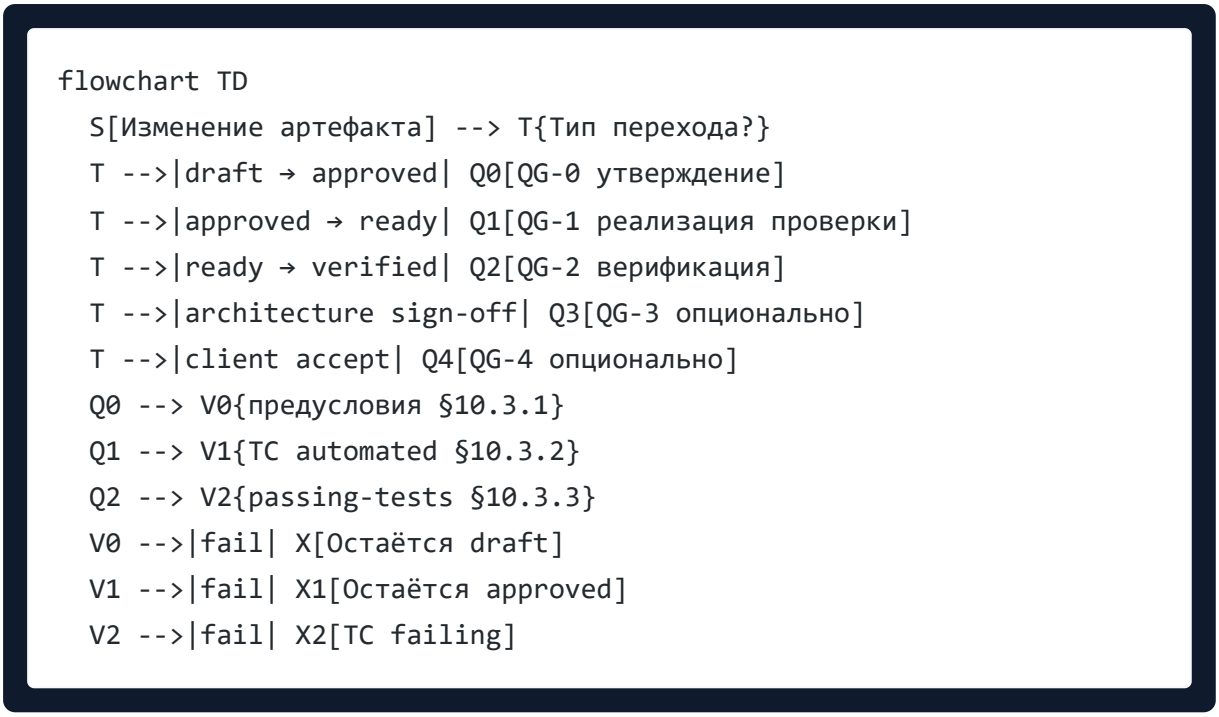
Все артефакты RENAR — требования, спецификации, ADAPT, тест-кейсы — живут не статично: каждый проходит через состояния (`draft → approved → verified → ...`), и каждый переход охраняется контрольной точкой качества (Quality Gate). Эта глава сводит воедино машины состояний всех артефактов и нормирует гейты: что именно проверяется перед переходом, кто и когда обязан это сделать.

Глава нормирует:

- **Понятие Quality Gate** — нормативное определение, предусловия и постусловия, кто и когда обязан проверять.
- **Канонический закрытый список gates** RENAR: QG-0 (утверждение), QG-1 (реализация проверки), QG-2 (верификация) — обязательные для conformance; QG-3 (архитектура) и QG-4 (приёмка) — опциональные расширения.
- **Lifecycle всех артефактов RENAR**: BR / SR / TR (требования, глава 6), SPEC (глава 8), ADAPT (глава 7), TC (глава 9) — консолидированные state machines с привязкой gate-id к каждому переходу.
- **Независимый от носителя enforcement** — нормативные требования к hook-механизмам через capabilities V1–V6 (глава 3). Конкретные реализации (нативная для носителя реализация hooks) выносятся в `guide/` и conformance manifest.
- **Closed list policy**: добавление нового типа Quality Gate производится только через formal change procedure стандарта RENAR (глава 13); project-local расширения списка gate-типов запрещены.

Глава не определяет frontmatter артефактов (это делают главы 6–09). Глава фиксирует **только** lifecycle states, переходы между ними и gates, контролирующие переходы.

10.1.1 Дерево решений: какой гейт сейчас (informative)



Walkthrough: guide/00. Плотные главы: reference/09.

10.2 Нормативное определение Quality Gate

10.2.1 Quality Gate

Quality Gate (gate) — нормативное условие, проверка которого обязана быть выполнена для разрешённого перехода артефакта из одного lifecycle-состояния в другое. Каждый gate состоит из:

1. **Идентификатор** — `QG-N` или `QG-<artifact>-<state>` (закрытый список §10.3, §10.4).
2. **Предусловие** — набор проверяемых утверждений об артефакте и связанных артефактах, которые обязаны быть истинны на момент запуска gate.
3. **Постусловие** — состояние, в которое переходит артефакт после успешного прохождения gate, и наблюдаемые эффекты (например, появление записи в лог переходов §10.13).
4. **Триггер** — кто или что инициирует проверку gate (актор: AI-агент / архитектор / автоматический runner; событие: одобрение / завершение прогона / поступление delta-T3).
5. **Enforcement-точка** — место в носителе, где проверка обязана быть автоматизирована (§10.11).

Gate не является событием успеха — это условие, которое **обязано быть проверено**. Прохождение gate может быть отрицательным (предусловие не выполнено) — в этом случае переход запрещён и артефакт остаётся в текущем состоянии.

10.2.2 Кто обязан проверять gate

Тип gate	Обязательный actor	enforcement носителя
Утверждение (QG-0)	Архитектор или авторизованный role-holder	Атомарная фиксация авторства и времени (V6, §3.3.6)
Реализация проверки (QG-1)	Автоматический runner (CI, eval-runner)	Атомарная фиксация результата прогона привязанного к версии артефакта (V5, §3.3.5)
Верификация (QG-2)	Автоматический runner с подтверждением `version-pin`	V5 + V6
Архитектура (QG-3, опционально)	Двойная подпись (клиент + архитектор)	V3 + V6
Приёмка (QG-4, опционально)	Stakeholder с полномочиями	V6

10.2.3 Связь с SENAR

SENAR §8 описывает Quality Gates как абстрактную концепцию для AI-управляемой разработки. RENAR **расширяет** SENAR в области requirements engineering:

- Сохраняет идентификаторы QG-0 / QG-1 / QG-2 как обязательные.
- Нормирует **формальные state machines** для каждого типа артефакта (SENAR этого не делает).
- Привязывает каждый переход в state machine к конкретному gate с предусловиями и постусловиями.
- Добавляет опциональные QG-3 / QG-4 для отраслей с расширенными требованиями к аудиту.

RENAR не противоречит SENAR; реализация SENAR-совместима с RENAR если выполнены требования §10.3 + §10.11.

10.3 Канонические RENAR gates (обязательные)

Закрытый список из трёх обязательных gates. Расширения вне этого списка — только опциональные §10.4 или через formal change procedure стандарта §10.10.

10.3.1 QG-0 — гейт утверждения

Назначение: разрешает переход артефакта из черновика в утверждённое для разработки состояние.

Предусловие (общая часть, дополняется per-artifact в §10.5–§10.9):

- Frontmatter артефакта валиден по schema своей главы.
- Идентификатор артефакта уникален в носителе (V1, §3.3.1).
- Adversarial review произведён; или явно зафиксировано отсутствие применимости — допустимо **только** для тривиальных артефактов (по критериям, объявленным в conformance manifest, §13) с записью причины в лог переходов (§10.13).
- Если артефакт ссылается на источник (`source.adapt` для BR/SR/SPEC, `verifies[]` для TC) — referenced артефакт существует в носителе в состоянии не ниже `approved` .

Постусловие:

- Артефакт переходит в `approved` (для requirements / SPEC) или `ready` (для TC) или `approved` ADAPT (§10.8).
- Запись в лог переходов (§10.13).
- Для requirements / SPEC: разрешается декомпозиция в дочерние артефакты (для BR — SR; для SR — TR + SPEC через `constrained-by` / `implements-spec`).

Триггер: явное одобрение архитектором / role-holder в нативном для носителя механизме (V3 diff & review, §3.3.3).

Применимые артефакты: BR, SR, TR, SPEC, ADAPT, TC.

10.3.2 QG-1 — гейт реализации проверки (только TC)

Назначение: подтверждает, что для артефакта существует валидная реализация — код, конфигурация, инфраструктурный артефакт — пригодная для верификации.

Предусловие:

- Реализация привязана к версии артефакта через `version-pin` (V5, §3.3.5).

- `automation.status: automated` (с валидным `automation.location`) или `automation.status: manual-pending` (с указанным `manual-pending-until` и `manual-pending-reason`).
- Все статические проверки реализации агента носителя (типы, lint, схема) — пройдены.
- Pos/neg парность для покрываемых утверждений артефакта обеспечена (глава 9 §9.7).
- Все обязательные секции body TC (глава 9 §9.4) заполнены.

Постусловие:

- TC переходит в `ready`.
- Запись в лог переходов.

Триггер: одобряющий actor (one-click promote `draft → ready`) при подтверждении автоматического runner о прохождении dry-run.

Применимые артефакты: TC (`draft → ready`).

Примечание: TR не проходит отдельно гейт QG-1. Условия валидности реализации (impl scope, version-pin, статические проверки) для TR входят в предусловия QG-2 (§10.6.2).
QG-1 применим только к TC. Для BR / SR / SPEC переход `approved → verified` управляется единым QG-2; промежуточного гейта реализации QG-1 для требований и SPEC нет.

10.3.3 QG-2 — гейт верификации

Назначение: подтверждает, что наблюдаемое поведение системы соответствует артефакту: все TC из `verified-by` артефакта в состоянии `passing` на текущей версии артефакта.

Предусловие:

- Для BR / SR / SPEC: все TC из `verified-by` имеют `last-run.result = pass` и `last-run.requirement-version` (или эквивалент `spec-version` / `version`) совпадает с текущей `version` верифицируемого артефакта.
- Pos/neg парность по нормативным утверждениям артефакта — выполнена.
- Все обязательные spec-specific виды TC для типа артефакта присутствуют (глава 9 §9.8).

- Для TR: все его AC верифицированы привязанными TC (`last-run.result = pass`).
- Spec-specific дополнительные предусловия:
  - SPEC-UI / SPEC-AI: TC в состоянии `passing` с `judge-isolation` соблюдённой (глава 9 §9.13.4).
  - SPEC-SEC: TC `tc-type: security` присутствует и `passing` .

Постусловие:

- Артефакт переходит в `verified` .
- Запись в лог переходов с evidence-refs (список ID прогонов).
- Носитель обязан фиксировать `version` верифицируемого артефакта в evidence-записи (V5).

Триггер: автоматический runner подтверждает passing TC и инициирует promote-transition по запросу автора (one-click promote `approved → verified`).

Применимые артефакты: BR, SR, SPEC, TR.

10.4 Опциональные gates

QG-3 и QG-4 — нормативно описаны, но **не обязательны** для conformance (глава 13). Реализация может объявить в conformance manifest либо поддержку QG-3 / QG-4, либо их отсутствие. Conformance без QG-3 / QG-4 остаётся валидной.

10.4.1 QG-3 — гейт архитектуры (опционально)

Назначение: разрешает переход ADAPT из `answered` в `approved` (§10.8). Также применим к декомпозиционным решениям SPEC-ARCH в проектах с регулируемой архитектурной приёмкой.

Предусловие:

- Все обратные находки в ADAPT в статусе `resolved` (глава 7 §7.4.5).
- Двойная подпись готова: client signature + architect signature (глава 7 §7.5).
- Для SPEC-ARCH (если QG-3 применяется): декомпозиционное решение зафиксировано в носителе как ADR-like артефакт со ссылкой из SPEC-ARCH (форма ADR специфична для носителя — fits в guide/).

Постусловие:

- ADAPT переходит в `approved` (immutable наравне с T3).
- Запись в лог переходов с обеими подписями (V6 author + timestamp фиксирует обоих акторов).

Триггер: явный двойной approval; носитель обязан атомарно фиксировать обе подписи (V2 atomic change unit, §3.3.2).

Когда применять:

- ADAPT — всегда (но реализация может объявить QG-3 как локальный псевдоним для approval ADAPT, не выделяя его в отдельный gate).
- SPEC-ARCH — в проектах с регуляторными требованиями к архитектурной приёмке.

10.4.2 QG-4 — гейт приёмки (опционально)

Назначение: фиксирует приёмку клиентом бизнес-результата после релиза. Переход BR из `verified` в `accepted`.

Предусловие:

- BR в `verified` (QG-2 пройден).
- Измеримый бизнес-результат (`business-outcome` в frontmatter BR) — измерен; `current-value` зафиксирован.
- `achievement` ≥ project-configurable порога (по умолчанию 80%, фиксируется в conformance manifest).
- Формальная подпись stakeholder.

Постусловие:

- BR переходит в `accepted` (терминальный недеградируемый статус — обратный переход требует delta-T3).
- Запись в лог переходов с подписью stakeholder.

Триггер: формальная приёмка stakeholder по факту релиза.

Когда применять:

- Проекты с явной фиксацией post-release outcomes (продуктовые SaaS, регулируемые отрасли).
- При отсутствии QG-4 — статус `accepted` не используется; BR остаётся в `verified` до `deprecated`.

10.4.3 Соответствие стандарту с опциональными гейтами

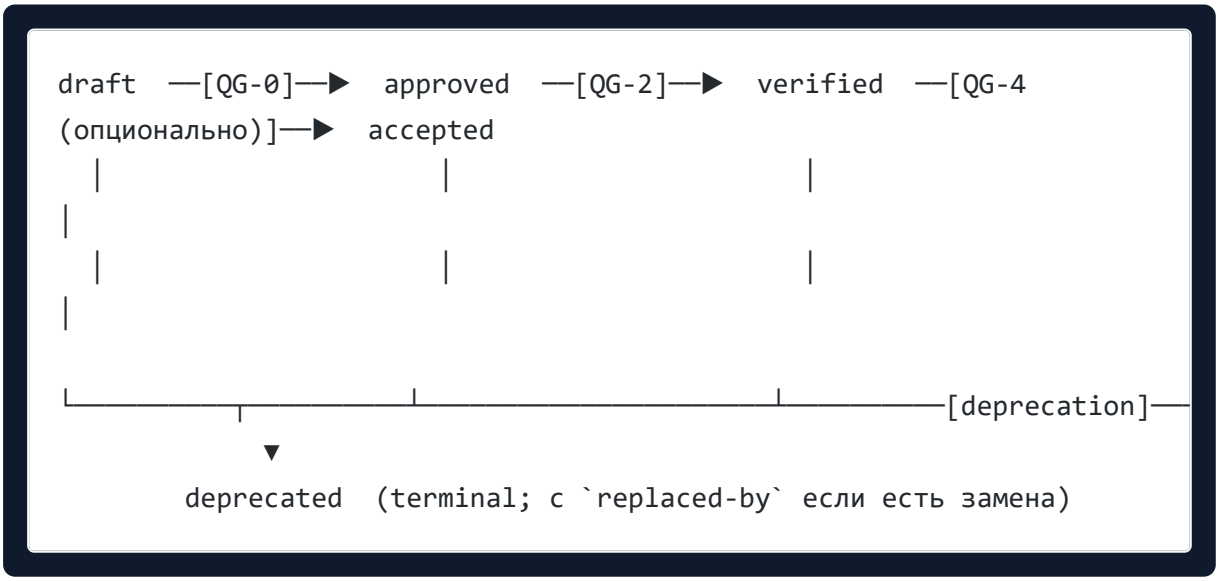
Conformance manifest (глава 13) обязан явно объявлять:

```
quality-gates:
  qg-0: required          # всегда required
  qg-1: required
  qg-2: required
  qg-3: declared          # required | declared | absent
  qg-4: declared
```

`declared` означает: реализация поддерживает gate; артефакты могут проходить его, но conformance не требует прохождения для всех артефактов. `absent` — gate не применяется в реализации; терминальное состояние артефакта — `verified` (без `accepted`).

10.5 Машина состояний BR / SR

10.5.1 Состояния и переходы



Статус	Семантика	Gate перехода
`draft`	Создан AI-агентом или архитектором; ещё не утверждён	— (создание)

`approved`	Утверждён к декомпозиции / реализации	QG-0 (§10.3.1)
`verified`	Все производные TC `passing` на текущей версии	QG-2 (§10.3.3)
`accepted`	Post-release бизнес-результат подтверждён	QG-4 (§10.4.2, опционально)
`deprecated`	Терминальный; не удаляется (V1 immutable history)	Deprecation transition (§10.5.3)

Frontmatter BR / SR (включая обязательные поля статуса) определяется в главе 6 §6.5.2 / §6.6.2. Этот раздел нормирует **только** переходы между состояниями и привязку к gates.

10.5.2 Предусловия по переходам

Переход	Gate	Дополнительные предусловия (поверх §10.3)
`draft → approved`	QG-0	BR: `business-outcome` заполнен. SR: `parent` BR в состоянии не ниже `approved`. Если SR ссылается на SPEC через `constrained-by[]` — все SPEC в `approved` или выше.
`approved → verified`	QG-2	Хотя бы один TC с `negative: true` в `verified-by`. Все `last-run.requirement-version` совпадают с текущей `version`.
`verified → accepted`	QG-4	Только если реализация объявила QG-4.
`* → deprecated`	Deprecation transition (§10.5.3)	См. §10.5.3

10.5.3 Deprecation transition

Предусловие:

- Артефакт находится в любом не-`deprecated` состоянии.
- `replaced-by` (если указан) существует в носителе и находится в состоянии не ниже `approved`.
- Нет активных дочерних TR в состоянии `approved` или активных задач исполнителя по этому артефакту (атомарная переадресация задач на replacement — обязательное условие, V2 atomic change unit).

Постусловие:

- `status: deprecated`.
- `deprecated-date` записан (V6).
- `replaced-by` указан, если есть замена.

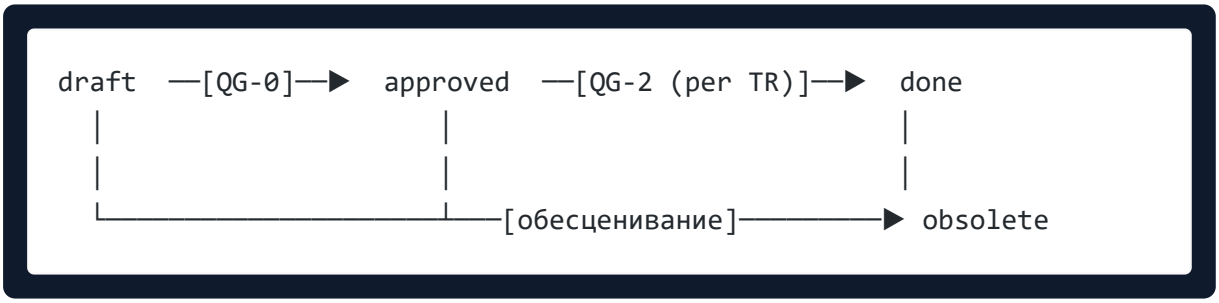
Триггер: архитектор или Product Owner.

10.5.4 Reverse-эволюция верификации

Если артефакт уже `verified`, но `version` инкрементировался (например, после применения delta-ADAPT, глава 7 §7.6) — статус обязан вернуться в `approved` до повторного прохождения QG-2 на новой версии. Этот переход обязателен и автоматический: носитель обязан инвалидировать `verified` при изменении `version` (глава 6 §6.5.4 reverse-эволюция).

10.6 Машина состояний TR

10.6.1 Состояния и переходы



Статус	Семантика	Gate
`draft`	TR создан; AC ещё не финализированы	—
`approved`	AC утверждены; разрешён старт работы исполнителя	QG-0 (§10.3.1) с предусловиями TR из §10.6.2
`done`	AC верифицированы; все привязанные TC `passing`	QG-2 (§10.3.3) с предусловиями TR из §10.6.2
`obsolete`	TR утратил актуальность до завершения (родительский SR изменился)	Обесценивание (архитектор)

10.6.2 Предусловия для TR

QG-0 для TR (`draft → approved`) — дополнительно к общей части:

- Цель сформулирована (`goal`).
- AC верифицируемы и независимы (каждый AC — отдельная проверка).
- Хотя бы один негативный сценарий присутствует.
- Установлена ссылка на parent SR (или BR для простых конфигураций) через `implements`.
- Если TR реализует SPEC — обязательное поле `implements-spec[]` (глава 8 §8.6.2).
- Если задача затрагивает безопасность — `threat-surface` задекларирован (глава 8 §8.5.8).

QG-2 для TR (`approved → done`):

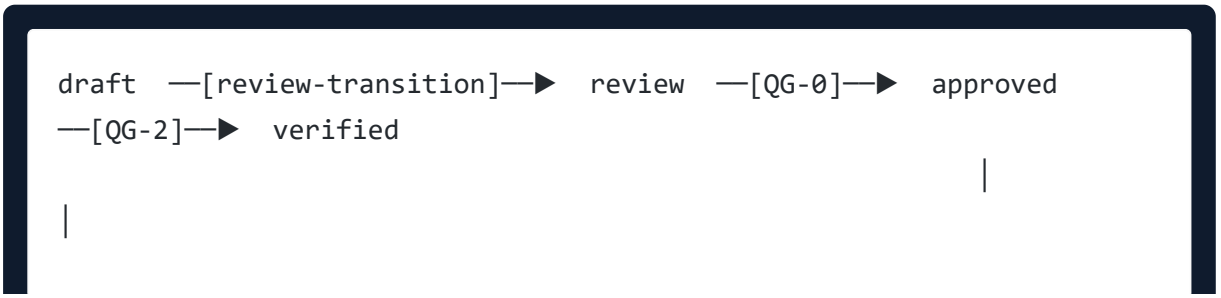
- Все AC TR подтверждены прохождением соответствующих TC (`last-run.result = pass`).
- Pos/neg парность по каждому AC.
- Для TR реализующего SPEC: TC соответствующего spec-specific вида существует и `passing` (глава 9 §9.8).

10.6.3 Обесценивание TR

Если родительский SR переходит в `deprecated` или его `version` изменилась так, что AC TR более не актуальны — TR переходит в `obsolete`. Это **не** деградация — это альтернативный терминальный путь. TR в `obsolete` не удаляется.

10.7 Машина состояний SPEC

10.7.1 Состояния и переходы



```
└──[обесценивание]──▶ obsolete  (terminal)
```

Статус	Семантика	Gate
`draft`	SPEC создан; обязательные frontmatter-поля заполняются	—
`review`	Обязательные body-разделы (§8.4.1) и type-specific (§8.5) присутствуют; готов к рецензированию	Review-transition (§10.7.2)
`approved`	Архитектор подтвердил; `depends-on[]` консистентен	QG-0
`verified`	Все обязательные spec-specific TC `passing`	QG-2
`obsolete`	Заменён или больше не актуален; `replaced-by` обязателен	Deprecation (§10.5.3 mutatis mutandis)

10.7.2 Review-transition (`draft → review`)

Review-transition не является полноценным gate в смысле §10.2.1 — это автоматическая проверка структурной полноты. **Предусловие**:

- Все обязательные frontmatter-поля §8.4 заполнены.
- Все обязательные body-разделы §8.4.1 присутствуют.
- Type-specific разделы §8.5 присутствуют для соответствующего `spec-type`.

Постусловие: `status: review`; артефакт виден архитектору для рецензирования.

Если review-transition не пройден — артефакт остаётся в `draft`; носитель обязан вернуть список отсутствующих секций (V3 diff & review поддерживает структурный фидбек).

10.7.3 Предусловия для SPEC

QG-0 для SPEC (`review → approved`) — дополнительно к общей части §10.3.1:

- `depends-on[]` граф ацикличен (глава 8 §8.6.3).
- Все SPEC из `depends-on[]` в состоянии не ниже `approved`.
- Если SPEC ссылается на ADAPT через `source.adapt` — ADAPT в состоянии `approved`.

QG-2 для SPEC (`approved → verified`):

- Все обязательные spec-specific виды TC для `spec-type` присутствуют и `passing` (глава 9 §9.8).
- Для SPEC-AI: pos/neg pair coverage по `evaluation-criteria` = 100%; judge-isolation соблюдена.
- Для SPEC-SEC: `tc-type: security` присутствует.
- Для SPEC-DATA: `tc-type: contract` присутствует для опубликованных interface-полей.

10.8 State machine ADAPT

10.8.1 Macro-состояния

```
draft  ──[review-transition]──▶  review  ──[backward-ready]──▶
client-ready

│

│ [client returns answers]

▼

answered

│

│ [QG-3]

▼

approved

│

│ [immutable; только delta-ADAPT или errata]
```

```
▼

frozen
```

Состояния ADAPT (`draft → review → client-ready → answered → approved → frozen`) определены в главе 7 §7.4. Этот раздел нормирует gates.

Переход	Gate	Предусловие
`draft → review`	Review-transition	Прямая интерпретация охватывает все разделы T3; первичные обратные находки зафиксированы в `open`
`review → client-ready`	Backward-ready	Все обратные находки переведены в `asked-to-client` ; пакет вопросов сформирован
`client-ready → answered`	Client-return	Все обратные находки в `answered` с author + timestamp ответа клиента (V6)
`answered → approved`	**QG-3** (§10.4.1)	Все обратные находки в `resolved` ; двойная подпись готова
`approved → frozen`	Freeze-transition	Автоматический после approve; ADAPT immutable; разрешается генерация BR / SR / SPEC с `source.adapt = approved`

10.8.2 Вложенная машина состояний для записи об обратной находке

Каждая запись об обратной находке внутри ADAPT имеет собственную подчинённую машину состояний (глава 7 §7.4.5):

```
open  ──▶  asked-to-client  ──▶  answered  ──▶  resolved
──[approve ADAPT]──▶  frozen
                  ▲                  │
                  └──── revised ─────┘  (если ответ требует
уточнения)
```

Sub-state	Семантика
`open`	Записан инженером; не отправлено клиенту
`asked-to-client`	Отправлен клиенту; зафиксирована дата вопроса

`answered`	Клиент ответил; ответ записан (V6 author + timestamp)
`resolved`	Инженер интегрировал ответ в прямую интерпретацию
`revised`	Ответ расплывчатый; повторный вопрос (возврат к `asked-to-client`)
`frozen`	После approval ADAPT; изменения невозможны

Нормативное правило: QG-3 (approve ADAPT) **запрещён**, если хотя бы одна запись об обратной находке в `open` / `asked-to-client` / `answered` / `revised`. Все такие записи обязаны быть в `resolved` (глава 7 §7.4.5).

10.8.3 QG-3 для ADAPT — детализация

Предусловие (полная):

- Все разделы прямой интерпретации заполнены (критерий forward complete).
- Все записи об обратных находках в `resolved`.
- Client signature получен и зафиксирован нативным для носителя механизмом (V3 + V6).
- Architect signature получен и зафиксирован.
- Если ADAPT — delta-ADAPT: parent-ADAPT в `frozen` (глава 7 §7.6).

Постусловие:

- ADAPT переходит в `approved`.
- Запись в лог переходов с обеими подписями.
- Носитель обязан атомарно (V2) зафиксировать обе подписи: частичная подпись (только клиент / только архитектор) **не** переводит ADAPT в `approved`.

Триггер: явное одобрение обоими акторами.

10.8.4 Errata для frozen ADAPT

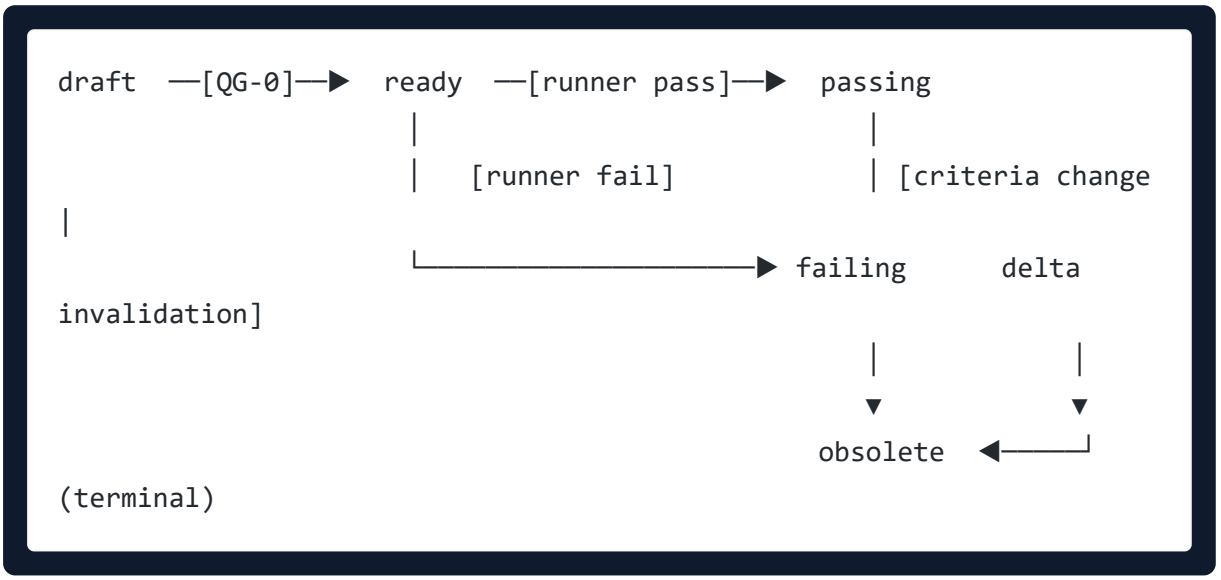
`frozen` — терминальное недеградируемое состояние. Изменения возможны только через:

1. **Delta-ADAPT** (если T3 содержит ambiguity, обнаруженную поздно) — новый артефакт с явной связью `parent-adapt`.
2. **Errata-ADAPT** (если ошибка интерпретации инженера) — отдельный артефакт с подписью клиента (если меняется contractual outcome) или только архитектора (если cosmetic).

В обоих случаях frozen ADAPT **не редактируется**. Это требование V1 (immutable history) для контрактных артефактов (глава 7 §7.6.3).

10.9 Машина состояний TC

10.9.1 Состояния и переходы



Статус	Семантика	Gate / триггер
`draft`	TC создан; реализация в работе	—
`ready`	dry-run runner прошёл; pos/neg парность подтверждена	QG-0 (§10.3.1) с предусловиями TC из §10.9.2
`passing`	`last-run.result = pass` на текущей `requirement-version`	runner pass; bot-managed
`failing`	`last-run.result = fail`	runner fail; bot-managed
`obsolete`	Покрываемое поведение более не существует	Deprecation (§10.9.4)

Frontmatter TC и pos/neg pairing определяются в главе 9.

10.9.2 Предусловия для TC

QG-0 для TC (`draft → ready`) — дополнительно к общей части:

- `automation.status: automated` (с валидным `automation.location`) *ИЛИ* `automation.status: manual-pending` (с `manual-pending-until` ≤ +1 sprint и заполненным `manual-pending-reason`).
- Pos/neg парность по покрываемым утверждениям подтверждена (§9.7).
- Dry-run runner прошёл (только структурная валидность; не путать с production-прогоном).
- Все обязательные секции body TC (§9.4) заполнены.
- Citation на `verifies[]` — артефакт существует в носителе в состоянии не ниже `approved`.

Постусловие:

- `status: ready`.
- Runner разрешён к production-прогону.

10.9.3 Переходы под управлением runner (не Quality Gates)

`ready → passing`, `ready → failing`, `passing → failing`, `failing → passing` — переходы, происходящие **только** по факту прогона runner (глава 9 §9.12 `last-run` bot-managed). Эти переходы **не являются** Quality Gates в смысле §10.2.1: они — нормативные следствия результатов прогона, а не прохождение гейта с предусловиями и постусловиями. В частности, проверка совпадения `last-run.requirement-version` с текущей версией верифицируемого артефакта (см. §9.10) — это проверка согласованности под управлением runner, а не отдельный gate.

Постусловие каждого runner-перехода:

- `last-run` обновлён: `result`, `timestamp`, `requirement-version`, `evidence-refs`.
- Носитель обязан запретить ручную модификацию `last-run` (только runner-actor).

10.9.4 Обесценивание TC

TC переходит в `obsolete` если:

1. Артефакт из `verifies[]` переходит в `deprecated` / `obsolete`.
2. Delta-T3 инвалидирует тестовое поведение (глава 9 §9.16).

Постусловие: `status: obsolete`. TC не удаляется (V1 immutable history).

10.9.5 Change-of-criteria — отдельный нормативный путь

Изменение `## Pass-критерий` или `## Fail-критерий` в TC — **не обычный transition**; это специальный путь, требующий отдельного approval workflow (глава 9 §9.13). Подробности enforcement — §10.11.3.

10.10 Политика закрытого списка

10.10.1 Нормативное правило

Закрытый список Quality Gates RENAR — обязательные {QG-0, QG-1, QG-2} и опциональные {QG-3, QG-4}. Изменение списка возможно **только** через formal change procedure стандарта RENAR (глава 13).

Настоящая политика является специализацией §1.7 Closed list policy для квалити-гейтов; общее правило для всех закрытых списков RENAR и master-индекс — §1.7.5.

10.10.2 Что запрещено

Действие	Запрещено?	Почему
Project-local создание нового gate-типа `QG-N`	Запрещено	Нарушает закрытый список; делает conformance не-портируемой
Локальное переопределение предусловий канонического гейта	Запрещено	Делает conformance несравнимой между реализациями
Дополнительное ужесточение предусловий локального гейта	Разрешено	Conformance manifest может объявить более строгие пороги (например, `qg-2.required-negative-tc: true`)
Локальное ослабление предусловий канонического гейта	Запрещено	Нарушает контракт стандарта
Объявление QG-3 / QG-4 как `absent` в conformance manifest	Разрешено	Опциональные gates — §10.4

Объявление QG-0 / QG-1 / QG-2 как `absent`	Запрещено	Нарушает conformance §10.4.3

10.10.3 Расширение списка

Добавление нового gate-типа возможно через:

1. Запрос на изменение стандарта (formal change request) с обоснованием — research-draft с типологией и сравнением с canonical gates.
2. Public review (срок и форум фиксируется политикой стандарта, глава 13).
3. Включение в следующую minor-версию стандарта (`v1.X` или `v2.0`).

Project-local расширения остаются за пределами conformance — они допустимы как internal-практики, но **не** влияют на conformance manifest.

10.11 Независимый от носителя enforcement

10.11.1 Нормативные требования

Носитель, реализующий RENAR, обязан обеспечить автоматическую проверку предусловий гейтов на следующих точках:

Точка enforcement	Что обязано быть проверено	Опирается на capabilities
Promote-transition (любой переход в более высокий статус)	Предусловия соответствующего gate (§10.3, §10.4, §10.5–§10.9)	V3 (diff & review) для блокировки перехода до approve; V4 (branching) для отделения WIP от утверждённой правды
Approve-transition (any approval action)	Authorship зафиксирован (actor) и временная метка	V6 (author + timestamp)
Reference-validation (любое создание/изменение артефакта с ссылкой на другой)	Referenced артефакт существует и в требуемом состоянии	V1 (immutable history) для stable identifier; V5 (version pin) для межносительных ссылок
Change-of-criteria для TC (§10.11.3)	Применён отдельный approval workflow	V3 + V6

Runner-transitions для TC (`ready → passing` / `failing`)	Только runner-actor может писать `last-run`	V6 (authorship); нативный для носителя ACL или role-based restrictions
Lifecycle invalidation (артефакт `verified` , версия инкрементировалась)	Артефакт автоматически переведён в `approved`	V5 (version pin) для детекции

10.11.2 Носитель без V3 / V4 / V6 — не conformant

Носитель, не обеспечивающий V3 (diff & review), не может реализовать gates: нет способа отделить «предложенное изменение» от «утверждённой правды» (глава 3 §3.3.3). Аналогично V4 (branching, §3.3.4) и V6 (author + timestamp, §3.3.6) — без них approval-механика невозможна. Носитель, не удовлетворяющий V3 / V4 / V6, **не реализует RENAR** независимо от других свойств.

10.11.3 Change-of-criteria для TC — особый enforcement

Изменение Pass / Fail критерия TC — высокорисковая операция (защита от test-fitting, глава 9 §9.13). Носитель обязан:

1. **Детектировать**: любое изменение секций `## Pass-критерий` / `## Fail-критерий` в TC-артефакте.
2. **Принудительно изолировать**: change-of-criteria обязано быть отдельным change-set (V4 atomic change unit), помеченным признаком, отличающим его от обычных правок (нативный для носителя механизм — частный случай V3 diff & review; форма признака специфична для носителя, выносится в guide/).
3. **Запретить совмещение**: одно и то же лицо не имеет права одобрить change-of-criteria и одобрение fix кода, который тестируется этим же TC. Носитель обязан проверять это правило при approve-transition.
4. **Регистрировать**: change-of-criteria записывается в audit-trail (§10.13) с явной типизацией события.

10.11.4 Формы нативной для носителя реализации

Конкретные нативные для носителя механизмы (как именно реализуется hook в данном носителе) — выносятся в `guide/` и conformance manifest. Стандарт не нормирует **форму** hook (это специфичное для носителя решение). Стандарт нормирует **что hook обязан проверять и в какой точке**.

Раздел `guide/` обязан содержать для каждого поддерживаемого носителя:

- Mapping enforcement-точек §10.11.1 на нативные для носителя механизмы.
- Примеры реализаций.
- Известные ограничения носителя относительно автоматизации каждой проверки.

10.12 Запрещённые переходы

Закрытый список переходов, нарушающих lifecycle. Носитель обязан их блокировать.

Из	В	Артефакт	Почему запрещено
`draft`	`verified`	BR / SR / SPEC	Пропускает QG-0; нет evidence approval
`draft`	`accepted`	BR	Same; и пропускает QG-2
`draft`	`done`	TR	Пропускает QG-0
`draft`	`passing`	TC	Пропускает QG-0 (нет dry-run runner)
`obsolete`	*	Любой	Терминальный статус; «реанимация» запрещена — нужен новый артефакт с `supersedes`
`deprecated`	*	Любой	Same
`frozen`	*	ADAPT	Same; изменения только через delta-ADAPT или errata (§10.8.4)
`verified`	`draft`	BR / SR / SPEC	Деградация через несколько ступеней — потенциальная потеря trace; если требуется доработка — `verified → approved` через delta или reverse-эволюция §10.5.4
`accepted`	`verified`	BR	Деградация после приёмки — недопустима без delta-T3
`accepted`	`approved`	BR	Same
`passing`	`draft`	TC	Деградация теряет history runs; используется `passing → failing → obsolete` или change-of-criteria путь
`ready`	`draft`	TC	Деградация теряет dry-run evidence (§10.9.2); ослабление TC через `[test-spec-change]` (глава 9 §9.13) — не путь возврата в `draft`

`failing`	`draft`	TC	Деградация теряет runner history; повторная диагностика — через новый прогон (`failing → passing` runner-managed) или `obsolete`

10.12.1 Реакция носителя

При попытке запрещённого перехода носитель обязан:

1. Заблокировать transition (V3 diff & review).
2. Вернуть вызывающему actor код ошибки с указанием конкретного нарушенного правила (по идентификатору строки этой таблицы).
3. Не создавать запись в лог переходов (§10.13).

10.13 Logging gate-passing events

10.13.1 Нормативное требование

Каждое успешное прохождение gate (любого типа: QG-0, QG-1, QG-2, QG-3, QG-4, runner-transition, deprecation, freeze-transition) обязано быть зафиксировано в носителе как immutable событие со следующими полями:

Поле	Семантика	Обязательность
`timestamp`	UTC ISO-8601 момент успешного прохождения	Обязательно
`artifact-id`	Идентификатор артефакта (immutable, V1)	Обязательно
`artifact-type`	`BR` / `SR` / `TR` / `SPEC-<type>` / `ADAPT` / `TC`	Обязательно
`artifact-version`	Версия артефакта на момент перехода (V5)	Обязательно
`from-status`	Исходное состояние	Обязательно
`to-status`	Целевое состояние	Обязательно
`gate-id`	`QG-0` / `QG-1` / `QG-2` / `QG-3` / `QG-4` / `deprecation` / `freeze` / `runner-pass` / `runner-fail` / `change-of-criteria`	Обязательно
`actor`	Идентификатор инициатора (V6); для двойной подписи — список акторов	Обязательно

`evidence-refs`	Ссылки на доказательства: ID прогонов runner, ID adversarial-review артефактов, ID подписей	Обязательно для QG-2 / QG-3 / QG-4
`notes`	Свободный текст	Опционально

10.13.2 Независимый от носителя формат

Формат хранения событий — специфичен для носителя (отдельный лог-стрим / append-only collection / иные формы). Стандарт нормирует только обязательные поля §10.13.1, не их сериализацию.

Conformance manifest обязан указать механизм хранения событий и формат экспорта (для audit-аудита, глава 13).

10.13.3 Retention

События **не удаляются** в течение всего lifecycle артефакта и после его перехода в `deprecated` / `obsolete` / `frozen`. Этого требует V1 (immutable history) и нормативные положения compliance (глава 13).

10.14 Связь с другими главами

Глава	Связь
02	SENAR QG-0..QG-2 — концептуальная основа; RENAR расширяет (§10.2.3)
06	frontmatter и body BR / SR / TR (§6.5–§6.7); state machines детализированы здесь (§10.5–§10.6)
07	frontmatter ADAPT (§7.8); backward sub-states (§7.4.5); двойная подпись (§7.5); delta-ADAPT (§7.6) — state machine здесь (§10.8)
08	frontmatter SPEC (§8.4); type-specific QG (§8.8); state machine здесь (§10.7)
09	frontmatter TC (§9.3); pos/neg pairing (§9.7); change-of-criteria (§9.13); state machine здесь (§10.9)
03	V1–V6 — фундамент enforcement (§10.11); без V3 / V4 / V6 — нет реализации gates
11	Уровни зрелости определяют объём применимых gates (например, RENAR-1 — QG-0 / QG-1 / QG-2 обязательны; на более высоких уровнях усиливаются предусловия QG-2: pos/neg парность, spec-specific TC)

13	Conformance manifest объявляет gate support (§10.4.3); хранит retention policy событий (§10.13.3)

---

title: "Модель зрелости" order: 11 lang: ru

11. Модель зрелости

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

11.1 Зрелость как лестница, а не ярлык

Глава 3 дала фундамент — носитель, способный хранить историю и происхождение требований. Но наличие фундамента ещё не говорит, насколько зрело команда им пользуется. Один проект просто держит требования в носителе; другой валидирует их по schema, гоняет парные тесты и заставляет вторую модель оспаривать первую. Это разные ступени одной лестницы — и эта глава их нумерует.

RENAR определяет **пять уровней зрелости** — `RENAR-1` .. `RENAR-5`, закрытый список. Уровень — это **измеримая характеристика** того, насколько формализовано управление требованиями в проекте. Его важно не путать с **заявлением о соответствии** (conformance): уровень описывает, *где находится* проект, а conformance — это процедура, которой он это *доказывает*. Механика заявления — в главе 13.

Каждая следующая ступень добавляет обязательства поверх предыдущей и закрывает свой класс расхождений. Поэтому читать главу стоит по порядку: §11.2 показывает, как RENAR-уровень соотносится с общей зрелостью по SENAR; §11.3 и пять секций §11.4–§11.8 разбирают ступени по одной; §11.9 сводит их в одну таблицу; §11.10–§11.11 отвечают на вопросы «с чего начать» и «как расти»; §11.12–§11.13 связывают уровни с метриками и остальным стандартом.

Граница главы строгая. Она нормирует **только** критерии уровней и переходы между ними. Процедуры заявления о соответствии — глава 13; метрики измерения уровня — глава 12; сами capabilities носителя — глава 3.

11.2 RENAR-M как предметно-специфичная размерность в модели зрелости SENAR

11.2.1 Модель зрелости SENAR (общая зрелость)

SENAR определяет одномерную модель из пяти уровней общей зрелости команды и методологии: `Стихийный → Супервизируемый → Измеримый → Управляемый → Оптимизирующий`. Эта модель оценивает **процессную зрелость команды в целом**, независимо от конкретной процессной области.

11.2.2 RENAR-M как отдельная размерность

RENAR-M — **отдельная размерность** в общей зрелости по SENAR, специализирующая её для процессной области «инженерия требований». Проект характеризуется **парой**:

```
проект ──▶ (SENAR-N, RENAR-M)

где N ∈ {1, 2, 3, 4, 5} — общая SENAR зрелость
    M ∈ {1, 2, 3, 4, 5} — RENAR-уровень управления требованиями
```

Пары `(SENAR-N, RENAR-M)` нормативно независимы: проект на SENAR-4 (Управляемый) может находиться на RENAR-2 (Зафиксированный, Documented) — команда зрелая в SENAR-практиках в целом, но **именно управление требованиями** формализовано слабо. Это допустимый и наблюдаемый сценарий.

11.2.3 Согласованность с SENAR Reference

SENAR Reference допускает предметно-специфичные размерности зрелости (аутентификация, безопасность, наблюдаемость и иные процессные области). RENAR-M — одна из таких размерностей; она не противоречит SENAR и не дублирует его.

В индустрии типична ситуация, когда в одной организации разные процессные области имеют различную зрелость. RENAR-M — нормативная размерность зрелости именно для области «инженерия требований», с критериями только из этой главы и смежных параграфов стандарта.

11.2.4 Соответствие как пара

Conformance manifest (§13.4) фиксирует **только** RENAR-M (поле `level`). SENAR-N остаётся в области SENAR conformance и в RENAR-манифесте не дублируется.

11.3 Закрытый список уровней RENAR-1..RENAR-5

Список из пяти уровней закрыт. Изменение списка возможно только через формальную процедуру изменения стандарта (§13.9.3).

Уровень	Краткое название	Семантика (одной строкой)
`RENAR-1`	Стихийный (Ad-hoc)	Носитель требований существует; артефакты ведутся без формального schema и lifecycle
`RENAR-2`	Зафиксированный (Documented)	Артефакты имеют базовый frontmatter и хранятся структурно; T3 зафиксировано
`RENAR-3`	Учитываемый (Tracked)	Полная schema frontmatter; lifecycle-статусы используются; delta-T3 workflow через ADAPT
`RENAR-4`	Верифицируемый (Verified)	100% `approved` имеют `verified-by`; pos/neg парность; QG-2 enforced; AI-provenance
`RENAR-5`	Оптимизирующий (Optimized)	Adversarial review как gate; несколько моделей для `priority: must`; граф знаний; метрики

Промежуточные уровни (`RENAR-3.5`) и project-local уровни запрещены (§13.9.2). Локальное ужесточение критериев в пределах объявленного уровня разрешено через `declared-stricter` (§13.4.2).

Каждый уровень включает нормативные критерии всех нижестоящих. `RENAR-M` подразумевает выполнение требований `RENAR-1` .. `RENAR-(M-1)` .

11.4 RENAR-1: Стихийный (Ad-hoc)

Первая ступень отвечает на один вопрос: **где живут требования?** На RENAR-1 ответ — «в носителе, а не в чьей-то голове, трекере задач или переписке». Формального schema и lifecycle ещё нет, но происхождение уже восстановимо: на вопрос «откуда это требование» отвечает носитель, а не память участника. Это минимум, ниже которого говорить о соответствии RENAR бессмысленно.

11.4.1 Нормативное определение

`RENAR-1` — нижний conformant уровень. Проект обязан удовлетворять:

- Носитель требований существует физически (V1–V6 (§13.3.2) обеспечены — это абсолютное mandatory clause независимо от уровня).

- Требования ведутся как артефакты внутри носителя требований (не только в трекерах задач или личной переписке).
- Существует ADAPT для каждого ТЗ (§13.3.3) — это mandatory clause, не уровневый.
- Применяется независимый от носителя язык для нормативных описаний (§2.5.4); специфичные для носителя детали выносятся в `guide/`.

> ***RENAR-1 ≠ нулевая инфраструктура.*** *«Стихийный» здесь — про отсутствие формального schema / lifecycle, а не про носитель: V1–V6 обязательны на любом уровне. Плоский файловый сервер, Notion, Google Docs или Confluence без immutable history / atomic change / version pin* ***не квалифицируются*** *даже на RENAR-1. Минимальный квалифицирующий носитель — distributed VCS или document-oriented store с V1–V6 (§3, guide/03–04).*

11.4.2 Что НЕ требуется на RENAR-1

- Стандартизированный frontmatter артефактов (допустим минимальный `id` + `title`).
- Lifecycle-статусы (`draft` / `approved` / `verified`) — артефакты могут жить без явных переходов.
- TC как отдельные артефакты — тесты допустимо вести в коде.
- COVERAGE — отсутствует.
- Нативные для носителя hooks lifecycle — не обязательны.

11.4.3 Наблюдаемые сигналы

- Артефакт-файлы BR / SR / ADAPT существуют в носителе (не в трекере, не в чате).
- На запрос «откуда это требование» ответ приходит через носитель, а не через память участника.
- Conformance manifest (§13.4) выпущен с `level: RENAR-1`.

11.4.4 Применение QG (enforcement)

QG-0/QG-1/QG-2 нормативно объявлены как required (§13.3.6), но enforcement через нативные для носителя hooks **не обязателен** на RENAR-1. Проверка на приёмке выполняется человеком вручную по мере необходимости.

11.5 RENAR-2: Зафиксированный (Documented)

RENAR-2 добавляет к существованию **структуру**. У каждого требования появляется базовый frontmatter, ТЗ зафиксировано как неизменяемый артефакт, у артефактов есть предсказуемое место. Требование теперь не просто существует — его можно найти, процитировать и сослаться на него. Машинной проверки ещё нет; дисциплину держат люди.

11.5.1 Нормативное определение

Поверх RENAR-1 проект обязан удовлетворять:

- Каждый артефакт BR / SR имеет frontmatter с обязательными полями (§6.5.2, §6.6.2) — допустим базовый минимум без строгой schema-валидации.
- ТЗ зафиксировано как immutable артефакт носителя (§7.4.2).
- Существует структурное хранение артефактов (логические папки или нативные для носителя коллекции).
- Дельта-ТЗ оформляется как явный артефакт в носителе, а не устно.

11.5.2 Что НЕ требуется на RENAR-2

- CI-валидация frontmatter по schema (`requirement-schema.md`).
- Полный lifecycle (статусы используются по желанию, не нормативно обязаны).
- Покрытие через ТС — расширения `tc-type` необязательны.
- Pos/neg парность ТС.

11.5.3 Наблюдаемые сигналы

- Все BR / SR имеют валидный (по обязательному минимуму) frontmatter.
- ТЗ найдено как один нативный для носителя артефакт.
- Дельта-ТЗ присутствует как явный change-set (§7.6).
- Conformance manifest с `level: RENAR-2` .

11.5.4 Применение QG (enforcement)

QG-0 (Approval, §10.3.1) применяется как процедурный gate (явный approval с нативной для носителя фиксацией V6 author + timestamp), но без автоматической блокировки нативными для носителя hooks.

11.6 RENAR-3: Учитываемый (Tracked)

На RENAR-3 включается **машина**. Frontmatter валидируется по schema, статусы lifecycle реально работают, дельта-ТЗ проходит через ADAPT. Здесь впервые появляется автоматический контроль: носитель сам блокирует ссылку на неутверждённый ADAPT и не даёт реализации сослаться на требование без привязки версии. Дисциплину держит уже не только человек.

11.6.1 Нормативное определение

Поверх RENAR-2 проект обязан удовлетворять:

- Frontmatter артефактов валидирован по schema (reference/02-schemas.md) нативным для носителя механизмом (§10.11.1).
- Lifecycle-статусы реально используются: каждый артефакт находится в одном из закрытых состояний (глава 10).
- ТС существуют для всех BR / SR / SPEC с `priority: must`; для `should` / `could` — желательно, но не обязательно.
- COVERAGE — нативный для носителя авто-генерируемый артефакт (§9.15) — обновляется при promote-переходах.
- Дельта-ТЗ оформляется как нативный для носителя change-set с явным анализом влияния (impact analysis, §9.16).
- Носитель обеспечивает reference-validation hook (§10.11.1): создание BR / SR / SPEC, ссылающегося на ADAPT в статусе ниже `approved`, автоматически блокируется.
- Каждый артефакт реализационного носителя, ссылающийся на BR / SR / SPEC, обязан привязываться через `verifies[].version` (§9.4, V5).

11.6.2 Наблюдаемые сигналы

- Hook валидации frontmatter возвращает экономный структурированный отклик при нарушении schema.
- Каждый артефакт имеет статус ∈ { `draft`, `approved`, `verified`, `deprecated`, `obsolete` } (§10.5).
- COVERAGE авто-генерируется и обновляется в течение разумного срока от последнего promote-перехода.
- При delta-ТЗ архитектор автоматически видит затронутые SR / SPEC / TC.
- Conformance manifest с `level: RENAR-3`.

11.6.3 Применение QG (enforcement)

QG-0 (Approval) и QG-1 (Implementation) enforced нативно для носителя. QG-2 (Verification) может быть enforced частично — проверка `verifies[]` обязательна, но pos/neg парность (§9.7) не обязана enforced быть автоматически.

11.7 RENAR-4: Верифицируемый (Verified)

RENAR-4 — рубеж **доверия к тестам**. Каждое нормативное утверждение покрыто парой pos/neg TC, QG-2 блокирует перевод в `verified` без зелёных тестов на актуальной версии требования, а спот-чек раз в итерацию ловит тесты, подделанные под код. Это уровень, на котором «тесты зелёные» наконец начинает означать «продукт верный».

11.7.1 Нормативное определение

Поверх RENAR-3 проект обязан удовлетворять:

- 100% артефактов в `approved` имеют `verified-by` со ссылкой на ≥ 1 TC (§9.4).
- Pos/neg парность (§9.7) выполнена для **каждого** нормативного утверждения верифицируемого артефакта (`tc-type: contract` / `security` / `eval` / иной). Single-TC-coverage допустим только для утверждений-инвариантов с негативной семантикой (§9.7 исключение).
- QG-2 (Verification, §10.3.3) **enforced** нативно для носителя: перевод артефакта в `verified` блокируется при отсутствии всех `passing` TC на текущей `requirement-version` .
- Все TC автоматизированы (`automation.status: automated`) либо явно помечены `manual-pending` с дедлайном и обоснованием (§9.5).
- Для `tc-type: ux` соблюдена изоляция судьи VLM (judge isolation, §9.13.4).
- Для `tc-type: eval` модель-судья отличается от модели реализации (Decision #8).
- AI-provenance во frontmatter артефактов по canonical-списку полей §4.10.1: минимум `ai-provenance.generated-by` (идентификатор модели) и `ai-provenance.generated-at` (отдельное поле UTC-времени) обязательны для AI-генерируемых артефактов; полный список полей и метрики стоимости на RENAR-5 — см. §4.10.1 + §11.8.1 (бюджет стоимости и задержки). Schemas frontmatter §6.5.2 / §6.6.2 обязаны включать все поля §4.10.1.

- Цитирование источника в теле артефакта: каждое нормативное утверждение имеет встроенную ссылку на источник в ТЗ или ADAPT (`[TZ-XXX §Y line Z]` или маркер `derived` со ссылкой).
- Hook непрерывной сверки (§2.4.2, continuous reconciliation) запускается нативно для носителя по расписанию (не реже раза в неделю).
- Спот-чек 5 случайных passing TC раз в итерацию (§9.14) фиксируется в audit-trail.

11.7.2 Наблюдаемые сигналы

- COVERAGE содержит метрику `verified-by-percent: 100%` для `approved` артефактов.
- При попытке перевести BR / SR / SPEC в `verified` без всех passing TC носитель возвращает блокирующую ошибку.
- Грубая выборка 5 случайных артефактов показывает цитирование источника на все нормативные утверждения.
- Conformance manifest с `level: RENAR-4` .

11.7.3 Применение QG (enforcement)

QG-0, QG-1, QG-2 enforced нативно для носителя. QG-3 (Architecture, §10.4.1) — declared, если проект использует ADAPT с двойной подписью (по умолчанию для регулируемых отраслей).

11.8 RENAR-5: Оптимизирующий (Optimized)

RENAR-5 замыкает контур **AI-надёжности**. Вторую модель ставят оспаривать первую, критичные требования генерируются несколькими моделями с обязательным разбором расхождений, частота галлюцинаций измеряется и удерживается ниже 1%. На этой ступени стандарт перестаёт быть дисциплиной, которую соблюдают, и становится системой, которая сама себя проверяет.

11.8.1 Нормативное определение

Поверх RENAR-4 проект обязан удовлетворять:

- Adversarial review (состязательная рецензия) — обязательный gate для перехода артефакта `draft → approved` : артефакт обязан пройти рецензирование

второй AI-моделью (отличной от модели генерации) с явной фиксацией согласия или расхождения в audit-trail.

- Согласование нескольких моделей (multi-model agreement) для артефактов с `priority: must`: артефакт генерируется не менее чем двумя моделями; расхождения помечаются маркером `[multi-model-disagreement]` и обязательны к ручному разбору.
- Бюджет стоимости и задержки (cost/latency budget) на каждый артефакт: для каждого AI-генерируемого артефакта зафиксированы `cost-budget` и `latency-budget`; превышение фактических значений запускает автоматическую декомпозицию артефакта на меньшие части.
- Граф знаний (knowledge graph, reference/05-knowledge-graph-schema.md) как первичный поиск: AI-агенты обязаны использовать запросы к графу как первичный источник контекста; полнотекстовый поиск — запасной путь.
- Непрерывная оценка (continuous evaluation) AI-критических компонентов: прогоны оценки по расписанию и при каждом изменении для всех `SPEC-AI` (§8.5.7).
- Метрика **Hallucination Rate** (глава 12) измеряется и поддерживается < 1%.
- Метрика **Multi-model Disagreement Rate** (глава 12) измеряется, тренды отслеживаются.
- Возврат улучшений шаблонов в `requirements-library` через нативный для носителя change-set — стандартная практика команды.

11.8.2 Наблюдаемые сигналы

- Каждый promote-нутый артефакт имеет в audit-trail запись об adversarial review.
- При `priority: must` для BR носитель показывает маркер `[multi-model-agreement]` либо `[multi-model-disagreement]`.
- Дашборд состояния графа знаний — нативный для носителя и доступен.
- Дашборд Hallucination Rate показывает < 1% на скользящем окне.
- Conformance manifest с `level: RENAR-5`.

11.8.3 Применение QG (enforcement)

Все обязательные gates (QG-0, QG-1, QG-2) enforced нативно для носителя. Adversarial review enforced как часть QG-0 (§10.3.1) — носитель блокирует переход `draft → approved` без подтверждения adversarial review. QG-3 (Architecture) — declared. QG-4 (Acceptance) — declared, если проект применяет post-release outcomes (§10.4.2).

11.9 Сравнительная таблица признаков

Всё, что разнесено по пяти секциям выше, в одной матрице. Заполнение: ❌ — не требуется; `частично` — частично; ✓ — нормативно обязательно.

Признак	RENAR-1	RENAR-2	RENAR-3	RENAR-4	RENAR-5
Носитель с V1–V6	✓	✓	✓	✓	✓
ADAPT для каждого T3	✓	✓	✓	✓	✓
Frontmatter стандартизирован	❌	частично	✓	✓	✓
Lifecycle-статусы используются	❌	частично	✓	✓	✓
Schema-валидация frontmatter (hook носителя)	❌	❌	✓	✓	✓
TC как полноценный артефакт	❌	❌	частично	✓	✓
Pos/neg парность для каждого утверждения	❌	❌	❌	✓	✓
COVERAGE авто-генерируется	❌	❌	✓	✓	✓
Reference-validation hook	❌	❌	✓	✓	✓
Привязка `verifies[].version` (V5)	❌	❌	✓	✓	✓
QG-0 enforced нативно для носителя	❌	частично	✓	✓	✓
QG-2 enforced нативно для носителя	❌	❌	частично	✓	✓

AI-provenance во frontmatter	❌	❌	❌	✓	✓
Цитирование источника в теле артефактов	❌	❌	❌	✓	✓
Adversarial review как gate	❌	❌	❌	❌	✓
Согласование нескольких моделей для `priority: must`	❌	❌	❌	❌	✓
Бюджет стоимости и задержки на артефакт	❌	❌	❌	❌	✓
Граф знаний как первичный поиск	❌	❌	❌	❌	✓
Непрерывная сверка	❌	❌	❌	✓ (базовая)	✓ (полная)
Непрерывная оценка (SPEC-AI)	❌	❌	❌	❌	✓
Hallucination Rate < 1%	н/п	н/п	н/п	н/п	измеряется и контролируется
Тренд Multi-model Disagreement Rate	н/п	н/п	н/п	н/п	отслеживается

11.10 Минимальный входной уровень (entry-level)

`RENAR-1` — нормативная **нижняя граница** conformance manifest. Проект, не выполняющий mandatory clauses (§13.3) — в частности, без носителя с V1–V6 или без ADAPT для каждого T3 — **не** имеет уровня RENAR-M вовсе; conformance manifest для такого проекта не выпускается.

Тип проекта	Рекомендуемый целевой уровень
Короткий эксперимент (spike), < 1 спринта, без контракта	RENAR-2 — достаточно для документирования

Внутренняя автоматизация, 1–3 месяца	RENAR-3 — учитываемый lifecycle
Клиентский продукт по договору	RENAR-4 — верифицируемый, с pos/neg парностью
AI-критическая компонента (зависит от eval)	RENAR-5 — обязательно
Регулируемые отрасли (медицина, финтех, госсектор)	RENAR-4 минимум, RENAR-5 рекомендовано

Стандарт допускает **разные уровни в разных проектах** одной организации — требования унифицировать уровень по портфелю нет.

Негативный сценарий: носитель, в котором отсутствует хотя бы одна capability из V1–V6 (§3.2), нормативно не может реализовать никакой RENAR-M — даже `RENAR-1`. Это структурное ограничение, не операционное; манифест такого проекта невалиден (§13.3.2).

11.11 Путь RENAR-1 → RENAR-5

Нормативная последовательность шагов между соседними уровнями. Времена — ожидаемый порядок (для небольшого проекта; это не нормативная гарантия).

11.11.1 RENAR-1 → RENAR-2

1. Создать структурное хранение артефактов в носителе (логические папки или нативные для носителя коллекции).
2. Перенести существующие требования из трекера задач, чата или документов в нативные для носителя артефакты с минимальным frontmatter (`id` , `title`).
3. Зафиксировать T3 как immutable артефакт (§7.4.2).
4. Договориться в команде: новые требования — только через носитель, не через трекер задач.

Ожидаемое время: 1–2 недели для небольшого проекта; 1–2 месяца для проекта с большим накопленным backlog.

11.11.2 RENAR-2 → RENAR-3

1. Привести frontmatter всех артефактов к schema (нативный для носителя нормализатор).
2. Включить hook носителя для schema-валидации (блокировать change-set при нарушении).
3. Внедрить lifecycle: пройтись по всем артефактам, проставить статусы.

4. Включить reference-validation hook (§10.11.1).
5. Сгенерировать первоначальный авто-генерируемый артефакт COVERAGE (§9.15).
6. Создать TC для артефактов с `priority: must`.
7. Внедрить привязку `verifies[].version` из реализационного носителя.

Ожидаемое время: 2–4 недели, включая обучение команды.

11.11.3 RENAR-3 → RENAR-4

1. AI-агент проходит по всем артефактам и генерирует пары pos/neg TC для каждого нормативного утверждения.
2. Реализация TC в коде / SPEC-runners — параллельно с продуктовой работой; растягивается на 1–2 квартала.
3. Включить hook носителя QG-2 (блокировать перевод в `verified` без всех passing TC).
4. Внедрить процесс спот-чека (§9.14).
5. Включить hook непрерывной сверки с еженедельным расписанием.
6. Внедрить AI-provenance во frontmatter (hook носителя блокирует при отсутствии для AI-генерируемых артефактов).
7. Внедрить цитирование источника в теле артефактов (hook носителя блокирует при отсутствии цитаты для нормативных утверждений).

Ожидаемое время: 1–2 квартала.

11.11.4 RENAR-4 → RENAR-5

1. Подключить adversarial review второй модели (отличной от первичной; принцип изоляции судьи §9.13.4); включить как enforcement QG-0.
2. Включить генерацию несколькими моделями для артефактов с `priority: must`.
3. Развернуть граф знаний (reference/05).
4. Настроить прицельные метрики Hallucination Rate и Multi-model Disagreement Rate (глава 12).
5. Внедрить бюджет стоимости и задержки с автоматической декомпозицией при превышении.
6. Закрепить практику возврата улучшений шаблонов в `requirements-library`.
7. Непрерывная оценка для всех артефактов `SPEC-AI` (§8.5.7).

Ожидаемое время: 1–2 квартала после RENAR-4.

11.11.5 Обратные переходы (понижение уровня, downgrade)

Нормативное понижение уровня (§13.8.2) — выпуск новой версии манифеста с уровнем ниже текущего — допустимо при наличии формального обоснования (например, вывод из эксплуатации AI-критической компоненты, упрощение носителя). Понижение обязано сопровождаться записью в audit-trail; скрытие понижения — нарушение mandatory clause (§13.3.1).

11.12 Связь с SENAR ADR (Adversarial Detection Rate)

SENAR §9 определяет ADR — Adversarial Detection Rate — как общую метрику способности процесса обнаружить ошибки до выхода в production. RENAR-M определяет, на каких уровнях существует нормативная инфраструктура adversarial review. RENAR-specific подкласс ADR для зоны требований — **ACR** (Adversarial Catch Rate, §12.3.6).

RENAR-M	Нормативная инфраструктура adversarial review	Измеримость ADR / ACR
RENAR-1, RENAR-2	Отсутствует (§11.4, §11.5)	н/п
RENAR-3	Нормативно отсутствует; команда может ввести adversarial review как локальное ужесточение (`declared-stricter` , §13.4.2)	опционально
RENAR-4	Pos/neg парность TC (§9.7) — структурный прокси ADR; adversarial review артефактов нормативно не требуется	опционально (покрытие pos/neg измеримо как прокси)
RENAR-5	Adversarial review как обязательный gate (§11.8.1) + согласование нескольких моделей для `priority: must`	нормативно измеримо (ACR, §12.3.6)

Зрелость RENAR-M связана с SENAR-метрикой ADR **непрерывно**, без дублирования: SENAR ADR задаёт понятие метрики; уровень RENAR-M определяет, какие состязательные циклы нормативны; ACR (§12.3.6) — предметно-специфичная метрика для зоны требований.

11.13 Связь с другими главами

Модель зрелости — это карта, по которой расставлены требования всех остальных глав: каждая глава «включается» на своём уровне. Таблица ниже показывает, где именно.

Глава	Связь
02 Положение в типологии методологий	§2.3 SoT inversion + §2.5 независимое от носителя версионирование — mandatory clauses независимо от уровня; §2.4 четыре отстройки — наблюдаются на всех уровнях, начиная с RENAR-2
06 Иерархия требований	frontmatter артефактов и обязательные поля проверяются на RENAR-3+; иерархия BR/SR/TR — на всех уровнях
07 ADAPT	ADAPT для каждого T3 — mandatory clause независимо от уровня (§11.4.1); двойная подпись QG-3 — declared на RENAR-4+
08 Specifications	Closed list 9 типов SPEC — mandatory; полное type-specific покрытие на RENAR-4+; непрерывная оценка SPEC-AI на RENAR-5
09 Test cases	TC для `priority: must` на RENAR-3; pos/neg парность на RENAR-4+; процесс спот-чека на RENAR-4+
10 Lifecycle и QG	QG-0/QG-1/QG-2 объявлены required на всех уровнях; enforcement носителя постепенный — частичный на RENAR-2, полный на RENAR-4+; QG-3 declared на RENAR-4+, если используется ADAPT
03 Версионирование носителя	V1–V6 — абсолютное mandatory clause для любого уровня; носитель без V1–V6 не имеет уровня RENAR-M (§11.10)
12 Метрики	метрики Hallucination Rate / Multi-model Disagreement Rate / Defect Escape Rate измеряются на RENAR-4+; конкретные определения — в главе 12
13 Conformance	§13.2 ссылается на эту главу за критериями каждого уровня; §13.5 self-assessment использует checklists §§11.4–12.8 (по одной секции на уровень); §13.9 closed list policy применяется к закрытому списку RENAR-1..5 (§11.3)

---

title: "Метрики" order: 12 lang: ru

12. Метрики

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

12.1 Что измерять в требованиях

SENAR уже измеряет процесс в целом — десять общих метрик (§9). Но «процесс в целом» не показывает, здорова ли именно работа с требованиями: как часто AI-агент придумывает требование без источника, ловят ли парные тесты реальные дефекты, как быстро дельта-Т3 доходит до кода. Эта глава добавляет **десять метрик, специфичных для инженерии требований** — поверх SENAR §9, а не вместо него.

Список метрик закрыт; для каждой задаются формула, цель по уровню зрелости (глава 11) и источник данных. Метрики собираются нативно для носителя через V1–V6 (глава 3); как именно рисовать дашборды — вопрос реализации, вынесенный в `guide/`. Глава не дублирует SENAR §9, а специализирует его, и не касается ROI или ценообразования — это не индикаторы процесса, а бизнес-эффекты (§12.5).

12.2 Связь с SENAR §9

SENAR §9 определяет десять метрик общего процесса: Throughput, Lead Time, FPSR (First-Pass Success Rate), DER (Defect Escape Rate), KCR (Knowledge Capture Rate), Cost Predictability, Cost-per-task, MIR (Memory Integrity Rate), Cycle Time, ADR (Adversarial Detection Rate).

RENAR §12 **не** редактирует и **не** заменяет эти метрики. REQ-специфичные метрики §12.3:

- **Уточняют** SENAR метрику для requirements phase (например, RDLT уточняет SENAR Lead Time на requirements phase).
- **Добавляют** наблюдения, специфичные для requirements engineering и не покрываемые SENAR §9 (Hallucination Rate, Multi-model Disagreement Rate).

Полный mapping — §12.7.

Закрытый список REQ-метрик (§12.3) сохраняется в рамках RENAR; SENAR §9 — отдельный закрытый список общих метрик. Изменение любого из двух списков — формально независимые change procedures соответствующих стандартов.

12.3 Закрытый список REQ-специфичных метрик

Закрытый список из десяти REQ-метрик. Изменение списка — только через formal change procedure стандарта (§13.9.3); общая closed-list политика и master-индекс — §1.7.5.

12.3.1 RDLT — время декомпозиции требования (Requirement Decomposition Lead Time)

Что: время от регистрации ТЗ в носителе до состояния «все BR / SR (parent цепочка из этого ТЗ) находятся в статусе `approved`, готовы для passage QG-0 (§10.3.1)».

Формула: `RDLT = timestamp(last BR/SR → approved) − timestamp(TZ registered)`. Измеряется в часах или днях.

Цели по уровням:

Уровень	Целевое значение
RENAR-3	< 1 неделя на 50-страничный ТЗ
RENAR-4	< 2 дня
RENAR-5	< 4 часа

Источник данных: audit-trail событий promote-transitions (§10.13) носителя.

Связь с SENAR: уточнение SENAR `Lead Time` для requirements phase.

12.3.2 Задержка «требование → задача» (Requirement-to-Task Latency)

Что: время от promote-transition SR в `approved` до создания первой TR со ссылкой `implements: SR-N` на этот SR.

Формула: `Latency = timestamp(first TR.created) − timestamp(SR → approved)`. Измеряется в часах.

Цели по уровням:

Уровень	Целевое значение
RENAR-3	< 3 дня
RENAR-4	< 1 день

RENAR-5	< 1 час (нативный для носителя auto-create TR после approval)

Источник данных: audit-trail носителя + межносительные ссылки реализационного носителя.

Связь с SENAR: уточнение SENAR `Cycle Time` для пары «requirement → executable task».

12.3.3 Доля галлюцинаций (Hallucination Rate)

Что: процент нормативных утверждений в AI-генерируемом артефакте (BR / SR / SPEC), которые не traceable к source (T3 / ADAPT / другой нормативный артефакт носителя). Source citation проверяется нативным для носителя citation parser (§13.3.1 RENAR-4 mandatory).

Формула: `Hallucination Rate = assertions_without_valid_citation / total_normative_assertions × 100%`. Измеряется per артефакт; агрегируется per project.

Цели по уровням:

Уровень	Целевое значение
RENAR-1, RENAR-2, RENAR-3	n/a (citation parser не обязателен)
RENAR-4	≤ 5%
RENAR-5	≤ 1%

Источник данных: нативный для носителя citation parser (AST-уровневый или regex по inline references вида `[TZ-XXX §Y]` / `[ADAPT-NNN §Z]`).

Связь с SENAR: новая метрика; SENAR не имеет аналога. Соответствует ISO/IEC 5338 «traceability requirement» для AI-generated artifacts.

Negative scenario (loss-of-conformance trigger): Hallucination Rate > 5% на RENAR-4 проекте — нормативный trigger loss of conformance (§13.8.1); проект обязан либо устранить нарушение через release recovery plan, либо формально downgrade до RENAR-3.

12.3.4 Доля расхождений multi-model (Multi-model Disagreement Rate)

Что: процент артефактов с `priority: must`, где две (или более) генерирующие AI-модели произвели нормативные утверждения с расхождением выше порога (по

умолчанию: embedding similarity < 85% на корпусе утверждений артефакта; порог фиксируется в conformance manifest `declared-stricter`).

Формула: `Disagreement Rate = BRs_with_high_disagreement / total_must_BRs × 100%`.

Цели по уровням:

Уровень	Целевое значение
RENAR-1..RENAR-4	n/a
RENAR-5	tracked; baseline устанавливается per-project в первом квартале

Источник данных: нативный для носителя multi-model runs log; embedding-similarity computed offline по парам «artifact from model A vs from model B».

Связь с SENAR: новая метрика; не покрывается SENAR §9.

Интерпретация: высокий disagreement rate — индикатор либо слабого prompt-engineering (нужно улучшить), либо реально сложного domain (нужно увеличить human review). Не сам по себе негатив — требует attention.

12.3.5 DRA — частота споров на приёмке (Dispute Rate at Acceptance)

Что: процент BR / SR, по которым на этапе passage QG-4 (§10.4.2) клиент заявил несогласие с интерпретацией или фактом покрытия требования.

Формула: `DRA = disputed_BRs_at_QG4 / total_BRs_in_release × 100%`.

Цели по уровням:

Уровень	Целевое значение
RENAR-3	≤ 10%
RENAR-4	≤ 5%
RENAR-5	≤ 2%

Источник данных: audit-trail QG-4 (§10.13) носителя — `gate-id: QG-4` event с `result: disputed` маркером.

Связь с SENAR: уточнение SENAR `DER (Defect Escape Rate)` для зоны requirements (defect = mismatch между требованием и ожиданием клиента).

Применимость: метрика применима только в проектах, использующих QG-4 (Acceptance Gate). При QG-4 = `absent` в conformance manifest (§13.3.6) — метрика не измеряется.

12.3.6 ACR — доля adversarial-находок (Adversarial Catch Rate)

Что: процент артефактов (BR / SR / SPEC), где AI-критик (вторая модель, отличная от primary; обязательна на RENAR-5 per §11.8.1) нашёл ≥ 1 high-severity issue до passage QG-0.

Формула: `ACR = artifacts_with_critic_high_findings / total_reviewed_by_critic × 100%`.

Цели по уровням:

Уровень	Целевое значение
RENAR-1..RENAR-3	n/a (adversarial critic не обязателен)
RENAR-4	optional (критик не нормативно обязателен на RENAR-4 per §11.7); если внедрён через `declared-stricter` (§13.4.2) — baseline 20–30% (значения ниже 20% — индикатор слабого критика или дублирования primary)
RENAR-5	tracked normatively; рост ACR требует attention к качеству prompts primary модели

Источник данных: audit-trail critic-runs носителя; severity classification (`high` / `medium` / `low`) фиксируется в critic output.

Связь с SENAR: подкласс SENAR `ADR (Adversarial Detection Rate)` для зоны requirements.

12.3.7 Расхождение версии теста и требования (Test-spec Drift Rate)

Что: процент TC в статусе `passing` (§10.9.1), у которых `last-run.requirement-version` (§9.12) отличается от текущей `version` верифицируемого артефакта (`verifies[]`).

Формула: `Drift Rate = stale_passing_TCs / total_passing_TCs × 100%`. Stale = `last-run.requirement-version` < текущей.

Цели по уровням:

Уровень	Целевое значение
RENAR-1..RENAR-3	n/a
RENAR-4	≤ 5%
RENAR-5	≤ 1% (нативный для носителя auto-rerun на delta-ADAPT)

Источник данных: COVERAGE-artifact (§9.15) auto-generated носителя.

Связь с SENAR: новая метрика; не покрывается SENAR §9.

12.3.8 Скорость покрытия верификацией (Coverage Velocity)

Что: темп перехода `approved` артефактов в `verified` (§10.5) за единицу времени (по умолчанию — за итерацию; носитель может определить другой интервал — фиксируется в conformance manifest проекта).

Формула: `Velocity = (verified_count(end) − verified_count(start)) / approved_count(start) × 100%`. Измеряется за выбранный интервал.

Цели по уровням:

Уровень	Целевое значение
RENAR-3	≥ 30% за итерацию
RENAR-4	≥ 50% за итерацию
RENAR-5	≥ 70% за итерацию

Источник данных: COVERAGE-artifact history (§9.15) носителя.

Связь с SENAR: уточнение SENAR `Throughput` для requirements phase.

12.3.9 Стоимость на одно утверждённое требование (Cost per Approved Requirement)

Что: стоимость AI-генерации (input tokens + output tokens × tariff модели) приведённая к одному артефакту в статусе `approved`, включая отвергнутые версии (которые остаются в носителе за счёт V1 immutable history).

Формула: `Cost-per-Requirement = total_AI_tokens_cost(period) / count(artifacts_approved_in_period)`. Измеряется в валюте проекта.

Цели по уровням:

Уровень	Целевое значение
RENAR-1..RENAR-3	n/a (затраты не отслеживаются обязательно)
RENAR-4	tracked; baseline устанавливается per-project
RENAR-5	tracked + optimization target (year-over-year снижение либо обоснование)

Источник данных: `ai-provenance.cost-budget` и `ai-provenance.cost-actual` поля frontmatter артефактов (§11.7.1) носителя.

Связь с SENAR: уточнение SENAR `Cost-per-task` для requirements phase.

12.3.10 Находки согласования за неделю (Reconciliation Findings per Week)

Что: количество issues, обнаруженных reconciliation-агентом (§2.4.2 continuous reconciliation) за неделю и зарегистрированных как backward findings в delta-ADAPT либо как direct change-set requirement.

Формула: `Findings/Week = count(reconciliation_findings_registered) / weeks_in_period`.

Цели по уровням:

Уровень	Целевое значение
RENAR-1..RENAR-3	n/a (reconciliation hook не обязателен)
RENAR-4	tracked; не ноль (если ноль — reconciliation hook не работает либо носитель потерял V5)

RENAR-5	trending **down** в долгосрочной перспективе (зрелый процесс не порождает новых drift)

Источник данных: audit-trail reconciliation runs носителя; backward findings list ADAPT (§7.4.5).

Связь с SENAR: новая метрика; не покрывается SENAR §9.

12.4 Сводная таблица целей по уровням

Закрытый список 10 метрик из §12.3 — целевые значения по применимым уровням.

Метрика	RENAR-3	RENAR-4	RENAR-5	Источник данных
RDLT (Decomposition Lead Time)	< 1 неделя	< 2 дня	< 4 часа	promote-transitions audit-trail
Requirement-to-Task Latency	< 3 дня	< 1 день	< 1 час	audit-trail + межносительные refs
Hallucination Rate	n/a	≤ 5%	≤ 1%	citation parser
Multi-model Disagreement Rate	n/a	n/a	tracked	multi-model runs log
DRA (Dispute Rate at Acceptance)	≤ 10%	≤ 5%	≤ 2%	QG-4 audit-trail (если QG-4 declared)
ACR (Adversarial Catch Rate)	n/a	optional (если critic declared-stricter)	tracked normatively	critic-runs audit-trail
Test-spec Drift Rate	n/a	≤ 5%	≤ 1%	COVERAGE-artifact
Coverage Velocity	≥ 30%/ итерация	≥ 50%/ итерация	≥ 70%/ итерация	COVERAGE history
Cost per Approved Requirement	n/a	tracked	tracked + optimized	ai-provenance fields
Reconciliation Findings/Week	n/a	tracked (> 0)	trending down	reconciliation audit-trail

Конкретное нативное для носителя представление этих метрик в dashboards — специфично для носителя и выносится в `guide/`.

> *Целевые значения для RENAR-4 / RENAR-5 — **provisional**: заданы как нормативные ориентиры направления, но подлежат калибровке по field-data в версии v1.1 (см. guide/07 §8.1). Это направляющие пороги, а не статистически валидированные значения.*

12.5 Бизнес-результаты (Business outcomes)

Шесть нормативных эффектов внедрения RENAR, ожидаемых на уровнях RENAR-3 и выше. Outcomes являются **нормативными ожиданиями стандарта**, не индикаторами процесса; их измерение — специфично для носителя и не обязательно (хотя §12.3 метрики косвенно их фиксируют).

> *ROI / cost-of-adoption — **ненормативная** тема и в нормативную главу не входит. Облегчённая **иллюстративная** (не гарантированная) модель стоимости и выгоды внедрения для лица, принимающего решение, — в guide/02-transition-guide.*

12.5.1 Результат 1 — Сокращение времени декомпозиции T3

Измеряется через §12.3.1 RDLT. Ожидаемое сокращение от baseline (без RENAR): порядок 5–10× на RENAR-4, 20–50× на RENAR-5.

12.5.2 Результат 2 — Снижение частоты споров на приёмке

Измеряется через §12.3.5 DRA. Ожидаемое снижение от baseline: с 15–30% до ≤ 5% на RENAR-4, ≤ 2% на RENAR-5.

12.5.3 Результат 3 — Аудиторская готовность

Audit-trail событий (§10.13) носителя + AI-provenance во frontmatter (§11.7.1) обеспечивают возможность compliance audit без отдельной подготовительной работы. Применимо для regulated industries (медицина, финтех, госсектор).

12.5.4 Результат 4 — Снижение стоимости работы с клиентскими delta-T3

Нативный для носителя impact analysis (§9.16) + reverse-эволюция верификации (§10.5.4) автоматизируют обработку delta-T3. Ожидаемое сокращение human attention: с десятков часов до часа на delta.

12.5.5 Результат 5 — Снижение потери знаний при смене команды

V6 author + timestamp (§3.3.6) фиксирует author всех артефактов и решений; SoT inversion (§2.3.1) переносит knowledge из головы в носитель. Ожидаемое сокращение onboarding нового члена команды: с недель до дней.

12.5.6 Результат 6 — Стандарт как продаваемый продукт

При достижении RENAR-4 / RENAR-5 нативная для носителя реализация стандарта может быть лицензирована / продана партнёрам как самостоятельный actionable продукт. Этот outcome — не процессная метрика, а структурное следствие formal standard.

12.6 Независимый от носителя сбор метрик

12.6.1 Нормативные требования

Носитель, реализующий RENAR на уровне RENAR-4 и выше, обязан обеспечить **автоматический сбор** §12.3 метрик через:

Источник	Опора на capabilities	Что обязано быть accessible
Audit-trail событий (§10.13)	V1 + V6	List of gate-passage events с timestamps, artifact-version, actor
COVERAGE-artifact (§9.15)	V5 + V1	Counts: approved / verified / total; pos/neg coverage percent; stale-rate
AI-provenance frontmatter fields	V6	cost-budget, cost-actual, generated-by (model id), generated-at
Reconciliation audit-trail	V1 + V6	List of reconciliation runs с findings list ID
Critic-runs audit-trail (RENAR-5)	V1 + V6	List of critic runs с severity classifications

Носитель, не обеспечивающий доступ к любому из источников выше, **не** может реализовать RENAR-4 или RENAR-5 (§11.7.1, §11.8.1).

12.6.2 Нативные для носителя панели мониторинга (dashboards)

Формат dashboards (UI / CLI / report-generation) — специфичен для носителя. Стандарт не нормирует визуализацию метрик. Носитель обязан экспортировать метрики в machine-readable формате (нативном для носителя) для целей внешнего аудита (§13.6 third-party assessment).

Конкретные шаблоны dashboard выносятся в `guide/` для каждого поддерживаемого носителя.

12.6.3 Агрегация по периодам (period aggregation)

Метрики §12.3 агрегируются:

- **Per артефакт** — Hallucination Rate, Cost per Approved Requirement.
- **Per period (sprint / неделя / месяц)** — Coverage Velocity, Reconciliation Findings per Week, ACR.
- **Per release** — DRA.
- **Continuous trending** — Multi-model Disagreement Rate, Test-spec Drift Rate.

Конкретные period boundaries фиксируются в conformance manifest (§13.4.2) `declared-stricter` или принимаются по default стандарта.

12.7 Сопоставление с метриками SENAR (mapping)

Полное сопоставление десяти метрик SENAR §9 с REQ-уточнениями из §12.3.

SENAR метрика (§9)	REQ-уточнение из §12.3
Throughput	+ Coverage Velocity (§12.3.8) — на уровне требований
Lead Time	+ RDLT (§12.3.1) — для requirements phase
FPSR (First-Pass Success Rate)	+ REQ-FPSR (доля артефактов, прошедших QG-0 без переделки) — производное, не отдельная метрика §12.3
DER (Defect Escape Rate)	+ DRA (§12.3.5) — defects на приёмке
KCR (Knowledge Capture Rate)	(используется как есть; косвенно усиливается через §12.5.5 результата)
Cost Predictability	+ variance Cost per Approved Requirement (§12.3.9)
Cost-per-task	+ Cost per Approved Requirement (§12.3.9) — для requirements phase

MIR (Memory Integrity Rate)	(используется как есть; усиливается через V1 + V6 на RENAR-4+)
Cycle Time	+ RDLT (§12.3.1) + Requirement-to-Task Latency (§12.3.2) — оба внутри SENAR Cycle Time
ADR (Adversarial Detection Rate)	+ ACR (§12.3.6) — adversarial для requirements зоны

Метрики, **не имеющие** SENAR аналога (новые в RENAR):

- Hallucination Rate (§12.3.3) — специфична для AI-generated artifacts.
- Multi-model Disagreement Rate (§12.3.4) — специфична для multi-model генерации.
- Test-spec Drift Rate (§12.3.7) — специфика requirement-version pinning V5.
- Reconciliation Findings per Week (§12.3.10) — специфика continuous reconciliation.

12.8 Связь с другими главами

Глава	Связь
02 Положение в типологии методологий	§2.3 SoT inversion + §2.4.2 continuous reconciliation — фундамент для §12.3.10 Reconciliation Findings
07 ADAPT	§7.4.5 backward findings — input для §12.3.10; delta-ADAPT — измеряется через §12.3.5 DRA
08 Specifications	§8.5.7 SPEC-AI continuous evaluation — связана с §12.3.4 Multi-model Disagreement Rate
09 Test cases	§9.12 last-run — input для §12.3.7 Drift Rate; §9.15 COVERAGE — источник для §12.3.8 Velocity и §12.3.7
10 Lifecycle и QG	§10.13 audit-trail событий — основа для всех §12.3 метрик; §10.4.2 QG-4 — gate, на котором фиксируется §12.3.5 DRA
03 Версионирование носителя	V1 + V5 + V6 — capabilities обязательные для нативного для носителя сбора §12.3 метрик (§12.6.1)
11 Модель зрелости	§12.3 цели по уровням RENAR-3/4/5 — конкретизация уровневых критериев из §11.4–§11.8
13 Соответствие	§12.3 метрики — вход для §13.5 самооценки; превышение порогов (например, Hallucination Rate > 5% на RENAR-4) — триггер потери соответствия §13.8.1

title: "Соответствие (conformance)" order: 13 lang: ru

13. Соответствие

> ***Часть RENAR Standard v1.0-draft*** *· ← Оглавление*

> ***Плотная глава:*** *начните с reference/08 kit; MVR↔§13.3 bijection — там же §1.*

13.1 Как доказать соответствие

Глава нормирует правила **заявления о соответствии** (conformance) к RENAR: кто, как и на каком основании может заявить, что проект реализует RENAR на уровне `RENAR-N`. Глава фиксирует:

- **Закрытый список уровней RENAR-1..RENAR-5** (§13.2); детальные критерии каждого уровня — в главе 11.
- **Mandatory clauses** (§13.3) — нормативные положения, обязательные для **любого** заявления о соответствии (conformance) независимо от уровня; нарушение хотя бы одного — отсутствие conformance.
- **Conformance manifest** (§13.4) — обязательный артефакт-декларация в носителе проекта; формат, обязательные поля, пример.
- **Самооценка** (§13.5, self-assessment) и **оценка третьей стороной** (§13.6, third-party assessment, опционально) — две процедуры подтверждения уровня.
- **Периодичность повторной оценки** (§13.7, re-assessment cadence) и **потеря соответствия** (§13.8, loss of conformance) — нормативный жизненный цикл manifest.
- **Политика закрытого списка уровней RENAR-N** (§13.9) — запрет project-local расширений списка уровней; путь добавления нового уровня через formal change procedure стандарта.

Маршрут читателя: §13.2 (семантическая сводка уровней) → §13.3 (mandatory clauses) → §13.4 (схема manifest) → §13.5–§13.6 (процедуры оценки) → §13.7–§13.8 (cadence и потеря conformance) → §13.9 (политика уровней) → §13.10 (связи).

Глава **не** определяет критерии каждого уровня RENAR-1..5 — это область главы 11. Глава **не** определяет нативные для носителя механизмы реализации проверок — это область главы 3 (capabilities) и `guide/06-compliance.md` (специфичный для носителя compliance).

13.2 Закрытый список уровней RENAR

Закрытый список из пяти уровней зрелости. Полные критерии — глава 11; ниже — нормативная **семантическая сводка** для заявления о соответствии.

Уровень	Краткая семантика	Статус conformance
`RENAR-1`	Ad-hoc (стихийный уровень): артефакты требований ведутся в носителе с V1–V6 (§13.3.2); без формального frontmatter schema, без lifecycle статусов	Минимальный входной уровень (§13.2.1)
`RENAR-2`	Documented (зафиксированный артефактами): носитель требований существует; артефакты имеют базовый frontmatter; T3 зафиксировано	Conformance декларируем (§13.2.2)
`RENAR-3`	Tracked (ведётся жизненный цикл и учёт связей): полная frontmatter schema; lifecycle статусы используются; delta-T3 workflow через ADAPT	Conformance декларируем (§13.2.2)
`RENAR-4`	Verified (верифицируемый): 100% `approved` артефактов имеют `verified-by` ; pos/neg парность (§9.7); QG-2 enforced; AI-provenance	Conformance декларируем (§13.2.2)
`RENAR-5`	Optimized (оптимизирующий): adversarial critic; multi-model генерация; knowledge graph; Hallucination Rate measured	Conformance декларируем (§13.2.2)

13.2.1 RENAR-1 как минимальный входной уровень (entry-level)

`RENAR-1` — нормативный входной уровень. Проект, в котором отсутствует носитель требований (требования живут только в ticket-системах или личной переписке), **не** заявляется как `RENAR-1` ; заявление соответствия стандарту начинается с фактического наличия носителя требований.

`RENAR-1` фиксируется в conformance manifest только если проект явно декларирует начало пути в RENAR; для проектов без намерения внедрять RENAR conformance manifest не требуется.

13.2.2 Закрытость списка

Новые уровни (`RENAR-6` , `RENAR-N+`) **не** создаются project-local. Изменение списка уровней — только через formal change procedure стандарта (§13.9).

Реализация может ужесточить требования относительно уровня (declare-stricter — см. §10.10.2); это не меняет уровень и не выводит проект за пределы closed list.

13.3 Обязательные положения (mandatory clauses), универсальные для всех уровней

Нормативные положения, обязательные для **любого** заявления о соответствии независимо от объявленного уровня (включая `RENAR-1`). Нарушение хотя бы одного — отсутствие conformance к стандарту RENAR в целом.

13.3.1 Инверсия источника истины (SoT inversion)

Реализация обязана соблюдать **Source of Truth inversion** (§2.3): иерархия артефактов требований — источник истины о поведении системы; код — производный артефакт реализации.

Эквивалентные нарушения:

- Reverse-engineering поведения из кода в SR без обоснования bug-fix (§2.3.3 (1)).
- Молчаливая адаптация SR под наблюдаемое поведение кода (§2.3.3 (4)).

13.3.2 Возможности носителя V1–V6

Носитель проекта обязан удовлетворять возможностям V1–V6 (глава 3 §3.3):

Capability	Статус для conformance
V1 — неизменяемая история	**Обязательно абсолютно**: без V1 невозможен audit trail и V5
V2 — атомарная единица изменения	**Обязательно абсолютно**: без V2 невозможна согласованность delta-ADAPT
V3 — сравнение различий и рецензирование	**Обязательно абсолютно**: без V3 нет gates, нет approval (§10.11.2)
V4 — ветвление / набор изменений	**Обязательно абсолютно**: без V4 неотделимы WIP и SoT (§10.11.2)
V5 — сквозная фиксация версии между носителями	**Обязательно абсолютно**: без V5 невозможен `verifies[].version` и QG-2 (§10.3.3)

V6 — автор и отметка времени	**Обязательно абсолютно**: без V6 невозможна двойная подпись ADAPT, AI-provenance (§10.11.2)

Negative scenario: носитель, в котором отсутствует хотя бы одна capability из V1–V6, нормативно **не** может реализовать никакой RENAR-N — включая `RENAR-1`. Это структурное ограничение (§3.2), не операционное.

13.3.3 ADAPT для каждого T3

Каждое T3 обязано иметь ровно один корневой ADAPT в статусе `approved` (§7.4.1). Производство BR / SR / SPEC из T3 без прохождения через approved ADAPT — нарушение стандарта; нативные для носителя hooks обязаны блокировать создание таких артефактов (§10.11.1 reference-validation).

При наличии delta-T3 — каждый delta-T3 обязан иметь свой delta-ADAPT (§7.6).

13.3.4 Closed list 9 SPEC types

Тип SPEC обязан принадлежать закрытому списку девяти типов (§8.3): `SPEC-ARCH`, `SPEC-API`, `SPEC-DATA`, `SPEC-INT`, `SPEC-PROC`, `SPEC-UI`, `SPEC-AI`, `SPEC-SEC`, `SPEC-OPS`.

Проект не имеет права создавать новые типы SPEC локально. Изменение списка возможно только через formal change procedure стандарта (§8.3.1, §13.9).

13.3.5 TC pos/neg парность для нормативных утверждений

Каждое нормативное утверждение верифицируемого артефакта (BR / SR / SPEC / TR), охватываемое хотя бы одним TC, обязано иметь парный negative TC (§9.7). Single-TC-coverage допускается только в одном случае: утверждение само описывает negative invariant (например, `SPEC-SEC` STRIDE-категория).

QG-2 (§10.3.3) обязан блокировать promote артефакта в `verified` при отсутствии хотя бы одного парного negative TC.

13.3.6 Закрытый список Quality Gates

Closed list Quality Gates (§10.3, §10.4):

Gate	Статус в conformance manifest
QG-0 — Approval	**Обязательно required**

QG-1 — Implementation	**Обязательно required**
QG-2 — Verification	**Обязательно required**
QG-3 — Architecture (опц.)	`declared` (поддерживается) или `absent`; для проектов с обязательным ADAPT — фактически всегда `declared`, поскольку ADAPT approval оперирует двойной подписью QG-3 (§10.4.1)
QG-4 — Acceptance (опц.)	`declared` или `absent`; при `absent` терминальный статус артефакта — `verified` (без `accepted`)

Локальное создание новых gate-типов запрещено (§10.10.2). Локальное ужесточение pre-conditions canonical gate разрешено (declare-stricter); локальное ослабление — запрещено.

13.3.7 Closed lists backward findings и SPEC-decompositions

- Список категорий backward findings в ADAPT закрыт (§7.4.4) — 7 категорий; добавление новых — formal change procedure (§13.9).
- Closed list типов SPEC дополнительно фиксирует §13.3.4.
- Closed list lifecycle-состояний артефактов (глава 10) — не расширяется project-local.

13.4 Манифест соответствия (Conformance manifest)

13.4.1 Расположение и формат

Conformance manifest хранится в корне носителя требований проекта под именем `RENAR-CONFORMANCE.yaml`. Формат — YAML 1.2; альтернативная сериализация (`.json`) допустима как **дополнительный** артефакт, не замена.

Manifest является **immutable** в смысле V1: каждое заявление о соответствии создаёт новую версию manifest (`manifest-version` инкрементируется); предыдущие версии не удаляются, остаются в носителе как audit trail. Замена manifest через `replaced-by` указывает на новую версию.

13.4.2 Обязательные поля

```yaml
# RENAR Conformance Manifest schema (mandatory fields, v1.0)
renar-version: "1.0"           # версия стандарта RENAR, к которой заявляется conformance
senar-version: "1.0"           # версия SENAR, на которую опирается реализация (обязательно, MVR-7)
manifest-version: 3            # инкрементируется при каждом обновлении; не пере-используется
manifest-id: "CFM-2026-001"    # стабильный нативный для носителя идентификатор (V1)

# Уровень conformance
level: "RENAR-3"               # из closed list §13.2 (RENAR-1..RENAR-5)
level-target: "RENAR-4"        # опционально: следующий целевой уровень (для path planning)

# Assessment metadata
assessment-mode: "self"        # self | third-party
assessment-date: "2026-05-13"  # ISO-8601 даты завершения текущей оценки
assessor:
  id: "architect-andrey-y"     # V6 author identifier
  role: "architect"            # роль из §13.5 (architect | authorized-role-holder | external-assessor)
  signature-ref: "<нативный для носителя pointer на signature event>"
next-assessment-due: "2026-08-13"  # §13.7

# Mandatory clauses подтверждение (§13.3)
mandatory-clauses-confirmed:
  sot-inversion: true                              # §13.3.1
  substrate-v1-v6: { v1: true, v2: true, v3: true, v4: true, v5: true, v6: true }   # §13.3.2
  adapt-per-tz: true                               # §13.3.3
  spec-types-closed-list: true                     # §13.3.4
  tc-pos-neg-pairing: true                         # §13.3.5
  quality-gates-closed-list: true                  # §13.3.6
  closed-lists-backward-findings: true             # §13.3.7

# Quality gates declaration (§10.4.3)
quality-gates:
```

```yaml
  qg-0: required               # required (обязательно)
  qg-1: required
  qg-2: required
  qg-3: declared               # required | declared | absent
  qg-4: absent

# Декларация возможностей носителя (§13.3.2)
substrate-capabilities:
  v1-immutable-history: declared
  v2-atomic-change-unit: declared
  v3-diff-review: declared
  v4-branching: declared
  v5-version-pin: declared
  v6-author-timestamp: declared
  substrate-id: "<нативный для носителя pointer на guide/03..06>"   # cross-ref на guide

# Spec types support (§13.3.4)
spec-types-supported: ["SPEC-ARCH", "SPEC-API", "SPEC-DATA", "SPEC-INT",
                       "SPEC-PROC", "SPEC-UI", "SPEC-AI", "SPEC-SEC", "SPEC-OPS"]
# Все 9 типов обязательны как minimum-supported; declaration «type X не используется в проекте» допустима,
# но НЕ может быть declaration «type X не поддерживается нативно для носителя».

# Опциональные поля
declared-stricter:             # §13.2.2, §10.10.2 — локальное ужесточение
  - clause: "QG-2"
    override: "required-negative-tc-per-clause"
    rationale: "regulated industry, double safety margin"
  - clause: "tc-pos-neg-pairing"
    override: "100% (без single-TC исключения для security)"
    rationale: "обязательное требование внутреннего ISMS"

exceptions: []                 # заявленные исключения относительно baseline уровня (с обоснованием в audit trail)

# Внешние записи conformance к §14.4 (опционально; поле значения см. ключ `claim` ниже — носитель §14.6)
```

```yaml
external-claims:
  - standard: "ISO/IEC 5338:2023"
    clause-ref: "§2.4.x"
    claim: "partial"           # full | partial | aligned
  - standard: "NIST AI RMF 1.0"
    clause-ref: "§2.4.x"
    claim: "aligned"

replaced-by: null              # нативный для носителя pointer на следующую версию manifest, если выпущена
replaces: "CFM-2026-001@v2"    # нативный для носителя pointer на предыдущую версию manifest
```

13.4.3 Семантика полей

- `renar-version` — версия стандарта; conformance заявляется к конкретной точке стандарта; при выпуске minor-версии стандарта (§13.9) — обязателен re-assessment (§13.7) с обновлением `renar-version`.
- `senar-version` — версия SENAR, на которую опирается реализация; обязательна по MVR-7 (§0.5.1); её отсутствие в manifest — non-conformant (§13.8).
- `manifest-id` — V1 immutable identifier; не переиспользуется даже после `replaced-by`.
- `level` — закрытый список §13.2; нарушение mandatory clauses (§13.3) — conformance отсутствует независимо от `level`.
- `level-target` — необязательное декларирование пути развития; не является нормативным обязательством.
- `assessment-mode` — `self` (§13.5) или `third-party` (§13.6); `third-party` обязан содержать формальную ссылку на внешний assessment-act.
- `assessor` — V6 author + role; для `third-party` — внешний actor с явной идентификацией.
- `next-assessment-due` — `assessment-date + cadence` (§13.7); просрочка — trigger loss of conformance (§13.8).
- `quality-gates` — обязательно содержит все пять gate-ids; статус `qg-0..qg-2` ∈ {required}; `qg-3, qg-4` ∈ {required, declared, absent}.

- `mandatory-clauses-confirmed` — каждое поле `true` обязательно для любого заявления о соответствии; `false` — manifest невалиден.
- `declared-stricter` — список локальных ужесточений; обязан содержать `clause`, `override`, `rationale`.
- `exceptions` — список заявленных исключений относительно baseline уровня; каждое исключение требует обоснования в audit trail и не может затрагивать mandatory clauses.
- `external-claims` — опциональный список записей соответствия внешним нормативным ссылкам (§14.4, §14.6); каждая запись содержит `standard`, `clause-ref`, поле **`claim`** (значение: full | partial | aligned) — технический ключ схемы, без замены термина в manifest.

13.5 Процедура самооценки (Self-assessment)

13.5.1 Участник (actor)

Self-assessment проводится **архитектором** проекта или **authorized role-holder** (глава 5 роли), явно объявленным как ответственный за conformance.

13.5.2 Методология

1. **Checklist по §13.3** — actor проверяет каждую mandatory clause; результат фиксируется в `mandatory-clauses-confirmed`. Хотя бы одно `false` — assessment не завершается, manifest не выпускается; формируется план остранения нарушений.
2. **Checklist по главе 11, §§11.4–12.8** — для объявленного `level`: §11.4 (RENAR-1), §11.5 (RENAR-2), §11.6 (RENAR-3), §11.7 (RENAR-4), §11.8 (RENAR-5). Каждый критерий уровня — отдельная проверка с фиксацией evidence в носителе.
3. **manifest заполнение** — все обязательные поля §13.4.2; `level` соответствует пройденному checklist уровню (не выше).
4. **Подпись и публикация** — actor подписывает manifest нативным для носителя механизмом (V6 author + timestamp); manifest становится частью SoT через V3 review (для self-assessment — самоодобрение допустимо, но evidence обязано фиксироваться).

13.5.3 Доказательная база (evidence)

Каждая проверка mandatory clause обязана сопровождаться evidence-ссылками в `audit-trail/CFM-<id>/<clause>.md` носителя проекта. Evidence — нативный для носителя pointer (V1 stable identifier) на конкретные артефакты, прогоны, события подтверждающие clause.

Evidence хранятся бессрочно (требование V1 + §10.13.3 retention).

13.5.4 Периодичность

Self-assessment проводится:

- При первом заявлении соответствия (kickoff).
- По cadence §13.7 (re-assessment).
- При срабатывании trigger §13.8 (loss of conformance).
- При выпуске minor-версии стандарта (§13.9).

13.6 Оценка третьей стороной (Third-party assessment, опционально)

Third-party assessment — опциональный путь подтверждения conformance внешним assessor. Применяется в контекстах с регуляторными требованиями к внешнему audit (медицина, финтех, госсектор, AI-критические системы) или при контрактных обязательствах перед клиентом.

13.6.1 Участник (actor)

External assessor — независимый actor с правом read-only доступа к носителю на период assessment. Формальная квалификация assessor специфична для носителя и фиксируется в `assessor.signature-ref` (например, внешний аудитор сертифицированной organisation).

13.6.2 Методология

Та же что §13.5.2 (checklist §13.3 + checklist главы 11 для уровня), плюс:

1. **Audit log** — все действия assessor (read-events) фиксируются нативно для носителя для прозрачности.
2. **Independent verification** — assessor самостоятельно проверяет evidence без полагания на self-assessment результаты проекта.
3. **Result** — signed manifest (assessor подписывает нативным для носителя V6 механизмом) **или** denial с обоснованием и списком конкретных нарушений

mandatory clauses / уровневых критериев.

13.6.3 Дополнительные поля manifest

При `assessment-mode: third-party` обязательны:

```
third-party:
  assessor-organisation: "<наименование>"
  assessor-qualification-ref: "<нативный для носителя pointer на
квалификационный документ>"
  audit-report-ref: "<нативный для носителя pointer на полный audit
report>"
  audit-log-ref: "<нативный для носителя pointer на read-events log>"
```

13.6.4 Соотношение self / third-party

Third-party assessment **не отменяет** self-assessment cadence (§13.7) — оба пути совместимы. Проект может вести self-assessment ежеквартально и third-party assessment ежегодно; manifest версионируется отдельно по каждому пути с разными `manifest-id` .

13.7 Период повторной оценки (Re-assessment cadence)

13.7.1 По умолчанию (Default)

По умолчанию cadence для self-assessment — **квартально** (каждые 3 месяца от `assessment-date`); для third-party — **ежегодно**. Конкретный cadence объявляется в `next-assessment-due` поле manifest.

13.7.2 Переопределение (Override)

Cadence может быть переопределена в `declared-stricter` :

```
declared-stricter:
  - clause: "re-assessment-cadence"
```

```
    override: "monthly"
    rationale: "AI-критический проект, eval-зависимость"
```

Ослабление cadence (`override: "annual"` для self при default `quarterly`) — запрещено как нарушение §13.3.1 SoT inversion (скрытое ослабление cadence == hiding lost-conformance event; audit trail SoT обязан фиксировать факт затягивания re-assessment).

13.7.3 Немедленные триггеры повторной оценки

Помимо cadence, немедленная повторная оценка обязательна при:

1. Выпуск minor-версии стандарта RENAR (`renar-version` сдвигается).
2. Нарушение mandatory clause (§13.3) — trigger loss of conformance (§13.8).
3. Существенное изменение носителя (например, замена носителя целиком) — V1-V6 declaration пересматривается.
4. Существенное изменение **области применения** проекта (новый класс артефактов, новый SPEC-type начинает использоваться).

13.8 Потеря соответствия (Loss of conformance)

13.8.1 Триггеры

Conformance считается **потерянным** при наступлении любого из:

Trigger	Описание
Mandatory clause нарушена	Любая из §13.3.1–§13.3.7 перестала выполняться (например, появилась BR без ADAPT; носитель потерял V3 после миграции)
Деградация возможности носителя	V1, V2, V3, V4, V5 или V6 более не обеспечивается носителем (например, миграция на носитель без diff & review)
Manifest expired	`next-assessment-due` прошёл без выпуска новой версии manifest
Level criteria нарушены	Критерии заявленного уровня (см. главу 11) перестали выполняться без формального downgrade

Metric threshold violation	Критическая метрика главы 12 превысила нормативный порог для уровня (например, Hallucination Rate > 5% на RENAR-4 — §12.3.3 line 94 explicit trigger)
External denial	Third-party assessor вернул denial с обоснованием

13.8.2 Процедура

1. **Регистрация loss-event** — нативная для носителя запись о факте потери conformance в audit trail manifest (`audit-trail/CFM-<id>/loss-events/<timestamp>.md`).
2. **Формальный downgrade или декларация unknown-state** — два варианта:
   - **Downgrade**: выпуск новой версии manifest с `level` ниже текущего (если нарушены критерии заявленного уровня, но mandatory clauses выполнены).
   - **Unknown-state**: явная декларация в публичных коммуникациях проекта что conformance временно отсутствует; manifest помечается `replaced-by: "<unknown-state>"` (sentinel-value, отличается от default `null` активного manifest); повторный assessment обязателен для восстановления conformance.
3. **Recovery plan** — обязательная регистрация плана восстановления (нативный для носителя артефакт `recovery/CFM-<id>-<timestamp>.md`) с указанием срока и mandatory clauses / уровневых критериев, требующих восстановления.
4. **Re-assessment** — после реализации recovery plan; обязательно self-assessment (§13.5) с обновлённым manifest. Для third-party-mode — обязательно повторное external assessment.

13.8.3 Публичные коммуникации

Потеря conformance, если уровень заявлен публично, обязана быть зарегистрирована публично в течение разумного срока (специфичный для носителя notification cadence фиксируется в `guide/`). Скрытие факта потери — нарушение mandatory clause §13.3.1 (audit trail SoT).

13.9 Политика закрытого списка уровней RENAR-N

13.9.1 Нормативное правило

Закрытый список уровней RENAR-1..RENAR-5 (§13.2) **не расширяется** project-local. Любой проект, заявляющий соответствие, обязан указать `level` ∈ {RENAR-1, RENAR-2, RENAR-3, RENAR-4, RENAR-5}.

Настоящая политика — специализация §1.7 Closed list policy для уровней зрелости; общее правило для всех закрытых списков RENAR и master-индекс — §1.7.5.

13.9.2 Что запрещено

Действие	Запрещено?	Почему
Project-local создание уровня (`RENAR-6` , `RENAR-PRO`)	Запрещено	Нарушает закрытый список; conformance становится не-портируемой
Локальное переопределение критериев уровня (например, ослабление RENAR-4 без формального downgrade)	Запрещено	Нарушает контракт стандарта
Локальное ужесточение критериев уровня (declare-stricter)	Разрешено	См. §13.4.2 `declared-stricter`
Заявление уровня выше фактически достигнутого	Запрещено	Нарушает mandatory clause §13.3.1 (SoT inversion для самой conformance machinery)
Заявление conformance без manifest	Запрещено	§13.4 mandatory
Промежуточные уровни типа `RENAR-3.5`	Запрещено	Список дискретный

13.9.3 Путь добавления нового уровня

Добавление нового уровня (например, `RENAR-6`) — только через formal change procedure стандарта:

1. **Research draft** — обоснование необходимости нового уровня; типология критериев; сравнение с существующими уровнями `RENAR-N` .
2. **Public review** — открытое обсуждение в форуме стандарта; срок и форма фиксируются политикой версионирования стандарта.
3. **Minor-version bump** — новый уровень включается в следующую minor-версию стандарта (`v1.X` или `v2.0`).

4. **Migration guidance** — для существующих conformant проектов выпускается миграционная инструкция (изменения в self-assessment checklist, новые поля manifest).

Project-local extensions остаются за пределами conformance — они допустимы как internal-практики (`declared-stricter`), но **не** влияют на формальный `level` в manifest.

13.10 Связь с другими главами

Глава	Связь
02 Положение в типологии методологий	§2.3 SoT inversion — mandatory clause §13.3.1; §2.7 conformance implication явно отсылает к этой главе
06 Иерархия требований	frontmatter артефактов (BR / SR / TR) — критерии level RENAR-2/-3 проверяются по §6.5–§6.7 (глава 11)
07 ADAPT	Mandatory clause §13.3.3 (ADAPT для каждого T3); §7.4.4 closed list backward findings — §13.3.7
08 Specifications	Mandatory clause §13.3.4 (9 SPEC types closed list); §8.3.1 closed list policy — §13.9
09 Test cases	Mandatory clause §13.3.5 (pos/neg парность); §9.10 QG-2 — §13.3.6
10 Lifecycle и QG	§10.3 canonical gates, §10.4.3 conformance manifest fragment — расширяется здесь до полного manifest schema; §10.10 closed list policy для gates параллельна §13.9 для уровней
03 Версионирование носителя	Mandatory clause §13.3.2 (V1–V6 declared); §3.4 mapping таблица — не-нормативная, информационная
11 Maturity model	Детальные критерии уровней RENAR-1..5 — здесь §13.2 содержит semantic-summary; полный checklist по уровню — в §§11.4–12.8 (по одной секции на уровень)
12 Metrics	Метрики, на которых строится self-assessment (`approved-without-verified` , `pos-neg-pairing-percent` , и др.) — конкретизируются здесь как input для §13.5

---

title: "Нормативные ссылки" order: 14 lang: ru

14. Нормативные ссылки

> ***Часть RENAR Standard v1.0-draft*** · *← Оглавление*

14.1 На какие стандарты опирается RENAR

RENAR не переписывает чужие стандарты — он на них опирается и заявляет соответствие в той или иной части. Глава разделяет два рода ссылок: **нормативные** (ISO/IEC 29148, 5338, 23894 и др. — их положения обязательны для RENAR-реализаций) и **информативные** (методологии и терминология для позиционирования, для conformance не обязательны). Полный расширенный каталог информативных сопоставлений — reference/11.

Глава перечисляет:

- **Нормативные ссылки** (§14.4) — стандарты, заявление о соответствии к которым делает RENAR (полностью или в части); их положения обязательны для реализаций, соответствующих RENAR.
- **Информативные ссылки** (§14.5) — методологии и терминология, которые RENAR использует для позиционирования; для conformance не обязательны.
- **Позиция по соответствию** (§14.3) — формулировка места RENAR среди родственных стандартов.
- **Что RENAR принципиально не принимает** (§14.7) — закрытый список практик, **не** заимствованных с обоснованием.

При расхождении между нормативной ссылкой и положением этой главы побеждает положение этой главы (RENAR — специализация и адаптация, не полное воспроизведение референсного стандарта). Заявление о соответствии внешнему стандарту RENAR делает **только** в указанной части, не целиком.

Все даты — дата опубликования референсной редакции; ссылка является **датированной** (§14.4.1).

14.2 SENAR как родительский стандарт

RENAR — **специализация SENAR** (§2.1) в области инженерии требований. RENAR не дублирует следующие положения SENAR; они применяются как есть:

Положение SENAR	Используется без переписывания
5 ценностей и 14 правил	Контекст всего RENAR; см. §2.1
QG-0 / QG-1 / QG-2 как концепция	RENAR конкретизирует машины состояний в §10
10 общих метрик процесса	RENAR добавляет 10 доменных в §12.3
5 уровней общей зрелости	RENAR-M — отдельное измерение (§11.2)
5 ролей (Supervisor, AI-агент, Architect / Tech Lead, Reviewer, Stakeholder)	RENAR не переопределяет роли; см. §5
Инструментирование агентов (уровни контроля, профили)	RENAR расширяет для специфики требований

RENAR начинается там, где SENAR заканчивается: SENAR — общая методология AI-нативной разработки; RENAR — нормативный документ управления требованиями для SENAR-совместимых систем.

14.3 Позиция по соответствию

> ***RENAR — управление требованиями, согласованное с ISO/IEC/IEEE 29148:2018, адаптированное для разработки с AI-агентами, построенное поверх методологии SENAR и совместимое с координацией SAFe 6.0.***

Эта формулировка — точка отсчёта для всех заявлений о соответствии, которые проекты выпускают на основе RENAR (§13.4).

14.4 Нормативные датированные ссылки

Каждая запись §14.4.2–§14.4.6 содержит блоки **«Что нормирует»**, **«Как RENAR соотносится»** и **«Заявление о соответствии»**. Блоки **«Что RENAR адаптирует»** и **«Что RENAR не принимает»** есть только у записей глубокой адаптации (29148 и 5338); остальные записи короче — это отражает глубину заявления, а не дефект структуры.

14.4.1 Понятие датированной ссылки

RENAR использует **датированные ссылки**: каждая нормативная ссылка указывает конкретную редакцию стандарта (с годом). Ссылки без редакции не применяются — семантика меняется между изданиями, и заявление о соответствии должно быть проверяемо относительно зафиксированной редакции.

14.4.1.1 Жизненный цикл нормативной ссылки

1. **Активная** — ссылка указана в действующей версии RENAR (`renar-version` в manifest соответствия, §13.4.2).
2. **Обновлена** — референсный стандарт выпустил новую редакцию; RENAR обновляется в разумный срок. manifest проекта фиксирует `renar-version`, которая, в свою очередь, фиксирует редакции внешних ссылок.
3. **Отозвана upstream** — референсный стандарт снят (как IEEE 830-1998). RENAR переносит ссылку в §14.5 и указывает правопреемника.

14.4.1.2 Триггеры немедленной переоценки

При выпуске новой редакции одной из ссылок §14.4 обязательна немедленная переоценка (§13.7.3) manifest проектов. Одной замены редакции в тексте RENAR недостаточно — требуется:

- проверка глав RENAR на согласованность с новой редакцией (запись в changelog RENAR);
- обновление `renar-version` в manifest проектов;
- запись в audit-trail носителя о переоценке.

Негативный сценарий: заявление о соответствии RENAR `renar-version: 1.0` при выходе новой редакции ISO/IEC 5338 **без** обновления `renar-version` в manifest — невалидно; hook носителя (§13.8.1) обнаруживает устаревшую версию.

14.4.2 ISO/IEC/IEEE 29148:2018 (инженерия требований)

Официальное название (EN): Systems and software engineering — Life cycle processes — Requirements engineering.

Что нормирует: международный стандарт инженерии требований — потребности стейкхолдеров, спецификация, валидация, верификация, атрибуты, трассируемость, жизненный цикл.

Как RENAR соотносится:

29148	RENAR	Тип

Классы требований: stakeholder, system, software	BR, SR, TR	Заимствует с переименованием
Атрибуты требования (18 в 29148)	Обязательный minimum во frontmatter (§6.5.2, §6.6.2) — 7–8 полей	Упрощает
Структура SRS	Структура носителя требований изоморфна	Заимствует
Методы верификации: inspection, analysis, demonstration, test	TC (§9) — полноценный артефакт; inspection — через workflow `[test-spec-change]` (§9.13)	Адаптирует

Что RENAR адаптирует:

- 29148 предусматривает 18 атрибутов; RENAR оставляет 7–8 обязательных, остальные — auto-derived (§4.12) или опциональны. Обоснование: при разработке с AI-агентами избыточная атрибутика увеличивает риск галлюцинаций (§12.3.3).
- 29148 не выделяет TC как отдельный артефакт. RENAR делает TC полноценным артефактом (§9).

Что RENAR не принимает: формальные review meetings и walkthroughs из 29148 — заменены на QG-0 / QG-2 и adversarial AI-review (§11.7 для RENAR-4+).

Заявление о соответствии: RENAR заявляет соответствие ISO/IEC/IEEE 29148:2018 в части классов требований, атрибутов, жизненного цикла и методов верификации (фиксируется в manifest, §13.4.2).

14.4.3 ISO/IEC 25010:2023 (модель качества продукта)

Официальное название (EN): SQuaRE — Product quality model.

Что нормирует: девять характеристик качества ПО (редакция 2023), включая новую Safety.

Как RENAR соотносится: характеристики 25010 — **обязательные категории** для нефункциональных SR. Frontmatter SR (§6.6.2) обязан содержать `quality-characteristic` из перечня 25010.

Заявление о соответствии: RENAR заявляет соответствие ISO/IEC 25010:2023 в части словаря категорий для NFR.

14.4.4 ISO/IEC 25022:2016 / 25023:2016 (меры качества)

Что нормирует: формальные меры для каждой характеристики 25010 (например, время отклика в мс).

Как RENAR соотносится: критерии Pass в TC (§9.11.1) обязаны быть выражены через меры 25022/25023, где применимо. Пример: «p95 < 200 мс при 100 RPS» вместо «производительность приемлемая».

Заявление о соответствии: RENAR заявляет соответствие в части измеримых Pass-критериев для TC.

14.4.5 ISO/IEC 5338:2023 (жизненный цикл AI-систем)

Что нормирует: первый международный стандарт жизненного цикла AI-систем (адаптация ISO/IEC 12207).

Как RENAR соотносится:

- **Журналы решений** — существенные решения AI-агента документируются; реализация: audit-trail (§10.13) + `ai-provenance` (§4.10.1).
- **Версионирование данных** — eval-datasets с version pin V5 (§3.3.5).
- **Версионирование модели** — `ai-provenance.generated-by` обязательно на RENAR-4+ (§11.7.1).
- **Непрерывная валидация** — eval-runs по расписанию (§11.8.1 для RENAR-5).

Заявление о соответствии: RENAR заявляет соответствие в части жизненного цикла AI-артефактов и генерации артефактов с участием AI.

Негативный сценарий: заявление о соответствии 5338 без `ai-provenance` (§4.10.1) — невалидно. RENAR обязан отказывать в выпуске manifest для таких проектов.

14.4.6 ISO/IEC 23894:2023 (управление рисками AI)

Что нормирует: классы AI-рисков и стратегии смягчения.

Как RENAR соотносится:

Риск (23894)	Смягчение в RENAR
Галлюцинации в выводе AI	Source citation (§4.10.2), метрика Hallucination Rate (§12.3.3)
Дрейф модели	pinning `last-run.requirement-version` (§9.12) + periodic re-run

Prompt injection через входные данные	Санитизация при импорте T3 (специфично для носителя, в guide/)
Bias в AI-генерации	Multi-model agreement для критических BR (RENAR-5, §11.8.1)
Adversarial inputs	Adversarial review (§11.7.1, §11.8.3)
Single point of failure (одна модель)	Multi-model agreement; изоляция judge-модели (§9.13.4)

Заявление о соответствии: RENAR заявляет соответствие в части идентификации и смягчения AI-рисков в области требований; полный реестр — reference/03.

14.5 Информативные ссылки

Информативные ссылки — методологии и терминология для позиционирования; заявление о соответствии к ним RENAR **не** делает.

14.5.1 Пять ключевых (начните здесь)

Источник	Зачем RENAR
SAFe 6.0	Маппинг иерархии Epic/Feature/Story → BR/SR/TR (§4.13.1)
Spec-Driven Development	SoT inversion (§2.3.1) как формальная парадигма
EARS (Mavin et al.)	Шаблоны формулировок SR и TC (§6.6.3)
BDD / Gherkin / Specification by Example	Prior art для полноценного TC (§9)
NIST AI RMF 1.0	Функциональное сопоставление Govern/Map/Measure/Manage

Краткие пояснения — в reference/11 §1–§2.

14.5.2 Расширенный каталог

Источник	Роль для RENAR
IEEE 830-1998 (deprecated)	Историческая ссылка; нормативный правопреемник — §14.4.2

BABOK v3	Терминология BA; gap по elicitation — в guide/
PMBOK 7	Принципы вместо процессов (§2.5)
ISTQB Foundation	Словарь тестирования, совместимый с `tc-type`
CMMI v2.0	Prior art для уровней зрелости (§11)
ISO/IEC 42001:2023	Организационный governance над RENAR
ISO/IEC 25059:2023	Качество AI-систем (расширение 25010)
EU AI Act (Reg. 2024/1689)	Поле `ai-act.risk-class`; юридическое соответствие — вне RENAR
SysML / MBSE	Prior art «требования как граф» (reference/05)

Детальные сопоставления — reference/11.

14.6 Сводка соответствий

14.6.1 Сводная таблица

Стандарт	Тип	Уровень	Главы RENAR
SENAR	Родительский	Специализация	Все
ISO/IEC/IEEE 29148:2018	Нормативный	Высокий	06, 09
ISO/IEC 25010:2023	Нормативный	Средний	06, 08
ISO/IEC 25022/25023	Нормативный	Средний	09
ISO/IEC 5338:2023	Нормативный	Высокий	04 §4.10, 11
ISO/IEC 23894:2023	Нормативный	Средний	11, reference/03
Остальные (§14.5)	Информативный	—	см. reference/11

14.6.2 Совокупные заявления

manifest (§13.4.2) может содержать **несколько** заявлений о соответствии к ссылкам §14.4 одновременно. Каждое проверяется независимо. Частичное заявление не предусмотрено: проект либо conformant через RENAR, либо нет.

14.6.3 Mandatory clauses и внешние стандарты

Mandatory clauses §13.3 — внутренние требования RENAR. Заявления §14.4 — внешние, опциональные сверх minimum (например, RENAR-1 без заявления к 5338, если нет AI-генерации артефактов).

14.7 Что RENAR принципиально не принимает

Закрытый список практик из родственных стандартов:

Практика	Источник	Почему не принимается
Тяжёлые документные review meetings, IEEE 1028 inspections	RUP, SWEBOK	Несовместимо со скоростью AI-агентов; заменено adversarial AI-review (§11.7.1)
Только ручная верификация (inspection meetings 29148)	ISO/IEC/IEEE 29148 §6.4	Hooks носителя (§10.11) + AI-review
Process-first CMMI (CCB, OSP)	CMMI v2.0	Принципы + автоматический enforcement (§2.5)
Формальные методы для всех требований (B, Z, TLA+)	Formal methods	Только critical safety domains, не baseline
Недатированные ссылки на стандарты	Industry practice	Только датированные (§14.4.1)
Самодекларированное соответствие без manifest	Отраслевая практика	manifest обязателен (§13.4)

14.8 Связь с другими главами

Глава	Связь
04 Terms	§4.13 — детальное сопоставление с родственными стандартами
02 Methodology	§2.3.4 SDD; §2.6 implications для mandatory clauses
06 Hierarchy	frontmatter — реализация атрибутов 29148
09 Test cases	TC — расширение 29148; терминология ISTQB-совместима

10 Lifecycle	QG — конкретизация SENAR QG-0..2
11 Maturity	Уровни RENAR-1..5
13 Conformance	`renar-version`, `external-claims[]`
reference/11	Полный информативный каталог §14.5
reference/03 AI risk	Реестр рисков по 23894 + NIST

---

title: "Быстрый старт" description: "30-минутный практический старт RENAR: T3 → ADAPT → SR → SPEC → TC → verified." order: 0 lang: ru version: "1.0-draft"

00. Быстрый старт

> *Один сквозной пример: маленький проект «email/password sign-up». Проходим полный цикл RENAR с минимальным набором артефактов: T3 → ADAPT → BR → SR → SPEC → TC. Время: ~30 минут чтения + проб на бумаге; ~2-3 часа если делать вживую на носителе артефактов. Предпосылки: знакомство с RENAR Core (5 правил + ADAPT + 2 QG). Язык RU-корпуса (canonical IDs + RU prose) — standard/00 §0.8.*

Что вы получите

После этого быстрого старта у вас будет:

- Понимание полного цикла RENAR на одном маленьком примере.
- Готовые YAML-шаблоны для каждого типа артефакта.
- Опыт прохождения двух шлюзов качества: QG-ADAPT-approve (педагогический псевдоним **QG-3 Architecture Gate**, см. §10.3) и **QG-2 Verification Gate**.
- Точка для масштабирования на реальный проект.

Предпосылки

Носитель (`substrate`) — система хранения и версионирования артефактов. RENAR независим от вида хранилища; для быстрого старта подойдёт любой носитель с capabilities V1-V6 (01-glossary §14.7):

- git с PR-ревью (по умолчанию для большинства проектов);
- документо-ориентированное хранилище (revision-based документы с API-подписями);
- любая БД с историей и подписями.

Структура папок (соглашение по умолчанию для git как носителя):

```
my-project.req/
  tz/                   # immutable T3 от клиента
  adapt/                # ADAPT артефакты
```

```
  br/                   # Business Requirements
  sr/                   # System Requirements
  specs/                # SPEC-* семейство
    arch/  api/  data/  ui/  sec/  ai/  proc/  int/  ops/
  tests/                # TC (полноценный артефакт; tc-type incl.
contract)
```

Для других носителей — эквивалентная организация по типам документов или по пространствам имён.

Шаг 1. ТЗ (5 минут)

Клиент даёт небольшое ТЗ. Это **договорной immutable** документ — после подписания не редактируется.

`tz/TZ-2026-001.md` (фрагмент):

```
---
id: TZ-2026-001
title: "Регистрация пользователей через email"
signed-date: "2026-05-15"
signed-by-client: "Иванов И.И., PM, ClientCo"
---

# ТЗ-2026-001

## §1. Контекст
Создать систему регистрации пользователей по email.

## §14. Требования
- Пользователь может зарегистрироваться через email и пароль.
- После регистрации пользователь подтверждает email (получает
письмо).
- После подтверждения — доступ в личный кабинет.

## §4. Ограничения
- Только web-приложение.
- Хранение в РФ (ФЗ-152).
```

ТЗ подписано клиентом → immutable. Все правки и интерпретации идут через ADAPT.

Шаг 2. ADAPT в статусе `draft` (10 минут)

Создаём `adapt/ADAPT-001-main.md`. AI-агент или инженер заполняет **Forward** (как поняли) и **Backward** (что неясно).

```yaml
---
id: ADAPT-001
title: "Адаптация ТЗ-2026-001 — Регистрация через email"
type: ADAPT
source-tz:
  id: TZ-2026-001
  signed-date: "2026-05-15"
  signed-by-client: "Иванов И.И., PM, ClientCo"
  document-ref: "<нативная для носителя фиксация версии>"
status: draft
created: "2026-05-16"
ai-provenance:
  generated-by: "anthropic-claude-opus-4-7@2026-05-16"
  prompt-template: "prompts/adapt-from-tz.md@v1.0"
  context-tokens: 1024
  output-tokens: 2048
  human-edits: false
---
```

2.1 Forward

```
## Forward §2 Требования

**Цитата:** «Пользователь может зарегистрироваться через email и
пароль.
После регистрации пользователь подтверждает email. После
подтверждения —
доступ в личный кабинет.»
```

```
**Интерпретация:** Регистрация — POST /auth/sign-up с {email,
password}.
Создаётся User со status `unverified`. Отправляется email с
verification
link. Клик по link → status `verified`. Только `verified` могут
войти.

**Достроенные сценарии:**
- sign-up: создание User в `unverified`, отправка email
- email verification: подтверждение → перевод в `verified`
- первый вход: только `verified` → редирект в личный кабинет

**Scope:**
- Входит: email/password, email verification, доступ в личный
кабинет.
- НЕ входит: OAuth (Google/Apple), SMS-верификация, 2FA, сброс
пароля.

**Forward links** (auto-populated после approval): BR-01, SR-01, SR-
02, SR-03.
```

2.2 Backward (3 записи)

```
## Backward

### B-001: gap — попытка входа неверифицированного

- **Категория:** gap
- **Статус:** open
- **Описание:** ТЗ не описывает поведение системы при попытке входа
  до верификации email.
- **Вопрос к клиенту:** показывать сообщение «подтвердите email» с
кнопкой
  повторной отправки — или возвращать 401 без объяснения?

### B-002: regulatory — ФЗ-152 хранение в РФ

- **Категория:** regulatory
- **Статус:** open
```

```
- **Описание:** ТЗ §3 требует «хранение в РФ». Что конкретно: дата-центр,
  юрисдикция provider, отдельная инсталляция?
- **Вопрос к клиенту:** уточнить scope ФЗ-152 для проекта.

### B-003: gap — повторная отправка email

- **Категория:** gap
- **Статус:** open
- **Описание:** Что если verification email потерян? Можно ли отправить
  повторно, сколько раз, есть ли rate limit?
- **Вопрос к клиенту:** policy на повторную отправку email.
```

Шаг 3. ADAPT в статусе `approved` (5 минут)

Клиент даёт ответы. Каждая backward запись проходит lifecycle:

```
open → asked-to-client → answered → resolved → frozen (после approval)
```

Резюме после ответов клиента:

```
### B-001: resolved
- **Asked-to-client:** 2026-05-17
- **Ответ клиента:** 2026-05-18, Иванов И.И.:
  «Показать сообщение + кнопку повторной отправки. 401 без объяснения — плохой UX.»
- **Resolution:** добавлен сценарий в Forward §2; создан SR-04.

### B-002: resolved
- **Ответ клиента:** 2026-05-18:
  «Дата-центр в РФ, юрисдикция РФ. Provider — выбирает исполнитель.»
- **Resolution:** добавлен пункт в Forward §2 о data-residency: ["RU"].

### B-003: resolved
```

```
- **Ответ клиента:** 2026-05-18:
  «Повторная отправка не чаще 1 раза в 5 минут, не больше 5 раз в
сутки.»
- **Resolution:** добавлены rate-limit правила в Forward §2; SR-04
уточнён.
```

Все backward → `resolved`, `open-questions-count = 0`. ADAPT готов к approval.

QG-ADAPT-approve — все 6 пунктов passed:

```yaml
status: approved
approval:
  client-signature:
    signed-by: "Иванов И.И."
    role: PM
    organization: "ClientCo"
    signed-at: "2026-05-18T15:30:00Z"
    signature-ref: "<нативная для носителя ссылка>"
  architect-signature:
    signed-by: "Петров П.П."
    role: architect
    signed-at: "2026-05-18T16:00:00Z"
ai-provenance:
  human-edits: true   # архитектор отредактировал AI-draft
open-questions-count: 0
resolved-questions-count: 3
```

После approval ADAPT **immutable** (frozen). Все BR/SR/SPEC ссылаются на approved ADAPT, не на ТЗ напрямую.

Шаг 4. BR-01 (3 минуты)

`br/BR-01-user-registration.md`:

```yaml
---
id: BR-01
```

```yaml
title: "Регистрация пользователей через email"
type: BR
status: approved
priority: must
source:
  adapt: "ADAPT-001"
  adapt-section: "Forward §2"
  tz-section: "§2"
business-context:
  stakeholder: "Иванов И.И. (PM, ClientCo)"
  business-goal: "Дать пользователям возможность создать аккаунт и
получить доступ к продукту"
business-outcome:
  measurement-type: kpi
  kpi-name: "registration-conversion-rate"
  measurement-method: "registered / visited_signup * 100%"
  baseline-value: 0
  target-value: 60
  target-met-by: "2026-09-01"
data-classification:
  contains-pii: true
  retention-days: 2555     # 7 лет по ФЗ-152
  data-residency: ["RU"]
compliance:
  - { standard: "ФЗ-152", article: "ст.6,12" }
ai-provenance:
  generated-by: "anthropic-claude-opus-4-7@2026-05-18"
  human-edits: true
---

# BR-01: Регистрация пользователей через email

Пользователь должен мочь самостоятельно создать аккаунт через
email/password
и получить доступ к личному кабинету после подтверждения email.
```

Шаг 5. SR-01..SR-04 (3 минуты)

Декомпозируем BR на verifiable SR. Каждый SR ссылается на approved ADAPT.

`sr/SR-01-sign-up.md` (frontmatter):

```yaml
---
id: SR-01
title: "Sign-up через email/password"
type: SR
status: approved
parent:
  id: BR-01
source:
  adapt: "ADAPT-001"
  adapt-section: "Forward §2"
constrained-by:
  - "SPEC-API-01"
  - "SPEC-DATA-01"
  - "SPEC-SEC-01"
quality-characteristic:
  - functional-suitability
  - security
---
```

Тело SR-01:

```
## Описание
POST /auth/sign-up принимает {email, password}.
- При валидных данных: создаёт User со status `unverified`,
отправляет
  verification email, возвращает 201.
- При невалидном email (regex/format): возвращает 422 с указанием
поля.
- При уже занятом email: возвращает 409.
- При слабом пароле (< 8 chars или попадает в blacklist): возвращает
422.
```

Аналогично создаются SR-02 (email verification), SR-03 (вход для verified), SR-04 (повторная отправка email, с rate limit из B-003 resolution).

Шаг 6. SPEC-* (3 минуты)

`specs/api/SPEC-API-01-auth.md` (фрагмент):

```yaml
---
id: SPEC-API-01
title: "REST API аутентификации"
type: SPEC-API
status: approved
source:
  adapt: "ADAPT-001"
api-style: rest
api-version: "v1.0.0"
versioning-strategy: url-path
authentication: bearer-jwt
rate-limits:
  - { endpoint: "POST /auth/resend-email", limit: "1/5min/user;
5/24h/user" }
contract-file:
  format: openapi-3.1
  location: "contracts/auth-api.yaml"
depends-on:
  - "SPEC-DATA-01"
  - "SPEC-SEC-01"
referenced-by: ["SR-01", "SR-02", "SR-03", "SR-04"]   # auto-derived
---
```

Аналогично — SPEC-DATA-01 (схема User), SPEC-SEC-01 (auth model + ФЗ-152 controls).

Шаг 7. TC pos/neg парность (5 минут)

Каждый SR — минимум 1 позитивный + 1 негативный TC. Каноничная схема frontmatter — reference/02 §8 (`type: TC` , `tc-type` , `requirement-version`).

`tests/TC-01-signup-success.md` (позитивный для SR-01):

```yaml
---
id: TC-01
title: "Sign-up: успешная регистрация"
type: TC
tc-type: system
status: ready
verifies:
  - id: SR-01
    requirement-version: "1.0"
negative: false
automation:
  status: automated
  location: "tests/auth/test_signup.py::test_signup_success"
  runner: pytest
---

## Given
- новый email (uniquely-generated@test.com)
- валидный password (длина ≥ 8, не в blacklist)

## When
POST /auth/sign-up {email, password}

## Then
- status 201
- response body содержит {"user_id": "<uuid>", "status":
"unverified"}
- в БД создан User с status `unverified`
- отправлен verification email (mock)
```

`tests/TC-02-signup-invalid-email.md` (негативный для SR-01):

```yaml
---
id: TC-02
title: "Sign-up: отклонить невалидный email"
type: TC
tc-type: system
status: ready
verifies:
```

```
  - id: SR-01
    requirement-version: "1.0"
negative: true
automation:
  status: automated
  location: "tests/auth/test_signup.py::test_signup_invalid_email"
  runner: pytest
---

## Given
- email "not-an-email" (без `@`)
- любой валидный password

## When
POST /auth/sign-up {email, password}

## Then
- status 422
- response body {"field": "email", "error": "invalid format"}
- User в БД НЕ создан
- email НЕ отправлен
```

Аналогично пары для SR-02, SR-03, SR-04. Дополнительно для SR-04 (с rate limit) — TC, проверяющий блок после 5-й попытки за 24 часа.

Шаг 8. Запустить TC и promote SR (2 минуты)

Test runner со стороны носителя запускает TC и обновляет `last-run`:

```
# tests/TC-01-signup-success.md (после прогона):
last-run:
  date: "2026-05-19T10:00:00Z"
  result: pass
  runner-id: "test-runner@1.0"
  run-ref: "<нативная для носителя ссылка>"
  requirement-version: "1.0"
```

Когда **все** TC из `SR-01.verified-by[]` зелёные, проводится выборочная проверка (Правило 5 Core) — инженер вручную запускает 1-2 случайных passing TC и сверяет результат с SR.

Если выборочная проверка пройдена → **QG-2 Verification Gate** пройден → SR-01 переводится в `verified`:

```
# sr/SR-01-sign-up.md:
status: verified
verified-by:
  - "TC-01"
  - "TC-02"
verified-at: "2026-05-19T14:00:00Z"
verified-by-engineer: "Петров П.П."
```

Аналогично — SR-02, SR-03, SR-04 → `verified`. Когда все SR в BR-01 verified → QG-4 (BR ready for acceptance).

Цепочка трассировки

В любой момент можно восстановить происхождение любого артефакта:

```
TC-02 (negative)
  └─ verifies SR-01 (sign-up)
       └─ derived from ADAPT-001 §14 Forward (email/password)
            └─ interprets TZ-2026-001 §2 (immutable)
       └─ constrained-by SPEC-API-01 (REST contract)
                                  └─ depends-on SPEC-DATA-01, SPEC-
SEC-01
```

При аудите через год: «откуда взялось требование возвращать 422 на невалидный email» — TC-02 → SR-01 → ADAPT-001 §14. Трассировка полная.

Что вы только что сделали

- Создали immutable договорной артефакт (T3).

- Прошли двустороннюю интерпретацию через ADAPT с тремя backward findings → клиент дал ответы → approved.
- Декомпозировали в BR + 4 SR, привязанные к approved ADAPT.
- Описали 3 SPEC (API, DATA, SEC) как параллельную ось структуры.
- Покрыли каждый SR pos+neg TC парой (минимум 8 TC).
- Прошли два шлюза качества: QG-ADAPT-approve (≡ QG-3 Architecture) и QG-2 Verification Gate.

Это полный цикл RENAR в минимальной форме. На реальном проекте scaling — больше BR/SR/SPEC, больше backward findings, делегирование на AI-агентов; опционально QG-4 (acceptance) для client sign-off.

Что дальше

Хотите...	Документ
Детальный сквозной пример на полномасштабном сценарии (login + profile + permissions)	01-walkthrough.md
Переход с legacy подхода (просто ТЗ → код) на RENAR	02-transition-guide.md
Git как носитель (commit-policy, PR-ревью, hooks)	03-tool-guide-git.md
Документо-ориентированный носитель	04-document-store-substrate.md
Сравнение RENAR с SAFe / BABOK / ISO 29148	05-safe-comparison.md
Compliance: GDPR, ФЗ-152, AI Act mapping	06-compliance.md
Failure modes — типовые провалы и их паттерны	07-failure-modes.md
Полная нормативная спецификация (15 глав)	`standard/`
Схемы артефактов и validation rules	`reference/02-schemas.md`
Глоссарий и mapping на отраслевые стандарты	`reference/01-glossary.md`

Быстрый старт RENAR 1.0-draft — renar.tech

---

title: "Сквозной пример" description: "Полноразмерный сквозной пример RENAR на проекте Login Flow для AcmeCorp." order: 1 lang: ru version: "1.0-draft"

01. Сквозной пример: Login Flow для AcmeCorp

> *Один полный цикл RENAR от подписанного T3 до accepted release. Пример — внутренний инструмент с регистрацией через корпоративный email и 2FA. Цель — показать **все этапы** на одном среднем по размеру проекте.*
>
> ***Контекст:** AcmeCorp, ~1 спринт работы команды, стек Next.js + FastAPI + PostgreSQL. RENAR-зрелость уровня RENAR-3+ (полный ADAPT + TC + adversarial). Пример **независим от вида хранилища**: операции через capabilities V1–V6; конкретная раскладка каталогов — 03-tool-guide-git или 04-document-store-substrate.*
>
> ***Предпосылки:** 00-quickstart, core/renar-core, reference/01-glossary.*

Маршрут читателя

Фазы 0–2 — сбор контекста, подписание T3, ADAPT. **Фазы 3–4** — декомпозиция в BR/SR/SPEC и генерация пар pos/neg для TC. **Фаза 5** — канонические шлюзы QG-0 (утверждение требований) и QG-1 (только переход TC `draft → ready`). **Фазы 6–7** — реализация TR и верификация (QG-2). **Фазы 8–9** — дельта-T3 при изменениях и приёмочный контур QG-4.

Фаза 0 — Сбор требований (elicitation)

До подписания T3. AI-агент проводит 2-3 интервью со stakeholder (Sales Director, IT Manager) и собирает контекст в структурированном виде.

Артефакты фазы 0 (справочные, не нормативные для RENAR Core):

```
acmecorp-requirements/          # носитель требований (раскладка —
guide/03 или guide/04)
  elicitation/
    domain-context.md            # домен и его особенности
```

```
    sales-director.yaml          # transcripts + structured findings
    it-manager.yaml
    findings-clustered.md        # сгруппированные находки
    critic-review.md             # adversarial review draft
требований
    multi-model-diff.md          # разногласия двух моделей
```

Фаза 0 не закреплена в Core (это область методологии elicitation, вне scope RENAR v1.0 — см. standard/01 §1.3).

Фаза 1 — Импорт ТЗ

1.1 Подписанный ТЗ

После итераций elicitation клиент подписывает `TZ-2026-042`:

```
# TZ-2026-042 — Login Flow для AcmeCorp Internal Tool
Дата подписания: 2026-05-03
Стороны: AcmeCorp + VendorCorp

## §1. Цели
Сократить время входа сотрудников AcmeCorp в инструмент до <2 минут
от первого захода до полного доступа.

## §14. Функциональные требования

### ФТ-001. Регистрация по корпоративному email
Сотрудник регистрируется через email из домена @acmecorp.com.
Email вне домена — отказ с пояснением.

### ФТ-002. Двухфакторная аутентификация (TOTP)
После регистрации обязательная настройка 2FA через TOTP.

### ФТ-003. Восстановление доступа через корпоративного
администратора
При потере 2FA устройства — recovery через ticket в IT support.

## §4. Нефункциональные требования
```

```
### НФТ-001. Производительность
Login <2 секунд (p95).

### НФТ-002. Безопасность
- bcrypt cost-factor ≥ 12.
- Логи входов хранятся 1 год.
- Блокировка после 5 неудачных попыток за 15 минут.

### НФТ-003. Юрисдикция
Все данные в РФ (гос-контракты).
```

ТЗ подписан → **immutable**. Любые правки идут через ADAPT (см. фазу 2) или delta-TZ (см. фазу 8).

1.2 Импорт в носитель требований

Runtime **обязан** зарегистрировать immutable ТЗ как ревизию в нотации носителя (V1 + V2) с AI-provenance (V6):

```
Operation: import-tz
  Input:  signed TZ-2026-042 (immutable source)
  Output: requirements-substrate/tz/TZ-2026-042.md
          изолированная единица изменения (ветка / черновая ревизия — по правилам носителя)
          ai-provenance record on import
```

Фаза 2 — ADAPT (двусторонняя интерпретация)

2.1 Primary agent генерирует draft ADAPT

```
Operation: adapt-from-tz
  Input:  TZ-2026-042 (immutable)
  Output: draft ADAPT-001
          Forward sections (по §2 ФТ + §3 НФТ)
```

```
          Backward findings (6 candidates)
  Provenance: model + prompt-template + token counts (V6)
```

2.2 Adversarial review backward записей

Отдельный critic-agent (другая модель) проверяет backward findings:

```
Operation: adversarial-review
  Input:  draft ADAPT-001
  Output: список находок (с тегом критичности)
          блокировать adapt-approve, пока критические находки открыты

Примеры находок critic-agent:
  [HIGH] B-001 reclassify gap → hidden-assumption
  [HIGH] missed backward: case-sensitivity email in ФТ-001
  [MEDIUM] B-004 terminology: define "сотрудник" via User.role
  [MEDIUM] B-006 feasibility: rate-limit scope (IP vs email vs
session)
```

2.3 Iterative resolution

Архитектор корректирует Forward и Backward, AI re-генерирует. После 2 циклов adversarial:

- 7 backward записей (B-001..B-007).
- Все либо resolved (после ответа клиента), либо reclassified.
- Forward охватывает §2 + §3 ТЗ полностью.

2.4 ADAPT в статусе `approved`

```
---
id: ADAPT-001
title: "Адаптация TZ-2026-042 — Login Flow AcmeCorp"
type: ADAPT
source-tz:
  id: TZ-2026-042
  signed-date: "2026-05-03"
```

```
  signed-by-client: "AcmeCorp PM"
status: approved
approval:
  client-signature:
    signed-by: "Иванова А.А."
    role: "Product Lead"
    organization: "AcmeCorp"
    signed-at: "2026-05-04T11:30:00Z"
  architect-signature:
    signed-by: "Петров П.П."
    role: architect
    signed-at: "2026-05-04T12:00:00Z"
generates-requirements: [BR-01, BR-02, SR-01, SR-02, SR-03, SR-04,
SR-05, SR-06, SR-07]
generates-specs: [SPEC-UI-01, SPEC-API-01, SPEC-DATA-01, SPEC-SEC-01]
open-questions-count: 0
resolved-questions-count: 7
ai-provenance:
  generated-by: "anthropic-claude-opus-4-7@2026-05-04"
  prompt-template: "prompts/adapt-from-tz.md@v2.1"
  human-edits: true
---
```

После approval ADAPT-001 — **immutable**.

Фаза 3 — Декомпозиция в BR / SR / SPEC

3.1 Декомпозиция

```
Operation: decompose
  Input:  approved ADAPT-001
  Output: draft BR (2), SR (7), SPEC (4)
  Затем: adversarial-review артефактов (тот же критический контур,
что в §14.2)
```

3.2 Adversarial review артефактов

```
[gpt-4-turbo:critic] Обзор черновиков BR, SR и SPEC…

[HIGH] BR-01 stakeholder поле пустое — кто owner business goal?
[HIGH] SR-05 говорит "bcrypt cost-factor 12" — это deployment detail,
       должно быть в SPEC-SEC-01, не в SR.
[MEDIUM] НФТ-003 (юрисдикция) не отражено в data-classification SR-
01.
[MEDIUM] SPEC-UI-01 не имеет accessibility-level — WCAG-AA минимум
для
       корпоративного инструмента.

→ 4 findings → fix → re-generate.
```

3.3 Финальный набор артефактов

```
acmecorp-requirements/           # носитель требований (раскладка —
guide/03 или guide/04)
├── br/
│   ├── BR-01-self-service-registration.md
│   └── BR-02-secure-mfa.md
├── sr/
│   ├── SR-01-email-domain-validation.md       (ФТ-001)
│   ├── SR-02-totp-enrollment.md               (ФТ-002 setup)
│   ├── SR-03-totp-verification.md             (ФТ-002 verify)
│   ├── SR-04-password-recovery-via-admin.md   (ФТ-003)
│   ├── SR-05-rate-limiting-failed-logins.md   (НФТ-002)
│   ├── SR-06-audit-logging.md                 (НФТ-002 audit)
│   └── SR-07-data-residency-ru.md             (НФТ-003)
├── specs/
│   ├── ui/SPEC-UI-01-login-flow.md
│   ├── api/SPEC-API-01-auth.md
│   ├── data/SPEC-DATA-01-user-model.md
│   └── sec/SPEC-SEC-01-auth-policy.md
└── tz/
    └── TZ-2026-042.md
```

3.4 Пример: SR-01 frontmatter

```yaml
---
id: SR-01
title: "Валидация домена email при регистрации"
type: SR
status: approved
parent:
  id: BR-01
source:
  adapt: "ADAPT-001"
  adapt-section: "Forward §2.1"
  tz-section: "§2 ФТ-001"
constrained-by:
  - "SPEC-API-01"     # endpoint POST /auth/register
  - "SPEC-DATA-01"    # схема User
  - "SPEC-SEC-01"     # auth policy
data-classification:
  contains-pii: true
  data-residency: ["RU"]
  retention-days: 365
compliance:
  - { standard: "ФЗ-152", article: "ст.13.1" }
ai-provenance:
  generated-by: "anthropic-claude-opus-4-7@2026-05-04"
  prompt-template: "prompts/decompose-adapt.md@v2.1"
  context-tokens: 12450
  output-tokens: 320
  human-edits: true
---

## Описание

Регистрация разрешена только если email принадлежит домену
`@acmecorp.com`. Остальные домены отклоняются с понятным пояснением.

## Поведение

- email НЕ из `@acmecorp.com` → 422 с body
  `{"error":"email-domain-not-allowed", "allowed-
```

```
domain":"acmecorp.com"}`.
  [ADAPT-001 §14.1 Forward; TZ-2026-042 §2 ФТ-001]
- email из `@acmecorp.com` → стандартная регистрация (см. SPEC-API-01).
- whitelist хранится в SPEC-SEC-01.allowed-domains, расширяется без релиза.
- сравнение домена — case-insensitive (ADAPT-001 §14.1 Forward).

## Ограничения

- `*.acmecorp.com` subdomain — отдельное решение архитектора (не входит).
```

3.5 SPEC-API-01 (фрагмент)

```
---
id: SPEC-API-01
title: "REST API аутентификации"
type: SPEC-API
status: approved
source:
  adapt: "ADAPT-001"
  adapt-section: "Forward §2"
api-style: rest
api-version: "v1.0.0"
versioning-strategy: url-path
authentication: bearer-jwt
rate-limits:
  - { endpoint: "POST /auth/login", limit: "5/15min/ip+email" }
contract-file:
  format: openapi-3.1
  location: "contracts/auth-api.yaml"
depends-on:
  - "SPEC-DATA-01"
  - "SPEC-SEC-01"
---

## Endpoints
```

```
### POST /auth/register
- body: `{"email": "<corp-email>", "password": "<strong>"}`
- 201 → `{"user_id": "<uuid>", "verified": false, "totp_setup":
false}`
- 422 → invalid email или weak password
- 409 → email уже зарегистрирован

### POST /auth/login
- body: `{"email": "<corp-email>", "password": "<password>", "totp":
"<6digits>"}`
- 200 → `{"access_token": "<jwt>", "expires_in": 3600}`
- 401 → invalid credentials
- 429 → rate limit (5/15min)

### POST /auth/totp-setup
- ... (см. SR-02)

## Error model
Единая структура: `{"error": "<code>", "details": {...}}`.
```

Фаза 4 — Генерация TC (пары pos/neg)

4.1 Генерация TC

```
Operation: tc-generate
  Input:  SR-01 (approved)
  Output: pos/neg TC pairs per testable assertion
          Пример: 2 утверждения → TC-001..TC-004
```

Pos/neg парность из Правила 4 RENAR Core: для каждого assertion в SR — 1 pos + 1 neg TC.

4.2 Пример TC-001 (позитивный)

```
---
id: TC-001
title: "Регистрация с разрешённым доменом — happy path"
type: TC
tc-type: system
status: ready
verifies:
  - id: SR-01
    requirement-version: "1.0"
negative: false
automation:
  status: automated
  location:
"tests/auth/test_registration.py::test_allowed_domain_succeeds"
  runner: pytest
---

## Given
- БД пуста; email alice@acmecorp.com не зарегистрирован.

## When
POST /auth/register {email: "alice@acmecorp.com", password:
"ValidPass123!"}

## Then (Pass)
- status 201
- response body содержит {"user_id": "<uuid>", "verified": false,
"totp_setup": false}
- в БД создан User(email="alice@acmecorp.com", verified=false,
totp_setup=false)
- verification email отправлен (mock SES получил вызов с template_id)

## Fail criteria
- status ≠ 201
- response body содержит plaintext password или его hash
- User в БД с другим email (case mismatch)
- email верификации не отправлен
- в логах plaintext password

## Not in scope
```

```
- TOTP setup → TC-005 (verifies SR-02)
- Rate limiting → TC-009 (verifies SR-05)
```

4.3 Пример TC-004 (негативный)

```
---
id: TC-004
title: "Регистрация с неразрешённым доменом — отказ с пояснением"
type: TC
tc-type: system
status: ready
verifies:
  - id: SR-01
    requirement-version: "1.0"
negative: true
automation:
  status: automated
  location:
"tests/auth/test_registration.py::test_disallowed_domain_rejected"
  runner: pytest
---

## Given
- email "bob@gmail.com" (вне whitelist).

## When
POST /auth/register {email: "bob@gmail.com", password:
"ValidPass123!"}

## Then (Pass)
- status 422
- response body == {"error": "email-domain-not-allowed", "allowed-
domain": "acmecorp.com"}
- User в БД НЕ создан
- email НЕ отправлен
- в логах audit-запись о rejected attempt (для SR-06)

## Fail criteria
- status ≠ 422
```

```
- User создан в БД
- email отправлен (security leak)
- audit-запись отсутствует
```

Фаза 5 — Шлюзы качества перед кодом (QG-0 и QG-1)

5.1 Сводка по контрольным примерам

После генерации TC для всех 7 SR — суммарно 26 записей контрольных примеров (пары pos/neg и дополнительные негативы для SR-05, SR-06).

5.2 QG-0 — утверждение BR-01 (`draft → approved`)

```
Operation: promote --to approved --requirement BR-01

QG-0 (утверждение BR/SR) — предусловия (выполняются все):
  ✓ source.adapt = ADAPT-001 (approved)
  ✓ дерево SR в состоянии, допустимом политикой проекта перед
утверждением BR
  ✓ adversarial-review выполнен успешно
  ✓ утверждения и связи cite разделы ADAPT-001

Постусловия:
  BR-01: draft → approved
  дочерние SR: при необходимости каскад draft → approved
```

5.3 QG-1 — только TC: `draft → ready`

По §10.3.2 **QG-1 применим только к TC** и отделяет подготовленный контрольный пример с исполнимой реализацией проверки от черновика.

```
Operation: promote --to ready --artifact TC-001   # для каждого TC в
наборе

QG-1 — предусловия для TC:
```

```
  ✓ зафиксирован version-pin реализации (V5)
  ✓ automation.status и location валидны
  ✓ статические проверки артефакта TC на носителе пройдены
  ✓ pos/neg парность по покрываемым утверждениям (глава 9 §9.7)
  ✓ обязательные секции body TC заполнены

Постусловие:
  TC: draft → ready
```

После **QG-0** на BR/SR и **QG-1** на каждом TC можно открывать работу по TR (фаза 6).

Фаза 6 — Реализация

6.1 Создание задач (TR)

```
Operation: sync-tasks
  Input:  verified SR/SPEC set
  Output: TR in implementation tracker (7 tasks)
          each TR: parent SR, implements-spec[], QG-0 ready (Goal +
AC)
```

6.2 Разработчик берёт TR-101

```
Operation: task-start TR-101
  QG-0 checks:
    ✓ Goal from SR-01
    ✓ AC list (4 items)
    ✓ parent.id resolves (approved)
    ✓ implements-spec present
    ✓ negative scenario in AC
  → work session allowed
```

6.3 Реализация (фрагмент)

```python
# acmecorp-login.src/src/auth/registration.py
from fastapi import HTTPException
from config import settings   # allowed-domains из SPEC-SEC-01

def validate_email_domain(email: str) -> None:
    domain = email.split("@", 1)[-1].lower()
    if domain not in settings.AUTH_ALLOWED_DOMAINS:
        raise HTTPException(
            status_code=422,
            detail={
                "error": "email-domain-not-allowed",
                "allowed-domain": settings.AUTH_ALLOWED_DOMAINS[0],
            },
        )
```

```python
# acmecorp-login.src/tests/auth/test_registration.py
def test_allowed_domain_succeeds(client, db, mock_ses):
    r = client.post("/auth/register",
        json={"email": "alice@acmecorp.com", "password":
"ValidPass123!"})
    assert r.status_code == 201
    assert "user_id" in r.json()
    user = db.query(User).filter_by(email="alice@acmecorp.com").one()
    assert user.verified is False
    mock_ses.send_email.assert_called_once_with(
        template_id="verification-email",
        to_email="alice@acmecorp.com",
    )

def test_disallowed_domain_rejected(client, db):
    r = client.post("/auth/register",
        json={"email": "bob@gmail.com", "password": "ValidPass123!"})
    assert r.status_code == 422
    assert r.json() == {
        "error": "email-domain-not-allowed",
        "allowed-domain": "acmecorp.com",
```

```
    }
    assert db.query(User).count() == 0
```

6.4 Хук валидации на стороне носителя

При попытке атомарной единицы изменения на носителе реализации — хук проверяет связь TR↔SR и наличие негативного TC:

```
[hook носителя] Проверка связей TR:
  ✓ TR-101 parent.id: SR-01 (approved)
  ✓ TR-101 implements-spec: [SPEC-API-01, SPEC-SEC-01]
[hook носителя] Негативные TC:
  ✓ SR-01.verified-by включает TC-002, TC-004 (negative)
✓ Изменение разрешено
```

Фаза 7 — QG-2 (шлюз верификации)

7.1 CI запускает TC

```
[CI] pytest acmecorp-login.src/tests/auth/test_registration.py
  test_allowed_domain_succeeds ..............  PASSED
  test_disallowed_domain_rejected ............  PASSED

[CI] Bot обновляет last-run в TC файлах:
  TC-001: last-run.result = pass, requirement-version = 1.0
  TC-002: last-run.result = pass, requirement-version = 1.0
  TC-003: last-run.result = pass, requirement-version = 1.0
  TC-004: last-run.result = pass, requirement-version = 1.0
```

7.2 Выборочная проверка (Правило 5 Core)

Раз в спринт инженер вручную запускает 5 случайных passing TC и сверяет фактический результат с SR:

```
Operation: tc-spot-check --count 5
  Selected: TC-001, TC-008, TC-012, TC-019, TC-024
  Result: 5/5 совпадают с ожидаемым поведением в SR
```

7.3 Promote SR-01 → verified

```
Operation: promote --to verified --requirement SR-01

QG-2 — предусловия:
  ✓ approved ADAPT linkage
  ✓ pos/neg TC passing; last-run.requirement-version зафиксирован
  ✓ выборочная проверка пройдена (текущий спринт)

Постусловие: SR-01 approved → verified; обновляется индекс покрытия
```

Фаза 8 — Дельта-ТЗ

8.1 Клиент через неделю

```
# TZ-2026-051 — Дополнение к TZ-2026-042
Базовый: TZ-2026-042

## §2 (изменение)
### ФТ-001 (расширение)
Дополнительно разрешить @subsidiary.acmecorp.com (дочерняя компания).
Whitelist расширяется до 2 доменов.
```

8.2 Delta-ADAPT

```
Operation: adapt-from-tz (delta)
  Input:  TZ-2026-051 + parent ADAPT-001
```

```
  Output: draft ADAPT-001-delta-1
          delta Forward + backward findings (e.g. B-008 scope)
```

После 1 итерации с клиентом — `ADAPT-001-delta-1` → approved.

8.3 Анализ влияния

```
Operation: impact-analysis --delta TZ-2026-051

Affected:
  BR-01 (расширение охвата)
  SR-01: verified → approved (TC rerun required)
  TC-001..004: re-pin 1.0 → 1.1; +2 new TC (subsidiary domain)
  TR-115: new implementation task
  SPEC-SEC-01: allowed-domains extended
```

8.4 Apply delta

Архитектор открывает изменения с маркером `[delta:TZ-2026-051]`. AI обновляет SR-01 (расширяет whitelist), генерирует 2 новых TC. Реализация в TR-115. CI прогоняет TC, бот обновляет last-run. После выборочной проверки — SR-01 v1.1 → verified снова.

Фаза 9 — QG-4 (приёмка)

9.1 Через 4 недели после release

```
Operation: evaluate-release v1.0
  Window: 4 weeks post-release

BR-01 KPI: Time-to-first-login — target <2min P95, actual 1.4min
(143%)
BR-02 KPI: 2FA adoption — target ≥95%, actual 97%
```

9.2 QG-4 report + adversarial

AI генерирует acceptance report. Adversarial critic находит: «recovery через admin не покрыт TC, верифицирующим full flow с tickets». Архитектор соглашается → создаёт mini-delta для добавления acceptance TC.

9.3 Sign-off

После закрытия findings клиент подписывает acceptance. BR → status `accepted`.

```
acmecorp-requirements/           # носитель требований (раскладка —
guide/03 или guide/04)
├── QG-4-REPORT-v1.0.md          # архив отчёта
└── lessons/2026-Q2.md           # lessons learned
```

Финальные артефакты

```
acmecorp-requirements/           # носитель требований (раскладка —
guide/03 или guide/04)
├── adapt/
│   ├── ADAPT-001-main.md                    (status: frozen)
│   └── ADAPT-001-delta-1.md                 (status: frozen)
├── br/
│   ├── BR-01-self-service-registration.md (status: accepted)
│   └── BR-02-secure-mfa.md                  (status: accepted)
├── sr/
│   ├── SR-01-email-domain-validation.md   (v1.1, verified)
│   ├── SR-02-totp-enrollment.md           (v1.0, verified)
│   ├── SR-03-totp-verification.md         (v1.0, verified)
│   ├── SR-04-password-recovery-via-admin.md (v1.0, verified)
│   ├── SR-05-rate-limiting-failed-logins.md (v1.0, verified)
│   ├── SR-06-audit-logging.md             (v1.0, verified)
│   └── SR-07-data-residency-ru.md         (v1.0, verified)
├── specs/
│   ├── ui/SPEC-UI-01-login-flow.md        (verified)
│   ├── api/SPEC-API-01-auth.md            (verified)
│   ├── data/SPEC-DATA-01-user-model.md    (verified)
```

```
│   └── sec/SPEC-SEC-01-auth-policy.md     (verified, v1.1)
├── tests/
│   └── TC-001..TC-028                     (28 TC, 100% passing)
├── tz/
│   ├── TZ-2026-042.md                     (initial, immutable)
│   └── TZ-2026-051.md                     (delta, immutable)
├── elicitation/                           # артефакты фазы 0
├── lessons/2026-Q2.md                     (уроки фазы 9)
└── QG-4-REPORT-v1.0.md                    (acceptance report)
```

Метрики проекта

После accepted release:

Метрика	Значение
RDLT (TZ signed → all SR verified)	11 days
Coverage Velocity	100% за 2 спринта
Hallucination Rate (детектированных)	0%
Adversarial findings caught (cycle 1)	4 high + 2 medium
Test-spec drift на delta-T3	0%
Acceptance disputes	0 (1 finding, resolved before sign-off)
Cost per BR	$0.46 (gen) + $0.18 (critic) = $0.64
Total AI cost	~$8.50
BRs accepted	2/2
Дней до accept	35

Что показывает этот пример

1. **Прозрачность** — каждый артефакт имеет provenance, каждый переход — шлюз с явными условиями.
2. **Скорость** — декомпозиция approved ADAPT — десятки секунд + 2 цикла adversarial.

3. **Трассировка** — от строки в ТЗ до passing TC за несколько операций запроса к носителю.
4. **Дельта-ТЗ** — затронутые SR/SPEC/TC/TR вычисляются автоматически.
5. **Замыкание контура** — QG-4 связывает результат с бизнес-метриками (KPI achievement).
6. **AI-нативность** — критик и генератор — разные модели (изоляция); выборочная проверка находит расхождения, которые может пропустить только автоматический прогон.

Что дальше

- Переход с legacy подхода (без ADAPT, без SPEC, без adversarial) — 02-transition-guide.md.
- Git как носитель артефактов — подробности в 03-tool-guide-git.md.
- Документо-ориентированный носитель — 04-document-store-substrate.md.
- Сравнение с SAFe / BABOK / ISO 29148 — 05-safe-comparison.md.
- Compliance mapping (GDPR / ФЗ-152 / AI Act) — 06-compliance.md.
- Failure modes — 07-failure-modes.md.

Сквозной пример RENAR 1.0-draft — renar.tech

---

title: "Переход на RENAR" description: "Поэтапная миграция команды от ручного workflow ТЗ→код к RENAR. От RENAR-1 к RENAR-5 без big-bang." order: 2 lang: ru version: "1.0-draft"

02. Переход на RENAR

> *Команды редко начинают с чистого листа. Эта глава — про то, как перевести существующий проект с ручного цикла «ТЗ → код» на RENAR постепенно, шаг за шагом, без остановки разработки. Каждый уровень даёт осязаемую пользу; команда выбирает, какого из* `RENAR-N` *достигать исходя из реальной ценности, а не из гонки за «полным соответствием» объявлениям на бумаге.*
>
> ***Предпосылки:*** *RENAR Core, standard/11-maturity-model (закрытый список уровней RENAR-1..RENAR-5), guide/07-failure-modes. Уже на раннем RENAR с legacy типами (* `INT-TC` *,* `AIC` *, ...) — см. 10-migration-v1.*

---

1. Оценка: где команда сейчас

Прежде чем мигрировать, оцените текущее состояние. Чек-лист «pre-RENAR»:

Признак	Pre-RENAR	RENAR-1 минимум
ТЗ существует как артефакт	В чате / Google Doc / Notion	Зафиксировано в носителе как файл
Кто-то ведёт требования	Только в трекере (Jira / Linear)	В носителе как BR / SR / ADAPT
Откуда взялось требование	По памяти / переспрашиваем	Любой может найти источник в носителе
Изменения требований	Устно на дейли	Через явный change-set (delta-ТЗ)
Тесты	В коде, привязки к требованиям нет	TC как артефакты или хотя бы в коде с упоминанием SR-ID

Если 3+ строк в колонке «Pre-RENAR» — команда **до RENAR-1**. Это нормальная стартовая точка для большинства проектов.

1.1 Сигналы готовности

Команда готова к миграции, если:

- Болит, что требования теряются между чатами / тикетами / документами.
- Disputed acceptances случаются регулярно — «мы не это просили».
- При onboarding нового инженера месяцы уходят на восстановление контекста.
- Используется AI для генерации требований / кода, но нет систематической проверки.

Если ничего из этого не болит — RENAR может быть преждевременной оптимизацией. См. §8 «когда не нужен».

---

2. Этап 1 — войти в RENAR-1 (Ad-hoc)

Цель уровня: требования живут в носителе, не в чате; для каждого ТЗ есть ADAPT.

Типичная длительность: 1-2 недели.

2.1 Что добавляется

1. Выбирается и заводится **носитель** артефактов (`substrate`): каталог `<project>.req/` в репозитории, рабочая область document store или иной носитель — RENAR к конкретной реализации не привязан; см. возможности **V1–V6** (глоссарий §2.7).
2. Существующее ТЗ переносится в эту среду как не подлежащий произвольным правкам артефакт (с датой и подписями).
3. Для каждого ТЗ создаётся `ADAPT` — артефакт-мост: раздел Forward («как мы поняли») и backward (вопросы заказчику).
4. Новые требования заносятся как BR / SR файлы в носителе, не только в трекер.

2.2 Что НЕ требуется на этом этапе

- Стандартизированный frontmatter (минимум — `id` + `title`).
- Lifecycle статусы (`draft` / `approved` /...) — артефакты могут жить без явных переходов.
- TC как отдельные артефакты.
- Hooks носителя.
- Нативный для носителя COVERAGE.

2.3 Типичные блокеры

- **«У нас уже есть тикеты в Jira»** — оставьте. RENAR-1 не требует миграции; tracker и носитель могут сосуществовать. Главное — носитель теперь источник

истины для **новых** требований.
- **«ADAPT — лишняя работа»** — на одно ТЗ ADAPT занимает 1-2 часа. Это окупается на первом же спорном acceptance.
- **«Где хранить?»** — любой носитель, удовлетворяющий **V1–V6**. См. guide/03-tool-guide-git или guide/04-document-store-substrate.

2.4 Когда переходить на RENAR-2

Когда **новые требования** перестали уходить в чаты и начали стабильно появляться в носителе. Это занимает 4-6 спринтов привычки.

---

3. Этап 2 — перейти на RENAR-2 (Documented)

Цель уровня: структура и frontmatter; delta-ТЗ как явный change-set.

Типичная длительность: 2-4 недели после RENAR-1.

3.1 Что добавляется

1. Каждый BR / SR получает frontmatter с обязательными полями (см. reference/02-schemas).
2. Папки структурируются: `br/`, `sr/`, `adapt/`, `dpia/`, ...
3. Изменения требований — только через delta-ТЗ (новый immutable артефакт), не через прямое редактирование.
4. ТЗ зафиксировано как immutable (изменения только через delta-ТЗ).

3.2 Шаблон: легализация существующих требований

Старые требования (которые уже были в носителе с RENAR-1) — пройти рецензирование и проставить frontmatter «как есть» без пересмотра содержания. Это **legalization**, не **rewrite**:

```
---
id: BR-12
title: "Регистрация сотрудника через корпоративный email"
status: approved          # уже работает в проде
created-at: "2025-12-01"  # ретроактивно
priority: must            # ретроактивная оценка
legacy: true              # маркер: требование не проходило ADAPT с
```

```
нуля
---
```

Поле `legacy: true` опционально, но рекомендовано — отличает «исторические» требования от новых, которые с самого начала прошли весь RENAR-пайплайн.

3.3 Типичные блокеры

- **«frontmatter на 100 требований — это месяц»** — да. Делайте в фоне, по 10-15 требований / неделю; новые требования сразу с frontmatter.
- **«Delta-T3 замедляет»** — на первых неделях да. После 2-3 итераций обычно становится быстрее, чем «давай просто поменяем».
- **«Не понимаем, какой priority ставить ретроактивно»** — оставьте `priority: should` для всего legacy; явно `must` ставится только при подтверждении со stakeholder.

3.4 Когда переходить на RENAR-3

Когда frontmatter валиден для 80%+ артефактов и команда привыкла к delta-T3.

---

4. Этап 3 — перейти на RENAR-3 (Tracked)

Цель уровня: автоматическая валидация + lifecycle enforcement + TC покрытие для priority=must.

Типичная длительность: 4-8 недель после RENAR-2.

4.1 Что добавляется

1. Hooks носителя валидируют frontmatter по schema на каждое изменение. Невалидные — блок integration.
2. Lifecycle статусы используются реально: каждый артефакт в одном из закрытых состояний (`draft` / `approved` / `verified` / `deprecated` / `obsolete`).
3. TC создаются для всех priority=must BR / SR / SPEC.
4. Нативный для носителя COVERAGE report auto-generated на каждый promote-transition.
5. Reference-validation hook: создание BR / SR с ссылкой на ADAPT в статусе ниже `approved` — блок.

6. Реализация ссылается на BR / SR / SPEC с pinned `verifies[].version`.

4.2 Шаблон: backfill TC

Для всех priority=must требований без TC:

1. Отсортировать по «частоте упоминания в incident reports» — где TC даст максимум value.
2. Покрывать по 3-5 требований / спринт, не пытаясь backfill сразу всё.
3. `TC` создаются как артефакты в вашем носителе со связью `verified-by`.

4.3 Типичные блокеры

- **«Старые требования упрямо не приводятся к schema»** — оставьте их в legacy-папке; новые сразу schema-valid. Hook носителя применяется только к новым / изменяемым артефактам.
- **«Hooks замедляют PR cycle»** — измерьте: если > 30s — оптимизируйте hooks (parallelization, caching); не «отключить».
- **«Команда обходит hooks через `--no-verify`»** — это organizational failure pattern; root cause — hooks слишком медленные / шумные. Чините, не запрещайте.

4.4 Когда переходить на RENAR-4

Когда COVERAGE для priority=must = 100%, frontmatter валиден везде, lifecycle действительно работает.

---

5. Этап 4 — перейти на RENAR-4 (Verified)

Цель уровня: для всех утверждённых артефактов есть успешно пройденные контрольные примеры (`TC`); переход под контрольной точкой **`QG-2`** (`Verification Gate`) обеспечивается носителем; соблюдена парность позитивных / негативных проверок; для материала от ИИ задан блок `ai-provenance`.

Типичная длительность: 6-12 недель после RENAR-3.

5.1 Что добавляется

1. 100% approved артефактов имеют `verified-by` ссылку на ≥ 1 TC.
2. Pos/neg парность для каждого нормативного утверждения.
3. Контрольная точка `QG-2` (`Verification Gate`) блокируется встроенными в носитель проверками: перевод в `verified` только при всех успешных `TC` для текущей `requirement-version` .
4. Все TC автоматизированы или явно `manual-pending` с deadline.
5. Для `tc-type: ux` — VLM-judge isolation.
6. Для `tc-type: eval` — judge-модель ≠ модель реализации.
7. `ai-provenance` для AI-сгенерированных артефактов: минимум `generated-by` + `generated-at` .
8. Source citation — каждое нормативное утверждение имеет pointer на источник в T3 или ADAPT.
9. Привязка согласования (reconciliation) запускается носителем не реже одного раза в неделю.
10. Spot-check 5 случайных passing TC раз в спринт.

5.2 Шаблон: поэтапное покрытие

Достижение 100% verified-by покрытия — не одним PR. Подход:

1. Сначала pos-only TC для всех priority=must (бенефит — быстрая обратная связь).
2. Затем neg-TC pairs (бенефит — отлов test-fitting drift).
3. Затем `tc-type` extensions (ux / eval / contract / security) по мере необходимости.

5.3 Типичные блокеры

- **«Изоляция judge-модели дорого»** — это правда. Но это структурный rate limit на AIR-06 test-fitting drift — нельзя обойти без потери verification integrity.
- **«Source citation замедляет авторов»** — автоматизируйте: AI-генератор должен сразу выдавать citation; ручная авторизация — только для legacy backfill.
- **«Reconciliation findings заваливают команду»** — tunable thresholds; начинайте с conservative defaults, постепенно ослабляйте.

5.4 Когда переходить на RENAR-5

Когда RENAR-4 стабильно работает 2-3 квартала, метрики стабильны, команда не «горит» от reconciliation noise.

---

6. Этап 5 — перейти на RENAR-5 (Optimized)

Цель уровня: adversarial review как gate; multi-model agreement для priority=must; cost/latency budgets; knowledge graph как primary search; continuous evaluation для AI-критических компонентов.

Типичная длительность: 12+ недель после стабильного RENAR-4.

6.1 Что добавляется

1. Adversarial critic — обязательный gate для `draft → approved`. Critic — модель, отличная от модели генерации.
2. Multi-model agreement для priority=must: артефакт генерируется ≥ 2 моделями; расхождения помечены и обязательны к разбору.
3. Cost / latency budget per artifact; превышение → автоматическая декомпозиция.
4. Knowledge graph как primary search для AI-агентов.
5. Continuous evaluation для SPEC-AI.
6. Метрика Hallucination Rate < 1%.
7. Метрика Multi-model Disagreement Rate отслеживается.
8. Возврат улучшений шаблонов в `requirements-library` — стандартная практика.

6.2 Когда RENAR-5 нужен

- Регулируемые отрасли (финтех, healthcare, AI-системы high-risk per AI Act).
- Когда продукт критически зависит от качества AI-генерации (генеративные продукты).
- Когда есть бюджет на continuous evaluation infrastructure.

Многие проекты могут остановиться на RENAR-4 — этого достаточно для **соответствия заявленному профилю** RENAR (conformance). RENAR-5 не обязательно «лучше», зато почти всегда **дороже и строже**. Выбирайте по потребности, а не по моде.

---

7. Миграция legacy-требований

Существующие тысячи требований в Jira / Confluence — как втягивать?

7.1 Стратегия не-миграции

Если legacy требования не активно меняются — **не мигрируйте**. RENAR применяется только к новым требованиям и активно изменяемым. Legacy остаётся в исходном месте

как read-only «исторический контекст».

7.2 Стратегия выборочной миграции

Для legacy, которые активно меняются:

1. На момент первого change — затянуть в носитель как BR/SR с `legacy: true` маркером и минимальным frontmatter.
2. Применить change как delta-T3.
3. Через 1-2 итерации требование «созревает» — становится full-RENAR (без legacy маркера, с полным frontmatter и TC).

7.3 Стратегия bulk-import

Когда нужно мигрировать сразу > 100 требований (например, при organizational reorganization):

1. Скриптовая выгрузка из tracker → frontmatter скелет (id, title, минимум).
2. Все импортированные требования — `legacy: true` + `status: approved` (как есть в проде).
3. Backfill TC и full frontmatter — спринт за спринтом, не блокируя current work.

---

8. Когда RENAR НЕ нужен

RENAR — overhead. Не применяйте к:

- **One-off скриптам и прототипам** — не доходят до production, или становятся production только через rewrite.
- **Очень маленьким командам (1-2 человека)** — overhead управления носителем может превышать benefit.
- **Проектам без AI-генерации требований / тестов** — главная ценность RENAR (защита от AI-specific failure modes) теряется.
- **Краткосрочным экспериментам (≤ 1 спринт)** — не успеете окупить ADAPT-setup.

Минимальный порог окупаемости: проект с ≥ 3 итерациями требований, ≥ 2 разработчиков, использует AI где-либо в pipeline (генерация требований / кода / тестов / документации).

---

9. Антипаттерны миграции

Чего избегать:

9.1 Миграция «big-bang»

Симптом: Команда останавливает разработку на 2 спринта чтобы «перейти на RENAR». Через 2 спринта получают burnout и брошенную миграцию.

Вместо: Гибридный режим. Новые требования сразу в RENAR, старые — по мере касания. Долго, но устойчиво.

9.2 Пропуск уровней

Симптом: «Давайте сразу на RENAR-4». Команда пропускает RENAR-2/3, ставит full hooks + ai-provenance + adversarial critic, но frontmatter не валиден и lifecycle хаотичен.

Вместо: Уровни идут по порядку. RENAR-4 без RENAR-3 базы — не работает.

9.3 Паралич «идеального frontmatter»

Симптом: Команда тратит недели на полирование одного frontmatter — «а правильно ли я выбрал `priority`?» Реальная работа стоит.

Вместо: frontmatter — «достаточно хорошо». Ошибки правятся в delta-T3. Главное — что-то быть в носителе, а не вылизанность.

9.4 Частичное принятие носителя

Симптом: Половина требований в носителе, половина — всё ещё в Jira. Никто не знает, где источник истины.

Вместо: Single source of truth должен быть **сразу**. Если нельзя перенести всё — перенесите только новые, явно объявите носитель владельцем «всего нового».

9.5 Подход «сначала tooling»

Симптом: Команда полгода пишет внутренние tooling вокруг RENAR (свой validator / своя CI / своя UI), не используя сам стандарт.

Вместо: Сначала практика на минимальном носителе (даже плоская папка с markdown). Tooling — когда становится больно вручную.

Стоимость и выгода внедрения (иллюстративно)

> *Секция **иллюстративная и ненормативная** — помогает прикинуть порядок величин. Конкретные числа зависят от проекта; реальную модель калибруйте по своим данным.*

Стоимость внедрения (что вкладывается):

- Обучение команды Core (5 правил + ADAPT) — порядка полдня-дня на инженера.
- Дисциплина ADAPT на каждое ТЗ — дополнительные часы на elicitation / backward до старта кода (окупаются отсутствием переоткрытия ТЗ позднее).
- Носитель уже есть (git) — отдельных лицензий не требуется; tooling-автоматизация опциональна и вводится постепенно.

Выгода (что возвращается):

- Сокращение времени декомпозиции ТЗ с AI-ускорением (порядок величин — standard/12 §12.5.1: 5–10× на RENAR-4, иллюстративно).
- Меньше споров на приёмке: ADAPT с двойной подписью фиксирует интерпретацию до кода.
- Меньше инцидентов из негативных сценариев — за счёт обязательной pos/neg-парности ТС.
- Audit trail «что сдавали по контракту» — бесплатно из носителя (V1 / V6).

Когда не окупается: короткоживущие прототипы, чистый продуктовый дискавери без договора, команды без compliance-давления и без внешнего клиента (см. §8 «когда RENAR не нужен»). Для них достаточно RENAR Core или ничего.

> *Количественная ROI-модель (порядок экономии на проект и т.п.) пока живёт в research-материалах; перенос откалиброванной версии в этот раздел запланирован в **бэклоге фазы 8** для v1.1.*

---

10. Связанные документы

- standard/11-maturity-model — нормативные определения RENAR-1..RENAR-5 уровней (closed list).
- 00-quickstart — 30-минутный sample для маленького проекта.
- 01-walkthrough — полный example на одном проекте.
- 07-failure-modes — что может пойти не так при миграции; organizational failure patterns §5.

- reference/02-schemas — frontmatter schemas, обязательные для RENAR-2+.
- 03-tool-guide-git — специфичный для носителя guide для git.
- 04-document-store-substrate — informative обзор носителя document-oriented store.

---

11. Зафиксированные решения для v1.0

- **Legacy backfill bulk-import — bypass запрещён.** На initial import артефакты вносятся со статусом `imported-legacy` (специфичный для носителя marker, не входит в нормативный closed list lifecycle §10.5–§10.8) и не участвуют в conformance assessment до промотирования через нормальный QG-0 flow. Это сохраняет §1.7.3 declared-weaker запрет: validation не обходится, а лишь отложена до момента, когда артефакт начнёт claim conformance.
- **Откат с уровня RENAR-3 → RENAR-2 допустим** через formal downgrade per §13.8.2: выпускается новая версия manifest с пониженным `level`. Recovery plan **не** обязателен (downgrade — intentional, не loss-of-conformance), но рекомендуется зафиксировать причину downgrade в audit trail.
- **Cross-org migration: каждая подсистема — свой manifest с собственным `level`** (§13.4). Organization-aggregate "RENAR-N" неформально определяется минимумом уровней подсистем; организационный manifest **не** нормируется RENAR v1.0.

11.1 Отложено на v1.1 (бэклог фазы 8)

- **Шаблоны миграции для команд 50+ инженеров.** Гайд ориентирован на группы 5–15 человек; для большего масштаба нужны полевые наблюдения. Ответственные: сопровождение стандарта RENAR и ранние внедренцы.

---

title: "Носитель: VCS (git)" description: "Реализация V1-V6 на git: layout .req репо, submodule pinning, PR/MR workflow, pre-commit hooks, delta-T3 flow." order: 3 lang: ru version: "1.0-draft"

03. Носитель: VCS — git

> *Конкретная реализация RENAR на git. Структура* `.req` *репозитория, submodule-pinning между* `.req` *и* `.src`*, PR/MR ревью workflow, pre-commit hooks для capability V1-V6, delta-T3 workflow. Этот guide — informative; нормативное содержание (capability требования, schemas, lifecycle) — в* `standard/`*. Альтернатива на document-oriented store — guide/04-document-store-substrate.*
>
> ***Предпосылки:*** *standard/03-substrate-versioning (нормативные capability V1-V6), reference/02-schemas (frontmatter schemas).*

---

1. Когда выбрать git как носитель

Git — носитель по умолчанию для:

- Открытых стандартов и проектов (нет зависимости от внутренней инфры).
- Внешних клиентов без выделенной document-store инфраструктуры.
- Команд с устоявшимся PR-workflow.
- Проектов, где требования и код в одной экосистеме (один и тот же VCS provider).

Git **не** оптимален, если:

- Нужны частые конкурентные правки одного артефакта несколькими авторами без merge conflicts.
- Нужен built-in полнотекстовый поиск без внешних инструментов.
- Требуется UI для non-technical stakeholder (PM, юристов) без git CLI.

В таких случаях рассмотрите document-oriented store — guide/04.

---

2. Layout двух репозиториев

Каноническая структура — два связанных репо.

```
<project>/
├── <project>.req/                ← репозиторий ТРЕБОВАНИЙ (отдельный
VCS repo)
│   ├── tz/                       ← TZ-YYYY-NNN.md (immutable после
регистрации)
│   ├── adapt/                    ← ADAPT-NN.md (bridge artefacts)
│   ├── br/                       ← BR-NN.md
│   ├── sr/                       ← SR-NN.md
│   ├── specs/                    ← SPEC-* по подпапкам (arch/, api/,
data/, int/, …)
│   │   ├── arch/  api/  data/  ui/  ai/  int/  proc/  sec/  ops/
│   ├── tr/                       ← TR-NN.md (task requirements)
│   ├── tests/                    ← TC-NN.md (контрольные примеры,
`TC` — самостоятельные артефакты)
│   ├── dpia/                     ← DPIA-NN.md (опционально для
regulated)
│   ├── library/                  ← templates, patterns
│   ├── docs/                     ← AI-generated документация
│   ├── COVERAGE.md               ← auto-generated (`[coverage]`
commits)
│   ├── REQUIREMENTS.md           ← auto-generated index
│   └── TEST-PLAN.md              ← auto-generated
└── <project>.src/                ← репозиторий РЕАЛИЗАЦИИ
    ├── src/                      ← код
    ├── tests/                    ← реализации TC (адресуются
`automation.location`)
    ├── requirements/             ← submodule → <project>.req @
<commit>
    ├── .gitmodules
    └── README.md
```

В `<project>.req/.gitattributes` для bot-generated артефактов:

```
COVERAGE.md      linguist-generated=true
REQUIREMENTS.md  linguist-generated=true
TEST-PLAN.md     linguist-generated=true
docs/**          linguist-generated=true
```

Это исключает их из статистики кода и сильно сжимает PR diff.

---

3. Capability mapping V1-V6 на git

Capability	Норматив	Git-механизм
V1 — Versioned content addressing	Каждый артефакт имеет stable identifier + content hash	git commit SHA + file path; `id:` поле во frontmatter (stable)
V2 — Schema validation	Структура артефактов проверяется автоматически	Pre-commit hook + CI job: `yamllint` + JSON Schema validator против `reference/02-schemas.md`
V3 — Lifecycle state enforcement	Переходы статусов следуют state machine	Pre-commit hook: проверка legal-transition (старый status → новый status) per standard/10-lifecycle-qg
V4 — Reporting / aggregation	Coverage / index / status reports auto-generated	Bot-commit на post-merge: `scripts/generate-coverage.js` + `[coverage]` тег
V5 — Reference integrity	Cross-references между артефактами не «висят»	CI job: парсит `parent`, `verified-by`, `constrained-by`, `verifies` поля; проверяет существование targets
V6 — Drift detection	Дрейф между артефактами и реализацией ловится автоматически	CI job (еженедельно + on-demand): walks frontmatter, проверяет соответствие с кодом, `last-run.requirement-version` ≥ current

> ***Состояние сценариев.*** *Приведённые ниже механизмы под* `git` *—* ***целевой образец v1.0****. Фактически готов только* `scripts/validate-frontmatter.js`*; остальные (* `validate-lifecycle`*,* `validate-references`*,* `generate-coverage`*,* `detect-drift` *и др.) — в* ***бэклоге фазы 8*** *(раздел §8 этого гайда). Пока скриптов нет, перечисленные возможности обеспечиваются вручную и код-ревью; автоматическое навязывание уровней RENAR-3+ «из коробки» под* `git` *пока недостижимо. Замечание относится и к document-store (guide/04).*

---

4. Submodule pinning

`<project>.src` фиксирует **конкретный commit** `<project>.req` через git submodule.

4.1 Как работает

- В `<project>.src` директория `requirements/` — submodule на `<project>.req`.
- При сборке / CI код знает: «я реализую требования по состоянию на commit `abc1234`».
- Разработчик в задаче открывает `requirements/sr/SR-05.md` через обычный `cat` или IDE — это файл в worktree.

4.2 Bump pattern

При обновлении требований:

1. PR в `<project>.req` с изменениями требований → ревью → merge.
2. **Отдельный** PR в `<project>.src`, который **только** двигает submodule pointer:

```
cd requirements
git pull origin main
cd ..
git add requirements
git commit -m "bump requirements: TZ-2026-042 delta + 3 new
SR"
```

3. Этот PR явно показывает: «требования обновились до коммита X».

4.3 Почему submodule, не subtree / monorepo

- **Provenance:** для любого коммита кода точно известно, какую версию требований он реализовывал.
- **Изоляция ревью:** ревью требований (в `.req`) и ревью кода (в `.src`) не смешиваются.

- **Атомарность delta-T3:** delta — это атомарный PR в `.req` + последующий submodule-bump в `.src`. Нет «частично применённой delta».
- **Совместимость с document store:** при переходе на document-oriented store submodule pinning превращается в revision-token pinning — концепция та же.

Альтернативы (subtree, monorepo): рассматривайте только если submodule не работает по причинам, специфичным для вашего VCS provider; во всех остальных случаях submodule — рекомендация.

---

5. PR/MR review workflow

Два уровня ревью, разделённые по репозиториям.

5.1 Ревью в `<project>.req`

Reviewer focus:

- Frontmatter schema-valid.
- Citation на T3 / ADAPT присутствует и валиден.
- `parent` / `verified-by` / `constrained-by` ссылки существуют.
- Lifecycle transition легитимна.
- Source citation в body для каждого нормативного утверждения (на RENAR-4+).
- Adversarial AI-review prompt пройден (на RENAR-5).

Approval = QG-0 (standard/10 §10.3.1).

5.2 Ревью в `<project>.src`

Reviewer focus:

- Submodule pointer соответствует merged commit в `.req`.
- Реализация ссылается на актуальные SR / SPEC через `verifies[].version`.
- Новые / изменённые TC соответствуют контракту в `.req`.
- Никаких изменений Pass/Fail-критериев TC без `[test-spec-change]` тега.

Approval = QG-2 (standard/10 §10.3.3) — после прохождения автоматизированных TC.

5.3 Запрещённые анти-паттерны

- **PR одновременно в `.req` и `.src`** — нарушает изоляцию ревью; запрещён hook носителя.
- **Изменение submodule pointer без merged commit в `.req`** — pointer указывает в untracked commit; CI блокирует.
- **`[test-spec-change]` без отдельного approval** — Pass/Fail-критерий TC изменён вместе с code-fix; CI блокирует merge.

---

6. Pre-commit и pre-merge hooks

Минимальный набор hooks носителя для RENAR-3+ на git.

6.1 Pre-commit (в `<project>.req`)

```
# Запускается на коммит в .req
# Capability V2: schema validation
yamllint --strict $(git diff --cached --name-only | grep '\.md$')
node scripts/validate-frontmatter.js $(git diff --cached --name-only
--diff-filter=AM)

# Capability V3: legal lifecycle transitions
node scripts/validate-lifecycle.js $(git diff --cached --name-only --
diff-filter=M)

# Capability V5: reference integrity (быстрая проверка только
изменённых)
node scripts/validate-references.js --changed-only $(git diff --
cached --name-only)
```

6.2 Pre-merge (CI job в `<project>.req`)

Полные проверки, которые медленны для pre-commit:

```
- name: Full reference validation (V5)
  run: node scripts/validate-references.js --all

- name: Coverage report regeneration (V4)
```

```
    run: node scripts/generate-coverage.js
    # commits as [coverage] bot user

  - name: Drift detection (V6, weekly)
    run: node scripts/detect-drift.js
    if: github.event.schedule == '0 0 * * 0'
```

6.3 Pre-merge (CI job в `<project>.src`)

```
  - name: Submodule points to merged commit
    run: |
      cd requirements
      git fetch origin
      git merge-base --is-ancestor HEAD origin/main

  - name: TC versions pinned to current requirement-version
    run: node scripts/validate-tc-version-pinning.js

  - name: No Pass/Fail change without [test-spec-change]
    run: node scripts/validate-test-spec-changes.js
```

---

7. ТЗ workflow на git

7.1 Forward-workflow (проект с нуля)

Первичное создание оси требований из нового ТЗ:

1. **branch в `.req`:** `git checkout -b init/TZ-2026-001` в `<project>.req`.
2. **Регистрация ТЗ:** `tz/TZ-2026-001.md` (immutable после регистрации; зафиксировать подпись клиента и дату).
3. **Создание ADAPT:** `adapt/ADAPT-001.md` — Forward (интерпретация по каждому § ТЗ) + Backward (вопросы клиенту), standard/07. Backward отрабатывается с клиентом до approval.

4. **Двойная подпись ADAPT → QG-ADAPT-approve:** ADAPT в `approved` (клиент + архитектор), `open-questions-count == 0`.
5. **Декомпозиция:** AI-агент выводит BR из approved ADAPT, затем SR (`source.adapt`), SPEC (`constrained-by[]`) и парные pos/neg TC.
6. **QG-0 approval** каждого артефакта (`draft → approved`); CI dry-run новых TC.
7. **PR в `.req` → merge;** bot regenerates REQUIREMENTS.md / COVERAGE.md / TEST-PLAN.md.
8. **Setup `.src`:** добавить submodule `requirements/ → <project>.req @ <commit>`, создать TR, реализовать, QG-2.

7.2 Delta-T3 workflow

Полная последовательность для применения нового delta-T3.

1. **branch в `.req`:** `git checkout -b change/TZ-2026-042` в `<project>.req`.
2. **Создание delta-T3 + delta-ADAPT:** `tz/TZ-2026-042-delta.md` (текст delta) и `adapt/ADAPT-NNN-delta.md` (forward интерпретация + backward findings, standard/07 §7.6). Delta-ADAPT обязан пройти двойную подпись прежде чем затронутые требования будут модифицированы.
3. **Impact analysis:** AI-агент находит затронутые BR / SR / SPEC / TC; помечает TC как `obsolete-pending`.
4. **Update artefacts:** AI-агент обновляет / создаёт BR / SR / SPEC и парные TC (pos+neg) в той же ветке.
5. **Adversarial critic review:** на RENAR-5 — обязательно (другая AI-модель).
6. **CI dry-run:** новые TC должны запускаться без ошибок инфры.
7. **Finalize:** version++, status: `approved`, regenerate REQUIREMENTS.md / COVERAGE.md / TEST-PLAN.md (bot).
8. **PR в `.req` → QG-0 approval → merge.**
9. **branch в `.src`:** `git checkout -b change/TZ-2026-042` в `<project>.src`.
10. **Bump submodule:** `cd requirements && git pull && cd ..`.
11. **Create TR / tasks:** для новых / изменённых SR (через `/task` или специфичный для носителя tooling).
12. **PR в `.src` «bump requirements + tests + new tasks» → ревью → merge.**
13. **Development:** взять TR → реализовать → CI → бот заполняет `last-run` в TC.

14. **QG-2:** при зелёных TC — approval → AI-агент переводит требование в `verified`.

Каждый шаг кроме (1) и (9) автоматизирован нативно для носителя через скрипты или CI.

---

8. Migration notes: scripts/ модернизация

Существующие scripts/ — bash + node helpers из ранней эпохи проекта. Состояние модернизации на v1.0-draft:

- ✅ **Legacy terms вычищены.** `req-branch.sh`, `req-finalize.sh`, `req-ai-instructions.md`, `req-use-template.sh` больше не используют устаревшие `INT-SR`, `AIC`, `UIC`, `tech-specs`, `ai-concepts`, `ui-concepts`. Closed list актуальных типов — standard/04 §4.4 (BR/SR/TR + 9 SPEC). Остаточные упоминания в reference/01-glossary.md и reference/02-schemas.md являются legacy-mapping для проектов, мигрирующих с предыдущих версий.
- ✅ **Schema validator создан.** `scripts/validate-frontmatter.js` — Node ES-module, проверяет frontmatter всех `.md` в `standard/`, `guide/`, `reference/`, `core/`: required fields `title` / `order` / `lang ∈ {ru, en}`. Запуск: `node scripts/validate-frontmatter.js [--quiet]`; exit 0 если все валидны, exit 1 при первой ошибке. Источник схемы — reference/02-schemas.
- ⏳ **ADAPT в forward-workflow.** Начальное создание (T3 → ADAPT → BR) сейчас не задокументировано как отдельный workflow рядом с §7 (delta-workflow); шаг 2 в §7 явно использует delta-ADAPT согласно standard/07 §7.6. Утверждённый forward-workflow — задача отдельного draft главы.
- ⏳ **Pre-commit hooks** (§6.1) пока частично разбросаны по `req-finalize.sh`. Целевое состояние — выделить в отдельные `scripts/validate-*.js`; `validate-frontmatter.js` — первый такой модуль.

Глава описывает **целевой набор скриптов** на выпуске v1.0; строки без отметки ✅ — работа для **фазы 8** (продолжение бэклога).

---

9. Common operations

Шорткаты для частых операций.

Операция	Команда
Найти SR по ID	`grep -r "^id: SR-12" sr/`
Найти TC, верифицирующий SR-12	`grep -rl "id: SR-12" tc/`
Найти orphan SR (без TC)	`node scripts/find-orphans.js sr`
Создать новый SR из шаблона	`cp library/templates/sr.md sr/SR-NN.md && $EDITOR sr/SR-NN.md`
Diff frontmatter за период	`git log --all --since=... -- sr/`
Текущая requirement-version SR-12	`yq '.version' sr/SR-12.md`
Список stale TC (last-run < current version)	`node scripts/list-stale-tc.js`

---

10. CI/CD integration patterns

10.1 Bot-commit conventions

Auto-generated артефакты коммитятся hooks носителя с conventional commit-message тегами для машинного парсинга:

Тег	Когда	Что коммитится
`[coverage]`	Post-merge в `.req`	Регенерация COVERAGE.md / REQUIREMENTS.md / TEST-PLAN.md
`[baseline-update]`	После approval baseline-change PR	Перегенерация PNG-эталонов для SPEC-UI
`[bump-req]`	Submodule bump в `.src`	Только submodule pointer + minimal metadata
`[reconcile]`	Reconciliation hook (V6) находит drift	Авто-fix очевидных рассогласований; flag для остальных

`[test-spec-change]`	Pass/Fail-критерий TC изменён	Manual; требует отдельный approval от reviewer

Hook носителя парсит commit message и применяет различные validation rules в зависимости от тега. `[coverage]` / `[bump-req]` / `[reconcile]` коммиты — от bot user; `[baseline-update]` / `[test-spec-change]` — от human + bot signature.

10.2 Bot-user setup

- Отдельный bot account (например `renar-bot@<org>`) с **write** permission в `<project>.req` и `<project>.src`.
- Bot commits signed (GPG / SSH commit signature).
- Bot user **не** может approve PR (separation of duties — approval всегда human).

10.3 branch protection

В обоих репо:

- `main` (или `master`) — protected. Push требует merged PR.
- Required status checks: schema validation (V2), reference integrity (V5), lifecycle validation (V3).
- Reviewers required: ≥ 1 для большинства; ≥ 2 для priority=must BR / SR / SPEC.

---

11. Cross-references

- standard/03-substrate-versioning — нормативные требования к носителю (capability V1-V6).
- reference/02-schemas — frontmatter schemas для validation hooks.
- 02-transition-guide — где этот guide вписывается в путь от pre-RENAR к RENAR-N.
- 04-document-store-substrate — informative обзор document-oriented store (альтернатива git).
- 07-failure-modes — что может пойти не так на git как носителе (схема обхода hooks, etc.).
- standard/10-lifecycle-qg — нормативные lifecycle переходы, которые validate-lifecycle hook должен enforce.

---

12. Open questions

- Стандартизировать ли минимальные реализации перехватов как **единое** спецификационное описание, на которое опираются и VCS, и document store?
- Submodule vs subtree: какие conditions делают subtree приемлемой альтернативой? Сейчас гайд однозначно за submodule.
- `[coverage]` bot user: best practice для signing / permission в multi-org проектах?
- Drift detection cadence: weekly — хороший default, но что для high-velocity teams (> 50 PR/неделя)?

---

title: "Носитель: document-oriented store (обзор)" description: "Informative обзор реализации V1–V6 на носителе document-oriented store — без vendor-specific деталей." order: 4 lang: ru version: "1.0-draft"

04. Носитель: document-oriented store (обзор)

> ***Informative appendix.*** *Нормативные capability V1–V6 — standard/03. Практический пример на distributed VCS (git) — guide/03.*

Document-oriented store — носитель, где артефакты RENAR хранятся как версионированные документы: стабильный идентификатор, цепочка ревизий (revision token), атомарное обновление, API-шлюз для lifecycle-переходов и подписей.

RENAR формально **не привязан** к классу систем document store — нормативно задаются только возможности (**V1–V6**) (§3.2–11.3).

---

1. Когда выбирать document-oriented store

Подходит, если:

- нужно централизованное enterprise-хранилище требований для нескольких проектов;
- non-technical stakeholders работают через web UI, а не через VCS;
- федерация межпроектных ссылок и поиск — **ключевое** требование к продукту;

Не подходит, если:

- команда уже стандартизирована на git workflow и PR/MR review;
- нет ресурсов поддерживать сервер document store + search index + API gateway;
- нужны **полноценные** контрольные примеры (`TC`) как объекты в той же среде без отдельной настройки пользовательских типов документов (см. §9 — наличие `TC` нужно для декларируемого соответствия RENAR);

---

2. Сопоставление возможностей V1–V6 (обобщённо)

Возможность	Типичный механизм document store

V1 — версионируемая идентичность	Идентификатор документа `_id` и цепочка ревизий
V2 — атомарная единица изменения	Одно целое приращение версии документа за шаг
V3 — сравнение и ревью (diff)	Поток утверждения через API и интерфейс; перехват прямых правок статуса
V4 — ветвление и слияние	Черновые документы либо ветви конфликтов (по возможностям продукта)
V5 — фиксация версии	Поле со ссылкой на ревизию в связанных артефактах
V6 — перехваты и события	Проверка при записи, веб-хуки на переходы жизненного цикла

Сравнение с распределённой VCS — §3.4 (таблица-пример; не является нормативным контрактом).

---

3. Миграция VCS ↔ document store

Миграция возможна, если в обеих реализациях носителя сохраняются:

- канонические идентификаторы артефактов (`BR` / `SR` / `SPEC` / `ADAPT`);
- связи трассируемости;
- состояния жизненного цикла по закрытому списку §10.

Процедура миграции — project-specific runbook; нормативный минимум — проверка V1–V6 до cutover (§3.5).

---

4. См. также

- 03-tool-guide-git.md — специфичный для носителя guide для distributed VCS (git)
- 03-substrate-versioning.md — нормативные V1–V6
- 02-schemas.md — canonical поля; нативное для носителя отображение — informative

---

---

title: "Сравнение с SAFe" description: "Mapping RENAR ↔ SAFe 6.0: WSJF, PI Planning, ART координация, RACI lifecycle, Built-in Quality." order: 5 lang: ru version: "1.0-draft"

05. Сравнение с SAFe

> *Mapping RENAR (стандарт **требований**) на SAFe 6.0 (стандарт **scaled agile координации**). Документы совместимы, но имеют разный scope: SAFe нормирует как команды координируют работу на масштабе; RENAR нормирует что собой представляет требование и как оно верифицируется. Эта глава — для команд, которые уже работают по SAFe (enterprise multi-team ART — типичный пример) и хотят сохранить SAFe ceremonies, добавив RENAR-артефакты как первичный источник истины о требованиях.*
>
> ***Предпосылки:** знакомство с RENAR Core (5 правил, ADAPT, 2 QG) и базовой SAFe 6.0 терминологией (Epic / Capability / Feature / Story, WSJF, PI, ART).*

---

1. Scope: что нормирует RENAR, что — SAFe

RENAR и SAFe пересекаются на уровне *артефактов работы* (Feature, Story), но решают разные задачи.

Аспект	SAFe 6.0	RENAR
Тип стандарта	Scaled Agile coordination framework	Стандарт инженерии требований
Первичный артефакт	Epic / Capability / Feature / Story	T3 → ADAPT → BR → SR → SPEC → TC
Что нормирует	Cadence, roles, ceremonies, flow	Schema, lifecycle, verifiability, drift control
Носитель артефактов	Выбор средства управления задачами (Jira / Rally / ADO / др.)	Без привязки к носителю; нормативно — возможности **V1–V6** (глоссарий §2.7)
Контрольные точки качества	Definition of Done на уровне Feature	`QG-0` (готовность к старту) + `QG-2` (проверено)
AI-нативность	Не нормирует процесс работы с ИИ	ИИ как полноценный участник (контрастная экспертиза *adversarial*, оценивающие сценарии, модели-судьи `judge`)

Ключевой принцип: SAFe и RENAR совместимы. SAFe говорит «как координировать команды на ART», RENAR — «что должно быть истинно про каждое требование, чтобы оно считалось `verified`». Feature в SAFe ≡ SR в RENAR — это **один и тот же артефакт**, описанный с разных сторон.

---

2. Главная таблица соответствия

SAFe artefact	RENAR artefact	Где живёт	Owner	Acceptance criteria
Strategic Theme	(вне scope RENAR — корпоративная стратегия)	стратегические доки	Executive	бизнес-уровень
Portfolio Epic	Группа BR одной системы	`<system>.req/br/` aggregated	Lean Portfolio Mgmt	Все BR в группе со статусом `verified`
Capability	BR подсистемы (если у подсистемы свой stakeholder)	`<subsystem>.req/br/`	Solution Architect	Все BR подсистемы `verified`
Program Epic	Опционально — крупная инициатива внутри ART, aggregated SR	`<system>.req/sr/` aggregated	Product Manager	Заданная outcome metric достигнута
Feature	SR (или связанная группа SR)	`<subsystem>.req/sr/`	Product Owner	TC из `verified-by` зелёные на текущей `requirement-version` (QG-2)
Story	TR (Task Requirement)	`<subsystem>.req/tr/` или task tracker (Jira, Linear, GitLab Issues)	Команда	Все AC выполнены, evidence зафиксирован, code merged

Enabler Epic	SPEC-AI / SPEC-ARCH / SPEC-OPS	`<system>.req/specs/`	Architect	Eval-метрики в пределах thresholds; baseline зафиксирован
Spike	TR с `level: research` + Decision в decision log	`<subsystem>.req/tr/` + decision log	Команда	Decision записан и связан с originating SR
Defect	TR + ссылка на нарушенный SR через `defect-of`	task tracker + `linked_defects` в SR	Команда	Negative TC проходит, regression test добавлен

Запрещённые соответствия: INT-SR — устаревший термин (§4.3 Terms), заменён `SR` с `constrained-by: [SPEC-INT-N]`. См. §6 ниже.

---

3. WSJF prioritization для RENAR

SAFe использует **WSJF (Weighted Shortest Job First)** как приоритизационный framework. RENAR адаптирует его для уровня BR / SR.

3.1 Формула

```
WSJF = (User-Business Value + Time Criticality + Risk Reduction & Opportunity Enablement) / Job Size
```

Каждый компонент оценивается по шкале 1-20 (modified Fibonacci); WSJF — относительная метрика, имеет смысл только при сравнении BR/SR внутри одного backlog.

3.2 Расширение frontmatter BR

```
---
id: BR-12
title: "Сократить время регистрации сотрудников до < 2 минут"
```

```yaml
status: approved
priority: must
prioritization:
  framework: WSJF
  components:
    user-business-value: 10
    time-criticality: 8
    risk-reduction-opportunity: 6
    job-size: 5
  wsjf-score: 4.8                # auto-calculated: (10+8+6)/5
  prioritized-at: 2026-05-03
  prioritized-by: "@product-owner"
---
```

Поле `prioritization` — опциональное в Core RENAR, обязательное на ART, который применяет SAFe.

3.3 Когда применяется

- **Обязательно** для BR с `priority: must` в проектах, координируемых через ART.
- **Рекомендуется** для всех BR в backlog, который проходит PI Planning.
- **Не применяется** для SR — SR наследует приоритет родительского BR. Перешафливание SR внутри одного BR — задача Product Owner на decomposition, не WSJF.

3.4 Альтернативы

Если команда не использует SAFe / WSJF — RENAR допускает другие frameworks:

- **MoSCoW** (Must / Should / Could / Won't) — простейший, для маленьких проектов. Уже есть как `priority` enum в RENAR Core.
- **RICE** (Reach × Impact × Confidence / Effort) — для product-driven команд (типично B2C).

RENAR нормирует **поле и его schema**; выбор framework — на проекте. См. также reference/02-schemas.md для допустимых значений `prioritization.framework`.

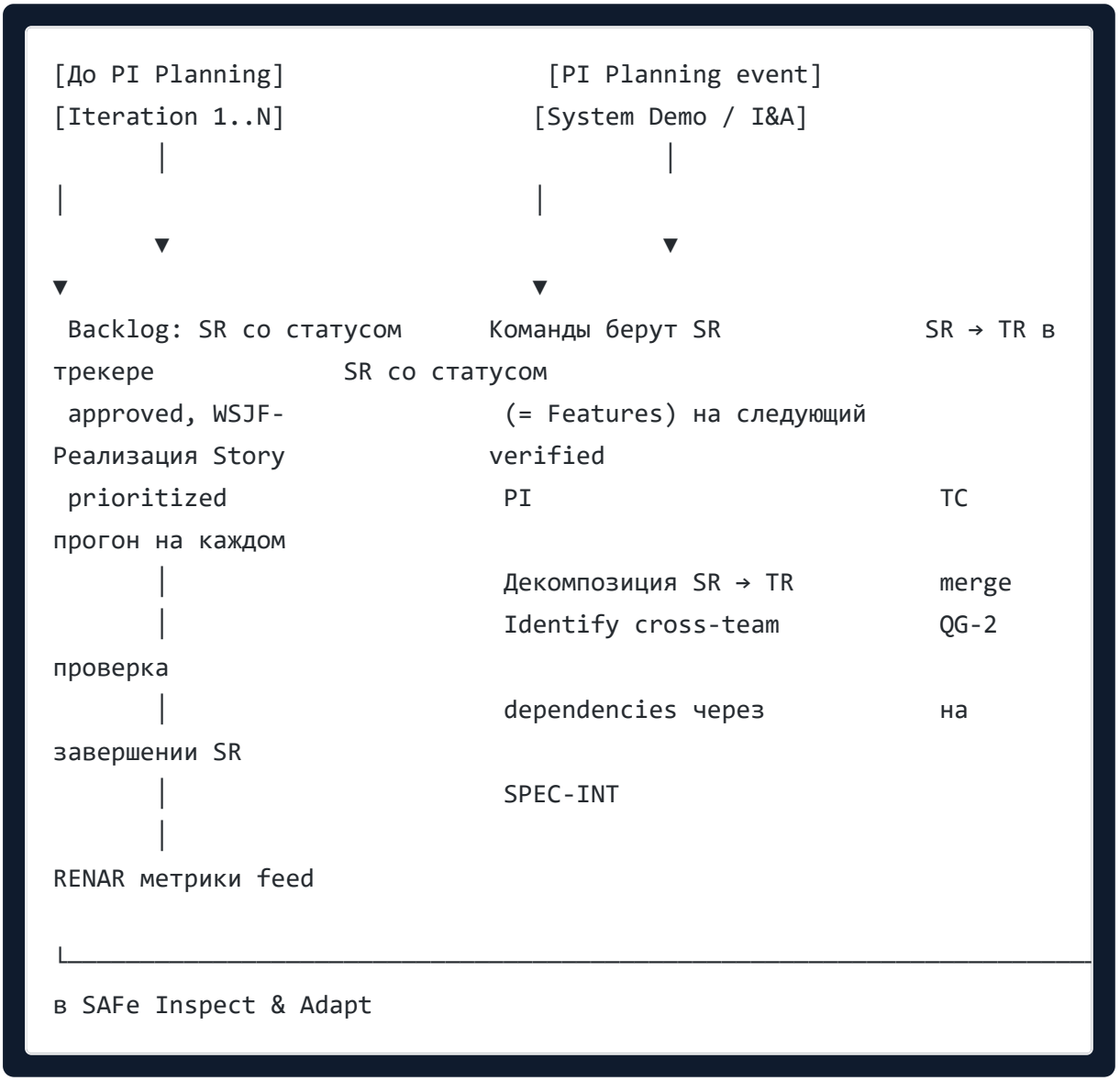
---

4. PI Planning интеграция

4.1 Что такое PI

Program Increment (PI) — фиксированный отрезок времени (обычно 8-12 недель) в SAFe, в течение которого ART коммитится на набор Features из общего backlog. PI Planning — двух- или трёхдневный event перед каждым PI, где команды совместно фиксируют commitment.

4.2 Где RENAR-артефакты попадают в PI flow



4.3 Артефакты RENAR в каждой ceremony

SAFe ceremony	RENAR input	RENAR output

Backlog refinement	BR / SR со статусом `proposed` или `approved`	Уточнённый ADAPT, обновлённый WSJF
PI Planning	WSJF-sorted SR backlog, ADAPT-доки	Commitment на SR в PI; SPEC-INT для cross-team
Iteration Planning	SR + декомпозированные TR	TR в `in-progress`
System Demo	SR со статусом `verified`	Evidence из TC last-run
Inspect & Adapt	Метрики RDLT, Coverage Velocity, Hallucination Rate	Корректировки backlog / процесса

---

5. ART координация и роли

5.1 SAFe роли ↔ RENAR ответственности

SAFe role	RENAR ответственность
RTE (release Train Engineer)	Координация cross-team dependencies через SPEC-INT; владелец PI Objectives ↔ RENAR метрик mapping
Product Manager	Owner портфельных Epic = групп BR на уровне системы
Product Owner	Owner Feature = SR на уровне команды; accountable за QG-0 (готовность к старту) и QG-2 (verified)
System Architect	Owner SPEC-* (особенно SPEC-ARCH, SPEC-INT); consulted при декомпозиции BR → SR
Tech Lead	Accountable за QG-2 на уровне SR; владеет TR-backlog команды
Business Owner	Approver BR / ADAPT с бизнес-стороны
Solution Architect	Owner BR на уровне подсистемы (когда подсистема имеет свой stakeholder)
Scrum Master	Owner ceremonies; не владеет RENAR-артефактами напрямую

5.2 Cross-team координация

В типичной SAFe-организации с несколькими командами в одном ART:

- **Каждая команда** владеет своими SR / TR в `<subsystem>.req/` .

- **RTE / Solution Architect** владеют SPEC-INT в `<system>.req/specs/int/` — общими integration contracts между подсистемами.
- **Изменение SPEC-INT** требует cross-team согласования (QG-0 от всех затронутых команд).

Правило: ART-уровневые роли (RTE) **не редактируют** SR в подсистемах напрямую. Координация — через SPEC-INT, который явно `constrained-by` для каждой затронутой SR.

---

6. Cross-team dependencies через SPEC-INT

Когда Feature в одной подсистеме блокирует / зависит от другой:

6.1 SR с зависимостью

```
# В <subsystem-a>.req/sr/SR-05.md
---
id: SR-05
parent: BR-12
title: "Регистрация пользователя через корпоративный email"
status: approved
constrained-by:
  - id: SPEC-INT-01
    ref: "specs/int/SPEC-INT-01-auth-handshake.md"
    requirement-version: "1.2"
verified-by:
  - TC-23
  - TC-24
---
```

6.2 SPEC-INT как контракт

```
# В <system>.req/specs/int/SPEC-INT-01-auth-handshake.md
---
id: SPEC-INT-01
type: SPEC-INT
```

```
title: "Auth handshake между Subsystem A и Subsystem B"
version: 1.2
participants:
  - subsystem: "subsystem-a"
    role: "client"
  - subsystem: "subsystem-b"
    role: "provider"
status: approved
verified-by:
  - TC-INT-01    # contract test
---
```

6.3 Breaking changes в SPEC-INT

Любое изменение `SPEC-INT.version` с breaking-семантикой (§4.11 Drift классы) требует:

1. ADAPT-уровневое обсуждение (зачем меняем).
2. Согласование от всех `participants` (QG-0 от каждой команды).
3. Migration plan для existing implementors (как старые SR останутся valid или будут адаптированы).

RTE **обязан** проверять SPEC-INT consistency между подсистемами регулярно (обычно раз в спринт) — это часть Inspect & Adapt и feed в conformance self-assessment (§13 Conformance).

---

7. Definition of Done на каждом уровне иерархии

DoD — **формальные** условия, проверяемые автоматически (hooks носителя). На каждом уровне иерархии — свой DoD.

Level	RENAR artefact	DoD criterion
Strategic Theme	—	(вне scope RENAR)
Portfolio Epic	Группа BR	Все BR в группе → `verified`, KPI impact подтверждён через outcome metric
Capability	BR подсистемы	Все BR со статусом `verified`; outcome metric измерен

Feature	SR	QG-2 passed: все TC из `verified-by` имеют `last-run.result = pass` на текущей `requirement-version`
Story	TR	Все AC выполнены, evidence зафиксирован, code merged, automated test exists
Enabler	SPEC-AI / SPEC-ARCH / SPEC-OPS	Eval-runs прошли пороги (для SPEC-AI); baseline зафиксирован (для всех)

Ключевое: на уровне Feature DoD в SAFe **совпадает** с QG-2 в RENAR. Нет двух разных «definition of done» — это один и тот же gate, описанный с разных сторон.

---

8. Built-in Quality ↔ RENAR mechanisms

SAFe принцип Built-in Quality: качество встроено в процесс, не ad-hoc. RENAR реализует Built-in Quality через нормативные механизмы:

SAFe Built-in Quality practice	RENAR mechanism
Continuous Integration	hooks носителя + CI на каждый change в требованиях (§13 Conformance); capability V6 (drift detection)
Test-First	TC создаются **до** реализации; QG-0 требует `verified-by` пустым только при `status: proposed`, не `approved`
Refactoring	Continuous reconciliation hook (§7.5 ADAPT); обнаруживает дрейф между требованиями и кодом
Pairing / Mobbing	AI-генератор + AI-критик (§5.2 Roles) — pair generation/review как нормативная роль
Definition of Done	QG-2 как формальный gate, проверяемый автоматически (§10 Lifecycle и QG)
Version Control	Возможность **V1** (адресуемость версионированного содержимого) без привязки к классу сред хранения
Automation	Capability V2-V6 — все верификации автоматизируемы

RENAR **не предписывает** instrument (Jenkins / GitLab CI / GitHub Actions / Tekton) — только **что** должно проверяться (capability), не **как**.

---

9. RACI lifecycle артефакта (с SAFe ролями)

Полный жизненный цикл RENAR-артефакта с распределением ответственности по SAFe ролям.

Активность	Responsible	Accountable	Consulted	Informed
Импорт ТЗ	AI-агент	System Architect	Business Owner	Команда, RTE
Декомпозиция ТЗ → ADAPT	AI-генератор	System Architect	Stakeholder, AI-критик	RTE, Команда
Декомпозиция → BR	AI-генератор	Product Owner	Business Owner, AI-критик	RTE, Команда
WSJF prioritization	Product Manager	Product Owner	RTE, Stakeholder	Команда
Декомпозиция BR → SR	AI-генератор	System Architect	AI-критик	Команда, RTE
Декомпозиция SR → SPEC-*	System Architect	System Architect	AI-критик, Tech Lead	Команда
Генерация TC	AI-агент	Test Architect	—	Команда
QG-0 approval	System Architect	Tech Lead	AI-критик	Business Owner, RTE
Выбор SR на PI	Product Owner	RTE	Команда (capacity)	Stakeholder
Декомпозиция SR → TR	Команда	Product Owner	Tech Lead	RTE
Реализация TR	Разработчик	Tech Lead	—	Команда
Прогон TC (automated)	hook носителя	—	—	Команда
QG-2 (SR verified)	System Architect	Tech Lead	—	Business Owner, RTE
System Demo	Команда + RTE	Product Owner	Stakeholder	RTE, Executive

Delta-T3 approval	System Architect + Stakeholder	Product Owner	AI-impact analysis, RTE	Команда
Spot-check 5 TC (audit)	System Architect	—	—	RTE
Reconciliation MR	AI-агент-reconciler	System Architect	—	Команда

---

10. PI Objectives ↔ RENAR метрики

PI Objectives — SMART outcomes на квартал. RENAR-метрики (§12 Metrics, reference/02-schemas.md) feed в PI Objectives:

PI Objective (example)	RENAR метрика
«Сократить время от подписания T3 до первого commit до < 2 дней»	**RDLT** (Requirement Decomposition Lead Time)
«Достичь Coverage Velocity ≥ 60% в спринт»	**Coverage Velocity** (% SR с `status: verified` per sprint)
«Снизить Hallucination Rate в новых требованиях до < 2%»	**Hallucination Rate** (% AI-сгенерированных утверждений, отклонённых на review)
«0 disputed requirements на acceptance в этом PI»	**Dispute Rate at Acceptance**
«Drift detection — все SR consistent с code в течение 24 часов после merge»	**Drift Lag** (capability V6)

Это превращает RENAR из «standard for documents» в **measurable contribution** к ART success. Метрики feed автоматически в Inspect & Adapt; RTE использует их при ретроспективе на завершении PI.

---

11. Что из SAFe оставить, что заменить

Для команд, переходящих на RENAR с уже работающего SAFe-процесса:

11.1 Оставить как есть

- **Cadence** (PI, Iterations) — RENAR не нормирует время; используйте свой ритм.

- **Ceremonies** (PI Planning, System Demo, I&A, Daily Standup) — RENAR-артефакты прозрачно вписываются в эти events.
- **WSJF** — оставить для приоритизации BR / SR; вписывается в `prioritization.framework: WSJF`.
- **ART, RTE, Scrum Master** — структура и роли сохраняются.
- **PI Objectives** — оставить; mapping на RENAR-метрики (§10) делает их измеримыми.

11.2 Заменить RENAR-эквивалентом

- **Feature description в Jira / Rally / ADO** → SR со полным frontmatter в `<subsystem>.req/sr/`. Tracker-запись становится **зеркалом** RENAR-артефакта, не первичным источником.
- **Acceptance criteria в Feature** → `verified-by: [TC-NN]` с автоматически проверяемыми TC.
- **Definition of Done на Feature** → QG-2 (нет двух разных DoD — это один gate).
- **Integration agreements между командами** → SPEC-INT (заменяет INT-SR из старых SAFe-реализаций).
- **Architecture Decision Records (ADR)** → SPEC-ARCH / SPEC-AI / SPEC-OPS (один из 9 типов SPEC, §4.4).

11.3 Добавить (новое в RENAR)

- **ADAPT** — bridge artefact между T3 и иерархией требований. В SAFe нет прямого аналога; реализуется как обязательный этап перед декомпозицией в BR.
- **AI-критик роль** — адверсариальная проверка AI-генерации. В SAFe не нормирована, в RENAR Core — обязательна для всех AI-сгенерированных артефактов.
- **Capability V6 (drift detection)** — непрерывная сверка требований с реализацией. В SAFe — manual reconciliation, в RENAR — нормативный механизм.

---

12. Negative: чего эта глава не утверждает

- **RENAR — не замена SAFe.** RENAR нормирует *требования*, SAFe — *координацию работы*. Команда может работать по RENAR без SAFe (маленький проект, одна команда) или с SAFe (enterprise multi-team ART scope).
- **RENAR — не стандарт планирования.** Cadence, capacity planning, velocity tracking — все за пределами scope. RENAR говорит «что должно быть истинно

про требование», не «когда команда должна его доделать».

- **RENAR не запрещает другие frameworks.** WSJF, MoSCoW, RICE — все совместимы. RENAR фиксирует **поле** `prioritization.framework`, не выбор framework.
- **RENAR не нормирует Jira / Rally / ADO workflow.** tracker-уровневая реализация — деталь носителя; нормативный источник — `<subsystem>.req/`.
- **RENAR не предписывает PI как обязательный.** Команда может работать по непрерывному flow без PI; в этом случае разделы 4 и 10 этой главы становятся informative.

---

13. Resolved decisions для v1.0

- `priority: must` **НЕ требует WSJF score даже в SAFe-проектах.** Per §12 этой главы: RENAR фиксирует `prioritization.framework`, не предписывает выбор framework. `priority: must` — это RENAR MoSCoW marker (reference/02-schemas), независимый от WSJF. WSJF — оптимизация SAFe-команд, не нормативное требование RENAR.
- **Feature/Capability/Story mapping — специфично для носителя.** RENAR нормирует closed list типов требований (BR/SR/TR + 9 SPEC) и не вводит SAFe Feature-уровни. Проекты, применяющие SAFe, фиксируют mapping в нативном для носителя `safe-mapping/` manifest (см. reference/02-schemas — informative extension).
- **PI Objectives — informative, вне scope RENAR.** PI — SAFe coordination artifact, RENAR не нормирует. Если команда хочет traceability, рекомендуется informative `cross-link.pi-objective-id` поле в BR-frontmatter — это не conformance-gating, но облегчает RTE-side queries.
- **Inspect & Adapt: метрики нативно для носителя автоматизированы.** Per §10.13 Logging + §12 — носитель обязан нативно для носителя выставлять COVERAGE / audit-trail. RTE использует те же данные через query API, manual extraction — anti-pattern (drift).

13.1 Отложено на v1.1 (бэклог фазы 8)

- **Эвристика ИИ для оценки поля WSJF Job Size** (шкала 1–20 по числу AC, сложности `TC` и площади кода). В v1.0 не нормируется; специфично для связки SAFe–RENAR. Расширенный informative mapping — reference/11.

---

14. Связь с другими главами

- 00-quickstart — базовый цикл RENAR без SAFe-надстройки.
- 01-walkthrough — полный example на small-scale проекте (без PI Planning).
- reference/01-glossary — точная семантика BR / SR / SPEC / TR / TC.
- reference/02-schemas — frontmatter schemas, включая `prioritization`.
- standard/04-terms — нормативные определения; mapping на SAFe закреплён в §4.13.
- standard/08-specifications — closed list 9 типов SPEC, включая SPEC-INT.

---

title: "Соответствие" description: "Mapping RENAR ↔ ISO 27001 / GDPR / ФЗ-152 / EU AI Act / NIST AI RMF / ISO/IEC 23894 / ISO/IEC 5338 / PCI-DSS, self-assessment checklists и auto-generated audit artifacts." order: 6 lang: ru version: "1.0-draft"

06. Соответствие

> *RENAR — стандарт инженерии требований; не compliance-стандарт сам по себе. Но RENAR предоставляет **инфраструктуру traceability**, которая делает соответствие другим стандартам (ISO 27001, GDPR, ФЗ-152, EU AI Act и др.) проверяемым автоматически. Эта глава — mapping артефактов RENAR на 8 ключевых compliance фреймворков + self-assessment checklists для compliance teams + список auto-generated artifacts для аудитора.*
>
> ***Предпосылки:** RENAR Core, reference/02-schemas.md (frontmatter schemas), reference/03-ai-risk-register.md.*

---

1. Принципы compliance в RENAR

1.1 Соответствие нормам как полноценное поле в метаданных

Каждое требование с регуляторным обоснованием **обязано** содержать поле `compliance` во `frontmatter`: трассировать соответствие в комментариях или во внешней таблице без связи с артефактом нельзя.

```yaml
compliance:
  - standard: "ISO 27001:2022"
    control: "A.5.34"
    rationale: "Privacy and protection of PII — требование шифрования персональных данных"
  - standard: "GDPR"
    article: "Art.32"
    rationale: "Security of processing"
  - standard: "ФЗ-152"
    article: "ст.19"
    rationale: "Меры по обеспечению безопасности персональных данных"
```

Поле повторяемое — одно требование может закрывать controls сразу нескольких стандартов.

1.2 Data classification как обязательное расширение

Каждое BR / SR, оперирующее данными, обязано декларировать классификацию:

```
data-classification:
  contains-pii: true                  # GDPR / ФЗ-152 trigger
  contains-financial: false           # PCI-DSS trigger
  contains-health: false              # HIPAA / 152-ФЗ спец.
категории trigger
  contains-children-data: false       # COPPA trigger
  retention-days: 1095                # 3 года
  data-residency: ["RU", "EU"]        # где разрешено физически
хранить
```

При `contains-pii: true` — автоматически становятся обязательными SR-encryption, SR-audit-log, SR-erasure (см. §4, §5, §2 ниже).

1.3 Traceability как audit foundation

RENAR traceability `BR → SR → SPEC → TC → реализация → CI run` — это и есть audit trail. Аудитор открывает coverage report и видит:

- Какие требования закрывают конкретный compliance control.
- Какие TC их верифицируют.
- Когда прошёл последний прогон (`last-run.date`).
- В какой `requirement-version` зафиксирована текущая редакция.

Время аудита: 1-2 дня вместо 2-3 недель. Capability V6 (drift detection) обеспечивает, что evidence не «протух» между аудитами.

---

2. ISO/IEC 27001:2022 — Information Security

2.1 Mapping Annex A controls

ISO 27001:2022 Annex A control	RENAR-артефакт
A.5.7 Threat intelligence	SPEC-SEC с threat model; SR с `compliance: A.5.7`

A.5.8 Information security in project management	Сам RENAR как process compliance
A.5.34 Privacy and protection of PII	SR с шифрованием + `data-classification.contains-pii: true`
A.6.3 Information security awareness	Onboarding процесс включает чтение RENAR
A.8.1 User endpoint devices	SR с device requirements (если в scope)
A.8.5 Secure authentication	SR-AUTH-* + SPEC-SEC с authentication flow
A.8.7 Protection against malware	SR + adversarial review (AI-критик)
A.8.10 Information deletion	SR с deletion logic + `data-classification.retention-days`
A.8.16 Monitoring activities	SR с logging + SPEC-OPS audit-log requirements
A.8.24 Use of cryptography	SR с явным crypto algorithm + ISO 25010 Security characteristic
A.8.25 Secure development life cycle	SENAR + RENAR как evidence
A.8.28 Secure coding	TC с security checks; SPEC-SEC с STRIDE coverage
A.12.1.2 Change management (наследие 27001:2013)	Delta-T3 процесс + Impact Analysis (§7.6)

2.2 Audit deliverable

Для ISO 27001 аудита auto-generates conformance report: scope (BR/SR/TC counts с compliance.iso27001 metadata) + Coverage by Annex A (control ↔ mapped SR ↔ status verified/approved/pending). См. §3 — список всех auto-generated артефактов. Специфичная для носителя команда — informative; нормативный механизм — capability V4 reporting from versioned artifacts.

---

3. GDPR (Regulation EU 2016/679)

3.1 Mapping articles

GDPR Article	RENAR-артефакт
Art.5 Principles	BR должен явно указывать lawful basis

Art.6 Lawfulness	BR с `gdpr.lawful-basis: consent / contract / legal-obligation / vital-interests / public-task / legitimate-interests`
Art.7 Conditions for consent	SR с consent management workflow
Art.13 Information to data subject	SPEC-UI с privacy notice экраном
Art.15 Right of access	SR с export-user-data функциональностью
Art.16 Right to rectification	SR с edit-user-profile
Art.17 Right to erasure	SR с delete-user-data + propagation на linked entities
Art.18 Right to restriction	SR с suspend-processing
Art.20 Right to data portability	SR с export в machine-readable формате
Art.25 Data protection by design and by default	`data-classification` обязателен на BR-уровне
Art.30 Records of processing activities	Auto-generated из BR с `data-classification.contains-pii`
Art.32 Security of processing	SR с encryption + access control
Art.33 Notification of personal data breach	SR с breach notification workflow + 72h timer
Art.35 Data Protection Impact Assessment (DPIA)	Для high-risk processing — отдельный `dpia/<feature>.md`

3.2 GDPR frontmatter

```yaml
gdpr:
  data-categories:
    - "identification: email, name"
    - "contact: phone, address"
    - "behavioral: usage logs"
```

```
  lawful-basis: contract              # Art.6
  retention-period-days: 1095
  cross-border-transfer: false        # если true → обязательны SCCs
или adequacy decision
  dpia-required: false                # если true → link на DPIA
  data-subject-rights:
    - access            # Art.15
    - rectification     # Art.16
    - erasure           # Art.17
    - portability       # Art.20
```

3.3 DPIA как RENAR-артефакт

Для high-risk processing создаётся `<system>.req/dpia/DPIA-NN-<slug>.md` с frontmatter `id / type: DPIA / status / gdpr-article: 35 / related-br[] / risks-identified / mitigations` и секциями: Описание процесса обработки, Необходимость и пропорциональность, Риски для прав и свобод субъектов, Меры снижения рисков, Согласование с DPO.

---

4. ФЗ-152 «О персональных данных» (РФ)

4.1 Mapping статей

ФЗ-152 Статья	RENAR-артефакт
ст.5 Принципы обработки	BR с явным целевым назначением (поле `business-context.business-goal`)
ст.6 Условия обработки	BR с `lawful-basis`
ст.9 Согласие на обработку	SR с consent management
ст.13.1 Хранение в РФ	`data-classification.data-residency: ["RU"]` для российских клиентов
ст.14 Доступ к информации	SR с export-user-data
ст.15 Право на уточнение	SR с edit-user-profile
ст.16 Удаление	SR с delete-user-data

ст.18 Обязанности оператора	Audit trail через RENAR traceability
ст.18.1 Локализация ПДн граждан РФ	`data-classification.data-residency` обязательно
ст.19 Меры защиты	SR с encryption + access control + ISO 25010 Security
ст.21 Уведомление о нарушении	SR с breach notification workflow
ст.22 Уведомление РКН	(operational, вне scope RENAR)

4.2 Особенность: data residency

Для проектов с российскими клиентами `data-residency` — **обязательное** поле в `data-classification`. Hook носителя (capability V6) проверяет:

- При `data-residency: ["RU"]` — все upstream SR (хранение, бэкапы, репликация) должны иметь evidence хранения в РФ.
- При добавлении хранения в зарубежной юрисдикции для RU-резидентного BR — блок integration (требование останавливается на QG-0).

---

5. EU AI Act (Regulation EU 2024/1689)

5.1 Risk classification

AI Act EU классифицирует AI-системы по уровню риска. RENAR обязывает явно указывать класс:

```
ai-act:
  risk-class: limited                # prohibited | high | limited | minimal
  rationale: "Generative AI for document drafting; no autonomous decisions affecting users' legal rights"
  high-risk-domain: false            # если true → требуется conformity assessment
  general-purpose-ai: true           # GPAI — базовая модель (Claude / GPT)
```

5.2 High-risk AI requirements (Art.9-15)

Для AI-систем класса `high` (financial scoring, hiring, public services, медицинская диагностика):

AI Act требование	RENAR-артефакт
Art.9 Risk management system	SPEC-AI + AI risk register (reference/03-ai-risk-register.md)
Art.10 Data and data governance	`eval-datasets/` с provenance + spot-check
Art.11 Technical documentation	SPEC-AI + ISO/IEC 5338 conformance (см. §7)
Art.12 Record-keeping	tool_event audit + ai-provenance во frontmatter
Art.13 Transparency to users	SPEC-UI с обозначением AI-обработки
Art.14 Human oversight	One-click approval + spot-check на QG-0 / QG-2
Art.15 Accuracy, robustness, cybersecurity	Eval-tests (`tc-type: eval`) + adversarial review

5.3 GPAI обязательства (Art.51-55)

Если в проекте используется General-Purpose AI Model (Claude, GPT, Gemini и др.):

- `ai-provenance` во frontmatter любого AI-сгенерированного артефакта (model id, version, prompt hash).
- Технический документ модели (если GPAI с systemic risk) — внешний документ + reference из SPEC-AI.

---

6. NIST AI RMF 1.0 (US-юрисдикция)

6.1 Mapping функций RMF

NIST AI RMF Function	RENAR-механизм
Govern	RENAR сам + роли из SENAR + compliance frontmatter
Map	BR с business-context, data-classification, ai-act risk-class

Measure	Eval-тесты с metric thresholds + RENAR-метрики (Hallucination Rate, Coverage Velocity)
Manage	Lifecycle с QG + reconciliation hook + AI risk register

6.2 RMF-специфичные расширения

```yaml
nist-ai-rmf:
  applicable: true
  govern: { role: "AI Governance Lead", policy-link: "..." }
  map:
    use-case-category: "content-generation"
    affected-stakeholders: ["clients", "internal-team"]
  measure:
    metrics-tracked: ["accuracy", "fairness", "robustness"]
    baseline-run-id: "eval-2026-04-01"
  manage:
    risk-tolerance: "low"
    incident-response-plan: "<link>"
```

6.3 Когда применять

Опциональное расширение — required только для проектов с US-юрисдикцией клиентов. Не противоречит ISO/IEC 23894 (см. §7); может применяться совместно.

---

7. ISO/IEC 23894:2023 — AI Risk Management

ISO/IEC 23894 — guidance по AI risk management. Не сертифицирует, но даёт structured framework для управления рисками AI-систем. Совместим с NIST AI RMF и AI Act EU.

7.1 Mapping на RENAR

ISO/IEC 23894 раздел	RENAR-артефакт
§6.4.1 Risk identification	AI risk register (reference/03-ai-risk-register.md)
§6.4.2 Risk analysis	SPEC-AI с risk assessment секцией

§6.4.3 Risk evaluation	Threshold для каждого риска в `eval-tests`
§6.4.4 Risk treatment	SR-mitigations + post-mitigation evaluation
§6.4.5 Communication and consultation	RACI matrix (см. 05-safe-comparison.md §9)
§6.4.6 Monitoring and review	Capability V6 (drift detection) + reconciliation hook

7.2 Что важно для ISO/IEC 23894 conformance

- Каждый AI-сгенерированный артефакт имеет `ai-provenance`.
- Для каждой AI use-case в SPEC-AI задокументированы:
  - Identified risks (минимум: hallucination, bias, robustness).
  - Mitigations (eval thresholds, adversarial review, human-in-the-loop).
  - Residual risks (что осталось после mitigation).
- Risk register обновляется при изменении SPEC-AI (capability V6 ловит drift).

---

8. ISO/IEC 5338:2023 — AI System Life Cycle Processes

ISO/IEC 5338 описывает life cycle процессы для AI-систем (extension к ISO/IEC/IEEE 12207). Удобный фреймворк для AI Act Art.11 technical documentation.

8.1 Mapping процессов на RENAR этапы

ISO/IEC 5338 process	RENAR-этап
Stakeholder needs and requirements definition	T3 + ADAPT + BR
Architecture definition	SPEC-ARCH + SPEC-AI
Design definition	SPEC-AI (model card, prompt design, RAG)
Implementation	TR + реализация
Verification	TC (`tc-type: eval` для AI)
Validation	Acceptance TC + spot-check
Operation	Capability V6 (drift detection) + reconciliation
Maintenance	Delta-T3 + ADAPT iteration

Disposal	Retirement workflow для AI-моделей (вне scope Core RENAR)

8.2 Conformance evidence

RENAR покрывает ISO/IEC 5338 если:

- SPEC-AI существует для каждой AI use-case с полной model card.
- Eval-tests привязаны к SPEC-AI через `verifies[]`.
- ai-provenance зафиксирован для всех AI-сгенерированных артефактов.
- Drift detection активен (capability V6).

---

9. PCI-DSS v4.0 — Payment Card Industry Data Security Standard

Если проект обрабатывает данные платёжных карт (cardholder data, CHD):

9.1 Mapping PCI-DSS requirements

PCI-DSS v4 requirement	RENAR-артефакт
Req.1 Network security controls	SPEC-OPS с network segmentation
Req.3 Protect stored CHD	`data-classification.contains-financial: true` + SR с encryption-at-rest
Req.4 Encrypt CHD transmission	SR с TLS + SPEC-API requirements
Req.6 Develop secure systems	RENAR + SENAR как process compliance evidence
Req.7 Restrict access by need to know	SR с RBAC + SPEC-SEC access control matrix
Req.8 Authenticate access	SR-AUTH-* + MFA где обязательно
Req.10 Log and monitor access	SR с audit log + SPEC-OPS observability
Req.11 Test security regularly	TC `tc-type: security` + pen-test (вне RENAR scope)

Req.12 Information security policy	RENAR + SENAR как documented policy

9.2 Особенность: CHD scope minimization

PCI-DSS требует минимизации scope CHD storage. Hook носителя (capability V6) проверяет, что новые SR с `contains-financial: true` явно обосновывают необходимость хранения CHD — без обоснования требование останавливается на QG-0.

---

10. Чек-листы самооценки

Короткие yes/no чек-листы для compliance-команд. Полный печатный kit для RENAR manifest — reference/08.

10.1 ISO 27001:2022

- [ ] Все BR с `contains-pii: true` имеют SR, закрывающий A.5.34
- [ ] Statement of Applicability задокументирован
- [ ] Каждый in-scope control имеет ≥ 1 verifying SR
- [ ] Coverage report зелёный для compliance-tagged SR
- [ ] Аудитор может пройти от control до passing TC за < 5 минут

10.2 GDPR

- [ ] Все обработки PII в Records of Processing (auto-generated)
- [ ] Lawful basis указан per processing activity
- [ ] Data subject rights Art.15–22 verifiable через SR + TC
- [ ] DPIA выполнен для high-risk processing
- [ ] Cross-border transfers обоснованы (SCCs или adequacy)

10.3 ФЗ-152

- [ ] Требования с PII граждан РФ имеют `data-residency: ["RU"]`
- [ ] hook носителя проверяет upstream SR хранения
- [ ] Согласие реализовано в SR с TC evidence
- [ ] Меры защиты ст.19 покрыты SR с passing TC

10.4 EU AI Act

- [ ] Каждый SPEC-AI имеет `ai-act.risk-class`
- [ ] Для `high` : Art.9–15 evidence в носителе
- [ ] Human oversight на QG-0 / QG-2 + spot-check
- [ ] Technical documentation auto-generated
- [ ] `ai-provenance` для GPAI-компонент

10.5 NIST AI RMF

- [ ] Все 4 функции (Govern / Map / Measure / Manage) имеют evidence
- [ ] AI Governance Lead определён в roles
- [ ] Eval baseline зафиксирован для SPEC-AI
- [ ] Incident response plan связан со SPEC-AI

10.6 ISO/IEC 23894

- [ ] AI risk register заполнен и связан со SPEC-AI
- [ ] Каждый риск имеет mitigation в SR
- [ ] Residual risks accepted владельцем
- [ ] V6 (author + timestamp) активна

10.7 ISO/IEC 5338

- [ ] SPEC-AI для каждой AI use-case с model card
- [ ] Eval-tests привязаны через `verifies[]`
- [ ] `ai-provenance` зафиксирован
- [ ] V6 активна

10.8 PCI-DSS

- [ ] SR с `contains-financial: true` имеют encryption-at-rest и in-transit
- [ ] CHD scope минимизирован
- [ ] RBAC в SPEC-SEC
- [ ] Audit log SR покрывает все CHD-touching flows
- [ ] Security TC прогоняются регулярно

10.9 Самооценка RENAR-conformance (за час)

Помимо compliance-фреймворков выше, проект декларирует **собственный** уровень RENAR-conformance через manifest `RENAR-CONFORMANCE.yaml` (standard/13 §13.4). Быстрый чек-лист самооценки (yes/no, один проход):

- [ ] Носитель требований обеспечивает все V1–V6 (distributed VCS / document store — да; Notion / Google Docs — нет, §11.4.1).
- [ ] На каждое T3 есть ровно один approved ADAPT с двойной подписью.
- [ ] Все 9 типов SPEC поддерживаются нативно для носителя (declaration «type не используется» допустима, «не поддерживается» — нет).
- [ ] Каждое нормативное утверждение, покрытое TC, имеет парный negative TC.
- [ ] QG-0 / QG-1 / QG-2 объявлены `required`.
- [ ] Манифест содержит `renar-version` + `senar-version` + `level` + подтверждение всех mandatory clauses (reference/08 §14).
- [ ] `assessment-date` свежее, `next-assessment-due` не просрочен.

Все семь — `yes` → проект conformant к заявленному `level`. Хотя бы один `no` → manifest не выпускается (§13.5.2).

Заполненный пример (RENAR-2, self-assessment) — полная схема в §13.4.2:

```
renar-version: "1.0"
senar-version: "1.0"
manifest-version: 1
manifest-id: "CFM-2026-014"
level: "RENAR-2"
level-target: "RENAR-3"
assessment-mode: "self"
assessment-date: "2026-05-20"
assessor: { id: "architect-team-lead", role: "architect", signature-
ref: "<нативный для носителя pointer>" }
next-assessment-due: "2026-08-20"
mandatory-clauses-confirmed:
  sot-inversion: true
  substrate-v1-v6: { v1: true, v2: true, v3: true, v4: true, v5:
true, v6: true }
  adapt-per-tz: true
  spec-types-closed-list: true
  tc-pos-neg-pairing: true
  quality-gates-closed-list: true
  closed-lists-backward-findings: true
quality-gates: { qg-0: required, qg-1: required, qg-2: required, qg-
3: absent, qg-4: absent }
spec-types-supported: ["SPEC-ARCH","SPEC-API","SPEC-DATA","SPEC-
INT","SPEC-PROC","SPEC-UI","SPEC-AI","SPEC-SEC","SPEC-OPS"]
```

```
exceptions: []
replaced-by: null
```

---

11. Автогенерируемые compliance-артефакты

RENAR предлагает генерируемые из существующих требований артефакты для аудиторов:

Artifact	Источник	Содержание
Records of Processing (GDPR Art.30)	BR/SR с `data-classification.contains-pii: true`	Категории данных, lawful basis, retention, recipients
Statement of Applicability (ISO 27001)	BR/SR с `compliance: ISO 27001:2022`	Какие Annex A controls в/вне scope, justification
Data Inventory	Все `data-classification` записи	Categories, residency, retention, owners
AI System Cards	SPEC-AI	Model card по NIST AI RMF / AI Act format
Audit Trail Report	Traceability BR → SR → SPEC → TC → last-run	Полная цепочка от контроля до доказательств верификации
DPIA Index	`dpia/` папка	Список DPIA + статус согласования с DPO
AI Risk Register Snapshot	AI risk register	Все идентифицированные риски + mitigations + residual

Нативная для носителя reporting operation: агрегировать evidence из versioned artifacts (capability V4) в compliance report по запросу оценщика.

11.1 Evidence pack — шаблоны (copy-paste)

> *Informative шаблоны для аудита. Полный E2E — guide/09 §E3.*

GDPR Art.15 — trace bundle (markdown export для auditor):

```
# GDPR Art.15 evidence — {project}

| Control | BR/SR | SPEC | TC | last-run |
|---|---|---|---|---|
| Art.15 access | BR-02 | SPEC-DATA-02 | TC-080, TC-081 | {date}
green |

Lawful basis: {from BR gdpr.lawful-basis}
Retention: {data-classification.retention-days}
Manifest: RENAR-CONFORMANCE.yaml §13.4
```

ФЗ-152 ст.14 — mapping table:

Требование 152-ФЗ	RENAR artifact	Evidence field
ст.14 субъект вправе получить сведения	SR-08, SR-09	`compliance: ФЗ-152 ст.14`
ст.19 меры защиты	SPEC-SEC-03	TC `tc-type: security`
учёт обращений	SPEC-OPS audit log	QG-2 `last-run`

ISO 29148 trace excerpt — см. reference/07; заполнить колонки «RENAR frontmatter» для каждого SR в scope.

Самооценка перед аудитом: reference/08 §14.

---

12. Что RENAR НЕ покрывает в compliance

RENAR — инфраструктура для compliance, но не **substitute** для:

- **DPO (Data Protection Officer)** — человеческая роль, юридическая ответственность.
- **Legal review** договоров и privacy notices.
- **Pen-testing** реализации (отдельный security workflow).
- **Bug bounty / vulnerability management** (operational).
- **Physical security** (датацентры, офис).
- **Operational security** — key rotation, incident response, monitoring runbook.
- **Cyber insurance**.

- **Regulatory filings** — уведомления РКН, GDPR registration с DPA, AI Act conformity assessment.

RENAR помогает делать compliance **в процессе разработки требований**. Operational compliance — отдельные процессы, которые могут *ссылаться* на RENAR-артефакты как на evidence.

---

13. Resolved decisions для v1.0

- **Compliance frontmatter — conditional mandatory.** Default — optional. Mandatory становится при `data-classification.contains-pii: true` (триггер GDPR / ФЗ-152) или `data-classification.contains-phi: true` (HIPAA-trigger). Это RENAR conformance-gating: артефакт с PII без compliance frontmatter — non-conformant per §13.3 (mandatory clauses расширения через manifest declared-stricter).
- **DPIA — mixed model.** Формальный DPIA — внешний документ (legal owns). RENAR-артефакт `dpia/<slug>.md` хранит machine-readable summary + pointer на formal document. Это сохраняет separation of concerns (юристы / engineers) при traceability.
- **Data residency — вне scope RENAR.** Per §1.3 (3) tech stack / infra — out of scope. Enforcement выполняется DevOps-уровневыми контролями (network policies, cloud regions). RENAR может фиксировать **требование** (`data-residency.region: eu-west`), но не enforcement.
- **Custom industry frameworks (ЦБ РФ финтех, HIPAA healthcare и др.) — отдельные документы.** Эта глава покрывает 8 cross-industry frameworks. Industry-specific addenda — отдельные `guide/06-compliance-<industry>.md` или внешние документы; они **могут** declared-stricter поверх RENAR-conformance.

13.1 Отложено на v1.1 (бэклог фазы 8)

- **Автовыгрузка доказательств по Schrems II и трансферам ЕС ↔ РФ.** Частично закрывается флагами `cross-border-transfer` и ссылками на SCC, но без правил уровня политики данных полная автоматизация недостижима (какие именно SCC применимы к конкретному трансферу решают юристы). Ответственные: связка legal-tech / адаптеры.

---

14. Связь с другими главами

- 00-quickstart — базовый цикл RENAR без compliance-надстройки.
- 05-safe-comparison — RACI с SAFe ролями (включая DPO как Consulted).
- reference/02-schemas — frontmatter schemas: `compliance`, `data-classification`, `gdpr`, `ai-act`.
- reference/03-ai-risk-register — AI risk register структура.
- reference/04-ai-style-guide — стиль AI-провенанса.
- standard/04-terms — SPEC-SEC, SPEC-AI, SPEC-OPS терминология.
- standard/13-conformance — RENAR-уровни conformance (не путать с compliance: RENAR-N оценивает зрелость *процесса*, compliance — соответствие *внешним нормам*).

---

title: "Режимы отказа" description: "8 классов дрейфа, 14 AI-рисков, organizational failure patterns. Для каждого: симптом, detection, prevention, recovery." order: 7 lang: ru version: "1.0-draft"

07. Режимы отказа

> *Систематический обзор всех известных способов, которыми RENAR-проект может выйти из строя: технический дрейф между артефактами и реализацией, AI-специфичные риски, и (главное) организационные паттерны, при которых процесс существует на бумаге, но не работает. Классы дрейфа нормированы в standard/00 §0.3.1; AI-риски — в reference/03. Для каждого failure mode — симптом, как обнаружить, как предотвратить, как восстановиться.*
>
> ***Предпосылки:*** *RENAR Core, reference/03-ai-risk-register.md.*

---

1. Карта failure modes

Три класса проблем:

Класс	Где живёт	Как обнаруживается
Drift	Несоответствие между разными представлениями одной сущности (frontmatter ↔ DB, требование ↔ код, TC ↔ требование)	Capability V6 (drift detection) + reconciliation hook
AI risks	Свойства AI-генерации (hallucination, bias, injection, model drift)	Adversarial review + eval-тесты + AI risk register (reference/03)
Organizational	Несоответствие между формальным процессом и реальными практиками команды	Поведенческие сигналы: pattern of approval, rate of dispute, frequency of bypass

Drift и AI risks ловятся механизмами носителя. Organizational — никаким носителем не ловятся; нужны human-уровневые рецензии процесса. Эта глава покрывает все три.

---

2. 8 классов дрейфа

Для каждого: симптом (как выглядит со стороны), detection (как обнаружить автоматически), prevention (как не допустить), recovery (что делать если уже случилось).

2.1 Schema drift

Симптом: Поля во frontmatter артефакта расходятся с теми, что носитель ожидает / поддерживает.

Detection: На каждое изменение артефакта носитель валидирует frontmatter по schema (reference/02-schemas.md). При расхождении — блок integration (QG-0 фейлит).

Prevention: Schema — single source of truth (closed list), не редактируется на проекте. Изменения schema — только через изменение полного RENAR Standard.

Recovery: Откатить frontmatter к schema-valid состоянию; если изменение нужно — открыть RFC на изменение стандарта.

2.2 Lifecycle drift

Симптом: Статусы (`proposed` / `approved` / `verified` / `obsolete`) и названия контрольных точек качества понимаются по-разному в разных подсистемах или у разных команд.

Detection: Сравнить переходы статусов в audit log с нормативной state machine (standard/10-lifecycle-qg). Аномалии (переход без соответствующих pre-conditions) — флаг.

Prevention: Переходы выполняются механизмом носителя, не ручной правкой frontmatter. Capability V3 (state machine enforcement).

Recovery: Откатить нелегитимный переход; провести QG-0 / QG-2 заново через корректный механизм.

2.3 Source-of-truth drift

Симптом: Одна и та же сущность редактируется в двух местах (например, и в `.req` директории, и в Jira-трекере). Версии расходятся.

Detection: Periodic reconciliation между носителем и tracker; diff показывает расхождения.

Prevention: В каждый момент времени для проекта **выбран ровно один SSoT-носитель**. Tracker — derived view, не источник истины. Hook носителя блокирует tracker-only изменения требований.

Recovery: Объявить один носитель-winner; смерджить второй в первый; убрать редактирование во второй до миграции.

2.4 Implementation drift

Симптом: Код в реализации ссылается на SR, которого больше нет (deprecated, удалён, переименован). Или: SR существует, но реализация ушла от него (поведение не соответствует).

Detection: Capability V6 (drift detection):

- Forward: пройти по requirement → найти реализующий код → запустить TC.
- Backward: пройти по коду → найти ссылки на SR / TC → проверить, что они существуют и `verified`.

Prevention: ID требований **immutable** — переименование запрещено. Deprecated требования остаются в репозитории со статусом `obsolete`, не удаляются.

Recovery: Открыть delta-T3, который явно adopts текущую реализацию (или, наоборот, требует откат кода до соответствия требованию).

2.5 Terminological drift

Симптом: «Verified», «implemented», «approved» означают разное у разных людей / команд.

Detection: Code review checklist: «использован термин не из глоссария?» — флаг. Аналогично — валидатор носителя проверяет, что значения enum-полей frontmatter только из closed list.

Prevention: Глоссарий — единственный источник терминов (reference/01-glossary). Каждый термин = ровно одно состояние lifecycle.

Recovery: Провести ревизию всех артефактов проекта на использование out-of-glossary терминов; заменить или подать RFC на расширение глоссария.

2.6 Order / provenance drift

Симптом: Delta-T3 #2 ссылается на SR, который был создан в Delta-T3 #1, но применение пошло в обратном порядке — SR не существовал на момент применения #2.

Detection: Delta-T3 нумеруются и применяются строго в порядке номеров. Hook носителя проверяет, что upstream delta уже применена.

Prevention: Delta-T3 нельзя перенумеровать. Каждый артефакт хранит `created-by-order` (delta-T3 создания) и `last-modified-by-order` (последний апдейт).

Recovery: Откатить out-of-order применение; перепримерить в правильном порядке.

2.7 TC ↔ requirement provenance drift

Симптом: TC верифицирует требование, но требование уже изменилось — `last-run.requirement-version` ниже текущей `version` требования. Тест зелёный, но проверяет устаревшее поведение.

Detection: Coverage report показывает категорию «Stale» — TC с устаревшей `last-run.requirement-version`. Capability V6 ловит это автоматически.

Prevention: В TC обязательное поле `verifies[].requirement-version` — pinned версия. QG-2 запрещает перевод требования в `verified`, если хотя бы один TC из `verified-by` имеет stale `last-run`.

Recovery: Перепрогнать stale TC на текущей версии требования; обновить, если TC сам устарел.

2.8 Test-fitting drift

Симптом: AI-агент имеет тривиальный путь зеленения failing-теста — ослабить Pass/Fail-критерий вместо исправления кода. Без защиты тесты дрейфуют от «строгий проверщик» к «зелёная пустота».

Detection: Изменение Pass/Fail-критериев в TC без явного `[test-spec-change]` тега — флаг носителя. Periodic spot-check 5 случайных passing TC раз в спринт.

Prevention:

- MR / change, изменяющий Pass/Fail-критерии, обязан иметь тег `[test-spec-change]` и отдельный approval инженера (не совмещённый с approval кода-фикса).
- Изоляция judge-модели: production-модель ≠ judge-модель.
- Метрика test-fitting drift rate trending.

Recovery: Восстановить старые критерии; провести root cause analysis — почему AI-агент выбрал зеленение вместо фикса; обновить prompt / system instructions.

---

3. 14 AI-рисков (краткая сводка)

Полные описания, mitigations и owner — в reference/03-ai-risk-register. Здесь — operational summary: id, название, severity, главный detection signal.

ID	Название	Severity	Detection signal
AIR-01	Hallucination в AI-генерируемых требованиях	High	Hallucination Rate metric > threshold; adversarial critic flags
AIR-02	Prompt injection через T3 от клиента	High	Suspicious pattern в imports; sandbox violation
AIR-03	Model drift / version change	Medium	diff regression при смене модели; eval baseline failure
AIR-04	Bias в AI-генерации требований	Medium	Stakeholder map gaps; missing accessibility/locale considerations
AIR-05	Single-model failure (no diversity)	Medium	Все артефакты с одним `ai-provenance.model`; нет multi-model agreement
AIR-06	Test-fitting / зеленение тестов	High	diff в TC Pass/Fail без `[test-spec-change]` тега
AIR-07	Hallucinated citations	Medium-High	Citation validator hook fails
AIR-08	Adversarial inputs в client data	High	Application-level (out-of-scope RENAR, tracked в SPEC-SEC)
AIR-09	Privacy leakage через AI logs	High	PII в tool_event audit; redaction скип
AIR-10	Knowledge graph poisoning	Medium	Incorrect edges; circular dependencies в graph
AIR-11	Reconciliation false positive overload	Low-Medium	Findings/week trending up без real issues; dismissal rate высокий
AIR-12	Cost runaway (uncontrolled AI spend)	Medium	Project AI cost approaching budget cap
AIR-13	Stakeholder не понимает AI-сгенерированные требования	Medium	Dispute rate at acceptance растёт; long approval cycles
AIR-14	Vendor lock-in to specific LLM provider	Medium	All prompts работают только на одном provider

Risk matrix и cadence рецензирования — reference/03 §5-§2.

3.5 Adversarial review (процедура)

> ***Informative.*** *Операционная процедура для WC-13; нормативные требования — standard/09 §9.4, standard/13 §13.2 (RENAR-5).*

Когда обязательно (normative): adversarial review — QG-0 для RENAR-5 (§11.8.1); для `SPEC-SEC` / `SPEC-AI` — external reviewer на QG-0 (§5); declared-stricter может расширить scope (standard/00 §0.7.2).

Шаг	Актор	Артефакт	Exit criterion
1. Scope	Architect	Список TC + связанные SR/SPEC	Каждый `approved` TC в scope имеет `tc-type` и `verified-by[]`
2. Critic pass	AI-критик (отдельная модель/промпт)	Findings log с id, severity, ссылкой на TC/SR	Findings traceable к конкретному clause §9.x; без «общих» рекомендаций
3. Triage	Architect + RE engineer	Disposition: fix / accept / reject	Каждый finding — owner + rationale; dismiss без rationale запрещён (см. §5.6)
4. Re-run	AI-агент или human	Обновлённые TC + diff	QG-2 pre-condition: `passing-tests / total-tests` для scope (§9.10)
5. Audit trail	носитель (V1)	commit/change unit с `adversarial-review` tag	provenance: model id, prompt version, findings hash (§10.13)

Claims discipline: метрики «100%» в §9 — это **target при QG-2**, не гарантия качества продукта. Severity AI-рисков — из reference/03, не editorial override.

Agent panel (без human reviewers): informative процедура — §4.5 (шаги 1–5); rubric и severity — reference/03.

---

4. Organizational failure patterns

Эти проблемы не ловятся механизмами носителя — это поведенческие паттерны команд. Появляются типично через 2-6 месяцев после внедрения RENAR.

4.1 ADAPT как формальность

Симптом: Клиент / stakeholder не вычитывает ADAPT перед подписанием. Backward-секция (вопросы клиенту) пустая или содержит yes/no без контекста.

Признак: ADAPT approved за < 24 часов после генерации; rate of disputed requirements at acceptance растёт.

Mitigation: Двойная подпись ADAPT (standard/05-roles §5.5) — обязательны и стейкхолдер, и архитектор. Backward-секция обязана содержать ≥ 1 не риторический вопрос. Spot-check ADAPT в I&A.

4.2 SPEC overload

Симптом: Команда создаёт SPEC для каждой задачи, даже когда SR + TR достаточно. SPEC-каталог разрастается; каждый PR обновляет 5+ SPEC.

Признак: Rate SPEC / SR > 1.5 (ожидаемое < 0.3 для проектов средней сложности).

Mitigation: Pre-review checklist: «нужен ли SPEC для этого изменения?» SPEC оправдан только когда есть несколько SR с общим constraint. См. standard/08-specifications.md §8.2 — когда SPEC обязателен.

4.3 Hooks как препятствие

Симптом: Команда регулярно обходит hooks носителя (`--no-verify` , манипуляции с timestamp, ручная правка statuses).

Признак: Git log / audit log носителя показывает частоту обхода commits; QG-0/QG-2 проходят за подозрительно короткое время.

Mitigation: Root cause — hooks слишком медленные / слишком noisy / слишком жёсткие. Не «запретить обход», а починить hooks. Метрика частоты обхода как trending — если растёт, провести retro с командой.

4.4 Drift detection без действия

Симптом: Capability V6 генерирует drift findings, но никто на них не реагирует. Findings backlog растёт, старые findings игнорируются.

Признак: Findings older than 14 days > 30; resolution rate < 20% / неделю.

Mitigation: Каждый drift finding — owner и SLA (resolve / accept / reject в течение N дней). Unresolved findings выше SLA — escalation. Capability V6 без human ownership = шум.

4.5 tracker as parallel universe

Симптом: Команда живёт в Jira / Linear / ADO; `.req` директория обновляется раз в неделю «для галочки». Tracker — реальный источник истины, RENAR — формальный артефакт для аудита.

Признак: diff `.req` vs tracker > 30% по any given week; commits в `.req` редкие и батчевые.

Mitigation: Single source of truth должен быть размещён в носителе, не tracker-resident. Tracker — derived view только. Если команда не может работать без tracker — носитель должен пушить *в* tracker, не наоборот.

4.6 Critic burnout

Симптом: AI-критик (adversarial review) генерирует много findings; постепенно дев / архитектор начинают игнорировать его output. Findings dismissed без рассмотрения.

Признак: Dismissal rate AI-критика > 80%; time-to-dismiss < 30 секунд per finding.

Mitigation: Tunable thresholds для критика. Если ratio false positive высокий — recalibrate prompt / model. Метрика «critic findings → real issue» (% от dismissed findings, которые потом всплыли как defect) — если 0%, критик useless.

4.7 Single-engineer dependence

Симптом: Только один инженер на проекте «понимает RENAR». Все QG-0 / QG-2 проходят через него. Если он в отпуске — процесс встаёт.

Признак: Bus factor RENAR-владения = 1. Distribution of QG approvals heavily skewed к одному человеку.

Mitigation: Парный onboarding (минимум 2 инженера на проекте умеют RENAR). Rotation QG-approver роли. Documentation проектных конвенций в `<project>.req/CONVENTIONS.md`.

4.8 Ad-hoc delta

Симптом: Изменения требований происходят без оформления delta-T3 — «давай просто поменяем SR-12 прямо в репозитории».

Признак: Direct commits в `<system>.req/sr/*` без соответствующего delta-T3; `created-by-order` поле пустое.

Mitigation: hook носителя блокирует mutation существующих требований без `delta-ref` в commit metadata. Все изменения — через delta-T3 workflow (standard/07-adapt §7.6).

4.9 TC abandonment

Симптом: TC создаются вместе с требованиями, но затем не прогоняются. `last-run` старше N месяцев; coverage report показывает «зелёные» TC, которые в реальности никогда не запускались за полгода.

Признак: Median `last-run` age > 90 дней; TC count растёт, run count не растёт.

Mitigation: носитель автоматически прогоняет TC по schedule (capability V4). TC без `last-run` за N дней автоматически помечаются `stale`; QG-2 блокирует, пока не перепрогнаны.

---

5. Failure recovery playbook

Что делать, когда система уже сломана. Последовательность общая для всех failure modes; specifics зависят от класса.

Шаг 1: Stop the bleeding

Найти и остановить ongoing damage:

- Drift: заморозить дальнейшие изменения в затронутой области.
- AI risk: приостановить AI-генерацию для затронутого класса артефактов.
- Organizational: вынести на retro / I&A — это не technical fix.

Шаг 2: Quantify

Измерить ущерб:

- Сколько артефактов в drift-состоянии?
- Сколько релизов с момента возникновения проблемы?
- Какие SR / SPEC / TC затронуты? (Capability V4 — coverage / drift report)

Шаг 3: Triage

Сегментировать ущерб на:

- **Critical** — уже в production, влияет на пользователей. Hot-fix.
- **Active** — в текущем PI, влияет на ongoing work. Block PI exit.
- **Historical** — старые артефакты, не активно используются. Batch fix.

Шаг 4: Fix

Для каждого класса — соответствующий fix:

- Schema drift → откат frontmatter; RFC если schema нужно расширить.
- Implementation drift → delta-T3 adopt OR откат кода.
- TC drift → repump TC на текущей requirement-version.
- Test-fitting → revert критерии; root cause AI-агента.
- Organizational → process retro + специфичные mitigations (§5).

Шаг 5: Prevent recurrence

- Усилить detection (нижний threshold, новая metric).
- Добавить mitigation в processed артефакт.
- Зафиксировать lessons learned в project decision log или ADAPT backward findings (категория `scope` / `terminology`).

Шаг 6: Verify

После fix — пройти QG-2 на затронутых артефактах заново. Drift detection должна показать clean state.

---

6. Negative: чего эта глава не покрывает

- **Security incidents** — breach response, forensics, regulatory notification. Это процесс уровня organization security, не RENAR scope.
- **AI red team / penetration testing** — отдельный security workflow; RENAR трекает только что соответствующие SR / SPEC-SEC должны быть.
- **Compliance breach response** — нарушение GDPR / ФЗ-152 / PCI-DSS требует юридического процесса с DPO / regulator, не technical recovery.
- **Production incidents** — outages, performance regressions. Это operational, см. SPEC-OPS runbook.
- **Stakeholder conflicts** — диспуты на acceptance, scope disagreements. RENAR даёт audit trail (кто что approved когда), но resolution — human process.

---

7. Связь с другими материалами о режимах отказа

Документ	Что в нём	Когда читать
reference/03-ai-risk-register	Полный реестр 14 AIR-рисков с mitigations	При планировании AI use-case; при review eval-стратегии
standard/04-terms §4.11	Closed list drift классов с нормативными определениями	При спорах о терминологии failure modes
05-safe-comparison §9	RACI matrix — кто accountable за каждую активность	При расследовании organizational failure
reference/04-ai-style-guide	Стиль AI-провенанса; минимальный контракт для AI-сгенерированных артефактов	При диагностике AIR-01 (hallucination), AIR-07 (citations)

---

8. Resolved decisions для v1.0

- **Набор шагов восстановления без привязки к платформе.** Последовательность из §2 носит универсальный характер. Детали «как именно заморозить изменения» для `git` и document-store — в 03-tool-guide-git §3 и 04-document-store-substrate. Объём нормативного минимума задан здесь же, в главе 7.
- **Подстройка критика событийно.** Повторная настройка промпта критика выполняется при выходе за порог метрик drift / галлюцинаций (§12.3.3); уровень RENAR-5 требует непрерывной оценки (§11.8.1), поэтому регулярный «общий пересмотр без причины» избыточен. По срабатыванию метрик — допустимо.

8.1 Отложено на v1.1 (бэклог фазы 8)

- **Числовые пороги для организационных паттернов (§5).** Сейчас даны качественные «признаки». Набор допустимых значений понадобится после накопления полевых данных. Ответственные: команда стандарта RENAR и внедренческие организации.
- **Формальный замер коэффициента «техавтобусности» для §5.7.** Вспомогательные средства не зафиксированы; возможный подход — графовый запрос по авторам коммитов в цепочке ревизий (встроенное в носитель сочетание **V6**). Ответственные: авторы средств для конкретных сред хранения.

---

title: "Руководство разработчика" order: 8 lang: ru

08. Руководство разработчика

> ***Пример только для среды*** `git`***.*** *Эта глава иллюстрирует* ***один*** *возможный сценарий на примере GitHub и разнесённых по репозиториям подсистем каталогах* `.req` *и* `.src`*.* ***RENAR к конкретной реализации среды не привязан****; ваши перехватчики и пайплайны могут быть устроены иначе. Общезначимые главы опираются на §6 Иерархия требований, §8 Спецификации, §10 Жизненный цикл и контрольные точки качества. Альтернатива без VCS как файловой базы — 04-document-store-substrate.md.*
>
> ***Выдуманная компания ActmeCorp:*** *платформа* `acme-platform` *и четыре подсистемы —* `acme-portal`*,* `acme-ai`*,* `acme-orchestrator`*,* `acme-site`*. Имена в примерах замените на свои; ссылки на нормативные главы этого не затрагивают.*

Содержание

1. Что нужно знать перед стартом
2. Первоначальная настройка локального окружения
3. Структура папок на локальной машине
4. Настройка рабочего пространства в IDE
5. Работа с требованиями
6. Работа с задачами
7. Git-процесс
8. Частые сценарии
9. Права доступа — кто что может менять
10. Быстрый справочник

---

1. Что нужно знать перед стартом

Два типа репозиториев

Каждая подсистема имеет два отдельных репозитория:

Репозиторий	Суффикс	Что внутри	Кто пишет
Требования	`.req`	BR, SR — что система должна делать	Архитектор, Tech Lead
Исходный код	`.src`	Код, тесты, CI/CD	Разработчик

Они разделены намеренно: требования версионируются независимо от кода, имеют другой цикл рецензирования и другие права доступа.

Иерархия

```
Система (acme-platform)
  └── Подсистема (acme-portal, acme-ai, acme-orchestrator, acme-site)
        └── Модуль (если есть)
```

Каждый уровень имеет свой `.req` репозиторий. Требования верхнего уровня — родители для требований ниже.

Три уровня требований

```
BR  — Бизнес-требование  (зачем это нужно бизнесу)
 └── SR  — Системное требование  (что делает система)
       └── TR  — Требование к задаче  (конкретика для реализации)
                  ↑ это поля вашей задачи в трекере, не файл
```

TR не является файлом. TR — это Goal + Acceptance Criteria в задаче трекера.

---

2. Первоначальная настройка локального окружения

Шаг 1. Определите свою подсистему

Уточните у Tech Lead, с какой подсистемой вы работаете:

- `acme-portal` — клиентский портал

- `acme-ai` — AI-pipeline
- `acme-orchestrator` — оркестратор
- `acme-site` — публичный сайт

Шаг 2. Создайте локальную структуру папок

```
mkdir -p ~/projects/acmecorp/acme-platform
cd ~/projects/acmecorp/acme-platform
```

Шаг 3. Клонируйте нужные репозитории

Обязательно для всех — родительские требования системы (только чтение):

```
git clone git@github.com:acmecorp/acme-platform/acme-platform.req
```

Обязательно — репозитории вашей подсистемы:

```
# Замените SUBSYSTEM на вашу подсистему: acme-portal / acme-ai /
acme-orchestrator / acme-site
SUBSYSTEM=acme-portal

mkdir -p $SUBSYSTEM
git clone git@github.com:acmecorp/acme-
platform/$SUBSYSTEM/$SUBSYSTEM.req $SUBSYSTEM/$SUBSYSTEM.req
git clone git@github.com:acmecorp/acme-
platform/$SUBSYSTEM/$SUBSYSTEM.src $SUBSYSTEM/$SUBSYSTEM.src
```

По необходимости — требования смежных подсистем (только чтение):

```
# Клонируйте только если работаете с интеграцией
mkdir -p acme-ai
git clone git@github.com:acmecorp/acme-platform/acme-ai/acme-ai.req
acme-ai/acme-ai.req
```

Шаг 4. Проверьте результат

```
find ~/projects/acmecorp -maxdepth 4 -name ".git" -type d | sed
's|/.git||' | sort
```

Ожидаемый результат для разработчика Acme Portal:

```
~/projects/acmecorp/acme-platform/acme-platform.req
~/projects/acmecorp/acme-platform/acme-portal/acme-portal.req
~/projects/acmecorp/acme-platform/acme-portal/acme-portal.src
```

---

3. Структура папок на локальной машине

Локальная структура **зеркалит** иерархию репозиториев в хостинге. Это важно: относительные пути `../..` между репозиториями становятся предсказуемыми.

> ***С раскладкой по каталогам (раскладка носителя) см. также guide/03 §14:*** *там задана каноническая схема репозитория и закрепление `.src` → `.req` через submodule (**возможность V5**, фиксация версии между носителями). Здесь — типичное локальное размещение для IDE: несколько рядом лежащих клонов без обязательного submodule в рабочей папке разработчика.*

```
~/projects/acmecorp/
  acme-platform/
    acme-platform.req/          ← требования системы (read-only для
разработчика)
    acme-portal/
      acme-portal.req/        ← требования вашей подсистемы
      acme-portal.src/        ← код вашей подсистемы  ← здесь вы
работаете
    acme-ai/
      acme-ai.req/          ← клонируется по необходимости
      acme-ai.src/
    acme-orchestrator/
      acme-orchestrator.req/
```

```
    acme-orchestrator.src/
  acme-site/
    acme-site.req/
    acme-site.src/
```

> ***Правило:*** *не меняйте имена папок. Они совпадают с именами проектов в git-хостинге. Это позволяет скриптам и CI работать с одними путями везде.*

---

4. Настройка рабочего пространства в IDE

VS Code Workspace

Создайте файл workspace в папке подсистемы:

```
cat > ~/projects/acmecorp/acme-platform/acme-portal/acme-portal.code-workspace << 'EOF'
{
  "folders": [
    {
      "path": "acme-portal.src",
      "name": "Code — Acme Portal"
    },
    {
      "path": "acme-portal.req",
      "name": "Requirements — Acme Portal"
    },
    {
      "path": "../acme-platform.req",
      "name": "Requirements — System (read only)"
    }
  ],
  "settings": {
    "files.exclude": {
      "**/.git": true
    }
  }
```

```
}
EOF
```

Открыть: `code ~/projects/acmecorp/acme-platform/acme-portal/acme-portal.code-workspace`

В боковой панели вы видите сразу три репозитория. Переключаться между кодом и требованиями — одно нажатие.

JetBrains (IntelliJ, WebStorm, PyCharm)

Откройте каждый репозиторий как отдельный модуль через **File** → **Attach Project** или используйте **Project Structure** → **Modules**.

---

5. Работа с требованиями

5.1 Как читать требования

Перед началом задачи прочитайте SR своей задачи:

```
acme-portal.req/
  sr/
    SR-01-auth.md       ← читаете свой SR
```

В frontmatter SR найдите поле `parent` — это ссылка на родительский BR:

```yaml
parent:
  id: BR-01
  repo: "acmecorp/acme-platform/acme-platform.req"
  file: "br/BR-01-order-ai-dev.md"
```

Откройте родительский BR в `acme-platform.req/br/BR-01-order-ai-dev.md` — это ответ на вопрос «зачем вообще существует эта функциональность».

5.2 Как читать трассировку

Полная цепочка от задачи до бизнес-цели:

```
Задача TASK-42 (ваш трекер)
  → SR-01 (acme-portal.req/sr/SR-01-auth.md)
    → BR-01 (acme-platform.req/br/BR-01-order-ai-dev.md)
```

Если что-то непонятно в задаче — поднимайтесь по цепочке вверх.

5.3 Как изменить требование

Разработчик предлагает изменение — создаёт Issue в `.req` репозитории с описанием несоответствия между SR и реальным поведением.

Tech Lead / Архитектор вносит изменение:

```
cd acme-portal.req
git checkout -b change/TZ-2026-002-totp

# Редактируете файл SR
# Обновляете version и updated в frontmatter
# Добавляете запись в source[]

git add sr/SR-01-auth.md
git commit -m "[delta:TZ-2026-002] SR-01 v1.2: добавить TOTP"
# Создаёте MR/PR в git-хостинге
```

> ***Запрещено:*** *коммитить изменения требований напрямую в* `main` *без MR/PR и ревью.*

5.4 Что нельзя делать с требованиями

- Менять `acme-platform.req` без согласования с архитектором
- Менять требования смежных подсистем (`acme-ai.req` , `acme-orchestrator.req`)
- Удалять файлы требований — только переводить в `status: deprecated`
- Создавать файлы версий (`SR-01-v1.md` , `SR-01-v2.md`) — история в `git log`

---

6. Работа с задачами

6.1 Перед тем как взять задачу

Задача готова к работе, если выполнены все пункты QG-0 Approval Gate (reference/01 §14.4, canonical имя; в pre-v1.0 материалах встречается legacy «Context Gate»):

- [ ] Сформулирована цель (Goal)
- [ ] Есть Acceptance Criteria — конкретные, тестируемые, независимые
- [ ] Есть хотя бы один негативный сценарий в AC
- [ ] Задача ссылается на SR (поле в трекере)
- [ ] Назначен тип работы
- [ ] Если затрагивает безопасность — Threat Surface задекларирован

Если чего-то нет — **не берите задачу в работу**. Вернитесь к Supervisor/Tech Lead.

6.2 Acceptance Criteria — как читать

Каждый AC — это отдельный тест. Хорошие AC:

```
✓ POST /auth/login возвращает 200 и JWT-токен при верных credentials
✓ POST /auth/login возвращает 401 при неверном пароле  ← негативный сценарий
✓ POST /auth/login возвращает 422, если поле email отсутствует
✓ JWT-токен истекает через 24 часа
```

Плохие AC (задачу брать нельзя):

```
✗ Логин должен работать корректно
✗ Обработка ошибок должна быть надёжной
✗ Система должна быть безопасной
```

6.3 Если AC непонятен или противоречит SR

1. Прочитайте SR, на который ссылается задача
2. Прочитайте родительский BR
3. Если противоречие есть — создайте комментарий в задаче с конкретным вопросом

Не интерпретируйте молча. AI интерпретирует молча — именно поэтому и нужны чёткие требования.

---

7. Git-процесс

7.1 Работа с кодом (`.src` репозиторий)

Стандартный feature-branch workflow:

```
cd acme-portal/acme-portal.src

# Создать ветку от main
git checkout main && git pull
git checkout -b feat/TASK-42-totp-auth

# ... работа ...

git add src/auth/totp.py
git commit -m "feat(auth): реализовать TOTP-аутентификацию (TASK-42)"

# Создать MR/PR в git-хостинге
```

Формат коммита: `<type>(<scope>): <описание> (<TASK-ID>)`

7.2 Работа с требованиями (`.req` репозиторий)

```
cd acme-portal/acme-portal.req

# Ветка для изменения требования
git checkout -b change/TZ-2026-002-totp

# Изменить файл, обновить frontmatter
git add sr/SR-01-auth.md
git commit -m "[delta:TZ-2026-002] SR-01 v1.2: добавить поддержку
TOTP"
```

7.3 Привязка MR/PR к задаче

В описании MR/PR всегда указывайте:

```
Closes #42
Related SR: SR-01 (acme-portal.req)
```

Если изменение затрагивает несколько `.req` репозиториев:

```
Related: acmecorp/acme-platform/acme-platform.req#15
```

7.4 Именование веток

Что делаете	Ветка
Новая функциональность	`feat/TASK-NN-slug`
Исправление бага	`fix/TASK-NN-slug`
Изменение требования	`change/TZ-YYYY-NNN-slug`
Новое требование	`feat/BR-NN-slug` или `feat/SR-NN-slug`

---

8. Частые сценарии

Сценарий 1. Начинаю новую задачу

```
1. Открыть задачу в трекере
2. Проверить QG-0 (все пункты выполнены?)
3. Найти ссылку на SR в задаче
4. Открыть SR в acme-portal.req/sr/
5. Прочитать SR + родительский BR если нужен контекст
6. Создать ветку feat/TASK-NN-slug в acme-portal.src
```

```
7. Реализовать + написать тесты по AC
8. MR/PR → ревью → merge
```

Сценарий 2. Обнаружил несоответствие между SR и требованием задачи

```
1. НЕ делать ничего молча
2. Добавить комментарий в задачу:
   "SR-01 §4 описывает X, задача требует Y — это противоречие.
    Прошу уточнить."
3. Дождаться ответа Supervisor/Architect
4. Не брать задачу в работу до устранения несоответствия
```

Сценарий 3. Нужно понять откуда взялось требование

```
# В .req репозитории
git log -- sr/SR-01-auth.md          # история изменений SR
git show <commit-hash>                # детали конкретного изменения
```

Или в frontmatter файла найдите поле `source` — там ссылка на ТЗ и его раздел.

Сценарий 4. Работаю с интеграцией (Acme Portal → Acme AI)

```
# Клонировать требования смежной подсистемы
cd ~/projects/acmecorp/acme-platform
mkdir -p acme-ai
git clone git@github.com:acmecorp/acme-platform/acme-ai/acme-ai.req
acme-ai/acme-ai.req

# Добавить в workspace
# Открыть acme-platform.req/specs/int/SPEC-INT-01-acme-portal-acme-
ai.md
# Там описан контракт интеграции
```

Интеграционные контракты canonical-формы — `SPEC-INT-NN` (standard/08 §8.5.4), не legacy `INT-SR`. SR подсистемы ссылается на `SPEC-INT-NN` через `constrained-by: [SPEC-INT-NN]`. `SPEC-INT` всегда живёт в `acme-platform.req/specs/int/` — не внутри подсистем.

Сценарий 5. Пришло дельта-ТЗ, меняются требования

Это работа архитектора/Tech Lead, но разработчику нужно:

```
1. Дождаться merge MR с обновлёнными SR
2. Сделать git pull в acme-portal.req
3. Прочитать tz/TZ-YYYY-NNN-index.md — там список изменённых
требований
4. Проверить, затрагивают ли изменения ваши текущие задачи
5. Если да — обновить или закрыть задачи по согласованию с Supervisor
```

Сценарий 6. Обновить локальные репозитории требований

```
# Скрипт обновления всех .req репозиториев
find ~/projects/acmecorp -name "*.req" -type d | while read repo; do
  echo "Updating $repo..."
  git -C "$repo" pull --ff-only
done
```

---

9. Права доступа — кто что может менять

Репозиторий	Разработчик	Tech Lead	Архитектор
`acme-platform.req` (системные BR/SR)	Только чтение	Только чтение	Запись через MR
`acme-portal.req` (SR подсистемы)	Только чтение	Запись через MR	Запись через MR
`acme-portal.src` (код)	Запись через MR	Запись + ревью	—

`acme-ai.req` (чужая подсистема)	Только чтение	Только чтение	—

Если нужно изменить требование — создайте Issue в `.req` репозитории, не коммитьте напрямую.

---

10. Быстрый справочник

Куда смотреть когда непонятно

Вопрос	Куда смотреть
Что должна делать система?	`acme-portal.req/sr/SR-NN-*.md`
Зачем это нужно бизнесу?	`acme-platform.req/br/BR-NN-*.md`
Откуда взялось это требование?	frontmatter `source` → ТЗ
Как изменилось требование?	`git log -- sr/SR-NN-*.md`
Как работает интеграция с Acme AI?	`acme-platform.req/specs/int/SPEC-INT-01-*.md`
Что изменилось по последнему ТЗ?	`acme-platform.req/tz/TZ-YYYY-NNN-index.md`

Чеклист перед созданием MR

- [ ] Код покрывает все AC из задачи
- [ ] Есть тесты на негативные сценарии
- [ ] MR/PR содержит ссылку `Closes #NN` на задачу
- [ ] Если изменилось поведение — создан Issue в `.req` для обновления SR

Команды Git которые используются чаще всего

```
# Обновить требования
git -C ~/projects/acmecorp/acme-platform/acme-portal/acme-portal.req
pull

# Посмотреть историю конкретного SR
git -C ~/projects/acmecorp/acme-platform/acme-portal/acme-portal.req
```

```
log -- sr/SR-01-auth.md

# Найти требование по ID
grep -r "id: SR-01" ~/projects/acmecorp/acme-platform/ --
include="*.md"

# Найти все задачи, ссылающиеся на SR-01
grep -r "SR-01" ~/projects/acmecorp/acme-platform/ --include="*.md"
```

Глоссарий

Термин	Значение
BR	Бизнес-требование — зачем нужна функциональность
SR	Системное требование — что делает система
TR	Требование к задаче — Goal + AC в трекере
AC	Acceptance Criteria — критерии приёмки задачи
QG-0	Approval Gate — проверка готовности артефакта к разработке (canonical v1.0; pre-v1.0 имя — «Context Gate»)
`.req`	Git-репозиторий с требованиями подсистемы
`.src`	Git-репозиторий с кодом подсистемы
SPEC-INT	Интеграционная спецификация между подсистемами (canonical v1.0; pre-v1.0 имя — `INT-SR`)
delta-T3	Дополнение к T3 при повторном заказе
deprecated	Устаревшее требование — не удаляется, только помечается

---

title: "Библиотека примеров" description: "Индекс E2E примеров RENAR: quickstart, login, GDPR export, API webhook, SPEC-AI eval, delta-T3." order: 9 lang: ru version: "1.0-draft"

09. Библиотека примеров

> *Шесть сценариев (E1–E6): три полных цикла in-doc (E3–E6) + два walkthrough (E1–E2). Каждый in-doc пример: **T3** → **ADAPT** → **BR/SR** → **SPEC** → **TC** → **QG**.*

#	Сценарий	Документ	Аудитория	Акцент
E1	Email/password sign-up (минимальный)	00-quickstart	Новичок	30 мин, минимум артефактов
E2	Login + profile + permissions (полный)	01-walkthrough	Tech Lead, QA	Полный lifecycle, AI-генерация TC
E3	Экспорт персональных данных (GDPR / ФЗ-152)	§14	Legal, PM, Architect	Compliance, SPEC-DATA/SEC
E4	Webhook idempotency (REST API)	§4	Backend, Architect	`tc-type: contract`, SPEC-API
E5	RAG-ассистент (SPEC-AI eval)	§5	AI engineer	`tc-type: eval`, judge isolation
E6	Delta-T3: scope dispute	§5	PM, Client, Architect	ADAPT backward, immutable TZ

---

2. E3 — Экспорт персональных данных (GDPR Art. 15 / ФЗ-152)

Контекст: SaaS «AcmeCRM» хранит PII клиентов. Регулятор и договор обязывают выдать машиночитаемый экспорт по запросу субъекта данных за ≤30 дней.

2.1 Фрагмент T3 (immutable)

```
# TZ-2026-002 — Data subject export

§3.1 Субъект данных может запросить полный экспорт своих PII через UI
«Privacy → Export my data».
```

```
§4.2 Формат: JSON + CSV bundle, zip, checksum SHA-256.
§4.3 SLA: готовность ссылки на скачивание ≤ 72 часа с момента
verified identity.
§4.4 Экспорт включает: profile, activity log 24 мес., marketing
consents.
§4.5 Запрос логируется; повторный экспорт — не чаще 1 раза в 30 дней
без override admin.
```

2.2 ADAPT (сокращённо)

```yaml
id: ADAPT-002
type: ADAPT
status: approved
source-tz: { id: TZ-2026-002, signed-date: "2026-05-01" }
```

Forward §3: экспорт асинхронный (job queue); identity — через существующий MFA flow.

Backward (resolved):

#	Finding	Resolution
B-01	«24 мес. activity» — calendar или rolling?	Rolling 730 days
B-02	Override admin — кто approves?	Role `privacy-officer` + audit log

2.3 BR + SR

```yaml
# br/BR-02-data-subject-export.md
id: BR-02
type: BR
title: "Субъект данных получает машиночитаемый экспорт PII"
status: approved
source: { adapt: ADAPT-002, adapt-section: "Forward §3" }
compliance:
  - { standard: GDPR, control: "Art. 15" }
  - { standard: "ФЗ-152", control: "ст.14" }
```

```
# sr/SR-08-export-request.md
id: SR-08
type: SR
parent: { id: BR-02 }
title: "Authenticated user инициирует export job"
status: approved
constrained-by: ["SPEC-API-04", "SPEC-DATA-02", "SPEC-SEC-03"]
```

```
# sr/SR-09-export-delivery.md
id: SR-09
type: SR
parent: { id: BR-02 }
title: "User скачивает готовый bundle по time-limited signed URL"
status: approved
constrained-by: ["SPEC-API-04", "SPEC-SEC-03"]
```

2.4 SPEC (выдержки)

SPEC-DATA-02 — schema export bundle (tables: `users`, `activity_events`, `marketing_consents`).

SPEC-SEC-03 — signed URL TTL 24h; rate limit 1 export / 30 days; role `privacy-officer` для override.

SPEC-API-04 — `POST /v1/privacy/export`, `GET /v1/privacy/export/{job_id}`.

2.5 TC (pos/neg для SR-08)

```
---
id: TC-080
title: "Export job создаётся для verified user"
type: TC
tc-type: system
status: ready
verifies:
```

```
  - id: SR-08
    requirement-version: "1.0"
negative: false
---
## When
POST /v1/privacy/export (session: verified-user)
## Then
- 202 + job_id; status pending; audit event recorded
```

```
---
id: TC-081
title: "Export отклонён без MFA-verified session"
type: TC
tc-type: security
status: ready
verifies:
  - id: SR-08
    requirement-version: "1.0"
negative: true
---
## When
POST /v1/privacy/export (session: password-only, no MFA)
## Then
- 403; job не создан; security audit event
```

Аналогичные пары для SR-09 (signed URL valid / expired).

2.6 Контрольные точки качества

Контрольная точка	Доказательства
QG-ADAPT-approve (по смыслу около «архитектурной точки»: `QG-3`)	ADAPT-002 в `approved`, замечания backward закрыты
QG-2 Verification Gate	`TC-080` … `081` успешны; в `last-run` совпадает `requirement-version` (`1.0`)

2.7 Что дальше

- Полный сценарий под `git` — 03-tool-guide-git
- Compliance mapping — 06-compliance
- Conformance manifest — reference/08

---

3. E4 — Webhook idempotency (SPEC-API)

Контекст: платёжный провайдер шлёт `POST /webhooks/payment` с `Idempotency-Key`. Дубликаты не должны создавать двойное списание.

ТЗ (фрагмент): «Повторный webhook с тем же key в течение 24h возвращает тот же `payment_id`, HTTP 200».

SR + SPEC:

```
id: SR-20
type: SR
constrained-by: ["SPEC-API-07"]
```

SPEC-API-07 — контракт: headers, body schema, response codes 200/400/409.

TC (contract pair):

```
id: TC-200
type: TC
tc-type: contract
verifies: [{ id: SR-20, requirement-version: "1.0" }]
negative: false
```

```
id: TC-201
type: TC
tc-type: contract
verifies: [{ id: SR-20, requirement-version: "1.0" }]
negative: true
```

Neg: второй key с другим body → 409 Conflict.

Контрольная точка `QG-2` : оба `TC` успешны; в `last-run` совпадает `requirement-version` (`1.0`).

---

4. E5 — RAG assistant (SPEC-AI)

Контекст: in-app чат отвечает по knowledge base; eval против golden Q&A set.

SPEC-AI-02 — model card, fallback, cost cap.

TC eval (judge ≠ production):

```yaml
id: TC-300
type: TC
tc-type: eval
verifies: [{ id: SPEC-AI-02, requirement-version: "1.0" }]
automation:
  judge-model: "eval-judge-v2"   # ≠ production chat model
negative: false
```

Neg eval: prompt injection в user message → refusal + audit event (`tc-type: eval` , adversarial negative).

См. standard/09 §9.6.2, guide/07 §4.5.

---

5. E6 — Delta-T3: scope dispute

Контекст: после signed TZ клиент просит «добавить экспорт в PDF» — вне scope ADAPT-002.

Правильный путь к источнику истины (SoT):

1. **Не** править immutable TZ и **не** молча расширять SR.
2. Оформить **delta-T3** → новый ADAPT-002b (forward + backward).
3. Backward finding: «PDF export не входил в Forward §3 ADAPT-002».
4. Клиент подписывает delta-ADAPT → новые SR/SPEC/TC.

Anti-pattern: direct commit в `sr/SR-08` без `delta-ref` — drift class 3.11.6 (standard/04 §4.11).

См. standard/07 §7.6, guide/02-transition-guide.

---

title: "Миграция на v1.0-draft" description: "Переход с pre-RENAR / v0.1-draft практик на RENAR v1.0-draft: deprecated типы, поля frontmatter, manifest." order: 10 lang: ru version: "1.0-draft"

10. Миграция на v1.0-draft

> *Для команд, которые уже вели требования «по-своему» или на ранних черновиках RENAR. Цель — **без big-bang**: сохранить immutable ID, заменить deprecated конструкции, выпустить новый conformance manifest. Нормативная база — standard/04 §4.14, CHANGELOG §Migration.*

Не путать с 02-transition-guide — там поэтапный вход в RENAR-1..5 с нуля; здесь — **breaking rename** и выравнивание схемы при переходе на текущую редакцию стандарта.

---

1. Когда нужна эта миграция

Ситуация	Действие
Проект без RENAR, но есть ТЗ + Jira	Сначала 02-transition-guide, не этот документ
Файлы с `INT-SR`, `INT-TC`, `AIC`, `UIC`, `TS`	Это руководство
`verifies[].version` без `requirement-version`	Обновить TC frontmatter (reference/02 §8)
Manifest `renar-version: 0.1-draft` или отсутствует	Выпустить новый manifest (reference/08)

---

2. Таблица замен типов (closed list v1.0-draft)

Deprecated	Canonical	Шаг миграции
`INT-SR`	`SR` + `constrained-by: [SPEC-INT-N]`	Переименовать type; создать/привязать SPEC-INT
`INT-TC`	`TC + tc-type: contract`	Добавить `type: TC`, `tc-type: contract`
`AIC`	`SPEC-AI`	Перенести body в SPEC-AI frontmatter

`UIC`	`SPEC-UI`	Перенести baselines в `specs/ui/baselines/`
`TS`	`SPEC-ARCH` или `SPEC-OPS`	По содержанию (архитектура vs ops runbook)
`TM` (module SR type)	`SR + level: module`	Убрать pseudo-type, задать level

ID не менять. Если filename содержит legacy prefix — допустимо поле `legacy-id` во frontmatter (informative traceability).

---

3. Пошаговый план (1–2 спринта)

Фаза A — инвентаризация (1–2 дня)

1. `grep` / search по носителю: `INT-SR`, `INT-TC`, `AIC`, `UIC`, `type: system` (ошибочный TC type).
2. Список артефактов с broken `verifies` / missing `source.adapt`.
3. Зафиксировать текущий `RENAR-CONFORMANCE.yaml` (если есть) как baseline.

Фаза B — проход по схеме (3–5 дней)

1. Batch-rename типов по таблице §14 (один PR на тип или один atomic change-set на delta-T3).
2. TC: `type: TC`, `tc-type`, `verifies[].requirement-version`, `last-run.requirement-version`.
3. SR: `constrained-by[]` на все используемые SPEC.
4. BR: `source.adapt` на approved ADAPT.

Фаза C — валидация (1–2 дня)

1. Hooks носителя / CI: frontmatter validator (reference/02).
2. Прогнать TC; обновить `last-run`.
3. Чек-лист самооценки — reference/08 §14.

Фаза D — manifest (1 день)

1. Bump `renar-version: "1.0-draft"`.
2. Increment `manifest-version`; новый `manifest-id`.
3. Подпись Architect / Tech Lead (V6).

---

4. Частые ловушки

Ошибка	Последствие	Исправление
Переименовать `SR-05` → `SR-05-v2`	Нарушение V1 immutable ID	`deprecated` + новый ID с `replaces`
Оставить `type: system` на TC	Validator fail; KG drift	`type: TC` + `tc-type: system`
Мигрировать код раньше `.req`	SoT inversion нарушена	Сначала SR/SPEC/TC approved, потом TR
Skip ADAPT «только для новых T3»	Non-conformant для legacy TZ	Ретроспективный ADAPT на каждое активное T3

---

5. Откат

Миграция идёт через историю носителя (V1). Откат — revert change-set / PR, **не** delete артефактов. Deprecated артефакты остаются с `status: deprecated` для audit.

---

6. Связанные документы

Документ	Зачем
02-transition-guide	RENAR-1..5 без schema breaking changes
09-worked-examples	Эталонные frontmatter после миграции
reference/07	Внешнее заявление ISO 29148
standard/13 §13.7	Re-assessment после миграции

Guide RENAR 1.0-draft — renar.tech

---

title: "Глоссарий" description: "Каноническая терминология RENAR и сопоставление с ISO/IEC 29148, BABOK, SAFe, ISTQB, SENAR." order: 1 lang: ru version: "1.0-reconciled"

Глоссарий RENAR

> ***Назначение:*** *единый источник канонических терминов RENAR с примерами и mapping на отраслевые стандарты. При расхождении формулировок* ***нормативно*** *побеждает* `standard/04-terms.md` *; этот глоссарий — informative lookup для читателя и assessor-а.*

---

1. Цепочка авторитетности

В случае разногласий по термину применяется порядок:

1. **Этот документ** — канонический для RENAR.
2. `standard/04-terms.md` — нормативные определения главы стандарта.
3. **ISO/IEC/IEEE 29148** — международный стандарт инженерии требований.
4. **BABOK Guide v3** — свод знаний по бизнес-анализу (*Business Analysis Body of Knowledge*).
5. **Изменение через предложение поправки в полный стандарт RENAR** — если все источники молчат.

Закрытые списки: главный индекс шестнадцати закрытых списков — `standard/01 §1.7.5` .

Не используются как источник терминов: тикеты в багтрекере, чаты команды (slang), устаревшие презентации, маркетинговые материалы.

---

2. Канонические термины

2.1 Уровни требований

Каноника v1.0 — закрытый список (см. `standard/04-terms.md §4.3`):

RENAR canonical	Полное название	RU (для UI)	Описание

BR	Business Requirement	Бизнес-требование	Бизнес-цель уровня заказчика: что организация хочет получить.
SR	System Requirement	Системное требование	Инженерное требование к ПО; поддаётся проверке. Один `SR` — одна проверяемая единица.
TR	Task Requirement	Требование к задаче	Требование уровня реализации, выводимое из `SR`; единица планирования работ.
TC	Test Case	Контрольный пример (TC)	Проверяемый артефакт, подтверждающий `SR` / `BR`. Описывает поведение, а не реализацию.

Правило идентификации: в frontmatter `id` — канонический идентификатор RENAR (`BR-01`, `SR-05`). При экспорте в другой носитель применяется целевое сопоставление.

Уточнения для `SR` (по полям frontmatter, не отдельные типы артефактов):

- **Module SR** — `SR` с `level: module` (`standard/06 §6.7`). Раньше отдельный label `TM` (§14.1.1).
- **Integration SR** — `SR` с `constrained-by: [SPEC-INT-N]`. Раньше отдельный label `INT-SR` (§14.1.1).
- **Contract TC** — `TC` с `tc-type: contract`. Раньше отдельный label `INT-TC` (§14.1.1).

Семейство `SPEC` (UX / ИИ / архитектура / интеграционные спецификации) — см. §14.5 Семейство SPEC.

2.1.1 Legacy labels (deprecated)

При миграции с материалов до v1.0 draft встречаются устаревшие метки. Их замены в каноне v1.0 (`standard/04-terms.md §4.14.1`):

Устаревший label	Замена в каноне v1.0	Пояснение
`TM` (Module/Submodule SR)	`SR` с `level: module`	Уточнение `SR`, не отдельный тип артефакта

`UIC` (UI Concept)	`SPEC-UI` (`standard/08 §8.5.6`)	Часть семейства SPEC (§14.5)
`AIC` (AI Concept)	`SPEC-AI` (`standard/08 §8.5.7`)	Часть SPEC family
`INT-SR` (Integration SR)	`SR` с `constrained-by: [SPEC-INT-N]`	Sub-classification SR
`INT-TC` (Integration TC)	`TC` с `tc-type: contract`	Sub-classification TC
`TS` (Technical Specification)	`SPEC-ARCH` или `SPEC-OPS` в зависимости от содержания	Часть семейства `SPEC`

Миграция существующих артефактов: встроенная в носитель привязка к набору изменений должна автоматически распознавать устаревшие метки и предлагать канонические замены. См. также `reference/06-ru-style-guide.md §1.10` — процедура прохода главы фазы 5.

2.2 ADAPT artifact family

Термин	Описание
ADAPT	Адаптивный документ, формулирующий проектное ТЗ (*Adaptive Document for Articulating Project's TZ*). Двусторонняя инженерная адаптация ТЗ: прямое направление (интерпретация) и обратное (вопросы). Утверждается двойной подписью.
delta-ADAPT	`ADAPT` для delta-ТЗ. Цепочка: ADAPT-001 → ADAPT-001-delta-1 → ADAPT-001-delta-2; применяются по порядку.
errata-ADAPT	Правка утверждённого `ADAPT` (наша ошибка интерпретации). Не меняет замороженный документ — добавляется отдельный артефакт.
Forward (в ADAPT)	Инженерная интерпретация каждого раздела ТЗ: цитата → интерпретация → достроенные сценарии → охват.
Backward (в ADAPT)	Список проблем и вопросов к клиенту. Жизненный цикл: `open` → `asked-to-client` → `answered` → `resolved` → `frozen`.
Term mapping (в ADAPT)	Таблица «термин клиента → инженерное понимание».

2.3 Backward categories (closed list)

ADAPT backward записи относятся к одной из 7 категорий. Список закрыт; новые добавляются только через изменение полного RENAR Standard:

Категория	Описание
`contradiction`	Противоречие внутри ТЗ.
`gap`	Гэп — про это ТЗ молчит.
`hidden-assumption`	Скрытое предположение инженера, требующее подтверждения.
`feasibility`	Технически нереализуемое или дорогое требование.
`regulatory`	Затрагивает законодательство / compliance.
`terminology`	Неясный или конфликтующий термин клиента.
`scope`	Уточнение границ scope (входит / не входит).

2.4 Контрольные точки качества (`Quality Gates` , закрытый список, каноника v1.0)

Закрытый список контрольных точек RENAR (по `standard/10-lifecycle-qg.md §10.3–10.4`):

Gate	Применяется к	Условие пропуска
QG-0 Контрольная точка утверждения (`Approval Gate`)	Перевод `BR` / `SR` / `SPEC` : `draft` → `approved`	Цель, критерии приёмки, не менее одного негативного сценария и охват; для `SR` — родительский `BR` в `approved` +; для `SR` с `constrained-by` — `SPEC` в `approved` +.
QG-1 Контрольная точка реализации (`Implementation Gate`)	Перевод `TC` : `draft` → `ready` (**только** для `TC`)	У `TC` поле `verifies` ссылается на артефакт в `approved` +; `requirement-version` совпадает; охват реализации и закрепление версии зафиксированы.
QG-2 Контрольная точка проверки (`Verification Gate`)	Перевод `SR` / `BR` : `approved` → `verified`	Все `TC` из `verified-by` зелёные; хотя бы один `TC` с `negative: true` ; у `last-run` совпадает `requirement-version` с текущей `version` ; выборочная проверка пройдена.
QG-3 Контрольная точка архитектуры	Утверждение `SPEC-ARCH` /	Зафиксирован артефакт в духе ADR в носителе; двойная подпись (клиент +

(`Architecture Gate` , *опционально*)	двойная подпись	архитектор). Не обязательна для декларируемого соответствия RENAR.
QG-4 Контрольная точка приёмки (`Acceptance Gate` , *опционально*)	Перевод `BR` : `verified` → `accepted`	Все `BR` покрыты проверенными `SR` ; приёмочные `TC` (`tc-type: acceptance`) зелёные; подпись клиента. Не обязательна для декларируемого соответствия RENAR.

Соответствие стандарту RENAR (conformance): `QG-0` / `QG-1` / `QG-2` обязательны для декларируемого соответствия (`standard/13 §13.3`). `QG-3` / `QG-4` — необязательные расширения.

Список закрыт; новые номера контрольных точек добавляются только через формальную процедуру изменения полного стандарта RENAR.

2.4.1 Legacy QG names (deprecated)

До v1.0 в RENAR использовались иные имена контрольных точек. Сопоставление по `standard/04-terms.md §4.14.1` :

Устаревшее (до v1.0)	Замена в каноникe v1.0	Пояснение
`QG-0 Context Gate`	`QG-0 Approval Gate`	Только переименование; смысл сохранён
`QG-1 Requirements Gate`	`QG-1 Implementation Gate`	**Сдвиг смысла:** ранее утверждение `BR` / `SR` , теперь только перевод `TC` : `draft → ready`
`QG-2 Implementation Gate`	`QG-1 Implementation Gate`	Перенумерация и слияние в одну `QG-1`
`QG-3 Verification Gate`	`QG-2 Verification Gate`	Перенумерация
`QG-4 Acceptance Gate`	`QG-4 Acceptance Gate`	Имя то же; теперь **опционально** для декларируемого соответствия RENAR

Миграция существующих артефактов: средства автоматизации преобразуют ссылки по таблице выше. См. `reference/06-ru-style-guide.md §1.10.2` — процедура фазы 5.

2.4.2 Локальные контрольные точки ADAPT

Жизненный цикл `ADAPT` (`standard/07-adapt.md`) использует локальные контрольные точки `ADAPT` параллельно каноническим `QG-N` . Это не отдельная нумерация `QG` , а локальные события жизненного цикла артефакта `ADAPT` :

Контрольная точка	Применение	Условие прохода
QG-ADAPT-draft	Создание `ADAPT`	Раздел Forward охватывает все разделы T3.
QG-ADAPT-review	Перевод в `review`	Все записи backward — `open` или `asked-to-client` ; нет состояния `draft` .
QG-ADAPT-client-ready	Передача клиенту	Все backward — `asked-to-client` ; пакет вопросов собран.
QG-ADAPT-answered	После ответа клиента	Все backward — `answered` .
QG-ADAPT-approve	Утверждение `ADAPT`	Все backward — `resolved` ; двойная подпись (клиент + архитектор).
QG-ADAPT-frozen	После утверждения	Неизменяемость; разрешена генерация `BR` / `SR` / `SPEC` .

Локальные контрольные точки `ADAPT` **не входят** в набор `QG-0` / `QG-1` / `QG-2` для декларируемого соответствия RENAR. Реализация может выразить `QG-ADAPT-approve` как локальный псевдоним для `QG-3` (`Architecture Gate` , если применимо) либо хранить отдельно — на усмотрение носителя.

2.5 Семейство SPEC (закрытый список)

Тип	Назначение	Источник
SPEC-ARCH	Архитектура системы / подсистемы: контексты, контейнеры, компоненты, представление развёртывания, атрибуты качества	Раздел Forward `ADAPT`
SPEC-API	Контракты API (REST / GraphQL / gRPC / асинхронные события); версионирование, модель ошибок, ограничения частоты	Раздел Forward `ADAPT`
SPEC-DATA	Модель данных: схема, ER-диаграмма, индексы, миграции, хранение, классификация	Раздел Forward `ADAPT`

	персональных данных	
SPEC-INT	Интеграция: взаимодействие между подсистемами и внешними системами; протоколы, контракты, SLA	Раздел Forward `ADAPT`
SPEC-PROC	Процесс / рабочий поток: бизнес-процессы, автоматы состояний, паттерны saga, оркестровка и хореография	Раздел Forward `ADAPT`
SPEC-UI	UI / UX: экраны, навигация, пользовательские сценарии, доступность, локализация, эталонные изображения	Раздел Forward `ADAPT`
SPEC-AI	ИИ / ML: карточки моделей, RAG, инженерия промптов, стратегия оценки, бюджет стоимости	Раздел Forward `ADAPT`
SPEC-SEC	Безопасность: аутентификация / авторизация, модель угроз, управление секретами, классификация данных	Раздел Forward `ADAPT`, регуляторные замечания backward
SPEC-OPS	Эксплуатация: развёртывание, наблюдаемость, SLO / SLA, runbook, аварийное восстановление	Раздел Forward `ADAPT`

Список закрыт — ровно девять типов (каноника по `standard/08 §8.3`); новые типы `SPEC` добавляются только через изменение полного стандарта RENAR.

2.6 Lifecycle статусы

Статус	Применим к	Значение
`draft`	все артефакты	В работе, не для использования другими
`review`	все	На ревью, возможны изменения
`approved`	ADAPT, SR, SPEC	Утверждён; неизменяем после двойной подписи (для `ADAPT`) или одной (для `SR` / `SPEC`)
`verified`	SR	Подтверждён успешными `TC` и выборочной проверкой
`frozen`	ADAPT	Утверждён и неизменяем; используется как источник для генерации `BR` / `SR` / `SPEC`
`deprecated`	все	Устарел, не используется в новых артефактах; замена (если есть) указывается полем `replaced-by`

`obsolete`	research/legacy	Полностью изъят из обращения

2.7 Возможности носителя (V1–V6)

RENAR не привязан к конкретному носителю артефактов. Артефакт может находиться в любом носителе, который удовлетворяет следующим возможностям:

Нормативное определение — `standard/03 §3.3` :

Возможность	Описание
V1 — неизменяемая история	Любое прошлое состояние артефакта можно адресно восстановить без потерь (цепочка ревизий для аудита сохраняется).
V2 — атомарная единица изменения	Изменение артефакта (или согласованной группы) фиксируется одной транзакцией: целиком успех или целиком откат; промежуточные состояния извне не видны.
V3 — сравнение и ревью (diff)	Предлагаемое изменение можно представить как разницу к базовой версии и принять или отклонить до попадания в утверждённое состояние (основа утверждения и контрольных точек `QG`).
V4 — ветвление и набор изменений	Незавершённая работа отделена от утверждённого источника истины; несколько независимых изменений ведутся параллельно без влияния на источник истины.
V5 — межсценарная фиксация версии	Можно зафиксировать конкретную версию артефакта из другого носителя как разрешимый идентификатор (основа поля `verifies[].requirement-version`).
V6 — автор и метка времени	Для каждой атомарной правки зарегистрированы однозначный автор и метка времени (основа подписи `ADAPT` и блока `ai-provenance`).

Примеры сред: распределённая VCS (`git` с ревью запросов на слияние), документоориентированное хранилище с цепочкой ревизий и подписями, любая СУБД с историей изменений и подписями. RENAR не требует `git` — только возможности **V1–V6**.

2.8 Провенанс ИИ (`ai-provenance` , канонические поля)

Во `frontmatter` любого артефакта, сгенерированного ИИ:

Поле	Тип	Описание
`ai-provenance.generated-by`	string	Модель в формате `<vendor>-<model>-<version>@<date>`. Пример: `anthropic-claude-opus-4-7@2026-05-15`.
`ai-provenance.prompt-template`	string	Путь к шаблону промпта и версия. Пример: `prompts/adapt-from-tz.md@v2.1`.
`ai-provenance.context-tokens`	integer	Число токенов во входном контексте.
`ai-provenance.output-tokens`	integer	Число токенов в выводе модели.
`ai-provenance.generation-time-ms`	integer	Время генерации в миллисекундах.
`ai-provenance.human-edits`	boolean	Значение «истина», если текст после генерации правил человек. **Для утверждённых артефактов обязательно `true`.**

2.9 Типы контрольных примеров

Закрытый список — шесть типов (каноника по `standard/09 §9.5`):

Тип TC (`tc-type` поле)	ISTQB соответствие	Применение
`acceptance`	Acceptance Testing	Проверка BR (приёмочный).
`ux`	расширение по удобству	Проверка `SPEC-UI` через модель-судью VLM / визуальное сравнение с эталоном (`baseline`).
`system`	System Testing	Проверка SR, SPEC-PROC, SPEC-ARCH.
`contract`	Component Integration Testing	Проверка SPEC-API / SPEC-INT / SPEC-DATA через contract testing (Pact и аналоги).
`eval`	специфично для ИИ	Проверка `SPEC-AI` через оценивающую LLM и метрики (BLEU, точность, доля галлюцинаций).
`security`	расширение по безопасности	Проверка `SPEC-SEC`: инварианты безопасности (STRIDE); нормативно только негативные сценарии (`standard/09 §9.6.4`).

2.10 Связи (frontmatter поля)

Поле	Назначение
`parent`	Родитель в иерархии (BR → SR → TR); один источник.
`children`	Дочерние артефакты (выводятся автоматически).
`source.adapt`	`ADAPT`, из которого выведен артефакт (каноника для `BR` / `SR` / `SPEC`).
`source.adapt-section`	Раздел `ADAPT` (Forward §N).
`source.tz-section`	Раздел ТЗ (справочно, для трассируемости).
`verifies` (в `TC`)	`SR` / `BR`, которые покрывает `TC`, с указанием `requirement-version`.
`verified-by` (в `SR`)	`TC`, подтверждающие `SR` (выводятся автоматически).
`derived-from`	Шаблон и версия (если артефакт создан из шаблона).
`replaces` / `replaced-by`	Замена при статусе `deprecated`.
`supersedes` (в новом)	Какое требование заменяется.
`linked-tasks`	Задачи, реализующие `SR` (через среду выполнения, не через файлы).

2.11 Соглашение об именах файлов (по умолчанию)

Тип	Шаблон	Пример
ADAPT	`adapt/ADAPT-NNN[-delta-N].md`	`adapt/ADAPT-001-main.md`, `adapt/ADAPT-001-delta-1.md`
BR	`br/BR-NN-<slug>.md`	`br/BR-01-notification-capture.md`
SR	`sr/SR-NN-<slug>.md`	`sr/SR-05-notification-feed.md`
SR подсистемы	`sr/<MODULE>-SR-NN.N-<slug>.md`	`sr/WMS-SR-01.2-pick.md`
SPEC-UI	`specs/ui/SPEC-UI-NN-<slug>.md`	`specs/ui/SPEC-UI-02-notification-feed.md`

SPEC-AI	`specs/ai/SPEC-AI-NN-<slug>.md`	`specs/ai/SPEC-AI-01-rag-strategy.md`
SPEC-INT	`specs/int/SPEC-INT-NN-<slug>.md`	`specs/int/SPEC-INT-01-auth-billing.md`
SPEC (generic)	`specs/<type>/SPEC-<KIND>-NN-<slug>.md`	`specs/api/SPEC-API-03-orders.md`
TC	`tests/TC-NN-<slug>.md`	`tests/TC-01-login-success.md`
T3	`tz/TZ-YYYY-NNN.md`	`tz/TZ-2026-001.md`
Дельта-T3	`tz/TZ-YYYY-NNN-delta-N.md`	`tz/TZ-2026-001-delta-1.md`
UX baseline	`specs/ui/baselines/SPEC-UI-NN-<scenario>.png`	`specs/ui/baselines/SPEC-UI-02-feed-default.png`
Eval dataset	`specs/ai/eval-datasets/SPEC-AI-NN-<slug>.jsonl`	`specs/ai/eval-datasets/SPEC-AI-01-typical-queries.jsonl`

Соглашение по умолчанию; хранилища, встроенные в конкретную среду, могут использовать иную раскладку при условии устойчивых идентификаторов (возможность **V3**).

2.12 Маркеры записи об изменении (справочно)

В носителе, где атомарные изменения сопровождаются метаданными (текст коммита в `git`, описание записи об изменении в документоориентированном хранилище), допустимы маркеры:

Маркер	Назначение
`[delta:TZ-YYYY-NNN]`	Изменение по delta-T3.
`[test-spec-change]`	Изменение критериев типа «пройдено / не пройдено» у `TC` (отдельное утверждение).
`[baseline-update]`	Обновление эталона UX или набора для оценки (отдельное утверждение).
`[coverage]`	Автоматическая регенерация сводок по покрытию, плану контрольных примеров и требованиям (бот).
`[reconciliation]`	Изменение от агента согласования (reconciliation).

`[multi-model-disagreement]`	Артефакт с расхождением ответов моделей ИИ — требует ручного ревью.
`[AI]`	Префикс для изменений, сгенерированных ИИ.

Механизм, встроенный в носитель, может выразить эти маркеры полями записи об изменении, метками или тегами — детали не нормируются.

---

3. Сопоставление со стандартами

3.1 Уровни требований

Метки каноники v1.0 RENAR и внешние стандарты:

RENAR	ISO/IEC 29148	BABOK	SAFe	Document store (example enum)
BR	Business Requirement	Business Need	Portfolio Epic / Strategic Theme	`BT`
SR	System / Software Requirement	Solution Requirement (Functional)	Feature	`ST`
SR (`level: module`)	(область подкомпонента, расширение)	(область подкомпонента)	Story (иногда)	`TM` (legacy
SR (`constrained-by: SPEC-INT-N`)	Требование к интерфейсу	Интерфейсное решение	Межфичевая интеграция	(связано)
TR	(нет прямого класса; детализация требования к системе / элементу системы)	Детализированное требование к решению	Story	`TK`

TC	Контрольный пример (Test Case)	Верификация	Приёмочный тест истории	`test_case`
TC (`tc-type: contract`)	Тест интерфейса	Component Integration Testing	Контрактный тест	(связано)
SPEC-UI	(между BR/SR — проектная спецификация)	Требование стейкхолдера (фрагмент UX)	(уровень проектирования)	(расширен
SPEC-AI	(расширение REQ)	(н/д)	Enabler	(расширен
SPEC-ARCH / SPEC-OPS	Описание проектирования	Компонент решения	Enabler tech spec	(расширен

Примечание: колонки «document store (пример перечисления)» и «SENAR (RU)» содержат исторические метки (`TM` , `UIC` , `AIC` , `INT-CT` и т.д.) для трассируемости с системами до v1.0. Колонка каноники RENAR — метки v1.0 согласно §14.1.1. При экспорте в document store или в носитель SENAR применяют целевое сопоставление.

3.2 Контрольные точки качества

Сопоставление канонических `QG-N` v1.0 RENAR с внешними моделями. Устаревшие имена `QG` (§14.4.1) в колонке SENAR сохранены для исторической трассируемости.

RENAR (v1.0)	SENAR (сопоставление с legacy)	Document store (пример)	Активность CMMI
QG-0 Контрольная точка утверждения	QG-0 (контекст)	`VK-1` (старт)	Проверка требований до обязательств
QG-1 Контрольная точка реализации	QG-2 (реализация в legacy)	`VK-1`	Базовая линия реализации (готовность `TC`)
QG-2 Контрольная точка проверки	QG-3 (верификация в legacy)	`VK-2`	Верификация
QG-3 Контрольная точка архитектуры *(опционально)*	(н/д)	`VK-3` (частично)	Утверждение архитектурного решения

QG-4 Контрольная точка приёмки *(опционально)*	QG-4 (Приёмка)	`VK-4`	Приёмка заказчиком

Примечание: до v1.0 `QG-1 Requirements Gate` фактически разделён между каноническим `QG-0 Approval Gate` (утверждение `BR` / `SR` / `SPEC`) и `QG-1 Implementation Gate` (готовность `TC`). См. §14.4.1 для полного сопоставления.

3.3 Lifecycle статусы

RENAR	Document store (example)	ISO/IEC 29148	CMMI
`draft`	`draft`	proposed	identified
`approved`	`approved`	agreed-to / baselined	committed
`verified`	`verified`	verified	validated
`deprecated`	`obsolete`	retired	obsolete

3.4 Артефакты процесса

RENAR	BABOK	SAFe	SENAR
ADAPT (Forward + Backward)	Requirements Analysis Document	Solution Intent (fixed + variable)	(n/a — RENAR-extension)
Work Order / T3	Stakeholder commitment artifact	Customer order	(контекст)
delta-TZ	Change Request	(n/a — handled via Solution Intent updates)	(контекст)
Impact Analysis	Impact Analysis (BABOK §8)	(derived)	(контекст)
Spot-check	Random sampling QA	(n/a)	Rule 9.5
Adversarial review	Independent verification	(n/a — REQ-extension)	(via ADR metric)
Reconciliation	Continuous improvement audit	Inspect & Adapt	Quality Sweep

3.5 Переводы в пользовательском интерфейсе

Поля `frontmatter`, идентификаторы и имена файлов — всегда канонические (латиница). В интерфейсе допустимы русские подписи:

Canonical	UI (RU)
Business Requirement	Бизнес-требование
System Requirement	Системное требование
Test Case	Контрольный пример (`TC`)
Quality Gate	Контрольная точка качества
Acceptance	Приёмка
Verified	Проверено
Approved	Утверждено
Deprecated	Устарело
Frozen	Замороженный
Backward finding	Замечание к ТЗ
Forward interpretation	Инженерная интерпретация

Перевод в интерфейсе не заменяет канонические идентификаторы в носителе.

---

4. Запрещённые / устаревшие термины

RENAR не использует следующие термины (даже если они встречаются в SENAR / отраслевой литературе):

Термин	Что использовать вместо	Почему
User Story как требование	SR	Story — единица планирования, не требование. Story может реализовывать SR, но сама требованием не является.
Use Case (формально)	SPEC-UI + SR	Use case mixes UX и behavior. RENAR разделяет SPEC-UI (UX-эталон) и SR (поведение).

Spec (без квалификатора)	SR / BR / SPEC-API / SPEC-DATA / ...	«Spec» неоднозначно. Используем точные термины.
Бизнес-логика как требование	SR	«Бизнес-логика» — термин кода, не требований.
Функциональность	`SR` / `TR`	Слишком широко; не поддаётся однозначной проверке.
Фича (mixed-lang)	Feature (SAFe context) или SR (RENAR canonical)	Mixed Russian/English; неоднозначно.
Хотелка	(никогда)	Договорный документ так не пишется.
Эпик как требование	BR (бизнес-уровень) или Portfolio Epic (SAFe)	Epic — единица планирования, не требование.
«Тестировать руками»	spot-check (Rule 5 Core) или manual TC с `type: acceptance`	Расплывчатое; не verifiable evidence.
«Доделать» (как статус)	`draft` / `review`	Не из closed list lifecycle.
TODO во `frontmatter`	запись backward в `ADAPT` (если речь о требовании) или задача в трекере (если о реализации)	Открытые вопросы живут в нужном артефакте, не в комментарии к полю.

При таких находках в локальных артефактах проекта включайте предупреждение на стороне носителя (`pre-commit` в `git` , правило валидации в document store).

---

5. Версионирование глоссария

Глоссарий — самостоятельный документ со своей версией. Изменение канонического термина — скачок major-версии (1.0 → 2.0) и сценарий миграции для всех проектных артефактов.

Текущая версия: 1.0-reconciled (согласование фазы 1.5 с `standard/04-terms.md §4.14.1`).

5.1 Открытые вопросы — закрыты (фаза 1.5, 2026-05-16)

Четыре открытых вопроса прежней редакции черновика закрыты согласованием с `standard/04-terms.md §4.14.1`:

#	Был открытым вопросом	Итог	Источник
1	Канонический язык: латиница (`BR` / `SR`) или русский (БТ/СТ)?	**Канон — латиница**; русский только в проекции UI (§4.5)	`standard/04 §4.13.3` + `reference/06-ru-style-guide.md §1.3`
2	`TM` как отдельная метка или уточнение `SR` ?	**`SR c level: module`** — уточнение, не отдельный тип артефакта	`standard/04 §4.14.1` (устарело `TM`) + §14.1.1
3	AIC ← AAC / AIA / AIC?	**`SPEC-AI`** (каноника v1.0); AIC — устаревшая метка	`standard/04 §4.14.1` + §14.1.1
4	`INT-TC` отдельный тип или соглашение об именах?	**`TC c tc-type: contract`** — уточнение, не отдельный тип	`standard/04 §4.14.1` (`INT-TC` deprecated) + §14.1.1

5.2 История согласований

Дата	Версия	Изменение
2026-05-16	1.0-reconciled	Согласование фазы 1.5 с `standard/04-terms.md §4.14.1` . Устаревшие метки перенесены из таблицы §14.1 в §14.1.1; имена `QG` приведены к канонике v1.0 в §14.4; устаревшие имена → §14.1. Обновлены таблицы сопоставления §4.1 и §4.2. Четыре открытых вопроса закрыты (§2.1). Задача: `ru-reconcile-glossary-vs-standard` .
(ранние черновики)	1.0-draft	Наполнение черновика в фазе 7; были открытые вопросы и расхождения с `standard/04 §4.14.1` . История коммита зафиксирована; заменено выпуском 1.0-reconciled.

5.3 Перекрёстные ссылки

- **Стиль редакции v1.0** (`reference/06-ru-style-guide.md`) — правила русской редакции нормативного текста; §1.9 задаёт канонический перечень терминов параллельно этому глоссарию. При расхождении приоритет формулировок для редакционных проходов — у руководства по стилю.

- **Канонические определения** (`standard/04-terms.md`); §4.14.1 — сопоставление устаревшие → каноника.

Глоссарий RENAR 1.0-reconciled — часть `reference/` *. См. также 02-schemas.md, 03-ai-risk-register.md, 06-ru-style-guide.md.*

---

title: "Схемы (formal)" description: "Canonical YAML frontmatter schemas for all RENAR artifact types + cross-field validation rules." order: 2 lang: ru version: "1.0-draft"

Схемы (формальные)

> ***Назначение:*** *машино-читаемые YAML frontmatter схемы для всех типов артефактов RENAR. Используются нативными для носителя валидаторами (CI hooks, document validators, application-level constraints) для проверки conformance.*
>
> ***Нормативные определения структуры*** *— в* `standard/06`, `standard/07`, `standard/08`, `standard/09`. *Валидация примеров:* `node scripts/validate-schema-examples.js`.

Этот документ — reference (informative lookup).

---

1. Common frontmatter (все артефакты требований и SPEC)

Поля, общие для BR/SR/TR/SPEC-* (canonical v1.0). Legacy типы `UIC` / `AIC` / `INT-SR` / `TS` — deprecated в v1.0; используются canonical замены (`../standard/04-terms.md §4.14.1`):

```
# === Identity ===
id: "<TYPE>-NN[.N]"                    # immutable; pattern зависит от типа
title: "<short, descriptive>"
type: BR | SR | TR | SPEC-ARCH | SPEC-API | SPEC-DATA | SPEC-INT | SPEC-PROC | SPEC-UI | SPEC-AI | SPEC-SEC | SPEC-OPS
slug: "<kebab-case>"                   # auto-derived from title

# === Scope ===
level: system | subsystem | module
scope:
  system: "<system-id>"
  subsystem: "<subsystem-id>"          # null если level=system

# === Lifecycle ===
status: draft | review | approved | verified | deprecated | obsolete
```

```
| frozen
                                        # замена артефакта =
deprecated + поле replaced-by (replaced — не статус, §10.5.3);
                                        # применимость по типам:
verified — BR/SR; frozen — только ADAPT (§7);
                                        # ready/passing/failing —
только TC (§8); см. canonical lifecycle standard/04 §4.6
priority: must | should | could       # MoSCoW; SAFe — через WSJF
(см. BR-specific)

# === Provenance: source ===
source:
  adapt: "ADAPT-NNN"                  # canonical источник (Forward
§N)
  adapt-section: "Forward §N.N"
  tz-section: "§N.N"                  # informative — для
traceability до ТЗ
  document-ref: "<нативная для носителя ссылка>"   # pinned ревизия
source-документа

# === Hierarchy (дерево требований) ===
parent:
  id: "<parent-id>"                   # required для SR (→BR), TR
(→SR); optional для BR
  ref: "<нативная для носителя ссылка>"
children: []                          # auto-derived

# === Связь с SPEC (граф) ===
constrained-by: []                    # для SR — список SPEC-*
типизированных ссылок
implements-spec: []                   # для TR — какие SPEC
реализуются
depends-on: []                        # между SPEC

# === Verification ===
verified-by: []                       # auto-derived; список TC IDs
verifies-business-goal: ""            # optional, ссылка на
BR.business-goal-id

# === AI provenance (RENAR-4+ обязательно для approved) ===
ai-provenance:
  generated-by: "<vendor>-<model>@<date>"
```

```yaml
  prompt-template: "<template-path>@<version>"
  context-tokens: integer
  output-tokens: integer
  generation-time-ms: integer
  generated-at: "<ISO-8601>"
  human-edits: boolean                # true required для approved

# === AI cost budget (optional) ===
ai-budget:
  context-tokens-target: integer
  context-tokens-actual: integer
  output-tokens-target: integer
  output-tokens-actual: integer
  generation-time-target-ms: integer

# === Замена ===
replaces: "<old-id>"
replaced-by: "<new-id>"
deprecated-date: "<ISO date>"

# === Schema versioning ===
schema-version: "1.0"
```

---

2. BR — Business Requirement

```yaml
# Extends common §1, дополнительно:

business-context:
  stakeholder: "<role>"               # required
  business-goal: "<short statement>"  # required
  kpi-impact:
    - kpi: "<KPI name>"
      direction: increase | decrease
      target: "<measurable>"

business-outcome:                     # required для QG-4
  measurement-type: kpi | survey | observation | usage
  kpi-name: "<KPI>"
```

```
  measurement-method: "<how>"
  baseline-value: number
  baseline-measured-at: "<ISO date>"
  target-value: number
  target-met-by: "<ISO date>"
  current-value:
    value: number
    measured-at: "<ISO date>"
    achievement: "<percent>"

prioritization:
  framework: WSJF | RICE | MoSCoW
  wsjf-score: number
  prioritized-at: "<ISO date>"
  prioritized-by: "<role>"

data-classification:
  contains-pii: boolean
  contains-financial: boolean
  contains-health: boolean
  contains-children-data: boolean
  retention-days: integer
  data-residency: ["RU" | "EU" | "US" | ...]

compliance:
  - { standard: "ISO 27001:2022", control: "<id>", rationale: "..." }
  - { standard: "GDPR", article: "Art.NN" }
  - { standard: "ФЗ-152", article: "ст.NN" }

ai-act:
  risk-class: prohibited | high | limited | minimal
  rationale: "<reason>"
  high-risk-domain: boolean
```

---

3. SR — System Requirement

```
# Extends common §1, дополнительно:
```

```yaml
parent:
  id: "BR-NN"                          # required

# источник ADAPT — через canonical source.adapt / source.adapt-
section (§1 common);
# отдельного поля derived-from-adapt нет (см. standard/06 §6.6.2)
constrained-by:                        # типизированные рёбра к SPEC-*
  - "SPEC-UI-NN"
  - "SPEC-API-NN"
  - "SPEC-DATA-NN"
  - "SPEC-SEC-NN"

quality-characteristic:                # ISO/IEC 25010:2023 (9
характеристик)
  - functional-suitability
  - performance-efficiency
  - compatibility
  - interaction-capability             # ранее usability (25010:2011)
  - reliability
  - security
  - maintainability
  - flexibility                        # ранее portability
(25010:2011)
  - safety                             # новая в 25010:2023

# Inherited from parent BR (если применимо):
# data-classification, compliance, ai-act
```

---

4. TR — Task Requirement

```yaml
# Extends common §1, дополнительно:

parent:
  id: "SR-NN"                          # required

implements-spec: []                    # SPEC-* которые реализуются
```

```
этой задачей
estimated-effort: "<short statement>" # optional, free-form
```

---

5. SPEC-* common schema

Все 9 типов SPEC делят общую структуру (см. §1) + следующие SPEC-specific поля:

```
type: SPEC-ARCH | SPEC-API | SPEC-DATA | SPEC-INT | SPEC-PROC | SPEC-
UI | SPEC-AI | SPEC-SEC | SPEC-OPS

referenced-by: []                    # auto-derived; SR/TR/SPEC
ссылающиеся на этот SPEC
depends-on: []                       # SPEC от которых этот зависит
compliance-refs: []                  # ISO / GDPR / ФЗ-152 / AI Act
/ NIST AI RMF
```

Обязательные разделы body:

```
## Назначение
## Scope (входит / не входит)
## <Type-specific sections — см. §6>
## Связь с требованиями
## Связь с другими SPEC
## Verification
## Open questions
```

---

6. SPEC type-specific extensions

6.1 SPEC-ARCH — Architecture

```
arch-style: monolith | microservices | modular-monolith | serverless
| hybrid
```

```
deployment-model: cloud | on-prem | hybrid | edge
tech-stack:
  languages: []
  frameworks: []
  data-stores: []
  message-brokers: []
quality-attributes:
  - { name: latency, target: "p95 < 200ms" }
  - { name: availability, target: "99.9%" }
```

Industry reference: arc42, C4 model, ISO/IEC/IEEE 42010.

6.2 SPEC-API — API Contract

```
api-style: rest | graphql | grpc | websocket | async-events
api-version: "v1.2.0"
versioning-strategy: url-path | header | query-param | content-
negotiation
authentication: bearer-jwt | api-key | oauth2 | mtls | none
rate-limits:
  - { endpoint: "*", limit: "1000/min/key" }
contract-file:
  format: openapi-3.1 | asyncapi-2.6 | proto3
  location: "contracts/<name>.yaml"
```

Industry reference: OpenAPI 3.1, AsyncAPI 2.6, gRPC IDL.

6.3 SPEC-DATA — Data Model

```
data-style: relational | document | graph | columnar | hybrid
storage-engine: postgresql | mysql | couchdb | mongodb | clickhouse |
...
schema-version: "1.4.0"
pii-classification:
  - { entity: User, fields: [email, phone], level: PII-high }
retention-policies:
```

```
  - { entity: Order, period: "7 years", basis: "tax law" }
migration-strategy: forward-only | reversible | dual-write
```

Industry reference: ISO/IEC 11179, JSON Schema.

6.4 SPEC-INT — Integration

```
integration-pattern: request-response | event-driven | message-queue
| webhook | file-transfer
direction: outbound | inbound | bidirectional
counterparty:
  system: "<external-system-name>"
  contract-owner: "<team-or-vendor>"
  contract-ref: "<external-spec-url>"
sla:
  availability: "99.5%"
  latency-p95: "500ms"
  fallback: "queue + retry; manual reconciliation after 24h"
idempotency: guaranteed | best-effort | none
```

Industry reference: Enterprise Integration Patterns (Hohpe), ISO/IEC 19944. **Заменяет** legacy `INT-SR`.

6.5 SPEC-PROC — Process / workflow

```
process-style: bpmn | state-machine | saga | choreography |
orchestration
state-count: integer
participants:
  - { role: customer, system: client-portal }
  - { role: agent, system: back-office }
sla:
  end-to-end: "2 business hours"
compensation: defined | not-applicable | manual
```

Industry reference: BPMN 2.0 (OMG), ISO/IEC 19510, Saga pattern.

6.6 SPEC-UI — UI / UX

```
ui-platform: web | mobile-ios | mobile-android | desktop | tv |
embedded
target-users:
  - { role: end-customer, persona: "ADAPT-NNN §X.Y" }
design-system: "<reference-or-internal>"
accessibility-level: WCAG-A | WCAG-AA | WCAG-AAA
i18n: required | not-required
mockup-links:
  - { tool: figma, url: "<link>", version: "v3" }
baseline-images:
  - "ai-concepts/baselines/SPEC-UI-NN-screen-01.png"
```

Industry reference: WCAG 2.2, Material Design / Apple HIG. **Заменяет** legacy `UIC`.

6.7 SPEC-AI — AI / ML

```
ai-pattern: rag | fine-tuning | prompt-engineering | tool-use |
multi-agent | embedding-only
production-model:
  vendor: anthropic | openai | google | local
  model: "<model-name>"
  version: "<exact-version>"
judge-model:                          # ОБЯЗАН отличаться от
production
  vendor: "<different-vendor>"
  model: "<different-model>"
context-strategy:
  embedding-model: "<model>"
  chunk-size: integer
  chunk-overlap: integer
  vector-store: pinecone | weaviate | pgvector | qdrant
eval-strategy:
  metric: accuracy | f1 | rouge | custom-rubric
  threshold: number
  baseline-dataset: "<path>"
cost-budget:
```

```
  tokens-per-request-target: integer
  tokens-per-request-ceiling: integer
  monthly-budget-usd: number
```

Industry reference: ISO/IEC 23894, NIST AI RMF. **Заменяет** legacy `AIC` . **Изоляция:** judge-model.vendor ≠ production-model.vendor (нормативно).

6.8 SPEC-SEC — Security

```
security-domains:
  - authentication
  - authorization
  - data-protection
  - audit
  - secrets-management
auth-model:
  authn: jwt-bearer | oauth2-pkce | mtls | passkey
  authz: rbac | abac | relbac
data-classification:
  - { class: PII-high, fields: [...] }
  - { class: PCI, fields: [...] }
threat-model-method: STRIDE | PASTA | OCTAVE
compliance:
  - ISO-27001
  - GDPR
  - Ф3-152
  - PCI-DSS-4
```

Industry reference: STRIDE (Microsoft), OWASP ASVS 4.0, ISO/IEC 27001, NIST 800-53.

6.9 SPEC-OPS — Operations

```
deployment-style: kubernetes | vm | serverless | docker-compose |
bare-metal
environments:
  - { name: dev, purpose: development, scale: minimal }
  - { name: staging, purpose: integration-testing, scale: half-prod }
```

```
  - { name: prod, purpose: production, scale: full }
slo:
  availability: "99.9%"
  error-budget-month: "43m"
  latency-p95: "300ms"
observability:
  logs: elastic | loki | cloudwatch
  metrics: prometheus | datadog | cloudwatch
  traces: jaeger | tempo | x-ray
disaster-recovery:
  rto: "<duration>"
  rpo: "<duration>"
```

Industry reference: Google SRE Book, ITIL v4, ISO/IEC 20000.

---

7. ADAPT schema

ADAPT — отдельный артефакт (см. `standard/07-adapt.md`).

```
# === Identity ===
id: ADAPT-NNN                          # immutable; sequential per
project
title: "Адаптация ТЗ <name>"
type: ADAPT

# === Source ===
source-tz:
  id: TZ-YYYY-NNN
  signed-date: "<ISO-date>"
  signed-by-client: "<name-role>"
  document-ref: "<нативная для носителя ссылка>"
parent-adapt:                          # для delta-ADAPT; null для
main
  id: ADAPT-NNN
  delta-tz: TZ-YYYY-NNN

# === Lifecycle ===
status: draft | review | approved | frozen | obsolete
                                       # подмножество §1:
```

```
verified/deprecated/review не применимы к ADAPT; см. §1 примечание
created: "<ISO-date>"
last-updated: "<ISO-date>"

approval:                              # required для approved
  client-signature:
    signed-by: "<name>"
    role: "<role>"
    organization: "<client-org>"
    signed-at: "<ISO-datetime>"
    signature-ref: "<нативная для носителя ссылка>"
  architect-signature:
    signed-by: "<name>"
    role: architect
    signed-at: "<ISO-datetime>"

# === Auto-derived ===
generates-requirements: []             # BR/SR выведенные из этого
ADAPT
generates-specs: []                    # SPEC-* выведенные
open-questions-count: integer          # должен быть 0 для approved
resolved-questions-count: integer

# === AI provenance ===
ai-provenance:
  generated-by: "<vendor>-<model>@<date>"
  prompt-template: "<template-path>@<version>"
  context-tokens: integer
  output-tokens: integer
  human-edits: boolean                 # true required для approved
```

Backward записи (внутри body):

```
# Каждая B-NNN запись:
id: B-NNN                              # immutable
category: contradiction | gap | hidden-assumption | feasibility |
regulatory | terminology | scope
status: open | asked-to-client | answered | resolved | frozen
tz-section: "§N.N"
description: "..."
```

```
asked-to-client: "<ISO-date>"
client-answer:
  signed-by: "<name>"
  signed-at: "<ISO-datetime>"
  channel: email | docusign | zoom-transcript | written-letter
  text: "..."
resolution: "..."                    # как ответ интегрирован в
Forward
```

---

8. TC — Test Case

```
# === Identity ===
id: "TC-NN[.N]"                      # immutable
title: "<descriptive>"
type: TC
slug: "<kebab-case>"

# === Classification ===
tc-type: acceptance | ux | system | contract | eval | security
negative: boolean                    # true для парного негативного
TC

# === Scope ===
level: system | subsystem | module
scope:
  system: "<system-id>"
  subsystem: "<subsystem-id>"        # null если level=system
  module: "<module-id>"              # null если level ≠ module

# === Lifecycle ===
status: draft | ready | passing | failing | obsolete

# === Verification mapping (≥1) ===
verifies:
  - id: "<requirement-id>"           # SR / BR / SPEC-*
    ref: "<нативная для носителя ссылка>"
    requirement-version: "<version-ref>"   # V5 pinning (см.
standard/03 §3.3.5)
```

```yaml
# === Pair link (mandatory если negative=false и существует парный)
===
paired-with: ["<TC-id>"]              # positive ↔ negative

# === Automation ===
automation:
  status: automated | manual-pending
  location: "<path-to-implementation>"      # mandatory если
automated
  runner: pytest | jest | go-test | playwright | vlm-judge | ragas |
pact | other
  manual-pending-until: "<ISO date>"        # mandatory если manual-
pending
  manual-pending-reason: "<why>"

# === Execution (mandatory если tc-type=ux | eval) ===
judge:                                # judge.vendor ≠ production
model vendor (§6.7, §9)
  vendor: "<provider>"
  model: "<model-id>"
  prompt-template: "<template-path>@<version>"
baseline:                             # mandatory для ux | eval
  artifact: "<нативный для носителя pointer>"
  perceptual-diff-threshold: float    # для ux
  metric-thresholds: {}               # для eval

# === Last run (auto-managed; bot-only) ===
last-run:
  date: "<ISO timestamp>"
  result: pass | fail | skipped | n/a
  runner-id: "<runner@version>"
  run-ref: "<нативная для носителя ссылка>"
  requirement-version: "<version-ref>"      # должна совпадать с
verifies[].requirement-version
  judge-report: "<for ux/eval>"

# === Замена / obsolescence ===
obsolete-pending: boolean             # true при detected delta-T3
инвалидации
replaces: "<old-id>"
replaced-by: "<new-id>"
```

```
obsoleted-date: "<ISO date>"

# === Inherited ===
ai-provenance: { ... }                # см. §1
```

---

9. Validation rules (cross-field)

Не выражаемые в чистой JSON Schema, требуют custom validator. Колонка «Формальная проверка» даёт исполнимый предикат или ссылку на готовый KG-запрос (reference/05 §5/§6), чтобы правило проверяла **машина**, а не только человек:

Правило	Описание	Формальная проверка (предикат / KG-запрос)
ID immutable	При изменении файла поле `id` не меняется.	`diff(prev.id, curr.id) == ∅` (change-diff носителя)
`parent` exists	Для SR — parent BR существует и в статусе ≥ `approved` .	`status(BR[SR.parent.id]) ≥ approved` ; orphans — 05 §6.1
`source.adapt` approved	Для BR/SR/SPEC — ADAPT в `source.adapt` в статусе `approved` / `frozen` .	`status(ADAPT[art.source.adapt]) ∈ {approved, frozen}`
`verified-by` consistency	TC в `verified-by` имеют `verifies[].id` = этот артефакт.	`∀ tc ∈ art.verified-by: art.id ∈ tc.verifies[].id`
`requirement-version` lock	`TC.last-run.requirement-version` = `verifies[].requirement-version` .	`tc.last-run.requirement-version == tc.verifies[].requirement-version` ; stale — 05 §5.4
`source.adapt` обязателен для BR/SR/SPEC	Canonical источник ADAPT; для TR — через parent SR (standard/06 §6.6.2).	`art.type ∈ {BR, SR, SPEC-*} ⇒ art.source.adapt ≠ null`
SPEC-AI requires ai-act	Для AI-артефакта `ai-act.risk-class` обязательно.	`art.type == SPEC-AI ⇒ art.ai-act.risk-class ≠ null`

Data residency consistency	RU в `SR.data-classification.data-residency` ⇒ то же в parent BR.	`'RU' ∈ SR.…data-residency ⇒ 'RU' ∈ BR[SR.parent].…data-residency`
Compliance hierarchy	SR.compliance ⊆ parent BR.compliance (или явный justification).	`SR.compliance ⊆ BR[parent].compliance ∨ exists(extension-justification)`
TC `automated` requires location	`automation.status: automated ⇒ automation.location` непустое.	`tc.automation.status == 'automated' ⇒ tc.automation.location ≠ ''`
Negative TC mandatory	На каждое нормативное утверждение — TC с `negative: true`.	`∀ assertion ∈ art: ∃ tc(negative: true)` (standard/09 §9.7)
ADAPT open-questions == 0 for approved	Approval блокируется при `open` / `asked-to-client` backward.	`adapt.status == approved ⇒ count(backward[status ∈ {open, asked-to-client}]) == 0` (05 §5.7)
Judge isolation (SPEC-AI)	`judge.vendor ≠ production-model.vendor`.	`tc.judge.vendor ≠ SPEC-AI[tc.verifies].production-model.vendor`
SPEC depends-on acyclic	Граф `depends-on` между SPEC — DAG.	cypher cycle-detection 05 §5.6: rows ≠ ∅ ⇒ нарушение

---

10. Изоморфизм носителя

Те же артефакты на разных носителях. Отображение для git (YAML frontmatter) ↔ document-oriented store (JSON document):

Поле (canonical)	git (YAML frontmatter)	Document store (JSON doc)
`id`	`id`	`_id = <project>:<doc-type>:<slug>`, поле `slug`
`type: BR`	`type: BR`	`level: "business"` (native enum — project mapping)
`type: SPEC-API`	`type: SPEC-API`	`doc_type: "spec_api"`

`parent.id`	`parent.id`	`parent`
`children`	(auto-derived)	`children` (auto-derived)
`status`	`status`	`status`
`priority`	`priority`	`priority`
`source.adapt`	`source.adapt`	`created_from_adapt`
`constrained-by[]`	`constrained-by`	`constrained_by` (subdoc array)
`verified-by[]`	(auto-derived)	`linked_tests`
`ai-provenance.*`	nested object	nested subdocument
`compliance`	`compliance` array	`compliance` subdoc
`data-classification`	nested object	nested subdoc
`business-outcome`	nested object	nested subdoc
`replaces` / `replaced-by`	string ID	`replaces` / `replaced_by`

Migration между VCS и document store использует эту таблицу. Нативные для носителя имена полей могут отличаться, но **семантика и invariants сохраняются** через capabilities V1-V6 (см. 01-glossary.md §14.7).

---

11. Schema versioning

Каждый артефакт имеет поле `schema-version` (semver). При несовпадении версии файла и текущей schema validator предлагает migration.

Изменение	Bump
Новое необязательное поле	minor (1.0 → 1.1)
Новое обязательное поле	major (1.0 → 2.0) + migration script
Удаление поля	major + migration script
Изменение enum	minor если добавление, major если удаление
Переименование поля	major + migration script

Текущая версия schemas: 1.0-draft (Phase 7 наполнение).

---

12. JSON Schema fragment example (BR)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://renar.tech/schemas/br.json",
  "title": "Business Requirement",
  "type": "object",
  "required": ["id", "title", "type", "status", "priority", "source",
"ai-provenance"],
  "properties": {
    "id": {
      "type": "string",
      "pattern": "^BR-[0-9]{2}(\\.[0-9]+)?$"
    },
    "title": { "type": "string", "minLength": 5, "maxLength": 100 },
    "type": { "const": "BR" },
    "status": {
      "enum": ["draft", "review", "approved", "verified",
"deprecated", "obsolete"]
    },
    "priority": { "enum": ["must", "should", "could"] },
    "source": {
      "type": "object",
      "required": ["adapt"],
      "properties": {
        "adapt": { "type": "string", "pattern": "^ADAPT-[0-9]{3}(-
delta-[0-9]+)?$" },
        "adapt-section": { "type": "string" },
        "tz-section": { "type": "string" }
      }
    },
    "ai-provenance": {
      "type": "object",
      "required": ["generated-by", "generated-at"],
      "properties": {
        "generated-by": {
          "type": "string",
```

```
          "pattern": "^[a-z]+-[a-z0-9-]+@[0-9]{4}-[0-9]{2}-[0-9]{2}$"
        },
        "human-edits": { "type": "boolean" }
      }
    }
  }
}
```

Аналогичные JSON Schemas для SR/TR/SPEC-*/TC/ADAPT — в `reference/schemas/` (TODO Phase 8).

Schemas reference RENAR 1.0-draft — см. также 01-glossary.md, `standard/06` - `09` для нормативных определений.

---

title: "Реестр AI-рисков" description: "Реестр AI-рисков для RENAR-проектов: 14 канонических рисков с mitigations." order: 3 lang: ru version: "1.0-draft"

AI Risk Register для RENAR-проектов

> ***Назначение:*** *реестр AI-специфичных рисков для проектов, использующих RENAR (где AI генерирует требования, спецификации и тесты). Основан на ISO/IEC 23894:2023 (AI Risk Management) и NIST AI RMF 1.0. Нормативные mitigation hooks — standard/09 §9.4, standard/07 §7.10.*

Не подменяет общий security risk register организации. AI-риски — отдельный класс из-за специфики генерации и непредсказуемости моделей.

---

1. Структура реестра

Каждый риск имеет:

```
id: AIR-NN                              # immutable
name: "<short name>"
category: hallucination | injection | drift | bias | sgnl-failure |
data-quality | adversarial | privacy
# значение category — ровно один член enum; допустим необязательный
уточнитель в скобках
# (напр. "sgnl-failure (process)"), при этом enum-значением считается
часть до скобки
severity: critical | high | medium | low
likelihood: high | medium | low
iso-23894-ref: "§N.N"
nist-rmf-function: govern | map | measure | manage
mitigations:
  - mechanism: "<description>"
    enforced-by: "<who/what>"
    automated: true | false
status: active | mitigated | accepted | monitoring | out-of-scope
owner: "@role"
```

```
last-reviewed: "YYYY-MM-DD"
related: ["<core rule N>", "<standard chapter>", "<other AIR-NN>"]
```

Список AIR-01..AIR-14 закрыт; новые риски добавляются только через изменение полного RENAR Standard.

1.1 Соответствие `category` ↔ NIST AI RMF trustworthiness characteristics

`category` риск-реестра отображается на характеристики доверенности NIST AI RMF 1.0 — для проверяемости заявления «основан на NIST AI RMF»:

category	NIST AI RMF trustworthiness characteristic
`hallucination`	Valid & Reliable
`injection`	Secure & Resilient
`drift`	Valid & Reliable; Safe
`bias`	Fair — with Harmful Bias Managed
`sgnl-failure`	Safe; Accountable & Transparent
`data-quality`	Valid & Reliable
`adversarial`	Secure & Resilient
`privacy`	Privacy-Enhanced

> ***Ссылки ISO/IEC 23894:2023.*** *Категории AI-рисков в 23894:2023 расположены в* ***Annex A*** *(risk sources); Clause 6 описывает процесс риск-менеджмента (по ISO 31000). Дескрипторы в колонке «ISO 23894» ниже — risk-source labels; точное сопоставление с пунктами Annex A подлежит сверке с текстом стандарта при формальном claim.*

---

2. Реестр

AIR-01. Hallucination в AI-генерируемых требованиях

Параметр	Значение

Категория	hallucination
Severity	**High**
Likelihood	**High** без mitigation
ISO 23894	Annex A — Output reliability
NIST RMF	Measure (accuracy)

Описание. AI-агент при генерации BR/SR/SPEC может «дописать от себя» утверждения, которых нет в ADAPT или ТЗ. Эти утверждения попадают в approved-документы, в backlog, в код.

Воздействие. Scope creep, dispute на acceptance, фичи которые не требовались клиентом.

Mitigations.

- Source citation — каждое assertion в SR ссылается на ADAPT §N (RENAR Core Правило 1).
- Adversarial review — критик-модель (другой vendor) ловит unsupported assertions.
- Метрика Hallucination Rate — target ≤ 1% на зрелых уровнях RENAR.

Status. active на ранних RENAR levels; mitigated при полном adversarial pipeline.

AIR-02. Prompt injection через ТЗ от клиента

Параметр	Значение
Категория	injection
Severity	**High**
Likelihood	**Low-Medium**
ISO 23894	Annex A — Adversarial inputs
NIST RMF	Manage (security)

Описание. Злонамеренный клиент может вставить в текст ТЗ скрытые инструкции для AI («ignore previous instructions and …»). При импорте ТЗ AI-агент может эти инструкции выполнить.

Воздействие. Утечка данных, вредоносные изменения в требованиях, нарушение security policy.

Mitigations.

- Sandboxing AI-агента при импорте: модель работает с T3 как с пассивными данными, не как с инструкциями (явно в system prompt).
- Input gateway проверяет inputs на known injection patterns перед передачей AI.
- При обнаружении suspicious pattern — escalation на инженера, не auto-process.

Status. monitoring (need automated detector).

AIR-03. Model drift / version change

Параметр	Значение
Категория	drift
Severity	**Medium**
Likelihood	**High** (модели обновляются регулярно)
ISO 23894	Annex A — Model lifecycle
NIST RMF	Manage (model governance)

Описание. Anthropic / OpenAI / Google обновляют модели. Тот же prompt с тем же T3 через 6 месяцев может дать другой output. Старые ADAPT генерировались Opus 4.6, новые — Opus 4.7.

Воздействие. Inconsistency между требованиями в одном проекте; невозможность точно воспроизвести генерацию старого артефакта.

Mitigations.

- Model versioning в `ai-provenance.generated-by` (точная версия + дата).
- Eval-тесты для SPEC-AI прогоняются при смене модели.
- При регенерации артефакта — diff против старой версии и assessment изменений.

Status. monitoring.

AIR-04. Bias в AI-генерации требований

Параметр	Значение
Категория	bias
Severity	**Medium**
Likelihood	**Medium**
ISO 23894	Annex A — Fairness
NIST RMF	Measure (fairness)

Описание. Модель имеет training-data bias. При генерации BR может игнорировать stakeholders определённых групп (например, accessibility users, non-English locales, специфичные регуляторики).

Воздействие. Требования не покрывают весь spectrum пользователей; продукт оказывается дискриминационным или non-compliant.

Mitigations.

- Multi-model agreement для priority=must BR (разные модели — разные biases).
- Stakeholder map обязателен в BR — explicit перечисление.
- Adversarial critic с prompt «check for missing stakeholders / accessibility considerations».

Status. active.

AIR-05. Single-model failure (no diversity)

Параметр	Значение
Категория	sgnl-failure
Severity	**Medium**
Likelihood	**High** без mitigation
ISO 23894	Annex A — Single point of failure
NIST RMF	Manage (resilience)

Описание. Если все артефакты генерируются одной моделью, её ошибки систематически проникают. Если она «галлюцинирует» определённый паттерн — все требования это унаследуют.

Воздействие. Систематическое искажение требований по проекту.

Mitigations.

- Multi-model для priority=must BR.
- Adversarial critic с другой моделью.
- Изоляция judge-модели от production-модели в eval-тестах (SPEC-AI: `judge-model.vendor ≠ production-model.vendor`).

Status. mitigated при полном RENAR pipeline.

AIR-06. Test fitting / зеленение тестов

Параметр	Значение
Категория	sgnl-failure
Severity	**High**
Likelihood	**Medium** без mitigation
ISO 23894	Annex A — Verification integrity
NIST RMF	Measure (validity)

Описание. AI-агент имеет тривиальный путь зеленения failing-теста — ослабить Pass-критерий. Это проходит code review, поскольку «тест зелёный».

Воздействие. False confidence; defects проходят в production.

Mitigations.

- Маркер `[test-spec-change]` обязателен для изменения Pass/Fail; требует отдельного approval.
- Spot-check 5 случайных passing TC раз в спринт (RENAR Core Правило 5).
- Метрика Test-fitting drift rate (отдельная от обычных metrics).

Status. mitigated при дисциплине spot-check.

AIR-07. Hallucinated citations

Параметр	Значение
Категория	hallucination
Severity	**Medium-High**

Likelihood	**Medium**
ISO 23894	Annex A — Output reliability
NIST RMF	Measure (accuracy)

Описание. AI-агент пишет citation `[TZ-2026-001 §4 line 142]`, но в реальном ТЗ §4 line 142 — про другое. Citation выглядит как evidence, но evidence ложный.

Воздействие. Source citation становится фикцией; trace chain рвётся при аудите.

Mitigations.

- Citation validator hook — парсит citation, открывает указанный документ, проверяет наличие соответствующего текста.
- Pre-commit/pre-approval блокировка при невалидной citation.

Status. monitoring (need validator implementation).

AIR-08. Adversarial inputs в clients data (runtime)

Параметр	Значение
Категория	adversarial
Severity	**High**
Likelihood	**Low**
ISO 23894	Annex A — Adversarial inputs
NIST RMF	Manage (security)

Описание. Клиент отправляет данные (через формы, API), специально сконструированные для манипуляции AI-компонент в runtime (не на этапе генерации требований).

Воздействие. Аналогично AIR-02, но в production runtime.

Mitigations.

- Input sanitization на API gateway.
- Constrained generation (structured outputs only).
- Rate limiting per user.

Status. out-of-scope for RENAR. Это application-level security; RENAR требует SR-уровень покрытия (SPEC-SEC threat model), но runtime защита — задача реализации, не нормирования требований.

AIR-09. Privacy leakage через AI logs

Параметр	Значение
Категория	privacy
Severity	**High**
Likelihood	**Medium**
ISO 23894	Annex A — Privacy
NIST RMF	Govern (privacy)

Описание. AI-агент при генерации артефакта имеет в контексте PII (из ТЗ или интервью). Логи генерации (tool events, audit records) могут хранить эти PII.

Воздействие. PII попадают в логи, в `ai-provenance`, в training data (если используется).

Mitigations.

- PII redaction в промптах перед отправкой в LLM (если возможно).
- `data-classification` tracking — куда какие данные физически идут.
- Disable training on conversations (Anthropic / OpenAI privacy settings, DPA).
- TTL на event logs с PII.

Status. active (need redaction tooling).

AIR-10. Knowledge graph poisoning

Параметр	Значение
Категория	data-quality
Severity	**Medium**
Likelihood	**Low**
ISO 23894	Annex A — Data integrity

NIST RMF	Map (knowledge integrity)

Описание. Если knowledge graph (KG) используется как primary search для AI-агентов, то incorrect edge может «отравить» все последующие AI-запросы, опирающиеся на этот граф.

Воздействие. AI генерирует требования на основе wrong context, систематически.

Mitigations.

- KG derived от frontmatter, не редактируется напрямую (см. 05-knowledge-graph-schema.md).
- CI-валидация графа на каждое изменение (нет circular dependencies, no orphan approved).
- Reconciliation-агент проверяет integrity графа еженедельно.

Status. monitoring.

AIR-11. Reconciliation false-positive overload

Параметр	Значение
Категория	sgnl-failure (process)
Severity	**Low-Medium**
Likelihood	**Medium**
ISO 23894	Annex A — Verification integrity
NIST RMF	Manage (operational)

Описание. Reconciliation-агент при слишком чувствительных правилах генерирует много false-positive findings. Архитектор начинает их игнорировать → real findings тонут.

Воздействие. Reconciliation теряет ценность, дисциплина не масштабируется.

Mitigations.

- Tunable thresholds в проектной конфигурации.
- Метрика Reconciliation Findings/Week — если растёт без real issues, нужна re-calibration.
- Архитектор может dismiss findings с обоснованием — это feedback для tuning prompt агента.

Status. monitoring.

AIR-12. Cost runaway (uncontrolled AI spend)

Параметр	Значение
Категория	sgnl-failure (operational)
Severity	**Medium**
Likelihood	**Medium** без budget tracking
ISO 23894	Annex A — Cost governance
NIST RMF	Manage (resource governance)

Описание. Без budget tracking AI-генерация (особенно с multi-model, adversarial, eval) может потреблять токены непропорционально размеру проекта.

Воздействие. Финансовые потери; нерентабельность практики.

Mitigations.

- `ai-budget` field в frontmatter артефакта (target + actual).
- Aggregated cost metric на уровне проекта.
- Cap на проект; alarm при approached.
- Recommendation engine: «использовать Sonnet/Haiku для рутины, Opus только для priority=must BR».

Status. active.

AIR-13. Стейкхолдер не понимает AI-сгенерированные требования

Параметр	Значение
Категория	data-quality (UX)
Severity	**Medium**
Likelihood	**Medium**
ISO 23894	Annex A — Transparency
NIST RMF	Govern (transparency)

Описание. AI генерирует SR в техническом стиле; клиент / non-technical stakeholder при рецензировании не понимает → approval становится формальностью.

Воздействие. QG-ADAPT-approve / QG-4 acceptance теряет смысл; dispute rate at acceptance растёт.

Mitigations.

- Style guide (04-ai-style-guide.md).
- BR пишется в business language; технологии — в SPEC-*, не в BR.
- Human-readable summary в каждом BR/SR — короткая секция, понятная без технического background.

Status. active.

AIR-14. Vendor lock-in to specific LLM provider

Параметр	Значение
Категория	sgnl-failure (operational)
Severity	**Medium**
Likelihood	**Low-Medium**
ISO 23894	Annex A — Vendor risk
NIST RMF	Govern (procurement)

Описание. Все промпты оптимизированы под конкретного провайдера (например, Anthropic Claude). Если provider меняет pricing / availability — переход требует переписывания всех промптов.

Воздействие. Operational risk, costs, business continuity.

Mitigations.

- Provider-agnostic prompts где возможно (избегать vendor-specific tool syntax).
- Multi-model уже enforced для priority=must (Принцип 4) — гарантирует что есть второй провайдер в pipeline.
- Periodic test runs на резервном провайдере.

Status. monitoring.

---

3. Risk matrix

```
Severity / Likelihood
         │  Low      Medium         High
─────────┼──────────────────────────────────
High     │  AIR-08   AIR-04, 06     AIR-01, 03
Medium   │  AIR-10   AIR-11, 13     AIR-07, 09, 14
Low      │  —        AIR-12         AIR-02, 05
```

Critical и High риски в top-right квадранте — приоритет mitigation.

---

4. Mitigation matrix

Какие mitigations покрывают какие риски:

Mitigation	Покрывает риски
Source citation (Core Правило 1)	AIR-01, AIR-07
Adversarial review (другая модель)	AIR-01, AIR-04, AIR-05
Spot-check passing TC (Core Правило 5)	AIR-06
Multi-model для priority=must	AIR-04, AIR-05, AIR-14
Judge isolation в SPEC-AI	AIR-05
AI provenance (model + version + date)	AIR-03, AIR-09
Citation validator hook	AIR-07
Input sandbox / sanitization	AIR-02, AIR-08
PII redaction + DPA	AIR-09
KG validation in CI	AIR-10
Reconciliation tunable thresholds	AIR-11
`ai-budget` field + project cap	AIR-12
Style guide + business-language BR	AIR-13

Compensating controls: одиночный mitigation редко достаточен; высокие риски требуют ≥ 2 независимых controls.

---

5. Operational governance

5.1 Review cadence

- **Monthly:** AIR-01, AIR-02, AIR-06, AIR-07, AIR-09 — high-impact runtime risks.
- **Quarterly:** все остальные.
- **On-incident:** любой риск, в который произошёл инцидент → root cause → mitigation.

5.2 Owner

Default owner — **Architect** проекта. Для AI-специфичных рисков — **AI Governance Lead** (если есть в организации).

5.3 Storage

Risk register проекта — отдельный артефакт по convention:

```
<project>.req/
  governance/
    ai-risk-register.md           # snapshot этого reference +
project-specific notes
    review-log.md                 # история ревью с датами и
подписями
```

На носителе, не поддерживающем директории — эквивалент namespacing.

5.4 Reconciliation-агент

Еженедельный run reconciliation-агента обновляет `status` и `last-reviewed` поля каждого AIR. Если статус меняется (e.g., monitoring → active) — alert архитектору.

---

6. Связь со стандартом

AIR	RENAR Core / Standard
AIR-01, 07	Core Правило 1 (ADAPT перед SR) + Standard ch.5

AIR-04, 13	Standard ch.4 (роли) + style guide §04
AIR-05	Standard ch.13 (AI generation) + SPEC-AI judge isolation
AIR-06	Core Правило 4 + Правило 5 + QG-2 Verification Gate
AIR-09	Standard ch.11 compliance + SPEC-SEC
AIR-12	Standard ch.13 (cost governance)

---

7. Что НЕ покрывает risk register

Этот реестр focuses на AI-специфичные риски процесса RENAR. **Не подменяет:**

- Общий security risk register организации (см. ISO 27001).
- Compliance risk register (см. 06-compliance.md).
- Application-level threat model конкретного проекта (см. SPEC-SEC).

Reg-обязательные требования регуляторов (AI Act high-risk, ФЗ-152) — отдельные artifacts; этот register — operational tier.

AI Risk Register RENAR 1.0-draft — renar.tech

---

title: "Руководство по AI-стилю" description: "Style, тон, структура, длина для AI-агентов, генерирующих RENAR-артефакты." order: 4 lang: ru version: "1.0-draft"

Руководство по AI-стилю — генерация RENAR-артефактов

> ***Назначение:*** *style, тон, структура, длина и лексикон для AI-агентов, генерирующих RENAR-артефакты (ADAPT, BR, SR, SPEC, TC, Impact Analysis). Снимает класс рисков «разные модели → разный стиль → drift восприятия» (см. AIR-04, AIR-13 в 03-ai-risk-register.md). Дополняет reference/06 RU Style Guide для human editors.*

Аналог в индустрии: Google Developer Documentation Style Guide, Microsoft Style Guide for technical writing, Diátaxis. Цель — единый ровный tone независимо от модели или промпт-инженера.

---

1. Принципы

1.1 Стиль = договорная точность

RENAR-артефакт — **договорной документ** между клиентом, командой и системой. Никаких метафор, никакого «красочного» языка, никаких ассоциаций. Только однозначные утверждения.

Хорошо:

> *Система должна принимать заявку на регистрацию через POST /auth/register.*

Плохо:

> *Пользователи смогут легко и удобно создавать аккаунты через современный API.*

1.2 Один артефакт = одна мысль

Каждый BR / SR / SPEC / TC покрывает **одну** концепцию. Если AI-агент склонен «упаковать» 3 темы в один SR — он должен декомпозировать, не упаковывать.

Сигнал упаковки: союз «и» в title, multiple actions в одном утверждении.

Плохо:

> *SR-05: Регистрация, авторизация и восстановление пароля*

Хорошо:

> *SR-05: Регистрация клиента SR-06: Авторизация клиента SR-07: Восстановление пароля*

1.3 Нет претензий на полноту

AI не должен «дорабатывать» требование от себя, если в источнике (ADAPT или T3) этого нет. Каждое утверждение либо имеет citation, либо помечено `derived` с явным обоснованием.

Плохо:

```
- При регистрации проверяется уникальность email.
- При регистрации проверяется уникальность телефона.   ← в ADAPT нет, AI домыслил
```

Хорошо:

```
- При регистрации проверяется уникальность email. [ADAPT-001 §14.1 Forward]
```

Если AI считает, что нужна проверка телефона — он не пишет её в SR, а добавляет в Critic-output: «возможно нужна проверка уникальности телефона — нет в ADAPT, требует backward finding к stakeholder».

1.4 Однозначность важнее красоты

Если выбор между точным длинным предложением и компактным неоднозначным — AI выбирает точное длинное. Длина — не enemy, ambiguity — enemy.

---

2. Структура и длина по типу артефакта

2.1 ADAPT

Body: 200-800 строк (зависит от объёма T3).

Обязательные секции:

- `## Краткое содержание` — 3-5 параграфов для одностраничного чтения клиентом.
- `## Term mapping` — таблица «клиент → инженер».
- `## Forward: интерпретация по разделам ТЗ` — раздел на каждый § ТЗ.
- `## Backward: обнаруженные проблемы` — записи B-NNN с lifecycle.
- `## Резюме backward findings` — таблица по категориям.
- `## Generated artifacts` (auto после approval).

Тон: двусторонний — Forward в инженерных терминах с явной интерпретацией, Backward в клиент-friendly формулировках.

2.2 BR — Business Requirement

Body: 30-80 строк.

Обязательные секции:

- `## Потребность` — одно предложение по шаблону `[Роль] должен [действие], чтобы [бизнес-цель].`
- `## Критерии успеха` — 3-7 измеримых пунктов.
- `## Контекст` — 5-15 строк.

Запрещено:

- Технологии («через REST API», «Postgres», «React»).
- Конкретные экраны или поля БД.
- Размышления автора («мы думаем», «возможно», «было бы хорошо»).

Тон: формальный, императивный («должен», не «может»).

2.3 SR — System Requirement

Body: 40-150 строк.

Обязательные секции:

- `## Описание` — одно предложение по шаблону `Система должна [поведение]. [Условие].`
- `## Поведение` — 5-20 пунктов с inline citations к ADAPT.

- `## Ограничения` — 3-10 пунктов.

Опциональные секции (когда применимо):

- `## Не входит в это требование` — explicit out-of-scope.
- `## Связанные SR` — cross-links.

Запрещено:

- Названия таблиц БД, конкретных функций, классов.
- Frameworks («через FastAPI», «React-компонент»).
- Конкретные UI-paths («кнопка в правом углу») — это в SPEC-UI.

Тон: формальный, точный, поведенческий («система отвечает 422», не «возвращается ошибка»).

2.4 SPEC-* (9 типов)

Body: 80-400 строк (varies по типу).

Общие обязательные секции (см. 02-schemas.md §2):

- `## Назначение`
- `## Scope` (входит / не входит)
- `## <Type-specific sections>`
- `## Связь с требованиями`
- `## Связь с другими SPEC`
- `## Verification`
- `## Open questions`

Тон по типам:

- **SPEC-ARCH / API / DATA / INT / SEC / OPS:** технико-аналитический. Допустимы конкретные технологии.
- **SPEC-UI:** пользователь-центричный нарратив с поведенческой точностью. «Менеджер видит ленту заказов, нажимает на заказ, открывается детальный экран», не «UI должен иметь listing component».
- **SPEC-AI:** model card style — capabilities, limits, failure modes, eval criteria.
- **SPEC-PROC:** procedural narrative с BPMN / state machine references.

2.5 TC — Test Case

Body: 30-80 строк.

Обязательные секции:

- `## Контекст` — на какой пункт верифицируемого артефакта ссылается TC.
- `## Предусловия` (Given) — 2-5 строк.
- `## Шаги` (When / действие) — 1-3 строки.
- `## Pass-критерий` (Then / Pass) — бинарный, наблюдаемый, воспроизводимый критерий.
- `## Fail-критерий` — список наблюдаемых признаков нарушения, **включая** возможные побочные эффекты (security leaks, missing audit logs).
- `## Постусловия` — ожидаемое состояние после прогона.
- `## Out of scope` — что намеренно не проверяется, с pointer на покрывающие TC.

> *Имена секций критериев (`## Pass-критерий` / `## Fail-критерий`) — canonical machine-detectable заголовки (standard/09 §9.4), детектируются enforcement-хуком (standard/10 §10.11.3); не переименовывать.*

Тон: исполнительный, чёткий («выполняется X», «возвращается Y»).

2.6 Impact Analysis

Body: 50-200 строк (зависит от размера дельты).

Обязательные секции:

- `## Затронутые требования` — таблица с типом изменения.
- `## Затронутые SPEC` — таблица с действием.
- `## Затронутые TC` — таблица с действием.
- `## Затронутые задачи в backlog`.
- `## Open questions` — что нужно уточнить со stakeholder через backward в delta-ADAPT.

Тон: аналитический, без оценок. Не «это плохое изменение», а «это изменение затрагивает X требований и Y задач».

---

3. Лексикон

3.1 Use (canonical)

- «Система должна …» (модальность).
- «должен» / «не должен» (RFC 2119 vocabulary — MUST / MUST NOT).
- «должна» / «не должна» (для системы).
- «возвращает» (HTTP/API behavior).
- «предусловия» / «постусловия» (формальные термины тестирования).
- «требование» / «утверждение» / «assertion».
- «критерий приёмки» (acceptance criterion).
- «backward finding» / «forward интерпретация» (RENAR canonical из ADAPT).

3.2 Запрещено

Слово/фраза	Заменить на	Почему
«Хочется чтобы»	«Система должна»	Модальность должна быть строгой.
«Удобно»	конкретный измеримый критерий	Subjective.
«Современный»	конкретные tech requirements в SR/SPEC	Marketing language.
«Качественный»	измеримые quality characteristics из ISO/IEC 25010	Subjective.
«Быстрый»	p95/p99 latency с числом	Vague.
«Надёжный»	uptime / reliability с числом	Vague.
«Возможно» / «Может быть»	(никогда в требованиях)	Hedging.
«Скорее всего»	(никогда)	Same.
«Желательно»	`priority: should` / `could` во frontmatter	Mixing channels.
«Понятный»	criteria for clarity (Flesch reading ease, etc.)	Subjective.
«Гладкий experience»	UX criteria в SPEC-UI	Marketing.
«Бесшовный»	конкретный технический критерий	Marketing.
«Поддерживать» (без объяснения как)	конкретный SR-уровень criteria	Vague.

«Интегрироваться с X» (без contract)	SPEC-INT с counterparty	Vague.

3.3 Use Cases vs Stories vs Requirements

RENAR использует **только canonical терминологию** из 01-glossary.md. AI-агент **не имеет права** называть требование «User Story», «Use Case», «Scenario», «Capability» — это запрещённые синонимы. Полный список запрещённых терминов и подстановок — в 01-glossary.md §5.

---

4. Шаблоны утверждений

4.1 BR-утверждение

Шаблон: `[Роль]` должен `[действие]`, чтобы `[бизнес-цель]`.

Пример:

> *Пользователь должен автоматически получать и хранить уведомления выбранных приложений в фоновом режиме, чтобы не пропустить важные сообщения.*

4.2 SR-утверждение

Шаблон: `<actor>` `<action>` `<condition>`. `<consequence>`.

Пример:

> *Система при первом запуске проверяет наличие разрешения NotificationListenerService. Если разрешение не выдано — отображается экран онбординга с кнопкой «Выдать доступ».*

4.3 TC Pass-критерий

Шаблон: `<actor>` `<action>` `<input>`. `<observable>` `<measurement>`.

Пример:

> *POST /auth/login с body `{"email":"x@y.z", "password":"correct"}`.*
> *Response status = 200, body содержит JWT с `exp = now + 24h ± 1m`.*

4.4 TC Fail-критерий

Список наблюдаемых признаков нарушения, **включая** возможные побочные эффекты:

```
- Response status ≠ 200 (ошибка авторизации).
- В 401-ответе указано конкретное поле, которое неверно (утечка информации).
- В системный лог записывается plaintext password (security violation).
- Email с уведомлением о входе НЕ отправляется при успешной авторизации (missing side effect).
```

Не допускается: «Pass не выполняется». Это не Fail-критерий, это negation.

---

5. Citation conventions

5.1 Inline citation

Каждое утверждение в BR/SR/SPEC имеет inline-citation в квадратных скобках:

> *При первом запуске отображается экран онбординга. [ADAPT-001 §14.1 Forward]*

Формат: `[<id> <section>]` или `[<id> <section> line <N>]` для точной строки.

5.2 Derived маркер

Если утверждение не цитата из ADAPT, а логически выведено:

> *Кнопка `Назад` блокируется на экране онбординга. [ADAPT-001 §14.1 Forward, derived: ADAPT требует «нельзя пропустить онбординг» → блокировка системного `back`]*

Объяснение `derived` обязательно.

5.3 Multi-source

Если утверждение поддерживается несколькими источниками:

> *Пароль хранится в зашифрованном виде. [ADAPT-001 §4 НФТ-002, ISO 27001 A.10.1.1]*

5.4 Что НЕ citation

- Ссылка на сам код («см. `auth/login.py`») — не источник требования.
- Ссылка на тикет в багтрекере — не источник требования (см. 01-glossary.md §1 Цепочка авторитетности).
- Ссылка на чат / устный разговор без записи в ADAPT backward → asked-to-client → answered.

---

6. System prompt для AI-генератора

Каждый AI-агент, генерирующий RENAR-артефакт, получает system prompt из:

1. **Role:** «Ты архитектор требований по стандарту RENAR».
2. **Style guide reference:** ссылка на этот документ.
3. **Glossary reference:** ссылка на 01-glossary.md.
4. **Constraints:**
   - Каждое утверждение либо с citation, либо `derived` с объяснением.
   - Использовать только canonical термины.
   - Структура и длина — согласно §14 этого документа.
   - Никаких запрещённых слов из §4.2.
5. **Output schema:** точный YAML frontmatter format (см. 02-schemas.md).
6. **Examples:** 2-3 примера good/bad для каждого типа артефакта.

Prompt-templates хранятся в `prompts/` директории организации с версионированием:

- `prompts/adapt-from-tz.md@v2.1`
- `prompts/decompose-adapt.md@v2.1`
- `prompts/generate-tc-pos-neg.md@v1.0`
- `prompts/critic-review-sr.md@v1.2`

`ai-provenance.prompt-template` во frontmatter артефакта содержит точную версию.

---

7. Стиль для разных моделей

Разные LLM имеют разные tendencies. Style guide учитывает это через prompt constraints:

Модель	Tendency	Mitigation
Claude Opus	Verbose, склонен к hedging («может быть», «возможно»)	Explicit constraint в prompt: «без hedging modal verbs».
Claude Sonnet	Конкретный, может пропустить edge case	Adversarial critic catches gaps.
GPT-4 / o-series	Marketing language tendency	Explicit blacklist слов из §4.2.
Mini / Haiku	Hallucinated citations	Citation validator hook (см. AIR-07).
Gemini Pro	Иногда смешивает RU/EN	Lang constraint в prompt + post-validation.

Style guide не запрещает использование любой модели, но требует enforcement через post-generation validation.

---

8. Validation pipeline

После генерации RENAR-артефакта запускается автоматическая валидация:

```
AI generates artifact
    ↓
Style validator (отдельный AI с другим prompt или rule-based)
    ├── citation check (each assertion has [...] или derived marker)
    ├── lexicon check (no forbidden words from §4.2)
    ├── structure check (required sections present)
    ├── length check (within bounds from §14)
    ├── canonical terms check (no forbidden synonyms)
    ├── modal verb check (нет hedging)
    ↓
On fail: regenerate с refined prompt или human review.
On pass: enter draft → ready transition (далее QG-1 / adversarial / approval).
```

Validator hooks — нативные для носителя. На git — pre-commit; на document store — pre-save document validator; на любом другом — equivalent gate.

---

9. Stylistic decisions для RU/EN

9.1 Body language

Body артефакта пишется на языке проекта:

- Российский клиент → RU.
- Международный → EN.
- Двуязычный → primary lang в `lang:` поле frontmatter; second lang — отдельный artifact с `replaces` / `replaced-by` links или translations подпапка.

9.2 frontmatter всегда канонический

Frontmatter поля — всегда canonical (английская латиница):

- `type: BR` (не `тип: БТ`).
- `status: approved` (не `статус: утверждено`).
- `priority: must` (не `приоритет: обязательно`).

UI отображает русские переводы (см. 01-glossary.md §4.5). Frontmatter — машинный, не для UI.

9.3 Mixed-lang запрещён

В пределах одного утверждения mixing RU+EN недопустим:

Плохо:

> *Система должна возвращать `JWT token` после successful login.*

Хорошо (RU):

> *Система должна возвращать JWT-токен после успешной аутентификации.*

Хорошо (EN):

> *The system must return a JWT token after successful authentication.*

Технические термины (`JWT` , `OAuth` , `gRPC` , `RBAC`) — допустимы в любом языке без перевода.

---

10. Перекрёстные ссылки

- Закрытый список canonical терминов и запрещённых синонимов — 01-glossary.md.
- Формальные frontmatter schemas — 02-schemas.md.
- AI-риски и mitigations — 03-ai-risk-register.md.
- Knowledge graph schema (используется validator для cross-reference checks) — 05-knowledge-graph-schema.md.

AI Style Guide RENAR 1.0-draft — renar.tech

---

title: "Схема графа знаний" description: "Канонические узлы и грани knowledge graph для RENAR-проектов." order: 5 lang: ru version: "1.0-draft"

Схема графа знаний

> ***Назначение:*** *формальная схема knowledge graph (KG) для RENAR-проектов — узлы, грани, properties, query patterns. KG служит **derived view** от RENAR-артефактов для семантических запросов AI-агентов и reconciliation. Machine-readable edge types — §4; нормативные поля связей — standard/06 §6.10, standard/08.*

KG — **не источник правды**. Source of Truth — артефакты (ADAPT / BR / SR / SPEC / TC) на носителе. Граф derived from frontmatter и не редактируется напрямую. Если граф противоречит артефактам → rebuild графа, не правка артефактов.

---

1. Зачем граф

1.1 Проблема без графа

Без KG AI-агент при генерации/анализе требования имеет flat поиск через:

- FTS5 / RAG поиск по `.md` файлам.
- Frontmatter `parent` / `children` (только direct hops).
- Keyword grep по коду.

Это даёт **синтаксический** контекст. Но AI часто нужен **семантический**: «дай все BR, влияющие на KPI X», «какие SR верифицированы tests с устаревшей requirement-version», «какие SPEC depends-on SPEC-DATA-03».

1.2 Use cases

Запрос	Без графа	С графом
Все BR, влияющие на Sales Cycle KPI	Grep + парсить frontmatter	Один Cypher-запрос
При изменении SR-05 — какие задачи и SPEC затронуты	Сканировать все ссылки	Single graph traversal
Какие SPEC-AI требуют high-risk classification по AI Act	Вручную	One query

Stakeholder map для проекта	Несколько запросов	Single subgraph extraction
Cross-project dependencies через INT-SR (заменено SPEC-INT)	Federation API calls	Federated graph query
Стало ли требование SR-12 деградировать (achievement % падает)	Manual analysis	Trend analytic on node properties
Stale TC — TC verifies SR с обновлённой версией, last-run на старой	Manual check	One query

---

2. Node types (closed list)

Type	Источник	Identity
`Requirement`	BR / SR / TR артефакты	`<id>`
`Specification`	SPEC-* артефакты (9 типов)	`<spec-id>`
`ADAPT`	ADAPT-NNN артефакты	`<adapt-id>`
`BackwardFinding`	B-NNN записи внутри ADAPT	`<adapt-id>:B-NNN`
`TestCase`	TC артефакты (вкл. `tc-type: contract`)	`<tc-id>`
`WorkOrder`	T3 и delta-T3	`<tz-id>`
`Stakeholder`	frontmatter `business-context.stakeholder`	`<stakeholder-id>`
`BusinessGoal`	frontmatter `business-context.business-goal` (deduplicated)	`<goal-id>`
`KPI`	frontmatter `business-outcome.kpi-name`	`<kpi-id>`
`Task`	TR / runtime task store	`<task-id>`
`CodeArtifact`	TC `automation.location`, code commits	`<repo>:<path>:<symbol>`
`Decision`	Architectural decision records	`<decision-id>`
`DeadEnd`	Failed approaches (опционально)	`<deadend-id>`
`RiskItem`	AI Risk Register entry	`AIR-NN`
`Compliance`	Compliance standard reference	`<std>:<control>`

`Template`	Requirements library template	`<template-id>`

Список закрыт; новые node types — только через изменение полного RENAR Standard.

---

3. Edge types (closed list)

3.1 Иерархия и derivation

Edge	From	To	Семантика
`parent`	Requirement	Requirement	A.parent = B → B is parent of A (BR → SR → TR)
`derived-from-adapt`	Requirement / Specification	ADAPT	Артефакт выведен из approved ADAPT section
`derived-from-template`	Requirement / Specification	Template	Шаблон (с version pin)
`replaces`	Requirement / Specification	Requirement / Specification	new replaces old (deprecated)
`supersedes`	Requirement / Specification	Requirement / Specification	strictly newer version (post-delta)
`parent-adapt`	ADAPT	ADAPT	delta-ADAPT → main ADAPT

3.2 ADAPT specifics

Edge	From	To	Семантика
`from-tz`	ADAPT	WorkOrder	source-tz pointer
`contains-backward`	ADAPT	BackwardFinding	B-NNN запись
`backward-asks-stakeholder`	BackwardFinding	Stakeholder	who answers
`resolved-by-answer`	BackwardFinding	(timestamp + author)	client-answer record

3.3 SPEC graph (parallel axis)

Edge	From	To	Семантика
`constrained-by`	Requirement	Specification	SR constrained by SPEC-* (typed edge)
`implements-spec`	Task	Specification	TR implements SPEC-*
`depends-on`	Specification	Specification	SPEC depends on another SPEC (DAG)
`referenced-by`	Specification	Requirement / Task	inverse (auto-derived)

3.4 Verification и реализация

Edge	From	To	Семантика
`verifies`	TestCase	Requirement / Specification	TC verifies artifact
`verified-by`	Requirement / Specification	TestCase	inverse (auto-derived)
`implements`	Task	Requirement	Task implements requirement
`realises-in`	TestCase	CodeArtifact	TC.automation.location
`linked-defect`	Requirement	Task (defect type)	Bug на этом требовании

3.5 provenance

Edge	From	To	Семантика
`from-order`	ADAPT / Requirement	WorkOrder	source.tz-section reference
`delta-from`	WorkOrder	WorkOrder	delta-T3 → base T3

`cited-in`	Requirement / Specification	WorkOrder	inline citation pointer на раздел T3 (для validator)
`decided`	Requirement / Specification	Decision	Decision при декомпозиции
`deadend`	Requirement / Specification	DeadEnd	Failed approach

3.6 Business / governance

Edge	From	To	Семантика
`owned-by`	Requirement	Stakeholder	business-context.stakeholder
`goal`	Requirement	BusinessGoal	business-context.business-goal
`impacts-kpi`	BusinessGoal	KPI	KPI driven by goal
`compliance-with`	Requirement / Specification	Compliance	compliance entry
`risk-mitigates`	Requirement / Specification	RiskItem	mitigates AIR-NN
`risk-introduces`	Requirement / Specification	RiskItem	introduces new risk (warning trigger)

3.7 Cross-project / integration

Edge	From	To	Семантика
`participates-in`	Specification (SPEC-INT)	Specification (SPEC-INT)	SPEC-INT participants — стороны интеграционного контракта (cross-project)
`cross-deps-on`	Task (project A)	Task (project B)	Cross-project dependency
`integrates-via`	Requirement	Specification (SPEC-INT)	Через SPEC-INT contract

3.8 Edge properties

Edges имеют properties (Cypher-style):

```
(SR:Requirement)-[v:verifies {requirement-version: "1.2", confidence:
"high"}]->(TC:TestCase)
(BR:Requirement)-[c:compliance-with {control: "A.5.34", rationale:
"PII protection"}]->(GDPR:Compliance)
(WO:WorkOrder)-[d:delta-from {effective-date: "2026-05-15"}]->(WO-
prev:WorkOrder)
(SR:Requirement)-[cb:constrained-by {since-version: "1.0"}]->
(SPEC:Specification)
```

---

4. Cypher-style query examples

4.1 BR, влияющие на KPI X

```
MATCH (br:Requirement {type:"BR"})-[:goal]->(g:BusinessGoal)-
[:impacts-kpi]->(k:KPI {name:"Sales Cycle Time"})
RETURN br.id, br.title, br.status
```

4.2 Затронутые задачи при изменении SR-05

```
MATCH (sr:Requirement {id:"SR-05"})<-[:implements]-(t:Task)
WHERE t.status NOT IN ["done", "cancelled"]
RETURN t.id, t.title, t.assignee
```

4.3 Все требования с PII в GDPR/ФЗ-152 scope

```
MATCH (r:Requirement)-[:compliance-with]->(c:Compliance)
WHERE c.standard IN ["GDPR", "ФЗ-152"] AND r.data-
classification.contains-pii = true
RETURN r.id, r.title, collect(c.control) as controls
```

4.4 Stale TC (criteria-version drift)

```
MATCH (r:Requirement)<-[:verifies]-(tc:TestCase)
WHERE tc.last-run.requirement-version < r.version
RETURN r.id, tc.id, tc.last-run.requirement-version, r.version
```

4.5 Multi-hop: code → test → SR → BR → KPI

```
MATCH (code:CodeArtifact {path:"src/auth/registration.py"})
      <-[:realises-in]-(tc:TestCase)
      -[:verifies]->(sr:Requirement {type:"SR"})
      -[:parent*1..2]->(br:Requirement {type:"BR"})
      -[:goal]->(g:BusinessGoal)
      -[:impacts-kpi]->(k:KPI)
RETURN code.path, sr.id, br.id, g.name, k.name
```

«Какие KPI зависят от этого файла кода?» — за один query.

4.6 SPEC dependency cycle detection

```
MATCH path=(s1:Specification)-[:depends-on*]->(s1)
RETURN path
```

Returning rows → нарушение DAG invariant (см. 02-schemas.md §9).

4.7 ADAPT с open backward findings

```
MATCH (a:ADAPT {status:"draft"})-[:contains-backward]->
(b:BackwardFinding {status:"open"})
RETURN a.id, count(b) as open_count
ORDER BY open_count DESC
```

---

5. Derivation rules

KG — **derived** от frontmatter артефактов. Не существует «прямого редактирования графа».

5.1 Node derivation

Node type	Источник в frontmatter
`Requirement`	`id`, `type ∈ {BR,SR,TR}`
`Specification`	`id`, `type ∈ {SPEC-ARCH..SPEC-OPS}`
`ADAPT`	`id`, `type: ADAPT`
`BackwardFinding`	ADAPT body parsing
`TestCase`	`id`, `type: TC`; derived: `last-run.*`, `last_modified` (§9.1), `criteria_changed_at` (§6.5)
`WorkOrder`	TZ-NNN frontmatter
`Stakeholder`	`business-context.stakeholder` deduplicated
`BusinessGoal`	`business-context.business-goal` deduplicated
`KPI`	`business-outcome.kpi-name` deduplicated
`Compliance`	`compliance.standard + compliance.control`

5.2 Edge derivation

Edge	Источник
`parent`	`parent.id` field
`verifies`	`verifies[].id` в TC
`constrained-by`	`constrained-by[]` в SR
`depends-on`	`depends-on[]` в SPEC
`implements-spec`	`implements-spec[]` в TR
`owned-by`	`business-context.stakeholder` → Stakeholder node
`compliance-with`	`compliance[]` массив

`derived-from-adapt`	`source.adapt`
`from-order`	`source.tz-section`
`delta-from`	`parent-adapt` (для ADAPT) или TZ delta

5.3 Refresh policy

Граф **rebuilt** при:

- Любое изменение в `.req` репозитории / collection (post-commit / post-save hook).
- Import TC last-run от CI бота.
- Создание/обновление task в task store.

Rebuild — incremental (только затронутые узлы и грани), не full rebuild каждый раз. Full rebuild — раз в неделю reconciliation-агентом.

---

6. Validation queries (для reconciliation)

Reconciliation-агент запускает graph queries для finding inconsistency:

6.1 Orphan approved requirements

```
MATCH (r:Requirement {status:"approved"})
WHERE NOT (r)-[:verified-by]->()
RETURN r.id  // approved без TC — warning
```

6.2 Broken citations

```
MATCH (r:Requirement)-[:from-order]->(wo:WorkOrder)
WHERE wo.status = "obsolete"
RETURN r.id, wo.id  // citation указывает на obsolete TZ
```

6.3 Circular parent chain

```
MATCH path=(r1:Requirement)-[:parent*]->(r1)
RETURN path
```

6.4 Stakeholder без owned BR

```
MATCH (s:Stakeholder)
WHERE NOT (s)<-[:owned-by]-()
RETURN s.id  // stakeholder в графе, но nothing assigned
```

6.5 Test fitting suspicious (AIR-06 signal)

```
MATCH (tc:TestCase)
WHERE tc.last_modified < tc.criteria_changed_at
  AND tc.last-run.result = "pass"
RETURN tc.id  // criteria недавно меняли, тут же passing — подозрительно
```

> *Свойства узла `TestCase`: `last_modified` — из node-таблицы (§9.1); `criteria_changed_at` — derived-свойство: timestamp последнего change-record с маркером `[test-spec-change]` (01-glossary.md §14.12). Оба заполняются при derivation графа (§2).*

6.6 SR без constrained-by (missing SPEC links)

```
MATCH (sr:Requirement {type:"SR", status:"approved"})
WHERE NOT (sr)-[:constrained-by]->(:Specification)
RETURN sr.id  // SR approved, но не привязан ни к одному SPEC — warning
```

6.7 SPEC без referenced-by (orphan SPEC)

```
MATCH (s:Specification {status:"approved"})
WHERE NOT (s)<-[:referenced-by]-()
  AND NOT (s)<-[:depends-on]-()
RETURN s.id  // возможный мёртвый SPEC
```

---

7. Federated queries (cross-project)

Multi-project координация через KG federation. Пример:

```
MATCH (s1:Specification {project:"auth", type:"SPEC-INT"})
      -[:participates-in]->(int:Specification)
      <-[:participates-in]-(s2:Specification {project:"billing"})
WHERE int.contract-version <> s1.implemented-version
RETURN s1.id, s2.id, int.id, int.contract-version, s1.implemented-
version
```

Отвечает на «какие cross-team integrations имеют version drift».

Federation — зависимая от носителя операция; реализуется через convention (межносительный query layer) или native multi-tenant graph.

---

8. AI prompt access pattern

8.1 Без графа

Промпт AI содержит:

```
Контекст: [полное содержимое нескольких .md файлов]
Задача: ...
```

Проблемы: токены, релевантность, hallucinations (см. AIR-01).

8.2 С графом

Промпт AI содержит **structured context**:

```
target-br: BR-05
parent:
  id: BR-05
  business-goal: "Reduce sales cycle"
  stakeholder: "Sales Director"
  data-classification: { contains-pii: true, residency: ["RU"] }
  compliance: ["ФЗ-152 ст.13.1"]
sibling-srs:
  - { id: SR-12, status: verified, scope: "lead intake" }
  - { id: SR-13, status: approved, scope: "lead qualification" }
related-specs:
  - SPEC-API-03 (auth contract, version 1.2)
  - SPEC-SEC-01 (auth model, RBAC)
related-decisions:
  - DEC-2026-04-15: "В этом проекте RLS на уровне БД, не applevel"
related-risks:
  - AIR-09 (privacy leakage) — mitigations active
```

AI генерирует SR с этим контекстом — больше шансов consistency, меньше hallucinations.

---

9. Нативные для носителя реализации

9.1 git как носитель (recommended for MVR)

Граф derived от frontmatter всех `.md` в `<project>.req/` + task store.

Пример реализации: embedded SQLite с graph schema:

```
CREATE TABLE nodes (
    id TEXT PRIMARY KEY,
    type TEXT NOT NULL,
    properties JSON,
    last_modified TIMESTAMP
);
```

```
CREATE TABLE edges (
    from_id TEXT,
    to_id TEXT,
    edge_type TEXT,
    properties JSON,
    PRIMARY KEY (from_id, to_id, edge_type)
);

CREATE INDEX idx_edges_from ON edges(from_id);
CREATE INDEX idx_edges_to ON edges(to_id);
CREATE INDEX idx_edges_type ON edges(edge_type);
```

Альтернатива для больших проектов: embedded graph DB (Kuzu, RedisGraph local).

9.2 Документный носитель (document-oriented store)

Документные БД с поддержкой графа имеют native graph queries через design views / Cypher-like языки. Не требуют separate infrastructure.

9.3 Schema invariants (независимо от носителя)

- Node types и edge types — fixed (closed list).
- Properties могут эволюционировать (minor schema bump).
- Удаление node / edge type — major schema bump + migration.

---

10. Implementation roadmap

Уровень зрелости	Покрытие графа
RENAR-1 / Core	KG опционален; парсинг frontmatter достаточно.
RENAR-2 / Foundation	Базовый граф: Requirement, TestCase, WorkOrder, Task; edges parent, verifies, verified-by, implements. Простые pre-built queries.
RENAR-3 / Team	Добавить: SPEC, ADAPT, BackwardFinding, Stakeholder, BusinessGoal, KPI, Decision. Edges: constrained-by, depends-on, derived-from-adapt, owned-by, goal, impacts-kpi. AI prompts с graph context.

RENAR-4 / Enterprise	Полная схема + reconciliation queries + federation. Visualization в UI.
RENAR-5	Trend analytics, межносительная federation, embedded graph DB для больших репо.

---

11. Перекрёстные ссылки

- Каноничные termini узлов и граней — 01-glossary.md.
- Формальные frontmatter schemas (источник derivation) — 02-schemas.md.
- AI Risk Register (AIR-10 KG poisoning) — 03-ai-risk-register.md.
- Style guide (validator использует KG для cross-reference checks) — 04-ai-style-guide.md.

Knowledge Graph Schema RENAR 1.0-draft — renar.tech

---

title: "Руководство по RU-стилю" description: "Canonical RU normative wording rules для RENAR Standard — terminology (§1) + RFC-2119 wording (§2) + formatting conventions (§3). Phase 3 pilot-validated normative artifact." order: 6 lang: ru version: "1.1.2" status: approved phase: "Phase 3 pilot validated — Phase 5 chapter pass unblocked" epic: ru-normative-pass-v1 task: ru-pilot-checkpoint compile-date: 2026-05-16 approval-date: 2026-05-17 pilot-commit: 564c8d4 pilot-chapter: standard/04-terms.md sources: - "research/ru-style-guide-draft-section-1-terminology.md" - "research/ru-style-guide-draft-section-2-rfc2119.md" - "research/ru-style-guide-draft-section-3-formatting.md"

RENAR RU Style Guide v1.0

> ***Normative artifact RENAR Standard.*** *Этот документ — canonical Style Guide для RU нормативного корпуса. Скомпилирован из трёх Phase 2 source drafts (§1 terminology + §2 RFC-2119 RU wording + §3 formatting).* ***Status:*** `approved` *— Phase 3 pilot validated 2026-05-17 (pilot chapter* `../standard/04-terms.md`*, commit* `564c8d4`*, 15 edits / +0.097% word delta, 0 semantic drift). Phase 5 mass chapter pass разблокирован.* ***Применение:*** *Phase 5 chapter pass + любая editorial работа над RU нормативным контентом RENAR Standard.*

§0 Introduction

§0.1 Назначение

RENAR RU Style Guide v1.0 — **normative companion** RENAR Standard, фиксирующий:

- canonical terminology rules для RU корпуса (12 buckets / 80+ closed-list terms / 4 directives) — §1;
- canonical RFC-2119 RU normative wording (lowercase regime + 11-keyword mapping + RFC 8174 carve-out) — §2;
- canonical formatting conventions (citations + code-fences + headings + tables + typography + frontmatter) — §3;
- integration protocol с RENAR Standard infrastructure (validate-frontmatter, Phase 5 chapter pass) — §4;
- change history + amendment procedure — §5.

Style Guide применяется ко всем RU normative artifacts RENAR Standard: `standard/`, `reference/`, `core/`, и (с soft application) `guide/`. Drafts в `research/` — author discretion, но Style Guide-compliance encouraged.

§0.2 Normative status

Style Guide v1.0 — **normative** в RENAR Standard sense:

- **Phase 5 chapter pass** обязан валидировать каждую главу против Style Guide §1-§3.
- **Phase 3 pilot checkpoint** — validation gate перед Phase 5 mass application.

- **Editorial tasks** (новые `research/` drafts, любые RU prose правки в normative artifacts) — apply Style Guide.
- **Нативный для носителя enforcement** — частичная: `node scripts/validate-frontmatter.js` (§3.7.4 / §4.3) + `node scripts/style-guide-check.js` (§4.3, v0.1.0 with known limitations).

Style Guide **не** dictates per-sentence wording — задаёт constraints внутри которых editor применяет judgement.

§0.3 Scope

§0.3.1 In scope

- RU prose, headings, tables, lists, code-fences, frontmatter в `standard/`, `reference/`, `core/`, `guide/`.
- Cross-file links между всеми directories.
- Normative semantic preservation (no MUST → SHOULD downgrade).
- RU typography (`«»` , em-dash, NBSP, числительные).

§0.3.2 Out of scope

- `standard/en/` translations (Phase 6/7 — separate EN Style Guide planned).
- Code в `scripts/`, `site/src/`, `.tausik/` — governed by project tech-stack conventions.
- Imagery / SVG (alt-text policy deferred per §3.13.7).
- `research/` drafts pre-publish (author discretion); frozen audits — Style Guide applies post-freeze.

§0.3.3 Hierarchy of authority

При конфликте между sources:

1. **`standard/04-terms.md §3.14.1`** (canonical legacy → v1.0 mapping) — **highest authority** для terminology. Style Guide §1 mirrors это.
2. **`standard/01-scope.md §1.7`** (closed-list policy meta-principle) — governs amendment procedure (§5.2).
3. **Style Guide §1-§3** — operational normative.
4. **Memory #31** (`ru-pass-principles`) — process principles, complementary.
5. **`research/ru-editorial-audit.md`** (Phase 1 deliverable) — input only, not authoritative post-lock-in.

§0.4 Application protocol

§0.4.1 Phase 5 chapter pass — invocation

```
Phase 5 editorial pass на главу X:

1. PRE-PASS:
   - read глава end-to-end (memory #31 принцип 7).
   - identify normative clauses, definitions, cross-refs, fences.
   - load Style Guide §1, §2, §3 as primary policy.

2. APPLY:
   - §1 terminology (per-bucket decisions, density-cap checks, F1
legacy labels).
   - §2 RFC-2119 (modal verb level preservation, no semantic
downgrade, EN UPPERCASE migration where applicable).
   - §3 formatting (citation conversion, fence lang tags, typography
normalization).

3. POST-PASS:
   - run §1.13 + §2.10 + §3.10 automated checks.
   - human spot-check 10% sample (memory #31 принцип 8).
   - validate-frontmatter passes (30/30 expected baseline).
   - cross-ref dead-link scan (0 expected).
   - word-count delta sanity (no inflation/deflation > ±15%).

4. COMMIT:
   - 1 chapter = 1 commit (memory #31 принцип 5).
   - commit message references Style Guide § applied.
   - Phase 5 task closes after commit.
```

§0.4.2 Subagent invocation

Phase 5 chapter pass через subagent — subagent prompt **обязан** включать explicit reference на Style Guide:

```
You are performing RU editorial pass on RENAR Standard chapter <X>.

PRIMARY POLICY: reference/06-ru-style-guide.md §1 + §2 + §3.
```

```
HARD RULES:
- §1.1.1: No mass find-replace. Per-occurrence judgement.
- §1.1.2: No semantic change (MUST/SHOULD/MAY level preserved).
- §2.1.4: No UPPERCASE Russian modals.
- §2.5: No modal verb LEVEL changes (downgrade forbidden).
- §3.3.2: All fences MUST have lang tag.
- §3.5.1: Bullets `-` only.
- §3.8.1: Кавычки `«»`, not `"`.

AI BIAS GUARD (§1.14): LLM trained on English skews:
- Inserts anglicism where RU equivalent exists.
- Tolerates mixed-lang prose.
- Polishes via semantic downgrade ("должен" → "следует").
WATCH OWN OUTPUT.

Output: per-clause diff with Style Guide § cited per change.
```

См. также §2.11 Subagent prompt template (RFC-2119-specific extended template).

§0.5 Change procedure

Изменения Style Guide v1.0 — formal procedure (mirrors `../standard/01-scope.md` §1.7 closed-list policy):

1. **Propose** — submitter создаёт `research/ru-style-amendment-NNN-<topic>.md` (mode: draft) с rationale.
2. **Review** — Style Guide editor (или project owner) review + accept / reject.
3. **Pilot** — accepted amendment проходит pilot validation на 1 chapter (Phase 3-style).
4. **Lock-in** — amendment merged в reference/06; version bump (v1.0 → v1.1).
5. **Phase 5 retroactive** — уже пройденные главы re-scanned для compliance с new amendment.

См. §5.2 Amendment procedure.

§0.6 Versioning

Version	Status	Phase	Date
v1.0-lock-in	proposed	Phase 2 lock-in	2026-05-16

v1.0	approved	Phase 3 pilot validated	2026-05-17 (этот документ)
v1.0.x	maintenance	Phase 5+ amendments	TBD
v1.1+	revision	Future epic	TBD

См. §5 Change history.

---

§1 Terminology canonicalization rules

> ***Source:*** `../research/ru-style-guide-draft-section-1-terminology.md` *(Phase 2 source artifact).*

§1.0 Назначение

§1 фиксирует **canonical terminology rules** для RU normative корпуса RENAR Standard. Регламентирует:

- какие термины остаются латиницей (`commit` , `frontmatter`),
- какие — кальками с RU инфлекцией (`реализация` , `спецификация`),
- какие — транслитом (`артефакт` , `репозиторий`),
- какие — нативно-русскими (`область применения` вместо `scope`),
- какие — устарели и требуют canonical замены (`UIC → SPEC-UI`).

§1 — **normative**. Phase 5 chapter pass обязан валидировать соответствие §1 при правке каждой главы (см. §1.13 Validation).

§1.1 Принципы

Шесть нормативных принципов §1, обязательные для применения Style Guide:

§1.1.1 No mass find-replace

Каждое term-decision контекстуальное. Пример: `session` в `../standard/06` (возможность носителя V2 — atomic change unit context) — это технический identifier, остаётся латиницей. `session` в `../standard/12` (метрическая единица, e.g. «session duration») — может быть переведено («длительность сеанса»). Слепая поиск-замена ломает нормативный смысл.

Применение: Phase 5 chapter pass — 1 chapter = 1 commit = 1 review (memory #31 принцип 5). Bulk find-replace инструменты запрещены.

§1.1.2 Preserved normative semantic

Если правка термина меняет MUST/SHALL/SHOULD/MAY semantics (RFC-2119 §2), normative MVR clauses, closed-list policy — это **content change**, требует отдельной задачи с явным rationale, не editorial. Editorial задача §1-§3 правит **только форму**, не содержание.

Применение: Style Guide §2 — pure form-level convention; semantic preservation guaranteed by §1.1.2.

§1.1.3 AI-bias guard

LLM-агент trained on web English склонна вписывать anglicism (имплементация / эссенциальный / латинизмы в prose) и tolerate их в editorial review. Style Guide применяется через **explicit checklist** + human spot-check, не через «выбор того, что нормально звучит».

Применение: каждая editorial правка Phase 5 проходит human spot-check на random sample (memory #31 принцип 8). Subagent invocations передают Style Guide §1 как primary policy в prompt.

§1.1.4 Closed list canonical terms

§1.9 фиксирует closed list canonical RENAR terms. Добавление новых — только через formal change procedure стандарта (mirrors `../standard/01-scope.md §1.7` closed list policy).

Применение: project-local создание новых canonical terms запрещено. Synonyms допустимы как pedagogical aid (UI projection, see `../standard/04 §4.13.3`).

§1.1.5 Single source of truth для terminology

Canonical normative источник: `../standard/04-terms.md` . Reference-каталог `01-glossary.md` обязан быть синхронизирован с canonical. При расхождении — `standard/04` побеждает; `reference/01` обновляется (см. §1.10 F1 reconciliation).

Применение: при Phase 5 chapter pass на `reference/01-glossary.md` — обязательная reconciliation против `standard/04 §4.14.1` mapping table.

§1.1.6 Domain-context preservation

Технические термины носителя, version-control и software-engineering (`commit` , `merge` , `hook` , `pipeline` , `runner` , `linter`) остаются латиницей, потому

что (a) являются конвенциями отрасли, (b) RU-эквиваленты теряют точность или вызывают двусмысленность, (c) удалить связь с возможностями носителя V1-V6 [§11.3] невозможно.

Применение: см. Bucket A whitelist §1.3.

§1.2 Категории терминов (шесть buckets)

Все термины RU normative корпуса RENAR классифицируются в одну из шести buckets:

Bucket	Категория	Policy	Section
A	Technical identifiers (носитель / VCS / SE domain)	Keep latin	§1.3
B	Status vocabulary (lifecycle статусы)	Keep latin (matches YAML schema)	§1.4
C	Organisational vocabulary	Rewrite в RU (prose); keep в YAML/code	§1.5
D	Accepted кальки	Keep, density-cap 1% per chapter	§1.6
E	Rewrite кальки	Per-term replacement table	§1.7
F	Translit	Triage: accepted / borderline / rewrite	§1.8

Bucket determination per term — Phase 2 closed list (§1.9). Phase 5 не пересматривает classification.

§1.3 Bucket A: technical identifiers (keep latin)

Rule: термины этой группы остаются латиницей в prose, code-fence content, frontmatter field names и section labels (когда technical).

Rationale: см. §1.1.6 Domain-context preservation. Дополнительно: возможности носителя V1-V6 `../standard/03` определены через эти термины; перевод нарушит referential integrity.

§1.3.1 Whitelist (closed list)

Term	Domain	RU UI projection (optional)
`commit`	VCS	фиксация

`merge`	VCS	слияние
`diff`	VCS	различие
`branch`	VCS	ветка
`push`	VCS	отправка
`pull`	VCS	получение
`hook`	VCS	перехватчик
`patch`	VCS	патч
`frontmatter`	Markdown	заголовочный блок
`manifest`	RENAR	манифест
`slug`	Markdown	слаг
`hash`	crypto / VCS	хеш
`trigger`	RENAR enforcement	триггер
`runner`	CI / testing	прогонщик
`pipeline`	CI	конвейер
`release`	RENAR	релиз
`deploy`	RENAR / DevOps	развёртывание
`build`	DevOps	сборка
`workflow`	RENAR / VCS	рабочий поток
`linter`	tooling	линтер
`parser`	tooling	парсер
`compiler`	tooling	компилятор
`tooling`	meta	оснастка
`tracker`	RENAR	трекер
`substrate` (только поля/код/имена файлов)	RENAR	**носитель** (prose canonical, см. ниже)
`provenance`	RENAR	происхождение

`fallback`	SE / RENAR ops	запасной вариант

RU UI projection — допустимо в multilingual UI (см. `../standard/04 §4.13.3`), но **canonical** — латиница. Frontmatter, ID, body normative paragraphs, scripts, CI hooks — всегда `commit/merge/...`.

> `substrate` → ***«носитель» (решение сессии #27).*** *В прозе канонический термин — **«носитель»** (склоняется). Латиницей `substrate` остаётся только в именах полей (`substrate-capabilities`, `substrate-v1-v6`), коде/YAML, именах файлов (`03-substrate-versioning.md`) и в layout-плейсхолдерах (`[requirements-substrate]/`). Поэтому строка `substrate` в whitelist выше помечена «только поля/код», а из §1.11 (prose-loanwords) термин убран. Канонический источник — `../standard/04 §4.8`.*

§1.3.2 Cross-bucket extension

Bucket A whitelist **не закрыт** в смысле §1.1.4. Это **открытый список идентификаторов домена носителя**: новые VCS / SE / CI / observability domain-terms могут быть добавлены proactively при необходимости. Phase 5 chapter pass: новые domain-terms идут в Bucket A автоматически, если match domain pattern «носитель / VCS / SE tooling».

Negative scenario: brand names (`GitHub`, `Astro`, `Markdown`, `JetBrains`) не Bucket A — они proper nouns. Treat как canonical references без перевода / транслитерации.

§1.4 Bucket B: status vocabulary (keep latin)

Rule: lifecycle status identifiers (`proposed`, `approved`, `verified`, `obsolete`, `frozen`, `active`, `planning`, `blocked`, `review`, `accepted`, `deprecated`, `done`, `ready`, `passing`, `failing`, `draft`, `client-ready`, `answered`, `resolved`, `revised`) остаются латиницей в prose, frontmatter, code-fence content.

Rationale:

1. Status — system-readable identifier, появляется в YAML frontmatter (`status: approved`); перевод prose-form вызывал бы dual-vocabulary («утверждённый → approved» mapping required for enforcement носителя).
2. State machines `../standard/10` определены через эти identifiers; перевод в prose разрывает референциальную цепь.

3. `../standard/04 §4.6` явно фиксирует canonical lowercase latin identifiers для всех типов артефактов.

§1.4.1 RU UI projection (для pedagogical/UI usage)

Status	RU UI projection (allowed in pedagogical guide/)
`draft`	черновик
`proposed`	предложенный
`approved`	утверждённый
`verified`	проверенный
`accepted`	принятый
`done`	выполненный
`obsolete`	устаревший
`deprecated`	удалённый
`frozen`	замороженный
`active`	активный
`planning`	планируется
`blocked`	заблокирован
`review`	на ревью
`ready`	готов
`passing`	проходит
`failing`	не проходит
`client-ready`	готов к клиенту
`answered`	отвечен
`resolved`	разрешён
`revised`	переуточнён

Pedagogical-only: RU projection допустим в `guide/` (where reader-facing narrative dominates), **не** в `standard/`, `reference/`, `core/`.

§1.4.2 Negative scenario — mixed-case mistakes

Распространённый paste-error: `proposed → Proposed → Предложенный` в одном файле. Style Guide требует **case consistency**: всегда lowercase `proposed`. Phase 5 chapter pass проверяет каждое status-mention.

§1.5 Bucket C: organisational vocabulary (rewrite в prose)

Rule: в **prose** заменить на RU; в **YAML field names, code-identifiers, fence content** — keep latin.

Rationale: organisational vocabulary не имеет domain-specific semantic; RU equivalents существуют, чтение естественнее.

§1.5.1 Replacement table

Latin (prose)	RU canonical	Notes
`scope`	область (применения) / охват	YAML key `scope:` keeps latin
`audit`	ревизия / проверка / контроль	«audit trail» — keep latin (§1.3.1 extension: домен носителя)
`draft` (как verb)	черновик / черновой	status `draft:` keeps latin (§1.4)
`policy`	правило / политика	«closed list policy» — keep latin (RENAR canonical phrase, §1.10)
`ownership`	владение / ответственность	—
`default`	по умолчанию	в YAML — keep `default:`
`bypass`	обход	—
`rollback`	откат	—
`override`	переопределение	—
`stub`	заглушка	—
`guide` (noun)	руководство	директория `guide/` — keep latin (path)
`walkthrough`	прохождение / пошаговый разбор	—
`quickstart`	быстрый старт	—

§1.5.2 Negative scenario — context discrimination

Перед применением каждого rewrite — проверить контекст:

- Pure prose («scope нормирует X») → replace («область определяет X»).
- YAML/code field name (`scope:`) → keep.
- Технический термин домена носителя («audit trail», «commit audit») → keep latin per §1.3.
- Cross-ref label («closed list policy» как RENAR canonical phrase) → keep latin.

Phase 5 chapter pass: для каждого Bucket C term — 100% manual context-discrimination (no automation).

§1.6 Bucket D: accepted кальки (keep, density-cap 1%)

Rule: accepted кальки остаются как есть; Phase 5 chapter pass проверяет **density-cap**: term frequency ≤ 1% prose-words per chapter. При превышении — Phase 5 reducer заменяет на synonym / pronominal reference / structural rewrite.

Rationale: эти термины давно accepted в RU IT, перевод не даёт reading-improvement. Но overuse снижает scan-readability (читатель видит wallpaper of «нормативный» + «реализация» + «спецификация» в каждом параграфе).

§1.6.1 Accepted кальки list

Term	Phase 1 count	Phase 5 density-cap action
`нормативный`	275	Reduce if >1% chapter density
`реализация`	111	Same
`спецификация`	25	Below cap
`формальный`	44	Below cap
`опциональный`	39	Below cap
`атомарный`	19	Below cap
`декомпозиция`	30	Below cap
`композиция`	30	Below cap
`верификация`	(low)	Below cap
`валидация`	(low)	Below cap

`идентификация`	(low)	Below cap
`идентификатор`	12	Below cap
`интеграция`	(low)	Below cap
`миграция`	29	Below cap
`авторизация`	(low)	Below cap
`аутентификация`	(low)	Below cap
`итерация`	15	Below cap

§1.6.2 Density measurement

Density = (term occurrences) / (prose words after strip) × 100. Strip: YAML frontmatter, fenced code blocks, inline backticks, markdown link URLs (см. `../research/ru-anglicism-inventory.md §2` методология). Cap: 1.0% per chapter.

Verification protocol: Phase 5 re-runs anglicism scanner (`d:/tmp/renar_anglicism_scan.py` ephemeral) на правленой главе; density delta — positive only (или, на best-of-class chapters, 0).

§1.7 Bucket E: rewrite кальки

Rule: заменить на RU canonical per replacement table. Phase 5 chapter pass — manual per-occurrence (no mass-replace per §1.1.1).

Rationale: имеется precise RU equivalent; калька — anglicism-creep без semantic value.

§1.7.1 Replacement table

Latin (calque)	RU canonical	Notes
`имплементация`	реализация (общее) / исполнение (active-voice)	Phase 5: 52 occurrences corpus-wide
`конформация`	соответствие	**Exception:** `conformance` — ISO 29148 term, keep latin в контексте домена носителя (см. `../standard/13-conformance.md`). RU «соответствие» — для prose discussion outside ISO context

`эссенциальный`	существенный / обязательный	depending on context: «obligatory» vs «central»
`дрифт` (translit form)	дрейф / расхождение	**Exception:** `drift` (latin) — RENAR canonical class name §3.11; keep `Schema drift` / `Lifecycle drift` латиницей. RU «дрифт» только в prose mentions outside §3.11 context
`трейс*` (in prose)	трасс* / отслеживание	«traceability» — RENAR canonical, keep latin
`аннотация` (когда AI-bias overuse)	пояснение / примечание	Context-dependent — see §1.5.2
`регуляция`	регулирование	Rare, AI-bias residue
`корреляция`	(Bucket D — keep)	Listed for clarification: it's accepted, not rewrite

§1.7.2 Exceptions and edge cases

При rewrite encountered:

- **Frozen quote / citation** — when calque appears inside quoted text from ISO/IEC/IEEE source — keep verbatim, add translator note if needed.
- **Field name in YAML schema** — never touch (canonical schema `02-schemas.md` is normative).
- **Cross-ref label** — if part of canonical RENAR phrase — keep latin per §1.3.1 extension.

§1.8 Bucket F: translit triage

Rule: триаж по 3 категориям — accepted / borderline / rewrite. Phase 2 финализирует triage; Phase 5 chapter pass применяет.

§1.8.1 Triage — Accepted (keep)

Транслит-термины, давно accepted в RU IT, keep:

Term	Phase 1 count	RU equivalent (not used)
`артефакт`	455	(RU equivalent «изделие» лишь утилитарно; «артефакт» canonical в IT)
`автор`	15	(Russian word — not anglicism per se)

`шаблон`	17	(Russian word — not anglicism)
`репозиторий`	18	(RU equivalents «хранилище» / «склад» не охватывают version-control semantics)

§1.8.2 Triage — Borderline (case-by-case)

Decide context-dependent:

Term	Phase 1 count	Decision rule
`ревью`	29	Keep when after «adversarial» (technical compound); rewrite to «рецензия» / «ревизия» when stand-alone
`кейс`	13	Keep in «edge case» / «test case» — technical compound; rewrite to «случай» в общем контексте
`стейкхолдер`	12	Keep как formal IT term; alternative «заинтересованная сторона» (longer, more formal — for `standard/`); «стейкхолдер» в `guide/` OK
`чекпойнт`	low	Keep если refers to RENAR/SENAR `/checkpoint` skill; rewrite to «контрольная точка» otherwise
`гейт`	low	Keep в normative RENAR context (Quality Gate); rewrite to «ворота» / «контроль» avoided (loses canonical meaning)
`онбординг`	low	Keep

§1.8.3 Triage — Rewrite (low-count, ambiguous)

Транслит с RU equivalents — rewrite:

Term	Phase 1 count	RU replacement
`фронтенд`	low	«клиентская часть» / «frontend» (latin) — pick latin
`бэкенд`	low	«серверная часть» / `backend`
`пайплайн` (RU form)	low	Keep `pipeline` (Bucket A latin) — don't use translit
`фолбэк`	low	Keep `fallback` (Bucket A) — don't translit
`оверхед`	low	«накладные расходы»
`апдейт`	low	«обновление» / «правка»

Negative scenario: Bucket F triage **не** rewrite того, что Bucket A keeps latin. Pick latin (`pipeline`) over translit (`пайплайн`). Translit forms — anti-pattern для терминов домена носителя.

§1.9 Closed list canonical RENAR terms

Закрытый список 50+ canonical RENAR terms, синхронизированный с `../standard/04-terms.md` . Изменение списка — formal change procedure стандарта.

§1.9.1 Артефакты requirements (canonical from `standard/04 §4.3`)

Canonical	RU UI projection	Latin	Source
`BR`	Бизнес-требование	Business Requirement	`standard/04 §4.3.3`
`SR`	Системное требование	System Requirement	`§3.3.4`
`TR`	Требование к задаче	Task Requirement	`§3.3.5`
`ADAPT`	Двусторонняя адаптация ТЗ	(canonical RENAR-extension)	`§3.3.2`
`ТЗ`	Техническое задание	(canonical RU)	`§3.3.1`
`delta-ТЗ`	дельта-ТЗ	—	`§3.3.1`

§1.9.2 SPEC family (closed list 9 types from `§3.4`)

`SPEC-ARCH` , `SPEC-API` , `SPEC-DATA` , `SPEC-INT` , `SPEC-PROC` , `SPEC-UI` , `SPEC-AI` , `SPEC-SEC` , `SPEC-OPS` — все latin.

§1.9.3 TC types (closed list 6 from `§3.5.2`)

`acceptance` , `ux` , `system` , `contract` , `eval` , `security` — все latin, lowercase.

§1.9.4 Quality Gates (canonical v1.0 from `§3.7`)

`QG-0 Approval Gate` , `QG-1 Implementation Gate` , `QG-2 Verification Gate` , `QG-3 Architecture Gate` (optional), `QG-4 Acceptance Gate` (optional).

Important: legacy names (`QG-0 Context Gate` , `QG-1 Requirements Gate` , `QG-2 Implementation Gate` , `QG-3 Verification Gate`) **deprecated**. См. §1.10 F1 reconciliation.

§1.9.5 Lifecycle статусы (canonical from `§3.6`)

См. §1.4.1 RU UI projection table.

§1.9.6 Возможности носителя (V1–V6)

`V1 Immutable history` , `V2 Atomic change unit` , `V3 diff & review` , `V4 Branching / change-set` , `V5 Cross-substrate version pin` , `V6 Author + timestamp` — все labels latin. RU описания допустимы в prose.

§1.9.7 Drift classes (closed list 8 from `§3.11`)

Все 8 классов — латиница в canonical name: `Schema drift` , `Lifecycle drift` , `Source-of-truth drift` , `Implementation drift` , `Terminological drift` , `Order / provenance drift` , `TC ⇔ requirement provenance drift` , `Test-fitting drift` .

§1.10 F1 reconciliation: legacy → canonical v1.0

Source: `../standard/04-terms.md §4.14.1` Устаревшие RENAR-specific labels.

Rule: при обнаружении в normative artifact — нативный для носителя hook обязан флагнуть; Phase 5 chapter pass правит на canonical.

§1.10.1 Legacy artifact labels

Устаревший	Canonical v1.0	Где применять
`UIC` (UI Concept)	`SPEC-UI`	All occurrences
`AIC` (AI Concept)	`SPEC-AI`	All occurrences
`INT-SR` (Integration SR)	`SR` с `constrained-by: [SPEC-INT-N]`	All occurrences
`INT-TC` (Integration TC)	`TC` с `tc-type: contract`	All occurrences
`TM` (Module/Submodule SR)	`SR` с `level: module`	All occurrences

`TS` (Technical Specification)	`SPEC-ARCH` / `SPEC-OPS` (context-dependent)	Manual per occurrence

§1.10.2 Legacy Quality Gate names

Устаревший (pre-v1.0)	Canonical v1.0
`QG-0 Context Gate`	`QG-0 Approval Gate`
`QG-1 Requirements Gate`	`QG-1 Implementation Gate` (semantic shift: ранее approval BR/SR, теперь только TC `draft → ready`)
`QG-2 Implementation Gate`	`QG-1 Implementation Gate`
`QG-3 Verification Gate`	`QG-2 Verification Gate`

§1.10.3 Affected file (Phase 5 priority 1)

`01-glossary.md`:

- §2.1 «Уровни требований» table — replace `TM` / `UIC` / `AIC` / `INT-SR` / `INT-TC` / `TS` per §1.10.1.
- §2.4 «Quality Gates» table — replace pre-v1.0 names per §1.10.2.
- §3.2 «Quality gates» cross-mapping table — same.

Phase 1.5 prerequisite: до Phase 5 запуска — `01-glossary.md` reconciliation должна быть выполнена. Recommend separate task `ru-reconcile-glossary-vs-standard`.

§1.11 Accepted technical loanwords

Cross-bucket whitelist для терминов домена носителя / IT-canonical, **которые не калька, не translit, а accepted technical loanwords**:

Term	Domain	Used in canonical RENAR
`state machine`	distributed systems	`standard/10` §10.5-§10.9
`atomic change unit`	distributed systems	`standard/03 §3.3.2` V2
`diff & review`	VCS / RENAR	`standard/03 §3.3.3` V3
`adversarial review`	RENAR / AI	`standard/00 §0.3`, `standard/04 §4.13.4`

`eval` / `evaluation`	AI testing	`standard/09 §9.6.2`
`runner`	CI / RENAR	`standard/04 §4.7`, `standard/10 §10.3.2`
`branching`	VCS	`standard/03 §3.3.4` V4
`version pin`	RENAR / носитель	`standard/04 §4.8.4`, `standard/03 §3.3.5`
`traceability`	RE general	`standard/01 §1.6`, `standard/04 §4.10.3`
`provenance`	RENAR AI	`standard/04 §4.10`, `standard/11 §11.7.1`
`closed list`	RENAR meta-policy	`standard/01 §1.7`

Rule: эти термины не подлежат rewrite в Phase 5; они canonical в RENAR domain и широко accepted в международной literature. AI-bias guard §1.1.3 особенно: не пытаться «перевести» в RU equivalents — это поломает domain accuracy.

§1.12 Process for adding new terms

Добавление нового canonical term:

1. **Identify need** — term появляется в нормативной главе ≥3 раз без §1.9 listing.
2. **Bucket classification** — определить bucket A-F (применить §1.2 criteria).
3. **Rationale draft** — submitter готовит rationale (3-5 sentences): why this term needed, why this bucket, what RU/EN alternatives rejected.
4. **Phase 2 review** — submit к Style Guide §1 editor (review same way как добавление в `standard/04-terms.md` per §4.14 formal change procedure).
5. **Phase 5 retroactive** — после approval, Phase 5 chapter pass нормализует existing occurrences.

Interim period (между submission и approval): new term допустим в `research/` drafts; в `standard/`, `guide/`, `reference/`, `core/` — только после approval.

§1.13 Validation

Phase 5 chapter pass обязан выполнить per глава:

§1.13.1 Automated checks

1. **Bucket A whitelist coverage** — grep latin terms vs §1.3.1 list; flag mismatches.
2. **Bucket B status consistency** — `proposed/approved/...` always lowercase latin; flag mixed-case.
3. **Bucket C latin-in-prose** — grep `scope|audit|draft|policy|...` вне code-fences / YAML; manual review each.

4. **Bucket D density-cap** — re-run anglicism scanner; verify <1% per term.
5. **Bucket E presence** — grep rewrite-target terms (`имплементация|конформация|эссенциальный|...`); manual review each.
6. **Bucket F triage** — grep translit forms; classify each per §1.8.
7. **Legacy labels (F1)** — grep `UIC|AIC|INT-SR|INT-TC|TM|TS` ; flag occurrence как current-term misuse. Citation contexts HE flagged (amendment 004): legacy→canonical mapping rows (legacy label + canonical replacement на одной строке), строки с deprecation/migration markers (`устаревш|legacy|deprecated|замен|migration|миграц|стар*`), строки внутри migration/legacy section heading (level-inherited через subsections). Backticked IDs (`` `UIC-NN` ``) уже stripped как code.
8. **Legacy QG names** — grep `QG-0 Context Gate|QG-1 Requirements Gate|QG-2 Implementation Gate|QG-3 Verification Gate` ; flag. Bare `Verification Gate` / `Implementation Gate` без QG-prefix — canonical post-amendment 003 (§1.9.4); HE legacy.

Tooling: `d:/tmp/renar_anglicism_scan.py` (Phase 1 ephemeral); Phase 5 lock-in version — `scripts/style-guide-check.js` v0.2.0 (§4.3).

§1.13.2 Human spot-check requirements

Per memory #31 принцип 8: **каждый Phase 5 chapter pass проходит human spot-check на random 10% правок**. Spot-check verifies:

- (a) AI-agent не вписал anglicism, не разрешённые §1.
- (b) Context-discrimination сделана правильно (§1.5.2).
- (c) Domain accuracy сохранена (§1.11 accepted loanwords).
- (d) Normative semantic preserved (§1.1.2).

§1.13.3 Phase 3 pilot checkpoint

До Phase 5 запуска (mass application) — Phase 3 pilot:

1. **Pick one chapter** (recommend: `standard/04-terms.md` — canonical-voice exemplar per `../research/ru-tone-sampling.md §4.1`).
2. **Apply full §1 ruleset.**
3. **User checkpoint:** review pilot diff, adjust §1 if needed.
4. **Lock-in Phase 2** after pilot approval.

§1.14 AI-bias guard

Этот § явно фиксирует known AI-bias failure modes при editorial pass:

§1.14.1 LLM English-bias

LLM trained on multilingual web with English dominance склонна:

- Inserting anglicism в prose где RU equivalent exists.
- Tolerating mixed-lang («корпоративный admin») as «normal».
- Rewriting RU into English-flavored RU («это implementation, которая verifies...»).
- Preferring кальку («имплементация») over native («реализация»).

Mitigation: Bucket E replacement table (§1.7.1) — hard-coded substitutions; agent invocation prompt includes «strict RU prose; bias toward §1.7 RU canonical».

§1.14.2 Confidence over precision

LLM может confidently rewrite phrase в форме, которая ломает normative semantic — например, заменять «должен» на «следует» «для читаемости» (это semantic downgrade per RFC-2119 §2).

Mitigation: §1.1.2 — normative semantic preserved; Phase 5 pilot checkpoint catches semantic drift.

§1.14.3 Pattern-completion drift

LLM, видя 50 examples «X нормирует Y», completes 51-й example с тем же pattern, даже если §1 требует RU rewrite.

Mitigation: spot-check на random samples (§1.13.2), не на «typical» examples.

§1.14.4 Human review obligation

Hard rule: Phase 5 chapter pass без human spot-check **запрещён**. Memory #31 принцип 8 — обязательно. Subagent invocations передают этот § в prompt.

§1.15 Negative scenarios

§1.15.1 No mass find-replace

Запрещено по §1.1.1. Tools (sed / awk / IDE find-replace) — banned for Phase 5 chapter pass. Manual per-occurrence approve required.

§1.15.2 No prescriptive Phase 5 decisions

§1 — terminology rules; **не** dictates wording per sentence. Phase 5 chapter pass — editorial judgment within §1 constraints, не mechanical rewrite.

§1.15.3 Closed list violation

Project-local создание новых canonical terms (`SR-CLIENT` , `BR-INTERNAL` , ...) — violation of §1.1.4 closed list policy. Нативный для носителя hook должен flag; Phase 5 pass — интеграция hook носителя TBD (future epic).

§1.15.4 Style Guide bypass

Phase 5 chapter pass без §1 application — **violates Phase 2 lock-in contract**. Style Guide §1 — normative, не recommendation. SENAR Rule 9.15 (AI Output QA) requires.

§1.16 Cross-link к sections §2, §3

- §2 **RFC-2119 RU normative wording** — regime для MUST/SHALL/SHOULD/MAY (EN UPPERCASE vs RU lowercase «должен/следует/может»). §1 не трогает this — §2 reserved.
- §3 **Formatting conventions** — citations bare/link, code-fence lang tags, headings, tables. §1 не дублирует.

---

§2 RFC-2119 RU normative wording

> ***Source:*** `../research/ru-style-guide-draft-section-2-rfc2119.md` *(Phase 2 source artifact).*

§2.0 Назначение

§2 фиксирует **canonical RU normative wording** для RENAR Standard. Регламентирует:

- regime для RFC-2119 keywords (UPPERCASE EN / UPPERCASE RU / lowercase RU) — закрывает Phase 1 F2;
- canonical mapping для всех 11 RFC-2119 keywords (`MUST/MUST NOT/SHALL/SHALL NOT/SHOULD/SHOULD NOT/MAY/REQUIRED/RECOMMENDED/NOT RECOMMENDED/OPTIONAL`);
- strict semantic preservation rule (no «должен → следует» downgrade под видом editorial polish);
- tense/mood normative rules (imperative-mood для normative clauses; canonical form для definitions; future-tense scope);
- migration plan для 15 existing EN UPPERCASE occurrences;

- canonical negation patterns (`не должен / запрещено / не допускается / не следует / не рекомендуется`).

§2 — **normative**. Phase 5 chapter pass обязан проверять каждую normative clause на соответствие §2 (см. §2.10 Validation).

§2.1 Принципы

Шесть нормативных принципов §2, обязательные для применения Phase 5 chapter pass:

§2.1.1 Semantic-fidelity первичен

RFC-2119 (RFC 2119) фиксирует **строгое semantic discrimination** между levels: MUST (absolute requirement) ≠ SHOULD (strong recommendation, allowed exceptions) ≠ MAY (optional). RU wording должен сохранить эту дискриминацию.

Применение: при выборе RU эквивалента для нормативной clause editor проверяет: совпадает ли семантика с RFC-2119 level? Если нет — это **content change**, требует отдельной задачи (см. §1.1.2 Preserved normative semantic).

§2.1.2 No semantic downgrade под видом editorial

LLM-агент склонен «полировать читаемость» путём «должен → следует», что даёт **семантический downgrade** (MUST → SHOULD). Это потеря normative strength.

Применение: Phase 5 editorial pass **не имеет права** менять modal verb level. Если original clause использует «должен» — replacement тоже imperative-mood («должен/обязан»). Если original — «следует» — replacement в пределах SHOULD level. Перенос между levels — content change.

Cross-ref: §1.14.2 Confidence over precision.

§2.1.3 RFC 8174 carve-out для RU

RFC 8174 уточняет: только **UPPERCASE** формы (MUST, SHALL, …) несут RFC-2119 нормативный смысл; lowercase английский («must», «shall») — обычный язык без normative weight.

RENAR Style Guide делает **явный carve-out** для русского:

> *RU lowercase modal verbs (`должен` , `следует` , `может` , `рекомендуется` , `допускается` , `требуется` , `обязательно` , `опционально` , `обязан` , `запрещено`), использованные в нормативных clauses внутри RENAR Standard artifacts (`standard/` , `reference/` , `core/` с normative role), **являются***

> ***canonical эквивалентом*** *RFC-2119 UPPERCASE keywords. Их normative weight равен соответствующему UPPERCASE keyword.*

Применение: §2.2 mapping table определяет канонические соответствия. Document RFC-2119/RFC-8174 в `../standard/14-normative-refs.md` cited; carve-out явно прописан в lock-in §2.

§2.1.4 No UPPERCASE convention в RU prose

Convention `ОБЯЗАН/ДОЛЖЕН/СЛЕДУЕТ/МОЖЕТ` (capitalized) — **запрещена**. Phase 1 audit фиксирует 0 occurrences в текущем корпусе; вводить новую convention в Phase 2 — против consistency.

Применение: RU normative modals — always lowercase. Capitalization — только когда начинают предложение (orthographic, не emphatic).

Negative scenario: если автор хочет визуально emphasize requirement — использовать `**должен**` (markdown bold) либо вынести в выделенный нормативный блок (e.g. таблицу MVR / SPEC requirements). UPPERCASE — табу.

§2.1.5 Imperative-mood для normative clauses

Normative clauses используют **imperative-mood** RU верб: «X должен Y», «X обязан Y», «X не должен Y», «X следует Y», «X может Y». Indicative form («X выполняет Y», «X is Y») — для descriptive prose, не для normative requirements.

Применение: при правке RU normative wording — keep imperative-mood. Если original imperative («должен») — replacement imperative («обязан» при contract obligations). Indicative form (`X — это Y`) допустим только для definitions (§2.4.2).

§2.1.6 MVR special handling

Minimum Viable Requirements (MVR-1..MVR-7) — закрытый list из 7 MVR в `../standard/00-introduction.md §0.5` . Текущие 10 SHALL в этой таблице — намеренный RFC-2119 идиом per ISO 29148 reading.

Decision (§2.6): MVR таблица мигрирует на RU lowercase canonical (`обязан` для SHALL, `должен` для MUST), сохраняя semantic. ISO 29148 cross-link — в footnote или в `../standard/14-normative-refs.md` .

§2.2 F2 regime decision

§2.2.1 Three options considered

Phase 1 audit (editorial-audit §1.2 F2) выявил mixed regime. Style Guide §2 закрывает F2 одним из трёх options:

Option	Description	Pros	Cons
(A) EN UPPERCASE everywhere	All normative modals в RU prose → MUST/SHALL/SHOULD/MAY	Direct ISO 29148 compatibility; zero ambiguity per RFC 2119	Breaks RU readability (EN keywords inside RU sentence); 261 RU lowercase occurrences потребуют migration
(B) RU UPPERCASE	`ОБЯЗАН/ДОЛЖЕН/СЛЕДУЕТ/МОЖЕТ` capitalized	Visually emphatic; matches RFC 2119 UPPERCASE-bearing-semantic principle	0 precedent в корпусе; visual noise typical for shouting; не natural в RU normative tradition
(C) RU lowercase canonical	`должен/обязан/следует/может/...` lowercase	261/276 (95%) уже соответствуют; natural RU normative tradition; closer to ГОСТ Р normative style	Требует explicit RFC 8174 carve-out (см. §2.1.3); 15 EN UPPERCASE occurrences потребуют migration

§2.2.2 Decision: Option (C) RU lowercase

Decision recorded. RENAR Standard RU corpus использует **RU lowercase modal verbs** как canonical normative wording.

§2.2.3 Rationale

1. **De-facto majority** (95%) уже соответствует — minimal disruption.
2. **Natural RU normative tradition** — ГОСТ Р, ISO 29148 RU translations, ИСО серия используют lowercase modals. Стандарт мирового уровня в RU — должен следовать RU normative tradition.
3. **RFC 8174 carve-out** (§2.1.3) явно устраняет ambiguity: lowercase RU modals в RENAR normative contexts = canonical RFC-2119 keywords.
4. **MVR table** — единственный pocket EN UPPERCASE (10 SHALL); migration plan (§2.6) минимизирует regression. ISO 29148 cross-link не теряется

(`../standard/14-normative-refs.md` carries).

5. **Visual emphasis** где нужен — через markdown `**bold**` либо через outdent в выделенные normative tables, не через capitalization.

§2.2.4 What this decision is NOT

- §2.2.2 **не** запрещает EN UPPERCASE в `04-ai-style-guide.md` (descriptive RFC 2119 mention, не normative usage in RENAR Standard sense).
- §2.2.2 **не** запрещает RFC-2119 UPPERCASE keywords в EN translation (`standard/en/` future epic). EN version следует EN convention (UPPERCASE) per §2.2.5 — normative.
- §2.2.2 **не** mandates rewrite в каждой clause `«X необходимо Y»` → `«X должен Y»`. Phrase «необходимо» канонический synonym `«должен»` (см. §2.3) и допустим.

§2.2.5 EN translation convention

Rule: EN-перевод RENAR Standard (`standard/en/` , `guide/en/` , `reference/en/` , `core/en/`) использует **UPPERCASE RFC-2119 keywords** как canonical normative wording.

Rationale:

1. RFC 8174 (May 2017) явно фиксирует: только UPPERCASE формы (`MUST` , `SHALL` , `SHOULD` , `MAY` , `REQUIRED` , `RECOMMENDED` , `OPTIONAL` + negations) несут RFC-2119 normative weight в EN context. Lowercase EN modals не имеют normative weight.
2. RU carve-out (§2.1.3) — narrow exemption для русского, основанный на ГОСТ Р / ISO 29148 RU translation tradition. Carve-out **не распространяется** на EN.
3. International standard practice (ISO/IEC/IEEE 29148, 12207, 15288 в EN) — UPPERCASE consistent.

Применение:

- EN translator использует §2.3 mapping table как **bidirectional reference**: RU lowercase canonical → EN UPPERCASE canonical.
- `должен` (MUST) → `MUST` , `обязан` (SHALL) → `SHALL` , `следует` (SHOULD) → `SHOULD` , `может` (MAY) → `MAY` , etc.
- MVR таблица в `../standard/00-introduction.md §0.5` : RU `обязан` (SHALL) → EN `SHALL` (восстанавливает Phase 1 EN baseline в EN translation).

- Descriptive carve-out (§2.7): EN translation `04-ai-style-guide.md` сохраняет existing EN UPPERCASE в descriptive RFC mentions — без re-keyword миграции.

Anti-patterns в EN translation:

1. Lowercase EN modals (`must` , `should`) в normative clause — нет normative weight per RFC 8174. **Anti-pattern.**
2. Mixed UPPERCASE/lowercase в одной clause (`MUST must ...`) — paste-error. **Anti-pattern.**
3. RU lowercase в EN clause (`must должен ...`) — translation incomplete. **Anti-pattern.**

Pilot validation: при старте `en-standard` эпика — pilot chapter (recommended: `standard/en/04-terms.md` per RU pilot precedent §5.1) проверяет §2.2.5 applicability. Если pilot выявит edge-cases — отдельный amendment NNN.

§2.3 Canonical mapping table

11 RFC-2119/RFC-8174 keywords + RU canonical equivalents. Phase 5 chapter pass — primary reference.

Bidirectional usage: Таблица применяется в обоих направлениях per §2.2.5:

- RU normative pass (Phase 5): EN UPPERCASE → RU lowercase canonical.
- EN translation (Phase 6+): RU lowercase canonical → EN UPPERCASE canonical.

Semantic preservation rule (§2.5) одинаково применима в обоих направлениях — modal verb LEVEL changes запрещены.

§2.3.1 Mandatory levels

RFC-2119	RU canonical (primary)	RU canonical (synonym)	Semantics
`MUST`	должен	обязан / необходимо / требуется	Absolute requirement — нет исключений
`MUST NOT`	не должен	запрещено / недопустимо	Absolute prohibition
`SHALL`	должен	обязан	Equivalent к MUST per RFC 2119 §2

`SHALL NOT`	не должен	запрещено	Equivalent к MUST NOT
`REQUIRED`	обязателен / обязательно	необходим	Equivalent к MUST (adjective form)

Notes:

- `должен` — modal verb (predicate): «X должен соответствовать Y».
- `обязан` — synonym `должен` для contract-style obligation: «X обязан соблюдать Y» (часто в контекстах носителя / SPEC).
- `обязателен` / `обязательно` — adjective / adverb (REQUIRED): «Поле X обязательно».
- `необходимо` — semantic equivalent: «Необходимо обеспечить Y».
- `требуется` — passive-voice equivalent: «Требуется обеспечить Y».
- `запрещено` / `недопустимо` — adjective predicate (MUST NOT): «Mass find-replace запрещено».

§2.3.2 Strong recommendation level

RFC-2119	RU canonical (primary)	RU canonical (synonym)	Semantics
`SHOULD`	следует	рекомендуется	Strong recommendation; exceptions требуют explicit rationale
`SHOULD NOT`	не следует	не рекомендуется	Strong negative recommendation; exceptions требуют rationale
`RECOMMENDED`	рекомендуется	предпочтительно	Equivalent к SHOULD (predicate form)
`NOT RECOMMENDED`	не рекомендуется	нежелательно	Equivalent к SHOULD NOT

Notes:

- `следует` — modal: «X следует verify Y».
- `рекомендуется` — passive equivalent: «Рекомендуется verify Y».
- `предпочтительно` — adverbial equivalent: «Предпочтительно использовать Y».

- `нежелательно` — equivalent NOT RECOMMENDED, lower-emphatic чем «не рекомендуется».

§2.3.3 Optional level

RFC-2119	RU canonical (primary)	RU canonical (synonym)	Semantics
`MAY`	может	допускается	Truly optional; no preference
`OPTIONAL`	опционален / опционально	необязателен / по выбору	Equivalent к MAY (adjective form)

Notes:

- `может` — modal: «X может содержать Y».
- `допускается` — passive: «Допускается включить Y».
- `опционально` — adverb (часто в YAML schema descriptions): « `field: optional` → опционально».
- `необязателен` — adjective negation REQUIRED, equivalent OPTIONAL: «Поле X необязательно».
- `по выбору` — phrase для optional choices: «Bucket assignment по выбору editor».

§2.3.4 Negation table — canonical patterns

Form	RU canonical	Discrimination
MUST NOT	не должен / запрещено / недопустимо	Hardcoded prohibition
SHALL NOT	не должен / запрещено	Equivalent MUST NOT
SHOULD NOT	не следует / не рекомендуется	Strong negative recommendation
NOT RECOMMENDED	не рекомендуется / нежелательно	Equivalent SHOULD NOT
(MAY NOT — non-canonical RFC)	не требуется (как absence-of-requirement)	RFC 2119 не содержит MAY NOT; «не требуется» = «MAY omit»

Hard rule: «не должен» (MUST NOT) ≠ «не следует» (SHOULD NOT). Editor никогда не подменяет одну форму другой «для читаемости».

§2.4 Tense / mood normative rules

Phase 1 audit (style-inconsistencies §3.6) выявил: 67 present-copula + 52 imperative-mood + 15 future-tense. §2.4 фиксирует convention для каждого register.

§2.4.1 Normative clauses → imperative-mood

Rule: normative requirements используют imperative-mood (`должен/обязан/следует/может/...`). Indicative present (`X выполняет Y` , `X is Y`) — для descriptive prose только.

Canonical:

```
SR должен прослеживаться до родительского BR через `parent:` link.
```

Anti-pattern:

```
SR прослеживается до родительского BR через `parent:` link.
```

Indicative form читается как **statement of fact**, не как requirement. Reader не знает, является ли это constraint, который должен соблюдаться, или observation.

§2.4.2 Definitions → predicative form

Rule: definitions используют RU canonical predicative form `X — Y, нормированное Z` , где Y — категориальный класс, Z — distinguishing characteristic. Эта форма доминирует в `../standard/04-terms.md` и принята за canonical exemplar (editorial-audit §3 canonical voice).

Canonical:

```
ADAPT — двусторонняя адаптация ТЗ, нормированная процессом
согласования с заказчиком.
```

Acceptable variants:

- `X — это Y, …` (с подчёркивающим частицей «это»).
- `X — Y` (без `это` , если context самостоятельно сигналит definition).

Anti-pattern:

- `X является Y` — formal-overhead; допустим в legal-style, но reads heavy.
- `X представляет собой Y` — anglicism-style копи verb chains; rewrite в `X — Y`.

Discrimination: «является» (67 occurrences corpus-wide) — допустим в low-density (≤2 per chapter), rewrite when ≥3 occurrences per § (excess formality). Phase 5 chapter pass — per-occurrence judgement.

§2.4.3 Future tense — scope-limited

Rule: future-tense (`будет/будут` , 15 occurrences corpus-wide) **запрещён** в normative clauses. Допустим только в:

1. **Roadmap sections** — `../standard/00 §0.7` (planned chapters), version-history sections.
2. **Forward-looking limitations** — «Phase 5 chapter pass [будет] применять §1.13 validation».
3. **Conditional consequences** — «при нарушении X — verify носителя [будет] flag clause as drift».

Anti-pattern: normative requirements в future tense.

```
# WRONG (semantic ambiguous — это требование или прогноз?):
SR будет содержать parent link.

# CORRECT:
SR должен содержать parent link.
```

§2.4.4 Conditional clauses

Rule: conditional normative requirements (`if X then Y must Z`) RU pattern:

```
Если X, то Y должен Z.
```

Acceptable variants:

- `При условии X — Y должен Z.`
- `Когда X, Y обязан Z.`

- `X влечёт обязательство Y по Z.` (formal стиль домена носителя).

Anti-pattern: conditional + indicative («Если X, то Y делает Z») — теряется normative weight.

§2.5 Strict semantic preservation rule

Hard rule (per §2.1.2): Phase 5 editorial pass **не имеет права** менять RFC-2119 level clause's modal verb.

§2.5.1 Allowed transformations

✅ Permitted (semantic-preserving):

From	To	Reason
`должен`	`обязан`	Same level (MUST); synonym choice editorial
`следует`	`рекомендуется`	Same level (SHOULD); synonym choice editorial
`может`	`допускается`	Same level (MAY); synonym choice editorial
`необходимо`	`требуется`	Same level (MUST); same passive-voice register
`не должен`	`запрещено`	Same level (MUST NOT); emphatic synonym

§2.5.2 Forbidden transformations

❌ Forbidden (semantic-altering — это **content change**, не editorial):

From	To	Reason
`должен`	`следует`	MUST → SHOULD downgrade
`следует`	`может`	SHOULD → MAY downgrade
`должен`	`может`	MUST → MAY downgrade
`не должен`	`не следует`	MUST NOT → SHOULD NOT downgrade
Любое modal	indicative («X делает Y»)	Loss of normative weight

§2.5.3 Verification protocol

Phase 5 chapter pass:

1. **Pre-pass scan:** grep modal verbs в главе, count per level (MUST / SHOULD / MAY).
2. **Post-pass scan:** re-grep, compare counts.

3. **Delta acceptance:** sum-per-level не должна меняться. Если меняется — flag as **semantic drift**, revert, surface для content-review задачи.

§2.6 MVR table migration plan

`../standard/00-introduction.md §0.5` — closed-list MVR-1..MVR-7 таблица с 10 SHALL.

§2.6.1 Current state (Phase 1 baseline)

MVR	Current wording (excerpt)	RFC-2119 keyword
MVR-1	«... SHALL be unambiguously identifiable ...»	SHALL
MVR-2	«... SHALL trace to parent ...»	SHALL
MVR-3	«... SHALL ...»	SHALL
MVR-4	«... SHALL ...»	SHALL
MVR-5	«... SHALL ...»	SHALL
MVR-6	«... SHALL ...»	SHALL
MVR-7	«... SHALL ...»	SHALL

(Note: actual MVR text per `../standard/00-introduction.md` §0.5; Phase 1 audit зафиксировал 9 SHALL + 1 MUST в `standard/00` (10 total RFC-2119 keywords).)

§2.6.2 Migration target

MVR	Migration target (RU canonical)
SHALL be unambiguously identifiable	обязан иметь однозначный идентификатор
SHALL trace to parent	должен прослеживаться до родителя
SHALL ...	должен ... / обязан ... (по contract semantics)

Convention: для MVR table — `обязан` preferred (tone контрактного обязательства носителя), `должен` — fallback when «обязан» reads awkward.

§2.6.3 Migration procedure

Phase 5 chapter pass на `../standard/00-introduction.md` :

1. **Pre-migration snapshot:** save current MVR table verbatim.
2. **Translate per §2.6.2 mapping** — manual per-row.
3. **Cross-ref check:** какие задачи / другие главы ссылаются на MVR (по EN keyword) — обновить refs.
4. **ISO 29148 cross-link:** добавить footnote `[1]` к таблице — «Каждый MVR соответствует RFC-2119 SHALL per ISO/IEC/IEEE 29148:2018 §5.2.1.» Cross-link to `../standard/14-normative-refs.md`.
5. **Quality check:** `node scripts/validate-frontmatter.js` + reread end-to-end (memory #31 принцип 6).

§2.7 reference/04 descriptive carve-out

`04-ai-style-guide.md` line 171 содержит 3 EN RFC-2119 keywords (`MUST NOT=1` , `MUST=2`) в **descriptive** контексте (mentions RFC 2119 vocabulary, не uses normatively).

§2.7.1 Rule

Descriptive mention RFC-2119 keywords в reference / guide / core главах — **разрешено**, при условии:

1. Контекст явно descriptive: «RFC 2119 определяет `MUST` как absolute requirement» — это про-RFC reference, не RENAR normative clause.
2. Keyword обёрнут в `code-fence` (`` `MUST` ``) либо в blockquote / EN-citation context.
3. Phase 5 chapter pass distinguishes — descriptive mention не migrates на «должен», keeps EN form.

§2.7.2 Treatment в `reference/04`

`04-ai-style-guide.md` line ~171 — keep EN UPPERCASE forms (descriptive context). Phase 5 chapter pass не трогает.

Verification: Style Guide §2.7 explicit carve-out; subagent invocations в Phase 5 на reference/04 — pass §2.7 as constraint.

§2.8 EN UPPERCASE migration plan (15 occurrences)

Phase 1 audit (style-inconsistencies §3.1) фиксирует 15 EN UPPERCASE в 3 файлах. Per §2.6-§2.7 — migration / carve-out:

File	EN total	Treatment	Status

`../standard/00-introduction.md`	10 (1 MUST + 9 SHALL)	Migrate per §2.6 → RU lowercase	Pending Phase 5
`../standard/01-scope.md`	2 (1 MUST + 1 SHALL)	Migrate → RU lowercase	Pending Phase 5
`04-ai-style-guide.md`	3 (1 MUST NOT + 2 MUST)	Keep — descriptive carve-out §2.7	No action

Net migration: 12 occurrences → RU lowercase; 3 keep EN (descriptive).

§2.8.1 Phase 5 priority order

Per editorial-audit §6 — recommended order:

1. `01-glossary.md` (Phase 1.5 reconciliation prerequisite).
2. `../standard/01-scope.md` (2 MUST/SHALL — small-scope migration combined with other Phase 5 work).
3. `../standard/00-introduction.md` (10 SHALL → §2.6 MVR migration — substantial work, dedicated commit).

`04-ai-style-guide.md` — отдельная задача (descriptive carve-out documentation), low priority.

§2.8.2 Verification post-migration

After Phase 5 migration:

1. Grep `\b(MUST|SHALL|SHOULD|MAY)\b` in `standard/` — should return only `reference/04` matches (descriptive).
2. Grep `\b(должен|обязан|следует|может|рекомендуется|допускается|запрещено)\b` in `standard/` — expected count ≥276 (current 261 + ~15 migrated).
3. Manual diff review per memory #31 принцип 6 (word-count delta sanity).

§2.9 Negation patterns — canonical forms

§2.3.4 фиксирует negation forms. §2.9 expanding на edge cases.

§2.9.1 Direct negation pattern

Positive form	Direct negation
должен Y	не должен Y

обязан Y	не обязан Y / освобождён от Y
следует Y	не следует Y
может Y	не может Y / лишён права Y
допускается Y	не допускается Y / запрещено Y
требуется Y	не требуется Y

§2.9.2 Idiomatic negation

Idiom	Discrimination
`запрещено Y`	Absolute prohibition (MUST NOT level), emphatic. Used for hard rules.
`недопустимо Y`	Equivalent `запрещено`, normative-flavored adjective.
`не подлежит Y`	Specialized: removes object from scope of Y. «X не подлежит modification» = MUST NOT modify X.
`исключается Y`	Removes Y from consideration; ≈ MUST NOT, applied to actions / options.

§2.9.3 Negation anti-patterns

❌ Anti-patterns to avoid:

- `должен не Y` (orphan negation) — replace with `не должен Y` (proper Russian) or `должен исключить Y` (rephrase).
- `не разрешён Y` — non-canonical, prefer `не допускается Y` или `запрещено Y`.
- Double negation in normative («не может не Y») — replace with positive («должен Y»).
- `обязан не Y` — replace with `не должен Y` или `запрещено Y`.

§2.10 Validation

Phase 5 chapter pass обязан выполнить per главу:

§2.10.1 Automated checks

1. **EN UPPERCASE residual scan:** grep `\b(MUST|MUST NOT|SHALL|SHALL NOT|SHOULD|SHOULD NOT|MAY|RECOMMENDED|NOT`

`RECOMMENDED|REQUIRED|OPTIONAL)\b` — expected: 0 в `standard/`, controlled count в `reference/04` (descriptive).

2. **RU UPPERCASE scan:** grep `\b(ОБЯЗАН|ДОЛЖЕН|СЛЕДУЕТ|МОЖЕТ|ЗАПРЕЩЕНО|РЕКОМЕНДУЕТСЯ|ДОПУСКАЕТСЯ|ТРЕБУЕТСЯ)\b` — expected: 0 corpus-wide (§2.1.4).
3. **Modal-verb level integrity:** per-level counts (`должен/обязан/...` MUST; `следует/рекомендуется` SHOULD; `может/допускается` MAY) сохраняются ±0 после Phase 5 pass per chapter.
4. **Future-tense scan:** grep `\bбудет\b|\bбудут\b` в normative clauses — expected only in roadmap / forward-looking sections (§2.4.3).
5. **Indicative drift scan:** grep normative clauses without modal verb — manual review case-by-case (heuristic; precision ~70% per Phase 1 audit).

§2.10.2 Human spot-check

Per memory #31 принцип 8 — каждый Phase 5 chapter pass проходит spot-check на random 10% modal-verb правок. Spot-check проверяет:

- (a) Modal verb level preserved (§2.5).
- (b) Imperative-mood maintained для normative clauses (§2.4.1).
- (c) Definition form canonical (§2.4.2).
- (d) Future-tense scope (§2.4.3).

§2.10.3 Phase 3 pilot checkpoint

Pilot chapter (recommend: `../standard/04-terms.md` per editorial-audit §3 canonical-voice exemplar):

1. **Apply §2 ruleset** на pilot.
2. **Re-run §2.10.1 automated checks** — verify counts match expectation.
3. **User checkpoint:** review pilot diff (modal-verb level integrity, no semantic downgrade).
4. **Lock-in §2** after pilot approval; Phase 5 commences.

§2.11 Subagent prompt template

For Phase 5 chapter pass — subagent invocations need explicit §2 reference. Template:

```
You are performing RU editorial pass on RENAR Standard chapter <X>.

PRIMARY POLICY: reference/06-ru-style-guide.md §2 (RFC-2119 RU
```

```
wording).

HARD RULES from §2:
- §2.1.4: NO UPPERCASE Russian modals (ОБЯЗАН/ДОЛЖЕН/...). Forbidden.
- §2.2: RU lowercase modals (должен/следует/может/...) are CANONICAL
RFC-2119 equivalents.
- §2.5: NO modal verb LEVEL changes (MUST/SHOULD/MAY discrimination
preserved).
   - "должен" → "следует" — FORBIDDEN (downgrade).
   - "должен" → "обязан" — OK (same level, synonym choice).
- §2.4.1: Normative clauses use IMPERATIVE-mood (modal verb).
Indicative ("X выполняет Y") not for requirements.
- §2.4.3: Future tense ("будет/будут") forbidden in normative
clauses.

AI BIAS GUARD: LLM trained on English skews toward "polishing" via
semantic downgrade. Watch own output.

Output: per-clause diff with §2 rule cited.
```

---

§3 Formatting conventions

> ***Source:*** *`../research/ru-style-guide-draft-section-3-formatting.md` (Phase 2 source artifact).*

§3.0 Назначение

§3 фиксирует **canonical formatting conventions** для RU normative корпуса RENAR Standard. Регламентирует:

- citation conventions (bare `§X.Y` / markdown link / inline «глав* X») — закрывает Phase 1 F3;
- code-fence language tag policy (whitelist + plain-text + markdown-examples);
- heading conventions (standard/ dotted vs guide/ plain split + depth rule);
- tables / lists de-facto convention endorsement (bullets `-`, alignment-less `---`);
- inline code (backticks) vs emphasis (bold / italic) conventions;
- frontmatter YAML formatting (field order, quoting, lang field);

- RU typography (кавычки `«»`, em-dash, non-breaking space, числительные форматы);
- cross-file link conventions (relative paths, anchor case, dead-link protection).

§3 — **normative**. Phase 5 chapter pass обязан валидировать соответствие §3 по каждой главе (см. §3.10 Validation).

§3.1 Принципы

Шесть нормативных принципов §3, применяемых ко всем formatting decisions:

§3.1.1 Читаемость носителем первична

Markdown — читаемый носителем текстовый формат. Formatting decisions должны:

- сохранять git-blame readability (no force-wrap mid-sentence per character-count),
- не ломать syntax highlighting в site/PDF rendering,
- быть стабильными при нативной для носителя обработке (markdown lint, frontmatter validator).

Применение: see §3.3 (fences must have lang tags для PDF), §3.7 (frontmatter must validate per `scripts/validate-frontmatter.js`), §3.9 (links must use anchor format compatible с Astro static-site generator).

§3.1.2 Scan-readable вторичен

Reader сканирует normative document; формат — service of scanning:

- numbered sections для cross-ref (§3.4 dotted-number convention);
- consistent bullets (§3.5 `-` only);
- typographic quotes для readability (§3.8 `«»`).

Применение: see §3.4, §3.5, §3.8.

§3.1.3 No tool dependency

Convention должна быть applicable manual editor (Markdown в Notepad++ / VS Code / любой text editor). Не должна требовать specific IDE plugin.

Применение: §3.3 fence lang tags — manually addable; §3.7 frontmatter — validated post-edit, не at-edit; §3.8 typography — soft rule (replacement scripts допустимы pre-commit, не in-flow).

§3.1.4 Accessibility

PDF / HTML / screen-reader compatibility:

- alt-text для images (TBD when imagery introduced — Phase 6/7 epic);
- semantic heading hierarchy (no skip, H1 → H2 → H3 monotonic);
- typed code-fences для syntax-highlighting.

Применение: §3.3 fence lang policy + §3.4 heading depth rule.

§3.1.5 De-facto convention endorsement

Phase 1 audit зафиксировал 100% bullet consistency (`-` only) + 100% table alignment-less (`---`) + 0 heading depth jumps. Style Guide §3 **endorses** эти de-facto conventions explicitly без forced revisitation.

Применение: §3.5 — explicit endorsement.

§3.1.6 RU typography first-class

Не трогать RU prose с ASCII-only mindset. `«»` (typographic quotes), `—` (em-dash), non-breaking space — RU normative typography baseline.

Применение: §3.8 — explicit rules.

§3.2 Citations & cross-refs

Phase 1 audit (style-inconsistencies §3.2): 1 264 `§X.Y` refs total, 416 bare (32%), 1 041 markdown-file-links, 86 anchor-only, 152 inline «глав* X», 98 «см.», 0 «see».

§3.2.1 Convention — three classes

Class	Pattern	When
(A) Bare intra-file	`§3.5`	Reference внутри того же файла
(B) Markdown link cross-file	`[§4.5](../standard/04-terms.md#4.5)`	Reference на другой файл
(C) Anchor-only intra-file	`[§3.5](#35-tables--lists)`	Clickable intra-file reference (explicit anchor)

Hard rule: cross-file reference **обязан** быть markdown link (Class B). Bare `§X.Y` для cross-file ссылки — anti-pattern (не clickable, no PDF outline integration).

Soft rule: intra-file reference — Class A или Class C по choice. Class A — minimal syntactic overhead (acceptable in dense reference sections); Class C — clickable, better for sites/PDF.

§3.2.2 Inline «глав* X» pattern

Form	Treatment
«в главе 4»	Canonical для cross-file narrative reference
«глава 4 содержит» (no link)	Anti-pattern when cross-file; convert to (B)
«глава 4» when intra-file context already clear	Acceptable (Class A equivalent)

Phase 5 action: 152 inline «глав* X» occurrences (style-inconsistencies §3.2) — review per occurrence; cross-file → convert to link.

§3.2.3 «См.» (RU) — endorsed

Convention: «см.» (lowercase, RU) — canonical. «see» (EN) — banned. Phase 1 audit: 98 «см.», 0 «see». **Endorse de-facto.**

Pattern:

```
... (см. [§3.5](#35-tables--lists)).
```

«См.» allowed без link, когда intra-file context (parenthetic mention without need to navigate).

§3.2.4 Anchor format

Heading	Anchor (auto-generated by Astro/CommonMark)
`## §3.5 Tables / lists`	`#35-tables--lists`
`### §1.10.3 Affected file (Phase 5 priority 1)`	`#1103-affected-file-phase-5-priority-1`
`## Назначение`	`#назначение`

Rule: anchors lowercase, hyphens вместо spaces/dots, RU keywords lowercase Cyrillic.

Hard rule: anchor validity verified post-edit via cross-ref dead-link scan (memory #31 принцип 6).

§3.2.5 Phase 5 migration priorities

Per editorial-audit §6 — citation cleanup priority:

Order	File	Bare §-refs	Treatment
1	`../standard/01-scope.md`	69	Highest — intro chapter, many cross-file refs
2	`../standard/12-metrics.md`	48	Medium-high
3	`../standard/04-terms.md`	39	Medium — terminology cross-refs (intra-file often OK)
4	`../standard/13-conformance.md`	38	Medium
5	`../standard/10-lifecycle-qg.md`	35	Medium
6+	Remaining files	low-medium	Per-chapter as encountered

§3.3 Code-fences

Phase 1 audit (style-inconsistencies §3.3): 195 total fences, 77 no-lang (39%), 118 typed. Top no-lang offenders: `guide/01` (20), `guide/08` (13).

§3.3.1 Language tag whitelist

Допустимые language tags (endorse Phase 1 de-facto):

Tag	Purpose	Phase 1 count
`yaml`	frontmatter / config examples	64
`bash`	CLI commands / shell snippets	15
`cypher`	Knowledge graph queries (reference/05)	15
`markdown`	Markdown examples внутри docs	14
`json`	JSON schema examples / API payloads	7
`python`	Python script examples	2
`sql`	SQL queries	1
`text`	Plain text / tree fragments / generic output	(new — replaces no-lang)

Closed list policy (§1.1.4): добавление нового tag — formal Style Guide amendment. Phase 5 chapter pass не выбирает «exotic» tags.

§3.3.2 No-lang fences — anti-pattern

Hard rule: все fences обязаны иметь language tag. No-lang fences — anti-pattern.

Treatment per content type:

Content type	Lang tag
YAML frontmatter / config	`yaml`
Shell command / output	`bash`
Markdown structural example	`markdown`
Directory tree fragment	`text`
Plain output dump (e.g. tool output)	`text`
JSON payload	`json`
SQL query	`sql`
Cypher query	`cypher`
Python script	`python`
Mixed-language pseudo-code	`text`

§3.3.3 Migration plan для 77 no-lang occurrences

Per editorial-audit §6 priority:

Order	File	No-lang fences	Mostly
1	`../guide/01-walkthrough.md`	20	Tree fragments + directory layouts → `text`
2	`../guide/08-developer-guide.md`	13	Mixed CLI / tree → `bash` или `text`
3	`../standard/06-requirements-hierarchy.md`	7	Identifier schema examples → `yaml` или `text`
4	`../standard/10-lifecycle-qg.md`	6	State machine examples → `text`

5+	Remaining 8 files	1-5 each	Per-file judgement

Phase 5 procedure: для каждого no-lang fence — read 2-3 lines content, choose lang tag per §3.3.2 mapping, replace ``` → ```<tag>. No content change.

§3.3.4 Indentation policy внутри fence

Rule	Detail
YAML frontmatter inside fence	2-space indent (matches existing corpus)
Bash commands	No leading whitespace (each command starts at column 0 inside fence)
Tree fragment	Indentation reflects directory depth (4 spaces per level recommended)
Markdown example	Preserve markdown formatting authentically (no escape)

§3.3.5 Inline code (backticks)

Single backticks для:

- Identifiers: `SR-001`, `commit`, `frontmatter`.
- File / path references: `standard/04-terms.md`, `reference/01-glossary.md`.
- Type / field names: `parent: BR-001`, `status: approved`.
- Short CLI fragment: `git commit -m "..."`.

Anti-pattern: backticks вокруг whole sentences («This is `inline-coded sentence`»). Reserve backticks для technical identifiers.

Double-backticks `` ` ` `` — для literal backtick-containing content (rare).

§3.4 Headings

Phase 1 audit (style-inconsistencies §3.4): 1012 total headings, max depth H4 (3 occurrences), depth jumps 0 (monotonic).

§3.4.1 Numbering convention — endorse split

Directory	Convention	Rationale

`standard/`	Dotted-number (`## 3.5 Tables / lists`)	Normative clauses — numbered for cross-ref precision
`reference/`	Dotted-number (matches standard/ — reference is normative companion)	Same as standard/
`core/`	Plain (`## Что вы получите`)	Pedagogical narrative
`guide/`	Plain (`## Quickstart` / numbered «Phase 0..9»)	Narrative + numbered phase walkthroughs
`research/`	Plain (or numbered if explicit structure preferred per author)	Research drafts — author choice

Endorse Phase 1 de-facto (style-inconsistencies §3.4): 746 dotted-number / 266 plain. Most `standard/` dotted; `guide/00` , `guide/03` , `guide/08` , `reference/02-03` , `core/renar-core` plain.

§3.4.2 Depth rule

Rule	Detail
Max depth normative	H3 (3 levels)
H4 — exception	Allowed только для substructure внутри large H3 sections; ≤5 per chapter
H5/H6 — banned	Restructure parent H3/H4 instead
Depth jumps	**Banned** — no H1 → H3 без intermediate H2. Phase 1 audit: 0 jumps (excellent baseline).

Phase 5 chapter pass — preserve depth monotonicity. Если правка добавляет H4 — verify ≤5 per chapter.

§3.4.3 Heading text — RU canonical

Pattern	Examples
Noun phrase	`## §3.5 Tables / lists` (mixed RU/EN — EN domain word in title acceptable)
Subject-predicate	`## Что вы получите` (guide / core narrative)
Imperative	`## Установите зависимости` (guide / core)

Anti-pattern: sentence-as-heading («## Этот раздел описывает основные принципы»). Heading — short label, not sentence.

§3.4.4 RU casing

Headings — sentence-case RU (capitalize only first word + proper nouns). NOT title-case English («## Tables And Lists»).

Exception: `## §3.5 Tables / lists` — mixed-script when first word — number + EN domain term acceptable per §1.3 Bucket A.

§3.5 Tables / lists

Phase 1 audit (style-inconsistencies §3.5): 100% bullet consistency (`-` only), 100% table alignment-less (`---`), 0 mixed-bullet files.

§3.5.1 Bullet style — endorse `-` only

Hard rule: `-` only. `*` / `+` — banned. Mixed bullets within file — banned.

```
- item 1
- item 2
  - sub-item (4-space nested indent)
- item 3
```

§3.5.2 Ordered list `1.` — endorse

Phase 1: 274 ordered lists. Convention:

```
1. Step one.
2. Step two.
3. Step three.
```

Auto-numbered (`1.` для каждого item) acceptable per CommonMark — renderer auto-numbers. Manual `1.` , `2.` , `3.` — preferred (explicit count visible в source).

§3.5.3 Table alignment — endorse `---` only

Hard rule: alignment markers `:---` / `---:` / `:---:` — banned (Phase 1: 0 occurrences, 644 alignment-less, 100% consistency). Endorse de-facto.

```
| Col A | Col B | Col C |
|---|---|---|
| a1 | b1 | c1 |
```

Rationale: alignment markers вводят rendering variance per renderer; alignment-less — universal.

§3.5.4 Nested lists indent

Type	Indent
Unordered nesting	2 spaces (matches CommonMark default) — `-` continues at offset 2
Ordered nesting	3 spaces (matches CommonMark default) — `1.` continues at offset 3

Phase 5 chapter pass — verify indent consistency per file (Phase 1 не quantified, default assumption: existing corpus consistent).

§3.5.5 Table header normalization

Rule	Detail
First column — left-justified by default (no alignment marker)	Most readable
Right-justified columns — допустимо когда numerical column (Phase 1: 0 occurrences — endorse banning)	
Header row — bold не required (renderer adds visual weight automatically)	

§3.6 Inline code & emphasis

§3.6.1 Inline code (backticks)

См. §3.3.5.

§3.6.2 Bold — `**text**`

Использовать для:

- Term introduction (first definition mention): «**Closed list policy** — формальный механизм ...».
- Emphatic normative («**Hard rule:** ...»).
- Negative scenario flag («**Anti-pattern:** ...»).

Hard rule: bold **не используется** как replacement для UPPERCASE (§2.1.4). Bold — additive emphasis, не shouting.

§3.6.3 Italic — `*text*`

Restricted use:

- Foreign-language phrase: «*ad hoc*», «*de facto*» (latin).
- Title of external work: «*Software Requirements* (Wiegers)».
- Technical term in introduction где bold уже used: «*RFC 2119* — IETF RFC».

Anti-pattern: italic для general emphasis (use bold). Italic для длинных passages (читаемость падает).

§3.6.4 Code-fence vs inline-code discrimination

Content	Inline (backticks)	Fence
Single identifier (`commit` , `frontmatter`)	✓	Excess
Path (`standard/04-terms.md`)	✓	Excess (unless context-bearing)
Short command (≤80 chars, no newlines)	✓	Optional
Multi-line command / multi-statement	Excess	✓
YAML frontmatter example	Excess	✓
Markdown structural example	Excess	✓

§3.7 frontmatter YAML

§3.7.1 Field order

Canonical field order per RENAR convention (matches existing `research/` + `standard/` corpus):

```yaml
---
title: "Document title"
status: draft  # | proposed | approved | verified | obsolete | frozen | ...
phase: "Phase N description"  # optional, epic-context
epic: epic-slug  # optional
task: task-slug  # optional
draft-date: 2026-05-16  # ISO 8601
scan-date: 2026-05-16  # для audit artifacts
target-final-path: "reference/06-...md"  # для drafts с known lock-in target
lang: ru  # primary language
sources: 3  # для audit aggregations
corpus: "standard/ + guide/ + reference/ + core/"  # для audits scope
---
```

§3.7.2 Quoting policy

Value type	Quoting
Pure ASCII identifier (`status: draft` , `lang: ru` , `epic: ru-normative-pass-v1`)	No quotes
String с пробелами или special chars (`title: "..."` , `phase: "..."`)	Double quotes
ISO date (`2026-05-16`)	No quotes
Numeric (sources: 3)	No quotes
Boolean (rare, e.g. `frozen: true`)	No quotes

§3.7.3 Required fields per artifact type

Type	Required
`standard/*.md`	`title` , `lang` , optionally `status`
`guide/*.md`	`title` , `lang` , optionally `status`
`reference/*.md`	`title` , `lang` , optionally `status`

`research/*-audit.md`	`title`, `status` (`frozen` after publish), `phase`, `epic`, `task`, `scan-date`, `corpus`, `lang`
`research/*-draft-*.md`	`title`, `status` (`draft`), `phase`, `epic`, `task`, `draft-date`, `target-final-path`, `lang`

§3.7.4 Validation

`node scripts/validate-frontmatter.js` runs as quality signal (Phase 5 per chapter, also pre-commit). Validates: required-field presence + type correctness + closed-value-list (status, lang).

Phase 5 expectation: 30/30 чек должен остаться («frontmatter validator clean»). Если стандарт растёт — validator принимает новые files automatically.

§3.7.5 `lang:` field — closed list

Currently allowed: `ru` (primary), `en` (planned для `standard/en/` Phase 6/7).

Future-proof: при добавлении EN translations — validator должен distinguish `lang: ru` vs `lang: en` artifacts.

§3.8 RU typography

§3.8.1 Кавычки — «»

Canonical: ёлочки `«»` для outer quotes. `"..."` (ASCII straight) — anti-pattern в prose.

```
# Correct
… concept «closed list policy» — formal RENAR mechanism …

# Wrong
… concept "closed list policy" — formal RENAR mechanism …
```

Inner quotes (nested): `„..."` (German low-9 + high-9 quotes) — when sufficient. Phase 1 audit corpus — inner quotes rare; defer to author discretion case-by-case.

Exceptions: ASCII quotes допустимы:

- Внутри code-fences (preserve verbatim).
- В URL / file paths (preserve syntactic).
- В EN citation внутри RU prose («Per RFC 2119 §2, "the use of ..."»).

§3.8.2 Em-dash — `—`

Canonical: em-dash `—` (U+2014) для:

- Парные em-dashes — separators в complex sentences («Term X — described as Y — applies …»).
- Definition predicate: «X — Y» (§2.4.2 canonical form).
- List-item separator: «item A — description».

Anti-pattern: `-` (hyphen) или `--` (double-hyphen) instead of `—`.

Exceptions:

- Hyphen `-` correct в compound words («closed-list», «source-of-truth»).
- Hyphen `-` correct в file paths / identifiers (`ru-style-rfc2119`).
- En-dash `–` (U+2013) для number ranges («§3.5–3.7») — rare, acceptable.

§3.8.3 Non-breaking space

Required:

- Между числом и единицей: `100 ms` (NBSP between 100 and ms), `≥1 chapter`.
- Между initials и фамилией: «И. М. Сеченов» (NBSP after И.).
- Перед em-dash в pair: «слово — описание» (NBSP перед —).

Implementation: markdown source может содержать literal NBSP (U+00A0) или HTML entity ` `. Astro / CommonMark renders correctly.

Phase 5 — soft rule: NBSP insertion — author discretion; не gated check.

§3.8.4 Числительные форматы

Form	Convention
Whole numbers ≤999	No separator: `42`, `999`
Whole numbers ≥1000	Space separator: `1 000`, `50 649`
Percentages	`33%` (no space между number и `%`)
Decimals (rare)	Russian decimal comma `0,5` или English period `0.5` — pick one per file; corpus default English period

Ordinals	`1-й` , `2-й` (RU suffix) или `1st/2nd` (EN, rare)

Phase 1 baseline: `50 649` (space separator) used в editorial-audit. Endorse.

§3.8.5 Math operators

Symbol	Use
`≥` (U+2265) / `≤` (U+2264)	Inequality
`≠` (U+2260)	Not equal
`±` (U+00B1)	Plus-minus
`→` (U+2192)	Implication / transformation
`↔` (U+2194)	Bidirectional

Plain ASCII `>=` , `<=` , `!=` , `+/-` , `->` , `<->` — anti-pattern в prose; acceptable inside code-fences.

§3.9 Cross-file link conventions

§3.9.1 Relative paths

Cross-file links use **relative** paths from the linking file's directory:

```
# from reference/06-ru-style-guide.md
[§4.5 standard/04](../standard/04-terms.md#4.5)
[reference sibling](01-glossary.md)
```

Anti-pattern: absolute paths (`/standard/04-terms.md`) — break при site build / portable usage.

§3.9.2 Anchor case

Anchor portion lowercase, hyphens, RU keywords Cyrillic. См. §3.2.4.

§3.9.3 Link label conventions

Pattern	Use
`[§3.5](file.md#35)`	Cross-ref to numbered section

`[term name](file.md#term-name)`	Cross-ref to RU heading (sentence-case lowercase)
`[file path](path)`	Reference to file (label = path verbatim в backticks) — `[`standard/04-terms.md`](../standard/04-terms.md)`

§3.9.4 Dead-link prevention

Phase 5 chapter pass — после правки запустить cross-ref dead-link scan (memory #31 принцип 6). Expected: 0 dead links.

Tooling: Astro build обычно flags broken links; CI gate (future) verify pre-merge.

§3.10 Validation

Phase 5 chapter pass обязан выполнить per главу:

§3.10.1 Automated checks

1. **Bare §-ref scan:** grep `§\d+\.\d+` не внутри `[...]` — flag для cross-file refs.
2. **No-lang fence scan:** grep ```` ``` ```` followed by newline (no lang tag) — flag every occurrence.
3. **Heading depth scan:** count H1/H2/H3/H4 — verify monotonic + H4 ≤5.
4. **Bullet style scan:** grep `^\s*[*+]\s` — flag (must be `-` only).
5. **Table alignment scan:** grep `:---|---:|:---:` — flag (must be `---` only).
6. **ASCII-quote scan:** grep `[^/]\"[^\"]+\"[^/]` в prose (exclude code-fences) — flag (must be `«»`).
7. **Hyphen-as-em-dash scan:** grep `--` или ` - ` (with surrounding spaces, prose-context) — flag (must be `—`).
8. **frontmatter validation:** run `node scripts/validate-frontmatter.js`.

§3.10.2 Human spot-check

Per memory #31 принцип 8:

- (a) Anchor cases verified (no broken intra-file anchors).
- (b) Citation classes (§3.2.1) applied correctly.
- (c) Typography (§3.8) applied without over-editing existing correct usage.
- (d) frontmatter — required fields present, status correct.

§3.10.3 Phase 3 pilot checkpoint

Pilot chapter — `../standard/04-terms.md` (canonical voice exemplar):

1. **Apply §3 ruleset** в pilot.
2. **Re-run §3.10.1 automated checks** — verify counts drop.
3. **User checkpoint:** review pilot diff (citation conversion + fence tagging + typography normalization).
4. **Lock-in §3** after pilot approval; Phase 5 commences.

§3.11 Phase 5 priority order (combined §3 axes)

Per editorial-audit §6 + §3.2.5 / §3.3.3:

Order	File	Combined formatting issues
1	`01-glossary.md`	Phase 1.5 prerequisite (F1 reconciliation) + per-§3 cleanup
2	`../standard/01-scope.md`	69 bare §-refs (top citation hot)
3	`../standard/12-metrics.md`	48 bare §-refs
4	`../standard/04-terms.md`	39 bare §-refs (intra-file часто OK — judgement-heavy pass)
5	`../standard/13-conformance.md`	38 bare §-refs
6	`../standard/10-lifecycle-qg.md`	35 bare §-refs + 6 no-lang fences
7	`../guide/01-walkthrough.md`	20 no-lang fences (top fence hot)
8	`../guide/08-developer-guide.md`	13 no-lang fences
9	`../standard/06-requirements-hierarchy.md`	7 no-lang fences
10+	Remaining files	Per-chapter as encountered

---

§4 Integration with RENAR Standard

§4.1 Closed-list master index

Style Guide v1.0 — **17-й закрытый список** RENAR Standard. Per `../standard/01-scope.md §1.7` closed-list master policy:

#	Closed list	Source
1-9	SPEC family (9 types)	`standard/04 §4.4`
10	TC types (6)	`standard/04 §4.5.2`
11	Quality Gates (canonical v1.0, 5)	`standard/04 §4.7`
12	Lifecycle статусы	`standard/04 §4.6`
13	возможности носителя (V1-V6)	`standard/03`
14	Drift classes (8)	`standard/04 §4.11`
15	MVR-1..MVR-7	`standard/00 §0.5`
16	Artifact types (BR/SR/TR/ADAPT/T3/delta-T3)	`standard/04 §4.3`
17	**Style Guide bucket terminology** (§1.9)	этот документ

Cross-link: `standard/01-scope.md §1.7` обязан включать reference на `reference/06-ru-style-guide.md` как 17-й список при следующем Phase 5 chapter pass.

§4.2 standard/04-terms canonical source

`../standard/04-terms.md` — **single source of truth** для terminology (§1.1.5).

При расхождении между Style Guide §1.9 и standard/04 §4.x:

1. `standard/04` побеждает.
2. Style Guide updated through amendment (§5.2) для re-sync.
3. Phase 5 chapter pass — verifies sync до commit.

Reverse direction: добавление термина в Style Guide §1.9 БЕЗ standard/04 — **запрещено** (§1.1.4 closed list policy).

§4.3 validate-frontmatter integration

`scripts/validate-frontmatter.js` — основной нативный для носителя enforcement для §3.7.

Phase 5 chapter pass invocation:

```
node scripts/validate-frontmatter.js
# expected: 30/30 baseline (per memory #31 принцип 6)
# fail mode: any "Frontmatter validation FAILED" → block commit
```

Style-guide check (`scripts/style-guide-check.js` v0.1.0 — implemented 2026-05-19):

```
node scripts/style-guide-check.js              # all checks, full output
node scripts/style-guide-check.js --quiet      # findings only
node scripts/style-guide-check.js --check=<n>  # one check (see list below)
```

Six checks (closed-list-mirroring §1.3.1 / §1.7.1 / §1.13.1.7-8 / §2.10.1.1 / §3.10.1.2 / §3.10.1.6):

- `bucket-a` — mid-sentence Capitalized Bucket A terms (§1.3 / §1.13.1.1). Heuristic: sentence-start exempt; mid-clause flagged. Mid-sentence Capitalization не явно запрещено §1.3 («остаются латиницей» ≠ «strict lowercase»); findings — advisory pending §1.3 amendment.
- `bucket-e` — rewrite-target кальки (§1.7.1 / §1.13.1.5): `имплементаци*` , `конформаци*` , `эссенциальн*` , `дрифт*` , `трейс*` , `регуляци*` .
- `legacy` — legacy labels (§1.13.1.7-8): `UIC` , `AIC` , `INT-SR` , `INT-TC` , `TM` , `TS` , и legacy QG names `Context Gate` / `Requirements Gate` / `Implementation Gate` / `Verification Gate` . **Amendment 004:** short labels flagged только как current-term misuse — citation contexts (legacy→canonical mapping rows, deprecation/migration markers, migration/legacy section headings с level-inheritance) НЕ flagged.
- `en-uppercase` — RFC-2119 EN keywords в RU prose (§2.10.1.1). Carve-out: `reference/04-ai-style-guide.md` (§2.7.2) и `reference/06-ru-style-guide.md` (this file, §2 self-reference) exempt; chapters с `lang: en` skipped per §2.2.5.
- `fence-lang` — code-fences без lang tag (§3.10.1.2 / §3.3.2).
- `ascii-quote` — ASCII `"..."` в RU prose (§3.10.1.6). EN-citation carve-out: quoted inner content без Cyrillic и ≥3 latin chars skipped per §3.8.1.

Known limitations (script v0.2.0):

- Bucket A check produces 100+ baseline findings due to mid-sentence `frontmatter` capitalization corpus-wide; treat as advisory until §1.3 case-policy clarified.
- Legacy check narrowing (amendment 004) is context-heuristic: marker-word / mapping-row / section-heading based. Genuine misuse внутри migration section suppressed by design (migration sections cite legacy throughout); the over-suppression guard verifies bare misuse вне citation context всё ещё flagged.
- ASCII-quote heuristic conservative; false-negative possible (e.g. RU prose `"..."` где inner content имеет mixed RU/EN).
- Closed lists embedded in script — manual re-sync при Style Guide version bump.

§4.4 Phase 5 chapter pass — full invocation protocol

См. §0.4.1 Application protocol для invocation sequence.

Дополнительно:

§4.4.1 Pre-pass checklist

1. Confirm chapter — нет другого Phase 5 task in progress на ту же главу.
2. `git status` clean — нет uncommitted changes в репозитории.
3. Style Guide §1-§3 loaded (read this artifact end-to-end at start of pass).
4. Phase 3 pilot checkpoint approved (mandatory gate per memory #31 принцип 3).

§4.4.2 Post-pass checklist

1. `node scripts/validate-frontmatter.js` — 30/30 baseline.
2. Cross-ref dead-link scan — 0 dead links.
3. `renar verify --clause <id>` — gates pass (нативная для носителя операция verify).
4. Diff visualization (manual or `git diff --stat`) — no unintended changes.
5. Word-count delta sanity — ±15% threshold.
6. Commit с Style Guide § citation в message.

§4.4.3 Failure modes

- **Semantic drift detected** — revert; surface as content-review task.
- **Validate-frontmatter fail** — investigate root cause; fix; re-run.
- **Dead-link scan fail** — fix anchors; verify; re-run.
- **Word-count delta out-of-range** — investigate (excessive deletion or padding); restore or trim.

§4.5 Subagent invocations

См. §0.4.2 (basic template) + §2.11 (RFC-2119-extended template).

Hard rule: subagent invocation для Phase 5 chapter pass **обязана** включать explicit reference на reference/06-ru-style-guide.md как primary policy. Без этого reference — Phase 5 attempt **rejected** при review.

---

§5 Change history & versioning

§5.1 v1.0 (этот документ — approved)

Date: 2026-05-17 (approved). **Compile-date:** 2026-05-16 (lock-in). **Phase:** Phase 3 pilot validated — Phase 5 chapter pass unblocked. **Compiled from:**

- `../research/ru-style-guide-draft-section-1-terminology.md` (Phase 2 §1 draft, status:draft, commit `5da0832`).
- `../research/ru-style-guide-draft-section-2-rfc2119.md` (Phase 2 §2 draft, status:draft, commit `2e76599`).
- `../research/ru-style-guide-draft-section-3-formatting.md` (Phase 2 §3 draft, status:draft, commit `a7a5d93`).

Status progression:

- `proposed` (Phase 2 lock-in — commit `cc68733`, 2026-05-16).
- **`approved`** (Phase 3 pilot validated — pilot chapter `../standard/04-terms.md`, pilot commit `564c8d4`, 2026-05-17). ← *current*
- `verified` (post-Phase 5 mass application + retrospective audit — TBD).

Phase 3 pilot validation results (decision #32):

- Pilot chapter: `../standard/04-terms.md` (canonical-voice exemplar per editorial-audit §3).
- Pilot commit: `564c8d4` (15 edits / 30 lines / +3 words / +0.097% word delta).
- Edits breakdown: 2 fence lang tags (§3.3.2) + 13 cross-file Class B link conversions (§3.2.1) + 4 lexical rewrites (Bucket E §1.7 + AI-bias §1.14.1) + 1 typo.
- Validation: validate-frontmatter 31/31 ✓, modal-verb level counts preserved per §2.5.3 ✓, 0 semantic drift ✓, 13/13 cross-ref targets verified ✓.

- User checkpoint: approved 2026-05-17 («подтверждаю»), no adjustments requested.

Lock-in signal: canonical voice exemplar received minimal-edit pass — Style Guide §1+§2+§3 validated as non-over-editing. Phase 5 mass application (30 chapters per §3.11) теперь разблокирована.

v1.0 содержание:

Section	Source	Lines (draft)
§0 Introduction	new (lock-in)	~150
§1 Terminology	§1 draft	~530
§2 RFC-2119 RU wording	§2 draft	~490
§3 Formatting	§3 draft	~590
§4 Integration	new (lock-in)	~140
§5 Change history (этот раздел)	new (lock-in)	~80

§5.1.1 v1.0.1 (2026-05-19 — patch)

Type: patch (clarification — §5.3 «no retroactive Phase 5 scan»). **Amendment:** `../research/ru-style-amendment-001-bucket-a-c-fallback.md` — Bucket A/C `fallback` resolution.

Rationale: v1.0 классифицировал термин `fallback` в двух bucket одновременно — §1.5.1 Bucket C («rewrite в prose» → «отступление / запасной вариант») и §1.8 Bucket F triage row («Keep `fallback` (Bucket A) — don't translit»). §1.3.1 Bucket A whitelist при этом термин не содержал. Corpus reality: все 5 instances `fallback` в нормативном корпусе — техническое использование в домене носителя (failure-mode redirect mechanism); 0 hits для RU-проекции.

Delta:

- §1.3.1 Bucket A whitelist — добавлена строка `fallback | SE / RENAR ops | запасной вариант` (после `provenance`).
- §1.5.1 Bucket C replacement table — удалена строка `fallback | отступление / запасной вариант`.
- Frontmatter `version: "1.0"` → `"1.0.1"`.

Pilot impact: 0 affected chapters (corpus уже compliant — Bucket C row была paper-only). **Retroactive Phase 5 scan:** не требуется (§5.3 patch semantics).

§5.1.3 v1.1.1 (2026-05-19 — patch)

Type: patch (clarification — completes Session #21 audit-fix-qg-naming-unify decision). **Amendment:** `../research/ru-style-amendment-003-qg-gate-suffix.md` — QG canonical names "Gate" suffix unify.

Rationale: v1.1 фиксировал canonical форму QG names БЕЗ "Gate" суффикса (§1.9.4 + §1.10.2), но Session #21 (commit `7b0f6ae`, audit-fix-qg-naming-unify) уже унифицировал corpus к WITH-Gate форме (`QG-2 Verification Gate` etc) в 6 файлах. Style Guide §1.13.1.8 grep оф legacy QG names тогда ловил canonical post-Session #21 формы как false positives.

User decision (2026-05-19): «Оставить с Gate — принять Session #21». Canonical form = WITH "Gate" суффикс.

Delta:

- §1.9.4 — canonical list: `QG-0 Approval Gate`, `QG-1 Implementation Gate`, `QG-2 Verification Gate`, `QG-3 Architecture Gate`, `QG-4 Acceptance Gate`.
- §1.10.2 — Canonical v1.0 column обновлена с "Gate" suffix.
- §1.13.1.8 — grep narrowed к prefixed-legacy strings (`QG-0 Context Gate|QG-1 Requirements Gate|QG-2 Implementation Gate|QG-3 Verification Gate`). Bare `Verification Gate` / `Implementation Gate` без QG-prefix — canonical, не flagged.
- `scripts/style-guide-check.js` LEGACY_LABELS — full prefixed legacy strings.
- Frontmatter `version: "1.1"` → `"1.1.1"`.

Pilot impact: N/A — corpus уже compliant per Session #21. §1.13.1.8 false positives устранены. **Follow-up task (deferred):** `ru-qg-gate-suffix-corpus-sweep` — apply canonical update к standard/04-terms.md §3.14.1 (4 rows), standard/05-roles.md (5 mentions), standard/10-lifecycle-qg.md anchors, reference/01-glossary.md (lines 284-290).

§5.1.4 v1.1.2 (2026-05-20 — patch)

Type: patch (tooling + clarification — legacy-label false-positive narrowing + corpus genuine-misuse sweep). **Amendment:** `../research/ru-style-amendment-004-legacy-label-narrowing.md` — legacy-label checker citation narrowing.

Rationale: §1.13.1.7 grep (`UIC|AIC|INT-SR|INT-TC|TM|TS` , flag every occurrence) производил 57 findings, из которых ~39 — legitimate citations (migration sections `standard/06 §6.2.3` + `standard/08 §8.7` , legacy→canonical mapping rows, deprecation prose в glossary/guide), и ~18 — genuine misuse (incomplete Phase 1.5 glossary reconciliation: §2.5/§2.9/§2.10/§2.11 несли legacy labels как current types). Mirrors amendment 003 QG-name narrowing.

Delta:

- §1.13.1.7 — legacy-label check narrowed: flag только current-term misuse; citation contexts (mapping rows, deprecation/migration markers, migration/legacy section headings с level-inheritance) НЕ flagged.
- `scripts/style-guide-check.js` v0.1.0 → v0.2.0 — `checkLegacy` citation-aware (`LEGACY_CITATION_MARKER` + `LEGACY_SECTION_MARKER` + `LEGACY_CANONICAL` mapping-row map). Over-suppression guard: bare legacy label вне citation context всё ещё flagged.
- Corpus genuine-misuse sweep: `reference/01-glossary.md` (§2.5 source col, §2.9 TC col, §2.10 parent, §2.11 file-naming → canonical SPEC-UI/SPEC-AI/SPEC-INT paths, §4 anti-term), `standard/08` , `standard/09` , `reference/05` .
- `guide/08-developer-guide.md` — ASCII-quote `"..."` → `«»` (§3.10.1.6).
- Frontmatter `version: "1.1.1"` → `"1.1.2"` .

Impact: legacy findings 57 → 0; ascii-quote 1 → 0. style-guide-check baseline теперь bucket-a advisory only (115).

§5.1.2 v1.1 (2026-05-19 — minor)

Type: Minor (new rule per §5.3 — pilot deferred до старта `en-standard` эпика). **Amendment:** `../research/ru-style-amendment-002-en-translation-uppercase.md` — EN translation UPPERCASE convention.

Rationale: v1.0.1 содержал «(TBD)» в §2.2.4 для EN-translation RFC-2119 wording policy. EN epic (en-{standard,guide,reference,core}) при старте получал ambiguous Style Guide signal. RFC 8174 fixed canonical EN convention: только UPPERCASE формы несут normative weight.

RU carve-out §2.1.3 — narrow exemption, не распространяется на EN. International standard practice (ISO/IEC/IEEE 29148, 12207, 15288 в EN) — UPPERCASE consistent.

Delta:

- §2.2.4 — убран маркер «(TBD)»; clause sharpened с cross-ref на §2.2.5.
- §2.2.5 (new, normative) — «EN translation convention»: EN-перевод использует UPPERCASE RFC-2119 keywords. Rule + rationale + применение + anti-patterns + pilot trigger.
- §2.3 — добавлен «Bidirectional usage» note: таблица работает в обоих направлениях (RU pass + EN translation).
- Frontmatter `version: "1.0.1"` → `"1.1"`.

Pilot impact: 0 RU chapters affected (правило адресуется только EN корпусу). **Pilot trigger:** при старте `en-standard` эпика — pilot chapter (recommended: `standard/en/04-terms.md` per §5.1 RU pilot precedent). **Retroactive Phase 5 scan:** N/A — EN content ещё не существует.

§5.2 Amendment procedure

Style Guide amendments — formal procedure (mirrors `../standard/01-scope.md §1.7` closed-list amendment policy).

§5.2.1 Proposal phase

1. Submitter создаёт `research/ru-style-amendment-NNN-<topic>.md` (kebab-case, NNN — sequential).

2. **Frontmatter: ```yaml**

title: "RU Style Amendment NNN: " status: proposed phase: "Phase 2.x amendment" epic: ru-normative-pass-v1 task: ru-style-amendment-NNN- draft-date: YYYY-MM-DD target-section: "§X.Y of reference/06-ru-style-guide.md" lang: ru



3. Sections required:
    - Rationale (why amendment needed; what fails without it).
    - Proposed change (delta — old text → new text).
    - Pilot impact (estimate Phase 5 chapter passes affected).
    - Migration plan (если retroactive change).

§5.2.2 Review phase

Editor (project owner + 1 reviewer minimum):

1. Verify rationale not duplicating existing § policy.
2. Verify proposed delta — well-scoped, semantic-preserving (or explicit content-change tag).
3. Check pilot impact estimate vs. capacity.
4. Accept / reject / request revision.

§5.2.3 Pilot phase

Accepted amendment — pilot validation на 1 chapter (matches Phase 3 pilot mechanism):

1. Pick representative chapter (judgement per amendment scope).
2. Apply amendment + existing §1-§3 ruleset.
3. User checkpoint diff review.
4. Pass / fail.

§5.2.4 Lock-in phase

Pilot pass → Style Guide version bump:

1. Edit `reference/06-ru-style-guide.md` with amendment delta.
2. Frontmatter version: `v1.0` → `v1.0.1` (patch) или `v1.1` (minor) или `v2.0` (major — breaking).
3. Update §5 Change history entry.
4. Commit: `docs(reference): Style Guide vN.M — amendment NNN (<topic>)`.

§5.2.5 Retroactive phase

Уже-pass'd главы Phase 5 — re-scanned для compliance с new amendment:

1. Identify affected chapters (per amendment migration plan).
2. Schedule re-pass tasks.
3. Track в `ru-normative-pass-v1` epic.

§5.3 Semantic versioning

Version segment	Triggers
Major (v1 → v2)	Breaking — fundamental rule change (e.g. F2 regime reversal). Requires full Phase 5 re-pass.
Minor (v1.0 → v1.1)	New rule / closed list extension. Pilot + retroactive scan.
Patch (v1.0 → v1.0.1)	Clarification / typo / rationale extension. No retroactive Phase 5 scan.

---

§6 AC mapping (lock-in)

AC#	Criterion	Evidence
AC-1	Artifact `reference/06-ru-style-guide.md`, status:approved (post-Phase 3 pilot 2026-05-17)	этот файл, frontmatter
AC-2	§0 Introduction — normative status + scope + application protocol + change procedure	§0.1-§0.6
AC-3	§1 Terminology — full compile, intra-doc anchors adjusted	§1.0-§1.16
AC-4	§2 RFC-2119 RU wording — full compile	§2.0-§2.11
AC-5	§3 Formatting — full compile	§3.0-§3.11
AC-6	§4 Integration — closed-list 17th + standard/04 sync + validate-frontmatter + Phase 5 protocol	§4.1-§4.5
AC-7	§5 Change history — v1.0 + amendment procedure	§5.1-§5.3
AC-8	Path adjustments — cross-draft refs → intra-doc; research → ../research/; reference → basename	grep verified pre-commit
AC-9	Cross-link на 3 frozen inventories + editorial audit + 3 source drafts	§5.1 + body references
AC-10	AC mapping table	этот §
AC-11	Read-only invariant — единственный новый файл reference/06-ru-style-guide.md	git status

AC-12 (neg)	NO broken intra-doc anchors — grep `(ru-style-guide-draft-section-)` returns 0	post-commit grep
AC-13 (neg)	NO duplicate sections — `## §1.0 / §2.0 / §3.0` each appears 1×	post-commit grep
AC-14 (neg)	NO "DRAFT" labels — `(DRAFT) / Phase 2 draft` 0 matches	post-commit grep

§7 Limitations & follow-ups

1. **Status: approved** (2026-05-17 via Phase 3 pilot on `../standard/04-terms.md`, commit `564c8d4`). Phase 5 mass chapter pass unblocked.
2. **Phase 1.5 prerequisite:** `01-glossary.md` reconciliation (Phase 4 task `ru-glossary-cleanup-uic-aic`, also `ru-reconcile-glossary-vs-standard` wider scope) recommended до Phase 5 mass pass. Glossary drift propagation risk per editorial-audit §7 R4.
3. **tooling baseline:** `scripts/style-guide-check.js` v0.2.0 implemented 2026-05-19 (amendment 004 narrowing 2026-05-20) (§4.3) — 6 checks live. Known limitations: bucket-a advisory pending §1.3 case-policy clarification. Legacy check citation-narrowed (amendment 004): mapping rows / deprecation markers / migration sections не flagged. Manual grep checklist retained as backup для edge cases.
4. **EN translation:** out of scope для v1.0 (§0.3.2). `standard/en/` Phase 6/7 will need parallel Style Guide-EN (separate epic).
5. `research/` **drafts** — author discretion; Style Guide applies post-freeze (e.g. this artifact's source drafts).
6. **Imagery policy:** alt-text / SVG conventions deferred until imagery introduction (Phase 6/7 epic).
7. **Density-cap 1.0%** (§1.6.2) — initial threshold, not empirically validated. Phase 3 pilot may adjust.
8. **Subagent prompt templates** (§0.4.2 + §2.11) — initial versions; Phase 5 may iterate per subagent invocation feedback.
9. **MVR exact mapping** (§2.6.1) — generic placeholder; actual per-MVR migration in Phase 5 chapter pass на `standard/00`.
10. **Subagent prompt templates** не code; они content. Если subagent invocations meaningfully diverge — pull templates into `reference/04-ai-style-guide.md` (cross-link instead of duplicate).

§8 Источники

- Memory #31 `ru-pass-principles` (epic `ru-normative-pass-v1`)
- Memory #34 `Session #15 autonomy override` (Phase 2 self-commit разрешён для lock-in session)
- Decision #24 «RU normative pass v0.1→v1.0 — 7-фазный подход»
- Decision #26 «Bucket B keep latin»
- Decision #27 «F1 reconciliation = Phase 1.5 prerequisite для Phase 5»
- Decision #28 «Phase 1 audit triplet pattern»
- Phase 1 deliverables (frozen):
  - `../research/ru-anglicism-inventory.md` (commit `6b1a9dd`)
  - `../research/ru-style-inconsistencies.md` (commit `b00d635`)
  - `../research/ru-tone-sampling.md` (commit `b3d4bde`)
  - `../research/ru-editorial-audit.md` (commit `de25e8c`)
- Phase 2 source artifacts (status:draft в research/, compiled here as §1-§3):
  - `../research/ru-style-guide-draft-section-1-terminology.md` (commit `5da0832`)
  - `../research/ru-style-guide-draft-section-2-rfc2119.md` (commit `2e76599`)
  - `../research/ru-style-guide-draft-section-3-formatting.md` (commit `a7a5d93`)
- Canonical RENAR sources:
  - `../standard/04-terms.md` (especially §3.13.3 UI projection + §3.14.1 legacy mapping)
  - `../standard/01-scope.md §1.7` (closed-list master policy)
  - `../standard/14-normative-refs.md` (RFC 2119 + RFC 8174 + ISO 29148 citations)
- External normative:
  - RFC 2119 (https://www.rfc-editor.org/rfc/rfc2119)
  - RFC 8174 (https://www.rfc-editor.org/rfc/rfc8174)
  - ISO/IEC/IEEE 29148:2018
  - CommonMark Specification (https://spec.commonmark.org/)

---

title: "ISO/IEC 29148 — матрица трассировки" description: "Mapping атрибутов ISO/IEC/IEEE 29148:2018 на поля frontmatter RENAR (BR/SR/TR/SPEC/TC)." order: 7 lang: ru version: "1.0-draft"

ISO/IEC 29148 — матрица трассировки

> ***Назначение:*** *проверяемое соответствие заявления conformance к ISO/IEC/IEEE 29148:2018 (standard/14 §14.4.2). Нормативные определения полей — в standard/06, standard/08, standard/09, 02-schemas.md.*

RENAR упрощает набор обязательных атрибутов 29148 (18 → 7–8 на артефакт) и добавляет TC как полноценный артефакт, ADAPT и SPEC-ось. Таблица ниже — **полная** трассировка для оценщика conformance и для заполнения `external-claims[]` в manifest.

---

1. Классы требований (29148 §5)

ISO/IEC 29148 класс	RENAR артефакт	Нормативный источник
Stakeholder requirement	`BR`	§6.5
System requirement	`SR` (level: system / subsystem / module)	§6.6
Software requirement (implementation unit)	`TR`	§6.7
Interface / design specification	`SPEC-*` (9 типов)	§8
Verification item	`TC`	§9
Requirements validation (client interpretation)	`ADAPT`	§7

---

2. Атрибуты требования (29148 Table B.1 → RENAR)

#	ISO/IEC 29148 атрибут	RENAR поле / механизм	Обязательность	Примечание
1	Unique ID	`id` (immutable)	mandatory	V1; см. §3.3.1
2	Requirement statement	body (Потребность / Поведение / …)	mandatory	EARS-шаблон для SR: §6.6.3
3	Rationale	body «Контекст» + `source.adapt-section`	mandatory (BR/SR)	Traceability к ADAPT
4	Source	`source.adapt`, `source.tz-section`, `source.document-ref`	mandatory	V5 pinning через `document-ref`
5	Fit criterion	body «Критерии успеха» (BR) / Pass-критерии TC	mandatory	Измеримость через 25022/25023: §14.4.4
6	Priority	`priority` (MoSCoW)	mandatory	WSJF — informative в SAFe mapping
7	Owner	`owner` (BR/SR) / `business-context.stakeholder`	mandatory	§6.5.2
8	Status	`status` (lifecycle enum)	mandatory	State machines: §10
9	Verification method	`verified-by[]` → `TC` + `tc-type`	обязательно для verify	TC как полноценный артефакт — расширение 29148
10	Parent / child	`parent.id`, auto `children[]`	mandatory (SR/TR)	Иерархия BR→SR→TR
11	Traceability (derived)	`verified-by`, `constrained-by[]`, `implements-spec[]`, KG edges	derived	reference/05 §4
12	Version	нативная для носителя version + `requirement-`	mandatory (V5)	§3.3.5

		`version` в TC		
13	Author	V6 author + `ai-provenance`	mandatory (RENAR-4+ AI)	§4.10.1
14	Date created / modified	timestamps записи изменений носителя	derived (V6)	Audit trail: §10.13
15	Risk	`compliance[]` , AIR register link	optional / domain	03-ai-risk-register.md
16	Assumption	ADAPT backward findings `type: assumption`	via ADAPT	§7.4.4
17	Dependency	`depends-on[]` (SPEC), `constrained-by[]` (SR)	mandatory где применимо	DAG invariant: 02 §9
18	Approval authority	QG-0 / QG-2 + ADAPT dual signature	mandatory	Замена formal walkthrough: §14.4.2

Не принято из 29148: review meetings и inspection-only verification без TC evidence — см. §14.7.

---

3. Verification methods (29148 §6.4)

29148 method	RENAR реализация
Test	`TC` с `tc-type: system | acceptance | contract | …`
Demonstration	`tc-type: acceptance` + client sign-off (QG-4 optional)
Inspection	`[test-spec-change]` workflow + adversarial review (§9.13)
Analysis	SR с `quality-characteristic` + eval-TC (`tc-type: eval`) для SPEC-AI

---

4. Lifecycle processes (29148 §6)

29148 process	RENAR глава	Gate
Requirements elicitation	ADAPT backward + T3	QG-0 (ADAPT approve)
Requirements analysis	BR/SR decomposition	QG-0 (BR/SR approve)

Requirements specification	SPEC axis	QG-3 Architecture (optional/required)
Requirements verification	TC + QG-2	QG-2 Verification
Requirements validation	ADAPT client signature + QG-4	QG-4 Acceptance (optional)
Requirements management	Lifecycle §10 + носитель V1–V6	Continuous

---

5. Использование при оценке conformance

1. Для каждого заявления `ISO/IEC/IEEE 29148:2018` в manifest — пройти строки §2–§5.
2. Выборочно (≥10% артефактов или все BR/SR уровня system) проверить наличие обязательных полей и трассировку `source.adapt`.
3. Несоответствие любой **обязательной** строки §14 — частичное заявление недопустимо (§14.6.2).

Reference RENAR 1.0-draft — renar.tech

---

title: "Самооценка conformance" description: "Печатный чек-лист, взаимное соответствие MVR↔§13.3, шаблон RENAR-CONFORMANCE.yaml для self-assessment." order: 8 lang: ru version: "1.0-draft"

Самооценка conformance

> ***Назначение:*** *практический kit для Tech Lead / Architect перед выпуском RENAR-CONFORMANCE.yaml. Нормативная база — standard/13. Этот документ* ***informative****; при расхождении побеждает standard/13.*

---

1. Взаимное соответствие MVR ↔ обязательные пункты §13.3

MVR (§0.5.1)	§13.3 clause	Поле `mandatory-clauses-confirmed`
MVR-1 SoT inversion	§13.3.1	`sot-inversion: true`
MVR-2 V1–V6	§13.3.2	`substrate-v1-v6: { v1..v6: true }`
MVR-3 ADAPT per T3	§14.3.3	`adapt-per-tz: true`
MVR-4 9 SPEC types	§14.3.4	`spec-types-closed-list: true`
MVR-5 TC pos/neg	§13.3.5	`tc-pos-neg-pairing: true`
MVR-6 QG closed list	§13.3.6	`quality-gates-closed-list: true`
MVR-7 conformance manifest	§13.4 (artifact)	manifest exists + signed
— (closed lists policy)	§13.3.7	`closed-lists-backward-findings: true`

Все семь MVR + §13.3.7 обязательны для **любого** уровня RENAR-1..5.

---

2. Чек-лист самооценки (обязательные пункты)

Отметьте после проверки evidence в носителе:

§13.3.1 SoT inversion

- [ ] Иерархия BR/SR/SPEC/TC — authoritative source о поведении
- [ ] Нет SR, восстановленных из кода без bug-fix justification

- [ ] Drift-hooks / review policy блокируют silent SR←code adaptation

§13.3.2 Capabilities V1–V6 (носитель)

- [ ] V1 immutable history — включён
- [ ] V2 atomic change unit — включён
- [ ] V3 diff & review — включён
- [ ] V4 branching / change-set — включён
- [ ] V5 сквозная фиксация версии между носителями — включена (`verifies[].requirement-version`)
- [ ] V6 author + timestamp — включён

§13.3.3 ADAPT

- [ ] Каждое активное T3 имеет approved ADAPT
- [ ] Каждый delta-T3 имеет delta-ADAPT
- [ ] Dual signature (Architect + Client representative) зафиксирована

§13.3.4 SPEC types

- [ ] Все SPEC ∈ {ARCH, API, DATA, INT, PROC, UI, AI, SEC, OPS}
- [ ] Нет локальных `SPEC-CUSTOM-*`

§13.3.5 TC pos/neg

- [ ] Каждое верифицируемое утверждение имеет pos + neg TC (или negative invariant exception)
- [ ] QG-2 блокирует `verified` при нарушении

§13.3.6 Quality Gates

- [ ] QG-0, QG-1, QG-2 реализованы как `required`
- [ ] QG-3, QG-4 объявлены `required` | `declared` | `absent` в manifest
- [ ] Нет локальных custom gates

§13.3.7 Closed lists

- [ ] Backward finding types — только closed list §7.4.4
- [ ] SPEC decomposition types — только closed list §8

Правило: хотя бы один unchecked → manifest **не выпускать** (§13.5.1).

---

3. Чек-лист уровня (выберите целевой RENAR-N)

Минимум для заявления уровня — standard/11 §11.4–12.8. Краткая сводка:

Уровень	Ключевые доп. критерии
RENAR-1	Mandatory clauses only; frontmatter минимален
RENAR-2	Canonical frontmatter + lifecycle statuses enforced
RENAR-3	Full SPEC axis + hooks на все QG-0..QG-2
RENAR-4	ai-provenance mandatory; adversarial review для SPEC-SEC/AI
RENAR-5	Multi-model agreement; continuous eval; KG reconciliation

- [ ] Выбранный `level` в manifest **не выше** фактически пройденного checklist
- [ ] `declared-stricter` (если есть) документирован отдельно

---

4. Шаблон manifest (минимальный)

Сохранить как `RENAR-CONFORMANCE.yaml` в корне носителя требований:

```yaml
manifest-version: 1
manifest-id: "CFM-YYYY-NNN"
renar-version: "1.0-draft"
senar-version: "1.0"
level: RENAR-2
assessment-date: "2026-05-22"
assessment-type: self
next-assessment-due: "2026-08-22"

mandatory-clauses-confirmed:
  sot-inversion: true
  substrate-v1-v6: { v1: true, v2: true, v3: true, v4: true, v5:
true, v6: true }
  adapt-per-tz: true
  spec-types-closed-list: true
  tc-pos-neg-pairing: true
  quality-gates-closed-list: true
```

```
  closed-lists-backward-findings: true

quality-gates:
  qg-0: required
  qg-1: required
  qg-2: required
  qg-3: declared
  qg-4: absent

external-claims:
  - standard: "ISO/IEC/IEEE 29148:2018"
    scope: "requirements classes, attributes, lifecycle,
verification"
    evidence: "reference/07-iso29148-trace-matrix.md"

substrate:
  type: git
  capabilities-verified: "2026-05-22"
  project-req-ref: "<нативный для носителя pointer>"

signed-by:
  name: "<Architect / Tech Lead>"
  role: approver
  signed-at: "2026-05-22T12:00:00Z"
```

Полный список полей — §13.4.2.

---

5. Периодичность

- Самооценка: **квартально** (default)
- После delta-ТЗ с impact на mandatory clauses — **внепланово**
- Loss-of-conformance triggers — §13.8

Reference RENAR 1.0-draft — renar.tech

---

title: "Педагогическая плотность глав" description: "Normative density scores standard/00–14; signposts для

hotspots (WC-02)." order: 9 lang: ru version: "1.0-draft"

Педагогическая плотность (WC-02)

> ***Informative.*** *Оценка плотности нормативного текста для calibrating reader journey. Не изменяет mandatory clauses.*

Метод (2026-05-22): `density score = (RFC-2119 RU markers + closed-list refs + state machine tables) / 1000 lines`, ручная калибровка + line count. Шкала: **L** (low ≤3), **M** (3–6), **H** (6–9), **VH** (≥9).

По главам (оценки плотности)

Глава	Lines	Score	Tier	Signpost
00 Introduction	253	4.2	M	§0.6.4 — начните с core
01 Scope	267	5.8	M	closed lists §1.7 — см. glossary
02 Methodology	241	5.0	M	SoT — core Rule 2 сначала
03 носитель V1–V6	246	5.2	M	→ guide/03 или guide/04
04 Terms	416	7.2	H	→ reference/01 для примеров
05 Roles	294	4.5	M	→ guide/01 walkthrough
06 Hierarchy	564	8.4	H	→ guide/00 quickstart перед frontmatter
07 ADAPT	355	6.8	H	→ core ADAPT section
08 Specifications	426	7.9	H	→ guide/09 E3 SPEC examples
09 Test cases	550	9.1	VH	→ reference/02 §8 + §9.1.1 decision tree
10 Lifecycle QG	628	9.5	VH	→ §10.1.1 QG tree + guide/00
11 Maturity	361	4.8	M	→ guide/02 transition
12 Metrics	361	3.9	M	provisional targets §11.3.10
13 Conformance	394	8.0	H	→ reference/08 kit first

14 Normative refs	~220	2.8	M	→ reference/11 расширенный каталог; §14.5 «пять ключевых»

Топ-5 «горячих точек» (signposted)

1. **§10 Lifecycle** — см. quickstart QG flow
2. **§9 TC** — см. schemas §8 + adversarial §guide/07
3. **§6 Hierarchy** — см. quickstart + walkthrough
4. **§13 Conformance** — см. reference/08 kit
5. **§8 SPEC** — см. guide/09 E3

Reference RENAR 1.0-draft — renar.tech

---

title: "Профиль реализации для агента" description: "Абстрактный контракт исполнения RENAR для implementer-ов. Машиночитаемый индекс: normative-index.yaml." order: 10 lang: ru version: "1.0-draft"

Профиль реализации для агента

> ***Назначение:*** *informative contract для агента или нативного для носителя runtime, который* ***имплементирует*** *RENAR без догадок. Нормативный текст — `standard/`; этот документ — operational mapping без привязки к конкретному vendor tooling.*
> ***Machine-readable:*** `normative-index.yaml`

---

1. Как читать таблицу

Колонка	Смысл
`clause_id`	Стабильный ID из `normative-index.yaml`
`Действие агента (абстрактно)`	Что должен уметь runtime **без** vendor CLI
`Входы / выходы`	Артефакты носителя
`Gate`	Контрольная точка качества или runner-managed check

---

2. MVR ↔ действия агента

clause_id	Действие агента (абстрактно)	Входы	Выходы	Gate
MVR-1	Блокировать reverse-engineering SR/SPEC из кода без bug-fix justification; SoT = requirements hierarchy	code diff, SR/SPEC	audit record / blocked promotion	—
MVR-2	Проверить, что носитель декларирует V1–V6 в manifest; прогнать capability checks	`RENAR-CONFORMANCE.yaml`	pass/fail report	—

MVR-3	Обеспечить approved ADAPT на каждое активное T3; delta-T3 → delta-ADAPT	TZ, ADAPT draft	ADAPT approved	QG-ADAPT-approve
MVR-4	Отклонять SPEC с type ∉ closed list	SPEC frontmatter	validation error	QG-spec-approved
MVR-5	Обеспечить pos/neg пару TC на каждое нормативное утверждение	SR/SPEC, TC set	paired TC	QG-2
MVR-6	Отклонять custom gate types; требовать QG-0..2	project config	manifest	—
MVR-7	Требовать подписанный `RENAR-CONFORMANCE.yaml` с `senar-version`	manifest	conformance claim	—

---

3. Взаимное соответствие MVR ↔ обязательные пункты §13.3

Полное взаимное соответствие MVR ↔ §13.3 — `08-conformance-self-assessment.md §1`. Агент **не должен** считать проект conformant, если любое поле `mandatory-clauses-confirmed` = false.

---

4. Контракт gate runner (абстрактно)

Gate	Pre (проверки агента)	Post (записи агента)
QG-0	Schema valid; parent links; ADAPT exists for TZ-backed SR	`approved` transition + audit-trail
QG-1	TR links `implements-spec[]`; носитель реализации pinned (V5)	TR `in_progress` → `done` evidence
QG-2	All `verified-by` TC `passing`; pos/neg; `last-run.requirement-version` match	artifact → `verified`

Decision trees: standard/09 §9.1.1, standard/10 §10.1.1.

---

5. Правило нейтральности к носителю для implementer-ов

Runtime **должен** map-ить abstract actions на нативные для носителя primitives через `RENAR-CONFORMANCE.yaml#v1-v6-mapping` (§3.7). Vendor-specific команды **не** могут быть единственным способом выполнить mandatory clause.

---

6. Статус покрытия (v1.0-draft)

Область	Index entries	Profile rows	Примечание
MVR	7/7	7/7	complete
§13.3 mandatory	7/7	via bijection	complete
QG-0..2	3/3	3/3	QG-3/4 deferred optional
Обязательные clauses по главам	partial	—	expand in later pass

---